# LLD Handbook — Vol 5: Machine Coding, Revision and Practice

OOP · UML · SOLID · Clean Code · Refactoring · Design Patterns · Concurrency · 45+ case studies in Java

4 parts · one place to study LLD: DesignGurus courses + classic books + GitHub problem sets, combined

what it is | problem | how it works | trade-offs | real world | interview | self-check | TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

# Contents


1. **Chapter 21 — Case Studies: Machine-Coding Round Problems**
   - 21.0 How the machine-coding round works
   - 21.1 Design a Task Management System
   - 21.2 Design a Digital Wallet Service
   - 21.3 Design an Event Calendar (Flipkart)
   - 21.4 Design a LeetCode-like Coding Platform — "Coding Blox" (Flipkart)
   - 21.5 Design a Property Listing Site — "PropertyHunt" (ClearTrip)
   - 21.6 Design a Loan Ledger — "Ledger Co" (Navi)
   - 21.7 Design an Inventory Management System
   - 21.8 Design a Unix File Search (`find`)
   - Chapter summary
2. **Chapter 22 — Revision: Cheat Sheets & Glossary**
   - 22.1 How to revise
   - 22.2 The LLD interview answer framework (one page)
   - 22.3 OOP & relationships cheat sheet
   - 22.4 SOLID one-pager
   - 22.5 Other design principles and Clean Code rules
   - 22.6 Code smells → refactorings (all 24)
   - 22.7 Design patterns: all 23 GoF + extras
   - 22.8 Concurrency cheat sheet
   - 22.9 Case-study map
   - 22.10 Machine-coding and edge-case checklists
   - 22.11 Glossary
   - Self-check
   - TL;DR
3. **Chapter 23 — Interview Question Bank**
   - How to use this bank
   - Part A — "Design X" (LLD and machine coding) questions
   - Everyday objects
   - Booking and reservations
   - Games
   - Platforms and marketplaces
   - Components and infrastructure
   - Machine-coding round favourites
   - Part B — Concept questions
   - OOP basics
   - Relationships and UML
   - SOLID
   - Design principles and clean code
   - Creational patterns
   - Structural patterns
   - Behavioral patterns
   - Concurrency
   - Java specifics for LLD rounds
   - Part C — How to run a 60–90 minute machine-coding round
   - Sources of the question lists
4. **Chapter 24 — Mock Interview Drills**
   - 24.0 How to use these drills
   - 24.1 Drill — Parking Lot
   - 24.2 Drill — Elevator System
   - 24.3 Drill — Library Management System
   - 24.4 Drill — Amazon Locker Service
   - 24.5 Drill — Vending Machine
   - 24.6 Drill — Online Blackjack Game
   - 24.7 Drill — Ride Sharing (practice session)
   - 24.8 Drill — Meeting Scheduler
   - 24.9 Drill — Movie Ticket Booking System
   - 24.10 Drill — Car Rental System
   - 24.11 Drill — ATM
   - 24.12 Drill — Chess Game
   - 24.13 Drill — Hotel Management System
   - 24.14 Drill — Food Delivery (practice session)
   - 24.15 Drill — Amazon Online Shopping System
   - 24.16 Drill — Stack Overflow
   - 24.17 Drill — Restaurant Management System
   - 24.18 Drill — Facebook
   - 24.19 Drill — Online Stock Brokerage System
   - 24.20 Drill — Jigsaw Puzzle
   - 24.21 Drill — Airline Management System
   - 24.22 Drill — ESPNcricinfo
   - 24.23 Drill — LinkedIn
   - Chapter summary

# Chapter 21 — Case Studies: Machine-Coding Round Problems

> **Goal.** Chapters 14–20 taught you to *design* a system on a whiteboard. This chapter trains the harder cousin of that interview: the **machine-coding round**, where you get a written problem statement, 90–120 minutes, and must hand over **running code** that the interviewer will feed their own inputs into. You will learn how the round is scored, a project skeleton you can type from memory, a minute-by-minute time plan, and then eight real problems — each solved end to end with a working, demo-able (ready to run in front of the interviewer) Java program.

**How each problem is laid out:** *Problem & clarifying questions → Requirements → Use cases → Core entities → Class diagram → Key flows → Design decisions & patterns → Java implementation (+ real output) → Concurrency & edge cases → Extensions → In the interview → Self-check → TL;DR.*

All code is Java 21, standard library only (only the classes that ship with Java, no extra downloads), one package (Java's folder-like grouping of related classes) per problem under `code/lld/ch21/<name>/`. Run from `chapters/21-case-machine-coding/code`:

```
javac --release 21 -d /tmp/lld_21 $(find . -name '*.java')
java -cp /tmp/lld_21 lld.ch21.taskmgmt.Main       # or wallet, calendar, codingblox, propertyhunt, ledger, inventory, filesearch
```

Every demo uses a fixed clock, a seeded random generator (a random-number generator started from a fixed "seed" number, so it produces the same sequence on every run), or sorted output, so what you see below is exactly what you get on every run.

**Problem map**

| # | Problem | Where it is asked (source) | The one idea that makes it work | Patterns |
|---|---|---|---|---|
| 21.1 | Task Management System | awesome-low-level-design | Status *state machine* + composable filters | State (enum), Specification, Strategy, Observer |
| 21.2 | Digital Wallet Service | awesome-low-level-design | `BigDecimal` money, idempotency keys, lock ordering | Strategy (offers), Facade |
| 21.3 | Event Calendar | Flipkart (kumaransg/LLD) | Half-open intervals + a boundary *sweep* to find free slots | Strategy (representative choice), Repository |
| 21.4 | Coding Blox (LeetCode-like platform) | Flipkart (kumaransg/LLD) | Command dispatcher + level-based scoring rule | Command, Strategy, Repository |
| 21.5 | PropertyHunt | ClearTrip (kumaransg/LLD) | Optional search criteria → list of predicates; unit normalisation | Builder, Specification, Strategy |
| 21.6 | Ledger Co (loans & EMIs) | Navi (kumaransg/LLD, Geektrust) | Derive balances from facts instead of storing them | Strategy (interest), Command |
| 21.7 | Inventory Management System | Common (Amazon/Flipkart-style) | Append-only movement ledger + per-SKU lock | Strategy (allocation), Observer (reorder) |
| 21.8 | Unix File Search ( `find` ) | Common OOD/machine-coding question | Filters as Specifications combined by Composite AND/OR/NOT; Builder for criteria; a parser for `find` syntax | Specification, Composite (×2), Builder, Interpreter-style parser |

(Food ordering "FoodKart", another kumaransg/LLD problem, is solved in Chapter 19.)

> **Key terms.** *awesome-low-level-design* and *kumaransg/LLD* are public GitHub repositories of LLD problems; Geektrust is a hiring platform that sets such challenges; OOD = object-oriented design. Patterns (taught in Chapters 8–11): a **state machine** is a fixed set of states plus the allowed moves between them (here an enum, a Java type with a fixed list of named values); **Specification** wraps one

yes/no rule in an object so rules combine with AND / OR / NOT; **Strategy** puts each interchangeable rule in its own class behind one interface; **Observer** lets objects subscribe to events; **Facade** is one simple front class over several others; **Repository** hides storage behind save / find methods; **Command** turns each action into a handler looked up by name; **Builder** assembles an object with many optional settings step by step; **Composite** lets a group be treated like a single object; an **Interpreter-style parser** turns text in a small language into a tree of objects. Other words: **idempotency keys** (21.2); **lock ordering** = always taking several locks in one fixed order, so two threads can never block each other forever (a **thread** is one independent line of execution in a program; a **lock** is an object only one thread can hold at a time); a **half-open interval** `[start, end)` includes its start but not its end; a **sweep** walks sorted time points one by one; **unit normalisation** converts every input to one unit as soon as it is read; an **SKU** (stock keeping unit) is the id of one sellable item.

---

## 21.0 How the machine-coding round works

### What it is

A **machine-coding round** (also called an *LLD coding round*) is an object-oriented design interview (designing a program as **classes** — named code types that bundle data with the methods acting on it) in which **working code is the answer**: you get a one- or two-page problem statement (usually a PDF), your own IDE (integrated development environment: editor plus compiler and debugger, such as IntelliJ IDEA) and 90–120 minutes; the internet is for syntax only. There is **no database, no UI (user interface) and no web framework** (a library for building websites and web APIs; an **API** is the set of calls a program offers to other programs): all data lives in memory (maps and lists), and the program is driven by a `main` method, a list of commands, a file, or unit tests (small automated checks that compare a result with the expected one). It is not the **system design** interview, which is about servers, databases, caches and queues (System Design Fundamentals); this round never leaves one program — its vocabulary is classes, their fields and methods, and how they relate (OOD vs LLD vs machine coding vs HLD compared in §1.2). It is especially common at Indian product companies (Flipkart, Swiggy, PhonePe, Navi, Razorpay, CRED, Meesho, Cleartrip); several problems below are statements candidates have shared from those rounds.

### How it is judged

Interviewers use a rubric (a scoring checklist). The exact wording differs by company, but it always covers the same things. Read the "Expectations" section of any statement in this chapter (Coding Blox, PropertyHunt and Event Calendar all have one) and you will see these repeated almost word for word.

| # | Criterion | What the interviewer checks | How to score well |
|---|---|---|---|
| 1 | **Working, demo-able code** | Does it compile? Does the driver (the small piece of code that feeds input into the program and prints the results) run the sample input and print the expected output? Does it survive *their* inputs? | Get a thin end-to-end path working in the first hour. Never finish with code that does not compile. |
| 2 | **Functional completeness** | Every *mandatory* requirement done; bonus only after that. | Implement requirements in the statement's order; tick them off. |
| 3 | **Clean models** | Entities have one job, fields are private, invariants (rules that must always be true, such as "balance is never negative") live inside the entity (for example a task refuses an illegal status change). | Use enums for fixed sets, records (Java's compact, unchangeable data classes) for values, classes for things that change. |
| 4 | **Service interfaces / separation of concerns** (each class handles one kind of work) | Parsing, business logic (the product's actual rules) and storage are in different classes. No `System.out` inside services. | Driver → Service → Repository layers (skeleton below). |
| 5 | **Extensibility** | "Add a new offer / sort order / scoring rule" should mean *adding a class*, not editing `if` chains. | Strategy and Specification for things the statement says "can be X or Y". |
| 6 | **Edge cases & failure handling** | Unknown IDs, duplicates, invalid input, wrong user, double-run. The program prints a clear error and **keeps going**. | Custom exceptions (your own error classes; an exception is how Java signals an error) with good messages; the driver catches them per command. |
| 7 | **Readability** | Intuitive names, small methods, no copy-paste. | Name classes after nouns in the statement; name methods after verbs in it. |
| 8 | **Concurrency (when asked)** | Wallets, inventory, booking: can two threads corrupt state? | Say where the lock is and why; demo it with a thread pool (a fixed set of reusable threads) if time allows. |
| 9 | **Tests (bonus)** | Some companies give extra credit for unit tests. | A tiny `assert`-style self-check in `main` is enough if there is no JUnit (Java's standard unit-testing library). |

### The five-step method, and how the clock changes it

The whiteboard round's five steps — **requirements → actors & use cases → classes → relationships → code** — and its 45-minute budget (a quarter of the time before naming a class; code covers only 2–3 flows) are taught in §1.2. **Machine coding keeps the five steps but flips the proportions:** steps 1–4 shrink to about 20 minutes (time plan below) and code fills the rest, because every mandatory feature must actually run. Every problem in this chapter is laid out in that same order.

What still applies from §1.2: **make one thin path run end to end first, then widen** (a parking lot's ~12 classes have only three that carry the main flows — detailing all twelve equally leaves nothing demo-able); and the interviewer still grades **scope** (a bounded list), **one job per class**, the right **relationships** (hold / own / extend), **code that matches the model**, and **a trade-off said out loud** — not UML notation (the standard box-and-arrow notation for class diagrams).

The single most common failure is **running out of time with nothing that runs**. The second is a "god class" (a class that does everything): one `Manager` with every map and every method. The skeleton and time plan below avoid both.

### A reusable project skeleton

Type this from memory in the first five minutes; it is the same for every problem.

```
src/
  model/          entities & value objects      (Task, Wallet, Loan, Property ...)   - enums, records, small classes
  repository/     storage contracts + in-memory (TaskRepository, InMemoryTaskRepository)
  service/        business rules, one per area  (TaskService, WalletService)          - no printing, no parsing
  strategy/       pluggable rules               (Offer, AllocationStrategy, SortBy)
  exception/      domain exceptions             (NotFoundException, InsufficientFundsException)
  driver/         Main + CommandProcessor       (parses input, calls services, prints output)
```

| Layer | Knows about | Must NOT know about | Why |
|---|---|---|---|
| **model** | nothing but itself (and other models) | services, storage, console | Entities guard their own invariants (e.g. `Task.changeStatus` rejects `TODO → DONE`). |
| **repository** | model | services, console | Swapping `HashMap` for a DB (database) later touches only this layer (Dependency Inversion — services depend on interfaces, not on concrete classes; Chapter 4). |
| **service** | model, repository *interfaces*, strategy *interfaces* | concrete repositories (injected — handed in from outside rather than created inside), console | This is the part the interviewer reads most carefully. |
| **strategy** | model | services | Each "X or Y" rule in the statement becomes one interface + small classes (Open/Closed). |
| **exception** | — | — | One class per failure *kind*, with a message a user could read. |
| **driver** | services | repositories' internals | Parsing and printing are replaceable; business logic is not. |

In this book each problem lives in **one Java package** (e.g. `lld.ch21.wallet`) for brevity; in the real round create the sub-packages above — it costs nothing and the interviewer notices. Three reusable building blocks:

1. **A generic in-memory repository** — a `ConcurrentHashMap<K, V>` (a map that many threads can use at once; `K, V` are type parameters, so the same class works for any key and value type — that is what *generic* means) behind an interface with `save / findById / findAll / delete` (see `Repository<K,V>` in Coding Blox, 21.4).
2. **An ID generator** — an `AtomicInteger` (an integer threads can increment safely without a lock) per entity type (`"T" + seq.incrementAndGet()`), thread-safe and predictable, which matches statements that say "auto-increment id starting with 1".
3. **A command processor** — a `Map<String, Consumer<String[]>>` from command name to handler (`Consumer` is a function that takes an argument and returns nothing; a *handler* is the code that deals with one command), fed by a built-in `List<String>` of sample commands (Coding Blox, PropertyHunt, Ledger). Adding a command = adding one map entry (Command pattern, Chapter 10).

### The 90-minute time plan

| Minutes | Activity | Output you should have |
|---|---|---|
| 0–10 | Read the statement twice. Ask clarifying questions. Mark mandatory vs bonus. Write the sample input as your driver's input list. | A numbered requirement list, a list of questions answered. |
| 10–20 | Nouns → entities, verbs → service methods. Pick the 1–2 patterns the statement is begging for ("can be sorted by X or Y" → Strategy). | Skeleton packages created, enums and records typed. |
| 20–60 | Implement mandatory requirements **in order**, running the driver after each one. | Sample input produces the sample output. |
| 60–75 | Edge cases and validation: unknown ids, duplicates, wrong owner, invalid numbers. Custom exceptions. | Driver prints errors and continues. |
| 75–85 | Bonus features (only if all mandatory ones work). Thread-safety where money or stock is involved. | Bonus commands in the driver. |
| 85–90 | Clean up names, delete dead code (code that is never run), run everything one last time. Prepare a 2-minute walkthrough. | Code that compiles, a demo, a story. |

A good rule: **run the program at least every 15 minutes**. A compile error found at minute 88 is a disaster; the same error found at minute 30 is a 20-second fix.

### What to say in the first two minutes

1. "I'll keep everything in memory with a repository interface, so a DB can be added later without touching services."
2. "I'll drive it from `main` with the sample commands from the statement, and print errors without crashing."
3. "The mandatory list is A, B, C; the bonus is D, E. I'll do A–C first."
4. "Where does it need to be thread-safe?" (Ask. For wallets and inventory the answer is always yes.)

### Common mistakes in this round

| Mistake | Why it hurts | Do this instead |
|---|---|---|
| Starting with a perfect class diagram for 20 minutes | No time left for code | Enums/records first; the diagram lives in your head and in names |
| One `Manager` class with 15 maps | Fails "separation of concerns" | One service per area + repositories |
| `System.out.println` inside the service | Service cannot be reused or tested | Services return data; the driver prints |
| `double` for money | `0.1 + 0.2 != 0.3` — interviewers test this | `BigDecimal` (Java's exact decimal number type) with a fixed scale (number of digits after the decimal point; 21.2) or whole units (`long` paise; a paisa is 1/100 of a rupee) |
| Crashing on the first bad command | "Fails gracefully" is an explicit criterion | Catch domain exceptions (errors that describe a broken business rule) in the driver loop, print, continue |
| Implementing bonus before mandatory | Partial credit on the mandatory part | Mandatory in order; bonus last |
| Writing every class in full detail | Nothing works end to end when time runs out | Make the core flow run first; stub (write an empty placeholder for) or name the rest |
| `Random` without a seed | Output changes every run; hard to demo | Seed it, or put randomness behind a Strategy (21.4) |
| Hard-coding "today" with `LocalDate.now()` | Demo changes daily | Inject a `java.time.Clock` (Java's replaceable source of the current time; 21.1, 21.2) |

### Real-world examples

The skeleton above is the standard layering of real Java back ends (the server-side part of an application): in a Spring Boot (the most popular Java framework for building services) service, controllers (classes that receive web requests) play the "driver" role, `@Service` classes hold business rules, and `@Repository` interfaces (Spring Data) hide storage. (The `@Something` markers are annotations: labels a framework reads to know what a class is for.) The in-memory repository you write in the round is exactly the kind of fake that teams use in unit tests for those services.

### In the interview

> **30-second answer — "How do you approach a machine-coding problem?"** "I read the statement twice, list mandatory and bonus requirements and ask about anything ambiguous. I set up model / repository / service / strategy / exception / driver packages, put the sample input into the driver, and implement requirements one by one, running after each. Services return data and throw domain exceptions; the driver prints and keeps going. I finish with edge cases, thread-safety where state is shared, and bonus features if time allows."

**Follow-ups**

- *"Why an interface for an in-memory repository?"* — So services depend on an abstraction (an interface rather than a concrete class); a JDBC (Java's standard API for SQL databases) or Redis (a fast in-memory key-value store) implementation can replace it with no change to business code, and tests can use the in-memory one.
- *"How do you make random behaviour testable?"* — Put it behind an interface (a Strategy) and inject a seeded or scripted implementation for demos and tests.
- *"How much do you test?"* — At minimum, the driver replays the statement's samples and prints results I can compare with the expected output. With time, a few JUnit tests on the service layer.

### Self-check

- **Q:** Name the six packages of the skeleton. → **A:** model, repository, service, strategy, exception, driver.
- **Q:** Where should `System.out.println` live? → **A:** Only in the driver (command processor / `Main`).
- **Q:** What do you do at minute 60 if the bonus is tempting but one mandatory requirement is missing? → **A:** Finish the mandatory one; bonus only after all mandatory ones run.
- **Q:** Why inject a `Clock`? → **A:** So time-based logic (due dates, reminders) gives the same output on every run and can be tested.

### TL;DR

Working code first, clean layering second, patterns where the statement asks for variation, graceful errors everywhere. Skeleton: **model → repository → service → strategy → exception → driver**. Run often.

---

## 21.1 Design a Task Management System

### Problem & clarifying questions

Build the core of a tool like Jira, Asana or Trello (popular team task-tracking apps): users create tasks, assign them to people, move them through statuses, comment on them, and search/filter them; the system reminds people about tasks that are due. The awesome-low-level-design version asks for create / update / delete, title + description + due date + priority + status, assignment, reminders, search and filtering (by priority, due date, assignee), marking tasks complete, task history, and safe concurrent access (several users acting at the same moment without corrupting data).

Questions to ask (with the answers this solution assumes):

| Question | Assumed answer |
|---|---|
| Which statuses exist, and can any status go to any other? | TODO, IN_PROGRESS, BLOCKED, DONE, CANCELLED. Only listed transitions are legal (e.g. TODO cannot jump straight to DONE; CANCELLED is final; DONE can be re-opened). |
| One assignee or many? | One assignee (nullable: the field may be `null`, Java's "no value"). A list is an easy extension. |
| What does "reminder" mean in a console app? | A method a scheduler would call: "notify owners of open tasks due within N days or overdue". |
| Should due dates in the past be allowed? | No, not on creation. |
| How are notifications delivered? | Through a listener interface; the demo prints them. |
| "View task history" — history of what? | Two readings: each task's audit log (a permanent record of who changed what, when) — built here — and a user's list of finished tasks, which is just a search: `assignedTo(u).and(withStatus(DONE))`. |
| "Set reminders" — per task, or a sweep? | A sweep over due dates is built here; a per-task reminder time is an extension (below). |

## Requirements

### Functional

1. Register users.
2. Create a task (title, description, priority, due date); update title / priority / due date; delete a task.
3. Assign / reassign a task to a user.
4. Change status following the allowed transitions; mark done.
5. Comment on a task.
6. Search with any combination of filters (assignee, status, minimum priority, due before, open only, title contains) and a sort order (due date, priority, id).
7. Reminders for open tasks due soon or overdue.
8. Per-task activity history.

### Non-functional

- Thread-safe: two people editing the same task must not corrupt it.
- Extensible: new filters, sort orders and notification channels without editing existing code.
- Deterministic demo (injected `Clock`).

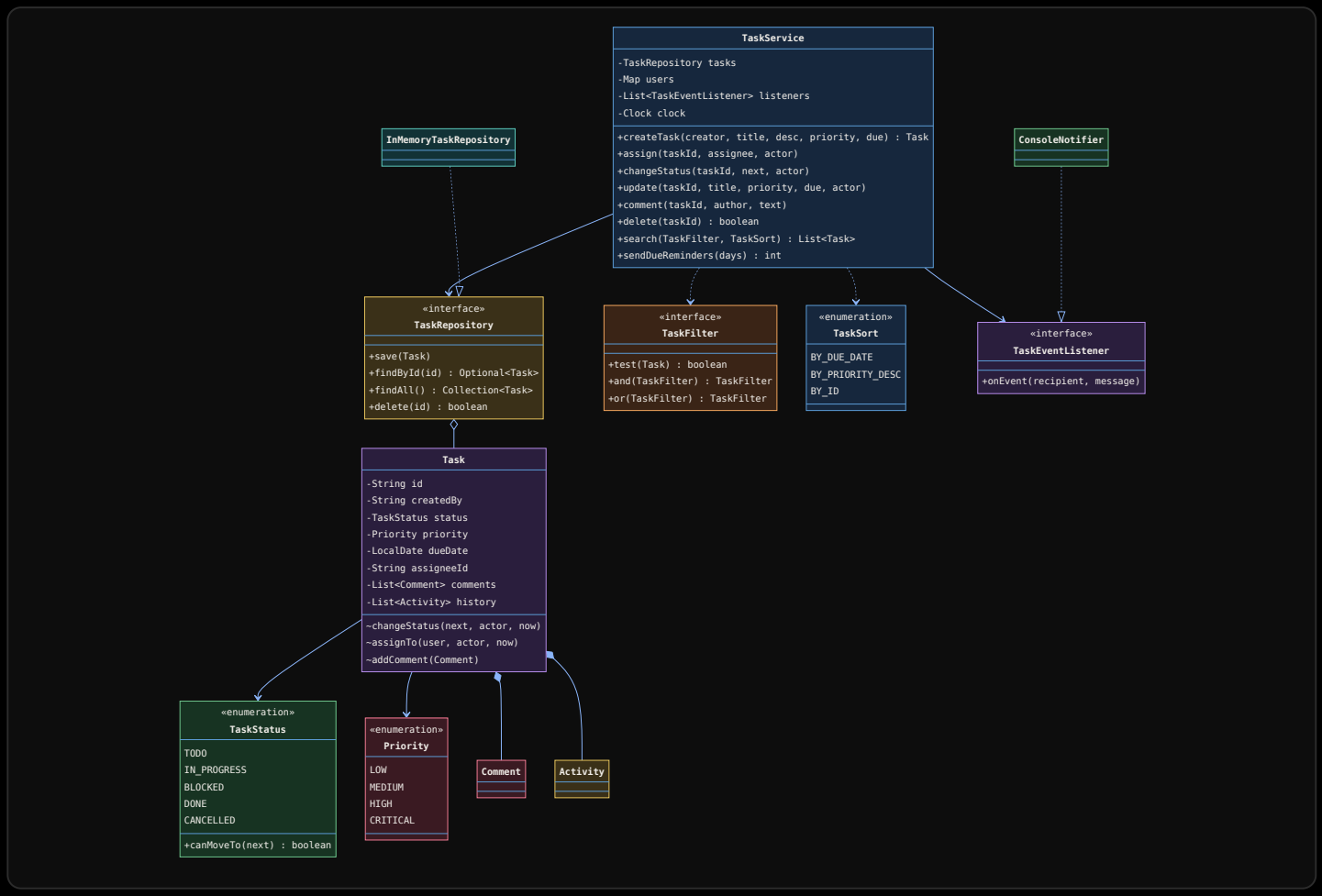
## Use cases

| Actor | Use cases |
|---|---|
| **Task creator** (any user) | create task, update, delete, assign, comment, search |
| **Assignee** | change status, comment, see own tasks |
| **Scheduler** (system actor) | send due-date reminders |
| **Notification channel** (external) | receives events (assigned, status changed, commented, reminder) |

### Core entities

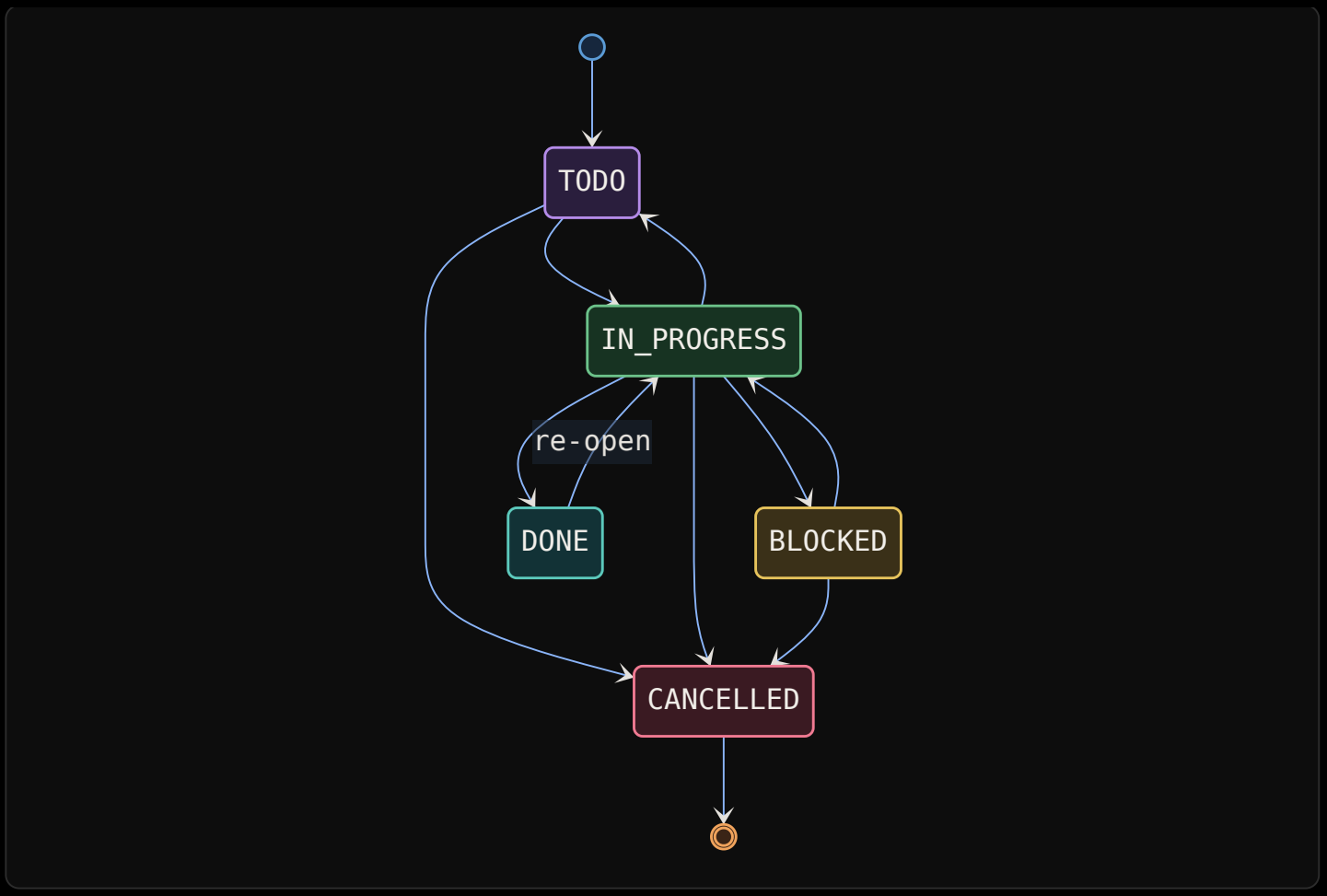
| Class | Kind | Responsibility |
|---|---|---|
| `Task` | class | Holds task data; enforces legal status changes; records its own history. All mutators (methods that change state) `synchronized` (only one thread at a time may run them on the same object). |
| `TaskStatus` | enum | Lists statuses and, for each, which statuses it may move to (a tiny state machine). |
| `Priority` | enum | LOW < MEDIUM < HIGH < CRITICAL (enum order is used for comparison). |
| `User`, `Comment`, `Activity` | records | Immutable (cannot change after creation) values. |
| `TaskRepository` / `InMemoryTaskRepository` | interface / class | Storage contract and a `ConcurrentHashMap` implementation. |
| `TaskFilter` | functional interface (an interface with one method, so a short lambda function can implement it) | Composable predicate (a yes/no test; Specification pattern) with factory methods (static methods that create objects) and `and` / `or`. |
| `TaskSort` | enum | Named comparators (objects that decide which of two items sorts first; Strategy). `BY_ID` compares the number inside the id, so `T2` sorts before `T10`. |
| `TaskEventListener` / `ConsoleNotifier` | interface / class | Observer for notifications. |
| `TaskService` | class | Facade with every use case; validates input, uses the clock, fires events. |
| `NotFoundException`, `InvalidTransitionException` | exceptions | Clear failure messages. |

## Class diagram

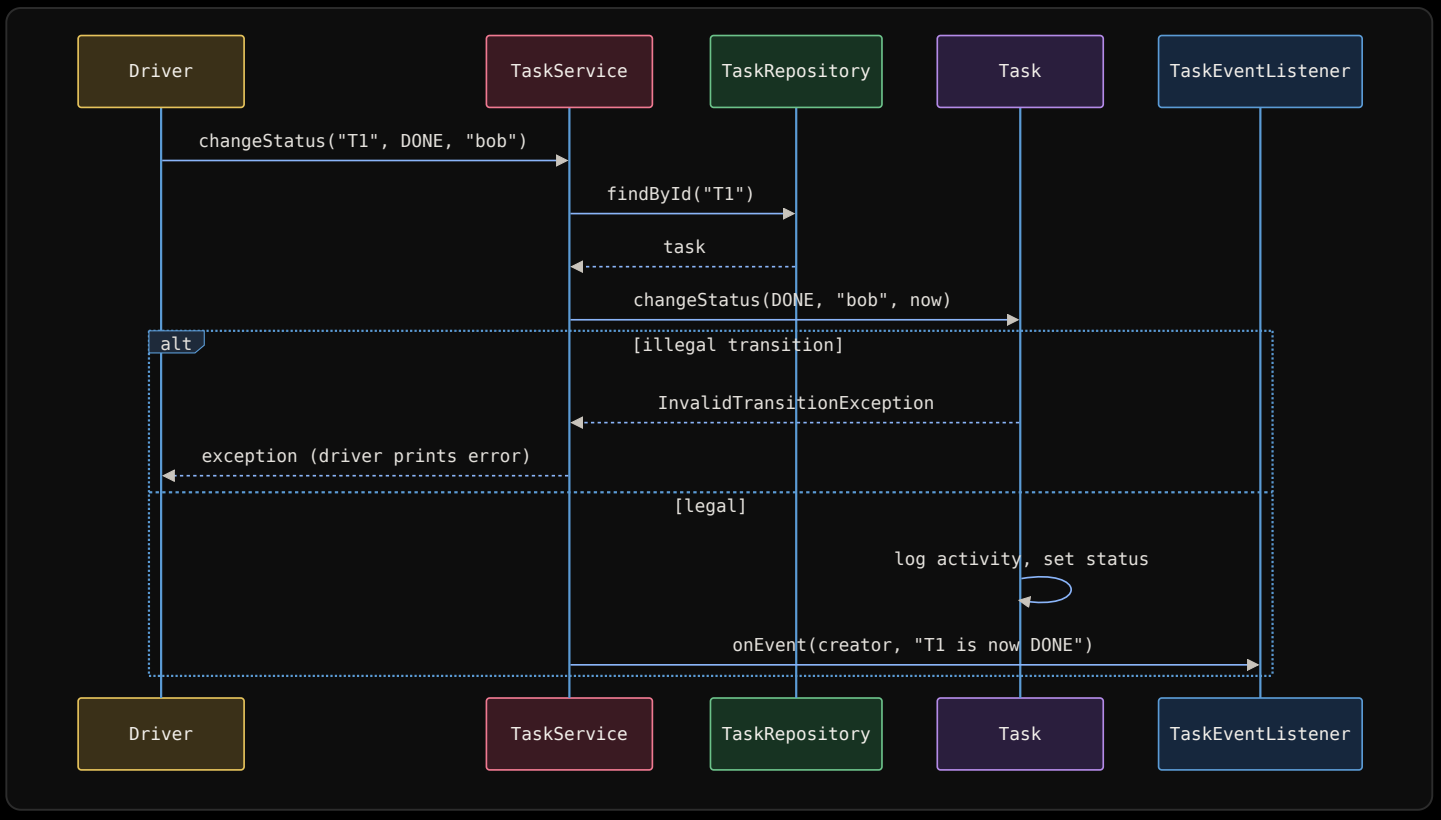



## Key flows

**Task status life cycle** — the rules live in `TaskStatus.allowedNext()`:




**Change status and notify:**



### Design decisions & patterns used

| Decision | Pattern | Why | Alternative rejected |
|---|---|---|---|
| Transitions stored in the `TaskStatus` enum | State (enum-based) | All rules in one place; `Task` just asks `canMoveTo`. | Full State pattern (one class per state, each holding that state's behaviour) with one class per status — overkill when states carry no behaviour besides transitions. |
| `TaskFilter extends Predicate<Task>` with `and/or` | Specification | Any combination of filters without a method per combination; a new filter is one factory method. | `search(String assignee, Status s, Priority p, ...)` with many nullable parameters — grows forever. |
| `TaskSort` enum of comparators | Strategy | Sort order is data the caller picks. | `switch` on a sort string inside the service. |
| `TaskEventListener` list | Observer | Email/Slack/push can be added without touching the service. | Calling `System.out` from the service. |
| `TaskService` as the only entry point | Facade | Driver sees one simple API. | Driver talking to repositories directly. |
| `Clock` injected | Dependency injection (a class receives the objects it needs from outside instead of creating them) | Deterministic reminders and tests. | `LocalDate.now()` scattered in code. |
| `synchronized` on `Task` mutators | Monitor per entity (each object's own lock guards its methods) | Fine-grained (many small locks instead of one big one): two different tasks never block each other. | One global lock on the service — simpler but serialises everything (forces all work to run one at a time). |

### Java implementation

Full runnable code: `code/lld/ch21/taskmgmt/` (16 files). Below are the files that carry the design; the rest are the `Priority` enum, the `User` / `Comment` / `Activity` records, two exceptions, the repository pair, `TaskSort`, the listener pair and `Main`.

```java
// File: lld/ch21/taskmgmt/TaskStatus.java
package lld.ch21.taskmgmt;

import java.util.EnumSet;
import java.util.Set;

/** Task lifecycle. Each status knows which statuses it may move to (a tiny state machine). */
public enum TaskStatus {
    TODO, IN_PROGRESS, BLOCKED, DONE, CANCELLED;

    public Set<TaskStatus> allowedNext() {
        return switch (this) {
            case TODO -> EnumSet.of(IN_PROGRESS, CANCELLED);
            case IN_PROGRESS -> EnumSet.of(BLOCKED, DONE, TODO, CANCELLED);
            case BLOCKED -> EnumSet.of(IN_PROGRESS, CANCELLED);
            case DONE -> EnumSet.of(IN_PROGRESS);          // re-open
            case CANCELLED -> EnumSet.noneOf(TaskStatus.class);
        };
    }

    public boolean canMoveTo(TaskStatus next) { return allowedNext().contains(next); }

    public boolean isOpen() { return this != DONE && this != CANCELLED; }
}
```

```java
// File: lld/ch21/taskmgmt/Task.java
package lld.ch21.taskmgmt;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

/**
 * The central entity. All mutators are synchronized so two threads editing
 * the same task cannot interleave (e.g. a status change racing a reassignment).
 */
public class Task {
    private final String id;
    private final String createdBy;
    private String title;
    private String description;
    private Priority priority;
    private LocalDate dueDate;
    private TaskStatus status = TaskStatus.TODO;
    private String assigneeId;                // null = unassigned
    private final List<Comment> comments = new ArrayList<>();
    private final List<Activity> history = new ArrayList<>();

    Task(String id, String createdBy, String title, String description, Priority priority, LocalDate dueDate) {
        this.id = id;
        this.createdBy = createdBy;
        this.title = title;
        this.description = description;
        this.priority = priority;
        this.dueDate = dueDate;
    }

    public String id() { return id; }
    public String createdBy() { return createdBy; }
    public synchronized String title() { return title; }
    public synchronized String description() { return description; }
    public synchronized Priority priority() { return priority; }
    public synchronized LocalDate dueDate() { return dueDate; }
    public synchronized TaskStatus status() { return status; }
    public synchronized String assigneeId() { return assigneeId; }
    public synchronized List<Comment> comments() { return List.copyOf(comments); }
    public synchronized List<Activity> history() { return List.copyOf(history); }

    synchronized void changeStatus(TaskStatus next, String actor, LocalDateTime now) {
        if (!status.canMoveTo(next)) throw new InvalidTransitionException(id, status, next);
        log(now, actor, "status " + status + " -> " + next);
        status = next;
    }

    synchronized void assignTo(String userId, String actor, LocalDateTime now) {
        if (!status.isOpen()) throw new IllegalStateException("Cannot assign a " + status + " task " + id);
        log(now, actor, "assigned to " + userId);
        assigneeId = userId;
    }

    synchronized void update(String newTitle, Priority newPriority, LocalDate newDue, String actor, LocalDateTime now)
{
        if (newTitle != null) { title = newTitle; }
        if (newPriority != null) { priority = newPriority; }
        if (newDue != null) { dueDate = newDue; }
        log(now, actor, "updated title/priority/due -> " + title + "/" + priority + "/" + dueDate);
    }

    synchronized void addComment(Comment c) {
        comments.add(c);
        log(c.at(), c.authorId(), "commented \"" + c.text() + "\"");
    }

    private void log(LocalDateTime at, String actor, String what) { history.add(new Activity(at, actor, what)); }

    @Override public synchronized String toString() {
        return "%s %-22s %-8s %-11s due=%s assignee=%s".formatted(id, title, priority, status, dueDate,
```

```
                assigneeId == null ? "-" : assigneeId);
    }
}
```

```java
// File: lld/ch21/taskmgmt/TaskFilter.java
package lld.ch21.taskmgmt;

import java.time.LocalDate;
import java.util.function.Predicate;

/**
 * Specification pattern: small, named predicates that combine with and()/or().
 * Adding a new filter = adding one factory method; the search code never changes (Open/Closed).
 */
@FunctionalInterface
public interface TaskFilter extends Predicate<Task> {

    static TaskFilter all() { return t -> true; }
    static TaskFilter assignedTo(String userId) { return t -> userId.equals(t.assigneeId()); }
    static TaskFilter withStatus(TaskStatus s) { return t -> t.status() == s; }
    static TaskFilter atLeast(Priority p) { return t -> t.priority().compareTo(p) >= 0; }
    static TaskFilter dueOnOrBefore(LocalDate d) { return t -> t.dueDate() != null && !t.dueDate().isAfter(d); }
    static TaskFilter open() { return t -> t.status().isOpen(); }
    static TaskFilter titleContains(String word) {
        return t -> t.title().toLowerCase().contains(word.toLowerCase());
    }

    default TaskFilter and(TaskFilter other) { return t -> test(t) && other.test(t); }
    default TaskFilter or(TaskFilter other) { return t -> test(t) || other.test(t); }
}
```

```java
// File: lld/ch21/taskmgmt/TaskService.java  (excerpt)
/** Facade for every use case. Holds no storage details and no printing. */
public class TaskService {
    private final TaskRepository tasks;
    private final Map<String, User> users = new ConcurrentHashMap<>();
    private final List<TaskEventListener> listeners = new CopyOnWriteArrayList<>();
    private final AtomicInteger seq = new AtomicInteger();
    private final Clock clock;

    public TaskService(TaskRepository tasks, Clock clock) {
        this.tasks = tasks;
        this.clock = clock;
    }

    // ... addListener, addUser, createTask, assign, update, comment, delete: validate, then call Task

    public void changeStatus(String taskId, TaskStatus next, String actorId) {
        requireUser(actorId);
        Task t = get(taskId);
        t.changeStatus(next, actorId, now());
        if (!t.createdBy().equals(actorId)) fire(t.createdBy(), t.id() + " is now " + next);
    }

    public List<Task> search(TaskFilter filter, TaskSort sort) {
        return tasks.findAll().stream().filter(filter).sorted(sort.comparator()).toList();
    }

    /** Reminder sweep: open tasks due within `days` days (or overdue). A scheduler would call this. */
    public int sendDueReminders(int days) {
        LocalDate limit = today().plusDays(days);
        List<Task> due = search(TaskFilter.open().and(TaskFilter.dueOnOrBefore(limit)), TaskSort.BY_DUE_DATE);
        for (Task t : due) {
            String who = t.assigneeId() != null ? t.assigneeId() : t.createdBy();
            String when = t.dueDate().isBefore(today()) ? "OVERDUE since " + t.dueDate() : "due " + t.dueDate();
            fire(who, "Reminder: " + t.id() + " '" + t.title() + "' is " + when);
        }
        return due.size();
    }

    // ... get, requireUser, today/now helpers
    private void fire(String to, String msg) { listeners.forEach(l -> l.onEvent(to, msg)); }
}
```

**Output** (of `Main.java`):

```
== create & assign
  [notify bob] alice assigned you T1 'Fix login bug'
  [notify carol] alice assigned you T2 'Write release notes'
== work on tasks
  [notify alice] T1 is now IN_PROGRESS
  [notify alice] bob commented on T1
  [notify alice] T1 is now DONE
== search: bob's open tasks, by priority
  T3 Upgrade to Java 21     CRITICAL TODO        due=2026-10-01 assignee=bob
== search: priority >= MEDIUM due within 3 days, by due date
  T4 Review PR #42          MEDIUM   TODO        due=2026-09-28 assignee=-
  T1 Fix login bug          CRITICAL DONE        due=2026-09-29 assignee=bob
  T3 Upgrade to Java 21     CRITICAL TODO        due=2026-10-01 assignee=bob
== reminders (due within 3 days)
  [notify carol] Reminder: T4 'Review PR #42' is due 2026-09-28
  [notify bob] Reminder: T3 'Upgrade to Java 21' is due 2026-10-01
  reminders sent: 2
== history of T1
  09:00 alice: assigned to bob
  09:00 bob: status TODO -> IN_PROGRESS
  09:00 bob: commented "Root cause found in PasswordValidator"
  09:00 bob: status IN_PROGRESS -> DONE
== edge cases
  error: Task T2 cannot move from TODO to DONE
  error: Task not found: T99
  error: User not found: dave
  error: Due date is in the past: 2026-09-27
  error: Cannot assign a CANCELLED task T4
  deleted T2? true, remaining=3
```

What to notice in the output: self-assignment (`bob` assigning T3 to himself) sends no notification; `T1` shows up in the priority/due search even though it is DONE (that search did not ask for `open()`), but it is **not** in the reminders, which only consider open tasks; the illegal `TODO → DONE` jump is rejected with a clear message; and the history of T1 is a complete audit trail.

### Concurrency & edge cases

- **Two users update the same task at once.** Each mutator of `Task` is `synchronized`, so a status change and a reassignment cannot interleave (have their steps mixed together); getters (methods that only read a field) are synchronized too so readers see a consistent value.
- **Storage.** `ConcurrentHashMap` for tasks and users; `findAll()` returns a copy so a search is never disturbed by a concurrent insert.
- **Listeners** (objects that asked to be told about events) are in a `CopyOnWriteArrayList` (a list that copies itself on each write, so reading needs no lock) — registering a listener while events fire is safe.
- **IDs** from an `AtomicInteger` — no duplicates under concurrency.
- **Edge cases handled:** unknown task / user / actor, blank title, past due date (on create and on update), illegal transition, assigning a closed (DONE/CANCELLED) task, self-assignment (no noise), comment notifications go to "the other party".
- **Not handled (say so):** lost updates between two *editors of the text fields* (last writer wins). Fix: optimistic locking with a `version` field — reject an update whose version is stale (out of date). Optimistic locking takes no lock up front; it only checks, at save time, that nobody else changed the record in the meantime.

### Extensions & follow-up questions

- **Sub-tasks / dependencies?** Add `parentId` and a `Set<String> blockedBy`; forbid DONE while a blocker is open (a check inside `changeStatus`).
- **Multiple assignees or watchers?** Replace `assigneeId` with `Set<String>`; notify all watchers through the same listener.
- **Recurring tasks?** A `RecurrenceRule` (daily/weekly) that creates the next task when one is marked DONE.
- **Custom workflows per project (like Jira)?** Move the transition table from the enum into a `Workflow` object (a `Map<Status, Set<Status>>`) injected per project.
- **A reminder time chosen by the user?** Store `remindAt` on the task; the scheduler's sweep fires every reminder whose time has passed and clears it.

- **Real reminders?** A `ScheduledExecutorService` (Java's tool for running tasks on a timer) calling `sendDueReminders(1)` every hour; store "already reminded" per task to avoid spam.
- **Search at scale?** Secondary indexes (extra lookup maps keyed by a field other than the id: `Map<String, Set<Task>>` by assignee, `TreeMap<LocalDate, Set<Task>>` by due date; a `TreeMap` keeps its keys sorted) or a search engine such as Elasticsearch (a popular open-source search server).

### Real-world example

In Jira, each project has a *workflow*: a set of statuses and the transitions allowed between them, and a transition can fire notifications to watchers. Search is done with JQL, a query language that combines conditions like `assignee = currentUser() AND status != Done ORDER BY priority DESC` (DESC = descending) — the same "compose filters, then sort" idea as `TaskFilter` + `TaskSort`, just with a text syntax on top.

### In the interview

Present in this order (30–45 min version): (1) statuses and transitions on the whiteboard — it shows you think about invariants; (2) `Task` and `TaskService` API; (3) filters as composable predicates — the pattern interviewers love here; (4) the observer for notifications; (5) run the demo. If asked "why not the full State pattern?", answer: statuses have no behaviour except which transitions are legal, so an enum with a transition table is simpler and still closed to illegal moves.

### Self-check

- **Q:** Where is the rule "TODO cannot go to DONE" coded? → **A:** In `TaskStatus.allowedNext()`; `Task.changeStatus` enforces it.
- **Q:** How do you add a "created by user X" filter? → **A:** Add `static TaskFilter createdBy(String u) { return t -> u.equals(t.createdBy()); }` — nothing else changes.
- **Q:** Why does the service take a `Clock`? → **A:** So "today" is controllable in demos and tests.
- **Q:** Why is `Task` synchronized per instance instead of a lock in the service? → **A:** Finer-grained: edits to different tasks never contend.

### TL;DR

Task = entity that guards its own state machine and history. Service = facade. Filters = Specification, sorts = Strategy, notifications = Observer. Thread-safety per task.

---

## 21.2 Design a Digital Wallet Service

### Problem & clarifying questions

Build the core of a wallet (an app account that stores money for payments) like Paytm, PhonePe or PayPal balance: users hold wallets, add money, transfer to other users and see a statement. The awesome-low-level-design statement also lists account/personal-info management, adding and removing payment methods, transfers to *external* accounts, multiple currencies, security, **concurrent transactions with data consistency**, and scale. Promotional **cashback** (money given back after a payment, as a reward) is not in that statement; it is added because wallet variants often ask for it, and it shows Strategy and **double-entry** thinking (every amount added somewhere is taken from somewhere else, so the books always balance). The interviewer focuses on three things: **money arithmetic**, **no double-spending under concurrency** (the same money spent twice by two simultaneous requests), and **no double-charging on retries** (idempotency).

> **Key term — idempotency:** an operation is **idempotent** if running it twice has the same effect as running it once. Networks fail in the middle of requests, so clients retry; if "transfer 100" is not idempotent, a retry moves another 100. The usual fix is an **idempotency key**: the client sends a unique id with the request, and the server remembers the result for each id and returns it again for a repeat instead of re-running the work. More in System Design Fundamentals.

| Question | Assumed answer |
|---|---|
| Money type? | `BigDecimal` with scale 2 (paise/cents). Inputs with more than 2 decimals are rejected, not silently rounded. |
| Can a balance go negative? | No. |
| What if a client retries a request (network timeout — it gave up waiting for the reply)? | Each request carries an **idempotency key**; a repeated key returns the original result and moves no money. |
| Offers? | Cashback strategies: "10% on first transfer, max 50" and "flat 25 on transfers ≥ 1000". Offers do not stack — the best one wins. |
| Where does cashback money come from? | From a **promotions wallet** (`PROMO`) that marketing tops up. Cashback is an ordinary transfer PROMO → sender, so money is never created from nothing. If the budget is used up, no cashback is paid and the transfer still stands. |
| Top-up source? | An external bank/card; modelled as a credit (money added to the wallet) with a reference. |
| Multi-currency? | Out of scope for the 90-minute version; discussed in extensions. |

## Requirements

### Functional

1. Create a wallet for a user.
2. Top up a wallet.
3. Transfer between two wallets (atomic: both sides or neither).
4. Transaction history (statement) per wallet, with running balance.
5. Apply the best eligible cashback offer after a transfer, paid out of the funded promotions wallet.
6. Idempotent top-up and transfer.

### Non-functional

- **Correctness under concurrency:** no lost updates (one thread's change silently overwritten by another's), no negative balances, no deadlocks (two threads each waiting forever for a lock the other holds), total money conserved (only top-ups add money; transfers and cashback only move it).
- Exact decimal arithmetic.
- New offers added without touching the transfer code.

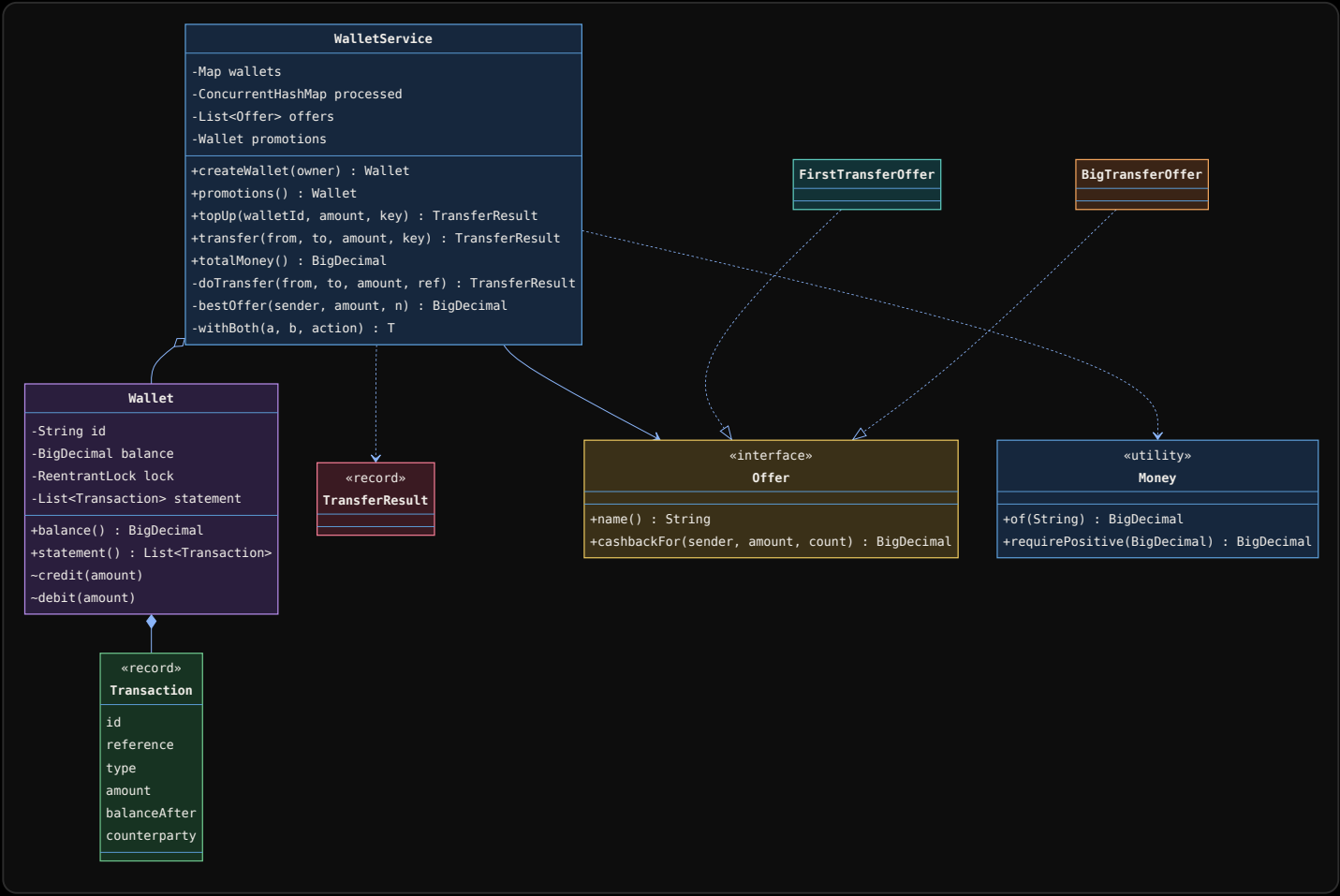
## Use cases

| Actor | Use cases |
|---|---|
| **User** | create wallet, top up, transfer, view statement |
| **Bank / card gateway** (external) | source of top-ups |
| **Marketing** (configures) | defines offers, funds the promotions wallet |
| **Client app** (retrying) | re-sends the same request with the same idempotency key |

## Core entities

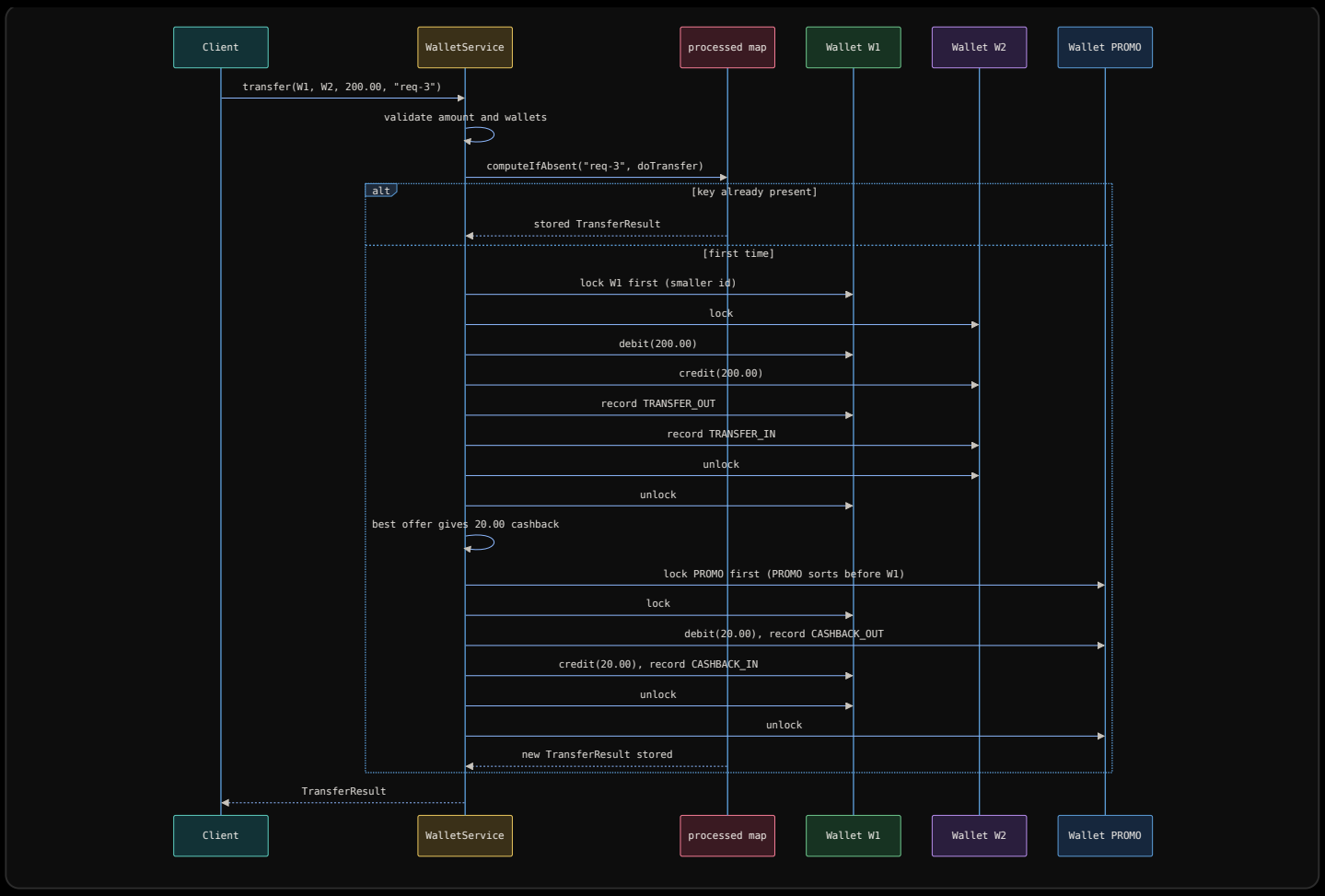
| Class | Kind | Responsibility |
|---|---|---|
| `Wallet` | class | Balance + own statement + its `ReentrantLock` (Java's explicit lock object). `debit` refuses to go below zero. The service owns one special wallet, `PROMO`, that funds cashback. |
| `Transaction` | record | Immutable ledger line: type, amount, balance after, counterparty (the other wallet involved), reference. |
| `TxnType` | enum | TOP_UP, TRANSFER_OUT, TRANSFER_IN, CASHBACK_OUT (on PROMO), CASHBACK_IN (on the user). |
| `TransferResult` | record | What the caller gets (stored for idempotent replays). |
| `Money` | utility | Normalises and validates `BigDecimal` amounts. |
| `Offer` + `FirstTransferOffer`, `BigTransferOffer` | interface + strategies | Compute cashback for a completed transfer. |
| `WalletService` | class | Facade: creates wallets, orchestrates (calls in the right order) locking, idempotency, offers. |
| `InsufficientFundsException`, `WalletNotFoundException` | exceptions | Failure reasons. |

## Class diagram




## Key flows

**Transfer with idempotency and lock ordering:**


Client
WalletService
processed map
Wallet W1
Wallet W2
Wallet PROMO
transfer(W1, W2, 200.00, "req-3")
validate amount and wallets
computeIfAbsent("req-3", doTransfer)
alt
[key already present]
stored TransferResult
[first time]
lock W1 first (smaller id)
lock
debit(200.00)
credit(200.00)
record TRANSFER_OUT
record TRANSFER_IN
unlock
unlock
best offer gives 20.00 cashback
lock PROMO first (PROMO sorts before W1)
lock
debit(20.00), record CASHBACK_OUT
credit(20.00), record CASHBACK_IN
unlock
unlock
new TransferResult stored
TransferResult
Client
WalletService
processed map
Wallet W1
Wallet W2
Wallet PROMO

### Design decisions & patterns used

| Decision | Why | Alternative rejected |
|---|---|---|
| **`BigDecimal`, scale 2, reject > 2 decimals** | `double` cannot represent 0.1 exactly; money must be exact. Rejecting `1.005` avoids silent rounding surprises. | `double` (wrong), `long` paise (also correct, and a fine choice — say so). |
| **Lock per wallet, acquired in id order** | Two opposite transfers (A→B and B→A) would deadlock if each locked its own source first. A global order (smaller id first) makes a cycle impossible. | One global lock — correct but every transfer in the system serialises. |
| **Debit before credit, both under both locks** | `debit` throws before any state changes, so a failed transfer leaves both wallets untouched (atomicity). | Credit then debit — would need a compensating action (a new step that undoes an earlier one) on failure. |
| **Idempotency via `ConcurrentHashMap.computeIfAbsent`** (creates and stores a value only if the key is missing, as one atomic step) | The check "seen this key?" and the execution are one atomic step per key, so two concurrent retries cannot both run. If the transfer throws, nothing is stored and the client can retry. | `if (!map.containsKey(k)) { run; put }` — check-then-act race (another thread can act between the check and the action). |
| **Cashback paid from a funded `PROMO` wallet** | Double-entry: every credit has a matching debit, so the sum of all wallets changes only on top-ups — which is exactly what the demo checks. A campaign cannot overspend its budget. | Crediting the user out of thin air — the books no longer balance and nobody can say what a campaign cost. |
| **Cashback as a second step, after the transfer locks are released** | Only two wallets are ever locked at once, and ordinary transfers never wait on the busy `PROMO` lock. | Locking sender, receiver and PROMO together for every transfer — correct, but every transfer in the system would queue on PROMO. |
| **`Offer` interface, best one wins** | Strategy: new promotions are new classes; the "no stacking" policy lives in one method. | `if/else` on promotion codes inside `transfer`. |
| **Statement stored in the wallet** | Reading a wallet's history needs no global scan. | One global transaction list filtered on read — fine for small data, slow at scale. |

`computeIfAbsent` detail worth knowing: the JDK (Java Development Kit: Java's compiler and standard library) documents that the whole call is atomic, that the mapping function runs only when the key is absent, that an exception from the function is rethrown with no mapping recorded, and that other threads updating the *same map* may block while it runs — so the function should be short and must not update the map itself. Our function only touches wallets, never `processed`, so it is safe. At higher scale, store a `CompletableFuture` (a handle to a result that will be ready later) with `putIfAbsent` and run the transfer outside the map (extension below).

### Java implementation

Full runnable code: `code/lld/ch21/wallet/` (12 files). Below are the files that carry the design; the rest are `TxnType`, the `Transaction` / `TransferResult` records, two exceptions, the two `Offer` strategies and `Main`.

```java
// File: lld/ch21/wallet/Money.java
package lld.ch21.wallet;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** Helpers that keep every amount at 2 decimal places. Never use double for money. */
public final class Money {
    public static final int SCALE = 2;
    private Money() { }

    public static BigDecimal of(String amount) { return normalize(new BigDecimal(amount)); }

    public static BigDecimal normalize(BigDecimal amount) { return amount.setScale(SCALE, RoundingMode.HALF_EVEN); }

    public static BigDecimal requirePositive(BigDecimal amount) {
        if (amount == null || amount.signum() <= 0) throw new IllegalArgumentException("Amount must be positive: " +
amount);
        if (amount.stripTrailingZeros().scale() > SCALE)
            throw new IllegalArgumentException("At most " + SCALE + " decimal places: " + amount);
        return normalize(amount);
    }
}
```

```java
// File: lld/ch21/wallet/Wallet.java
package lld.ch21.wallet;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.locks.ReentrantLock;

/**
 * Holds a balance and its own statement. The balance is only changed while `lock` is held;
 * the service decides lock order when two wallets are involved.
 */
public class Wallet {
    private final String id;
    private final String ownerName;
    private final ReentrantLock lock = new ReentrantLock();
    private BigDecimal balance = Money.normalize(BigDecimal.ZERO);
    private final List<Transaction> statement = new ArrayList<>();

    Wallet(String id, String ownerName) { this.id = id; this.ownerName = ownerName; }

    public String id() { return id; }
    public String ownerName() { return ownerName; }
    ReentrantLock lock() { return lock; }

    public BigDecimal balance() {
        lock.lock();
        try { return balance; } finally { lock.unlock(); }
    }

    public List<Transaction> statement() {
        lock.lock();
        try { return List.copyOf(statement); } finally { lock.unlock(); }
    }

    // --- called only with lock held ---
    void credit(BigDecimal amt) { balance = balance.add(amt); }

    void debit(BigDecimal amt) {
        if (balance.compareTo(amt) < 0) throw new InsufficientFundsException(id, balance, amt);
        balance = balance.subtract(amt);
    }

    BigDecimal rawBalance() { return balance; }
    void record(Transaction t) { statement.add(t); }
}
```

```java
// File: lld/ch21/wallet/Offer.java
package lld.ch21.wallet;

import java.math.BigDecimal;

/** Strategy: an offer looks at a completed transfer and returns the cashback (zero if not eligible). */
public interface Offer {
    String name();
    BigDecimal cashbackFor(Wallet sender, BigDecimal amount, int senderTransferCount);
}
```

```java
// File: lld/ch21/wallet/WalletService.java
package lld.ch21.wallet;

import java.math.BigDecimal;
import java.time.Clock;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Supplier;

public class WalletService {
    public static final String PROMOTIONS_ID = "PROMO";
    private final Map<String, Wallet> wallets = new ConcurrentHashMap<>();
    /** Idempotency store: request key -> result of the first successful execution. */
    private final ConcurrentHashMap<String, TransferResult> processed = new ConcurrentHashMap<>();
    private final Map<String, AtomicInteger> transferCounts = new ConcurrentHashMap<>();
    private final List<Offer> offers;
    private final AtomicInteger txnSeq = new AtomicInteger();
    private final AtomicInteger walletSeq = new AtomicInteger();
    private final Clock clock;
    /** Marketing's budget. Cashback is a real transfer out of this wallet, so no money is created. */
    private final Wallet promotions = new Wallet(PROMOTIONS_ID, "Promotions");

    public WalletService(List<Offer> offers, Clock clock) {
        this.offers = List.copyOf(offers);
        this.clock = clock;
        wallets.put(promotions.id(), promotions);
    }

    public Wallet promotions() { return promotions; }

    public Wallet createWallet(String ownerName) {
        Wallet w = new Wallet("W" + walletSeq.incrementAndGet(), ownerName);
        wallets.put(w.id(), w);
        return w;
    }

    public TransferResult topUp(String walletId, BigDecimal amount, String idempotencyKey) {
        BigDecimal amt = Money.requirePositive(amount);
        Wallet w = get(walletId);
        return processed.computeIfAbsent(idempotencyKey, key -> {
            w.lock().lock();
            try {
                w.credit(amt);
                w.record(txn(key, w, TxnType.TOP_UP, amt, null));
            } finally { w.lock().unlock(); }
            return new TransferResult(key, "BANK", walletId, amt, BigDecimal.ZERO);
        });
    }

    /**
     * Moves money atomically. computeIfAbsent makes the key check-and-run atomic, so a retried
     * request never debits twice. If the lambda throws, nothing is stored and the client may retry.
     */
    public TransferResult transfer(String fromId, String toId, BigDecimal amount, String idempotencyKey) {
        if (fromId.equals(toId)) throw new IllegalArgumentException("Cannot transfer to the same wallet");
        BigDecimal amt = Money.requirePositive(amount);
        Wallet from = get(fromId), to = get(toId);
        return processed.computeIfAbsent(idempotencyKey, key -> doTransfer(from, to, amt, key));
    }

    private TransferResult doTransfer(Wallet from, Wallet to, BigDecimal amt, String ref) {
        // Step 1: the transfer itself, under both wallet locks.
        int n = withBoth(from, to, () -> {
            from.debit(amt);                        // throws before any change if funds are short
            to.credit(amt);
            from.record(txn(ref, from, TxnType.TRANSFER_OUT, amt, to.id()));
            to.record(txn(ref, to, TxnType.TRANSFER_IN, amt, from.id()));
            return transferCounts.computeIfAbsent(from.id(), k -> new AtomicInteger()).incrementAndGet();
        });
        // Step 2: cashback is a second, ordinary transfer PROMO -> sender (same lock-ordering rule).
```

```
        BigDecimal offered = bestOffer(from, amt, n);
        BigDecimal cashback = offered.signum() == 0 || from == promotions ? BigDecimal.ZERO
                : withBoth(promotions, from, () -> {
                    if (promotions.rawBalance().compareTo(offered) < 0) return BigDecimal.ZERO;   // budget used up
                    promotions.debit(offered);
                    from.credit(offered);
                    promotions.record(txn(ref, promotions, TxnType.CASHBACK_OUT, offered, from.id()));
                    from.record(txn(ref, from, TxnType.CASHBACK_IN, offered, promotions.id()));
                    return offered;
                });
        return new TransferResult(ref, from.id(), to.id(), amt, cashback);
    }

    /** Locks two wallets, always the smaller id first, so two opposite operations can never deadlock. */
    private static <T> T withBoth(Wallet a, Wallet b, Supplier<T> action) {
        Wallet first = a.id().compareTo(b.id()) < 0 ? a : b;
        Wallet second = first == a ? b : a;
        first.lock().lock();
        try {
            second.lock().lock();
            try { return action.get(); } finally { second.lock().unlock(); }
        } finally { first.lock().unlock(); }
    }

    /** Offers do not stack: the single most generous one wins. */
    private BigDecimal bestOffer(Wallet sender, BigDecimal amt, int n) {
        return offers.stream().map(o -> o.cashbackFor(sender, amt, n))
                .max(Comparator.naturalOrder()).orElse(BigDecimal.ZERO);
    }

    public Wallet get(String id) {
        Wallet w = wallets.get(id);
        if (w == null) throw new WalletNotFoundException(id);
        return w;
    }

    /** Sum of every wallet, promotions included: only top-ups change it (cashback just moves money). */
    public BigDecimal totalMoney() {
        return wallets.values().stream().map(Wallet::balance).reduce(BigDecimal.ZERO, BigDecimal::add);
    }

    private Transaction txn(String ref, Wallet w, TxnType type, BigDecimal amt, String counterparty) {
        return new Transaction("TX" + txnSeq.incrementAndGet(), ref, w.id(), type, amt, w.rawBalance(),
                counterparty, clock.instant());
    }
}
```

**Output** (of `Main.java`):

```
== top-ups
  Alice=2000.00  Bob=300.50
== transfers
  TransferResult[reference=req-3, from=W1, to=W2, amount=200.00, cashback=20.00]
  retry returns same result? true
  TransferResult[reference=req-4, from=W1, to=W2, amount=1200.00, cashback=25.00]
  Alice=645.00  Bob=1700.50  PROMO=55.00
== edge cases
  error: Insufficient funds in W2: balance 1700.50, needed 99999.00
  error: Amount must be positive: -5
  error: At most 2 decimal places: 1.005
  error: Cannot transfer to the same wallet
  error: Wallet not found: W99
  failed req-5 can be retried later: 10.00
== statement for Alice
  TX2   TOP_UP        2000.00  bal= 2000.00  ref=req-1
  TX4   TRANSFER_OUT   200.00  bal= 1800.00  ref=req-3 with W2
  TX7   CASHBACK_IN     20.00  bal= 1820.00  ref=req-3 with PROMO
  TX8   TRANSFER_OUT  1200.00  bal=  620.00  ref=req-4 with W2
  TX11  CASHBACK_IN     25.00  bal=  645.00  ref=req-4 with PROMO
  TX13  TRANSFER_IN     10.00  bal=  655.00  ref=req-5 with W2
== statement for PROMO
  TX1   TOP_UP         100.00  bal=  100.00  ref=budget-1
  TX6   CASHBACK_OUT    20.00  bal=   80.00  ref=req-3 with W1
  TX10  CASHBACK_OUT    25.00  bal=   55.00  ref=req-4 with W1
  TX14  CASHBACK_OUT     1.00  bal=   54.00  ref=req-5 with W2
== concurrency: 1000 opposite transfers on 8 threads
  total before=2400.50 after=2400.50 conserved=true
  Alice=655.00  Bob=1691.50  PROMO=54.00
```

Reading the output: marketing first funds `PROMO` with 100.00; the retried top-up `req-1` did not add another 2000; the retried transfer `req-3` returned the *same object* and moved no money; Alice's first transfer earned 10% (20.00), her 1200.00 transfer earned the flat 25.00 (the first-transfer offer no longer applies); the failed `req-5` stored nothing, so re-using that key later succeeded (and, being Bob's first successful transfer, earned him 1.00). Every cashback appears twice — `CASHBACK_OUT` on PROMO (100.00 → 80.00 → 55.00 → 54.00) and `CASHBACK_IN` on the receiver. Finally 1000 concurrent opposite transfers kept the total (PROMO included) exactly constant — no lost updates, no deadlock. (Each side starts with more than 500.00, so no transfer in that race can fail, and the final balances are the same on every run.)

### Concurrency & edge cases

- **Race on one wallet:** every read-modify-write (read a value, compute a new one, write it back — three steps another thread can slip between) of `balance` happens with the wallet's lock held.
- **Deadlock:** avoided by a global lock order (lexicographic — dictionary order — wallet id; `"PROMO"` sorts before `"W1"`), enforced in one helper, `withBoth`; never more than two wallet locks held at once.
- **Double submit:** idempotency key → at most one execution per key.
- **Same key, different body** (client bug): today the stored result is returned. Production systems also store a hash (a short fingerprint computed from the data) of the request and reject a mismatch with an error.
- **Validation:** non-positive amounts, more than 2 decimals, same source and target, unknown wallet, insufficient funds.
- **Cashback** (a `PROMO` → sender transfer) runs after the transfer's locks are released, so for a moment the transfer is visible without its cashback — fine for a reward. If the budget cannot cover the offer, cashback is zero and the transfer is kept.

### Extensions & follow-up questions

- **Multi-currency?** Each wallet has a `Currency`; transfers across currencies go through a `FxRateProvider` (FX = foreign exchange) interface; store both amounts and the rate used in the transaction. Use `java.util.Currency` for codes and decimal places.
- **Payment methods (card, bank, UPI — India's instant bank-to-bank payment system)?** A `PaymentMethod` interface with `charge(amount)`; top-up calls it first, and only credits the wallet on success (or uses a pending → confirmed state if the gateway is asynchronous). Users add and remove methods on their account.

- **Transfers to external accounts (withdraw to bank)?** Debit the wallet into a `PENDING_PAYOUT` state, call the bank through a `PayoutGateway` interface, and either confirm, or post a compensating credit if the bank rejects it.
- **Reversal / refund?** Never edit a past transaction; post a new compensating transaction referencing the original.
- **Distributed system?** Wallet balances in a database with row locks (`SELECT ... FOR UPDATE` locks the rows it reads until the transaction ends) or optimistic versioning; idempotency keys in a table with a unique constraint (a database rule that refuses a second row with the same value); ledgers as double-entry journals.
- **Long transfers blocking the idempotency map?** Use `putIfAbsent(key, new CompletableFuture<>())`: the winner runs the transfer outside the map and completes the future; duplicates just wait on `future.join()` (blocks until the result is ready). On failure, remove the key so a retry can run.
- **Limits (daily cap, KYC tier — KYC, "know your customer", is the identity check that sets how much a user may move)?** A chain of validators (Chain of Responsibility) run before `doTransfer`.
- **Why not `AtomicReference<BigDecimal>` + CAS (compare-and-set: replace a value only if it still equals what you last read, in one indivisible step)?** Works for a single wallet, but a transfer touches two balances that must change together; CAS on two variables is not atomic, so locks are simpler.

### Real-world example

Stripe's (a large online-payments company) API accepts an `Idempotency-Key` header (a named piece of metadata sent along with an HTTP request): if a request times out and the client retries with the same key, Stripe returns the saved result of the first attempt instead of charging twice. Wallets such as Paytm and PhonePe run cashback campaigns with rules like "X% up to a cap on your first transaction" — exactly the shape of `FirstTransferOffer`.

### In the interview

Lead with the three risks: exact money, concurrency, retries (and, if offers come up, "cashback must come out of a budget wallet"). Show `Money`, then `doTransfer` with the lock ordering (draw the A→B / B→A deadlock and how ordering breaks the cycle), then idempotency. Run the concurrent demo and point at `conserved=true`. Offers are a 2-minute Strategy mention. If time is short, cut multi-currency and payment methods — say you would add them behind interfaces.

### Self-check

- **Q:** Why is `new BigDecimal("0.1")` preferred over `new BigDecimal(0.1)`? → **A:** `double` stores numbers in base 2, where 0.1 has no exact form, so the double constructor captures a long, inexact decimal; the String constructor is exact.
- **Q:** Two threads transfer W1→W2 and W2→W1 at once. Why no deadlock? → **A:** Both lock W1 first (smaller id), so one waits for the other instead of each holding what the other needs.
- **Q:** A transfer fails for insufficient funds. Can the same idempotency key be reused? → **A:** Yes — `computeIfAbsent` stores nothing when the function throws.
- **Q:** Where would a "max 3 transfers per minute" rule go? → **A:** A validator run before `doTransfer`, not inside the wallet.

### TL;DR

`BigDecimal` money, lock both wallets in a global order, debit before credit, idempotency key → run at most once, offers as Strategies paid out of a funded promotions wallet. Prove it with a concurrent demo where the total is conserved.

---

## 21.3 Design an Event Calendar (Flipkart)

### Problem & clarifying questions

This is a Flipkart machine-coding statement (kumaransg/LLD, `Event_calendar_flipkart`). In short: onboard users with **working hours**; group users into **teams** (a user belongs to at most one team); create **events** on one day with a start and end time, whose participants are users, teams, or both. A user can be in **only one event at a time**. For team participants the event creator says how many **representatives (N)** each team must

send; the system blocks the event against N *available* members of each team, and if any team cannot supply N the event is not created. Required and bonus features are listed under Requirements below. The statement says: no DB, no UI, no CLI (command-line interface) needed — a `main` or tests is enough — and "do not make assumptions, ask".

Clarifying questions (and the answers this solution uses):

| Question | Answer used |
|---|---|
| Is an event that ends at 15:00 in conflict with one that starts at 15:00? | No. Slots are **half-open** `[start, end)` (the start minute belongs to the slot, the end minute does not). |
| Must an event lie inside every attendee's working hours? | Yes (the statement's Event5 fails because F starts at 11:00). |
| Which team members become representatives? | The first N free members in team order (a pluggable strategy). |
| If a user is invited directly *and* is in an invited team, do they count as a representative? | No — a direct attendee is not picked again as a representative. |
| Free slots for a team with N representatives: what counts as "free"? | A moment is free for the team if at least N members are working and not busy then. |
| Recurring event: if one occurrence clashes? | Reject the whole series (all-or-nothing). |

**A note on the statement's sample.** For "free slots for user A and team T1 with 1 representative today" the sample answer says *10AM to 3PM and 4PM to 7PM*. But T1 = {C, E} and neither works before 11:00 (C starts 11:30, E 11:00), so by the statement's own rules the first slot starts at **11:00**; our program prints `11:00-15:00, 16:00-19:00`. Pointing this out politely ("I think the sample means 11AM; shall I go with the rules?") is exactly what "do not make assumptions, ask" tests. (Its step 1 also says "Onboard 5 Users" but lists six, A to F; the demo creates all six.)

## Requirements

### Functional

1. Create user with working hours.
2. Create team of existing users (one team per user).
3. Create event for users and/or teams with N representatives; reject if any attendee is outside working hours, already busy, or a team lacks N free members.
4. **Events of a user between two date-times** (direct + as team representative).
5. **Suggest free slots** for users/teams on a day.
6. Bonus: **update event time** (atomic: old booking kept on failure; report failure if impossible).
7. Bonus: **recurring events** (time, start date, end date, frequency daily/weekly).

### Non-functional

- The check "is everyone free?" and the booking must be **atomic**, or two concurrent creations could book the same person twice.
- Representative choice should be replaceable (fairness, least-busy, random).

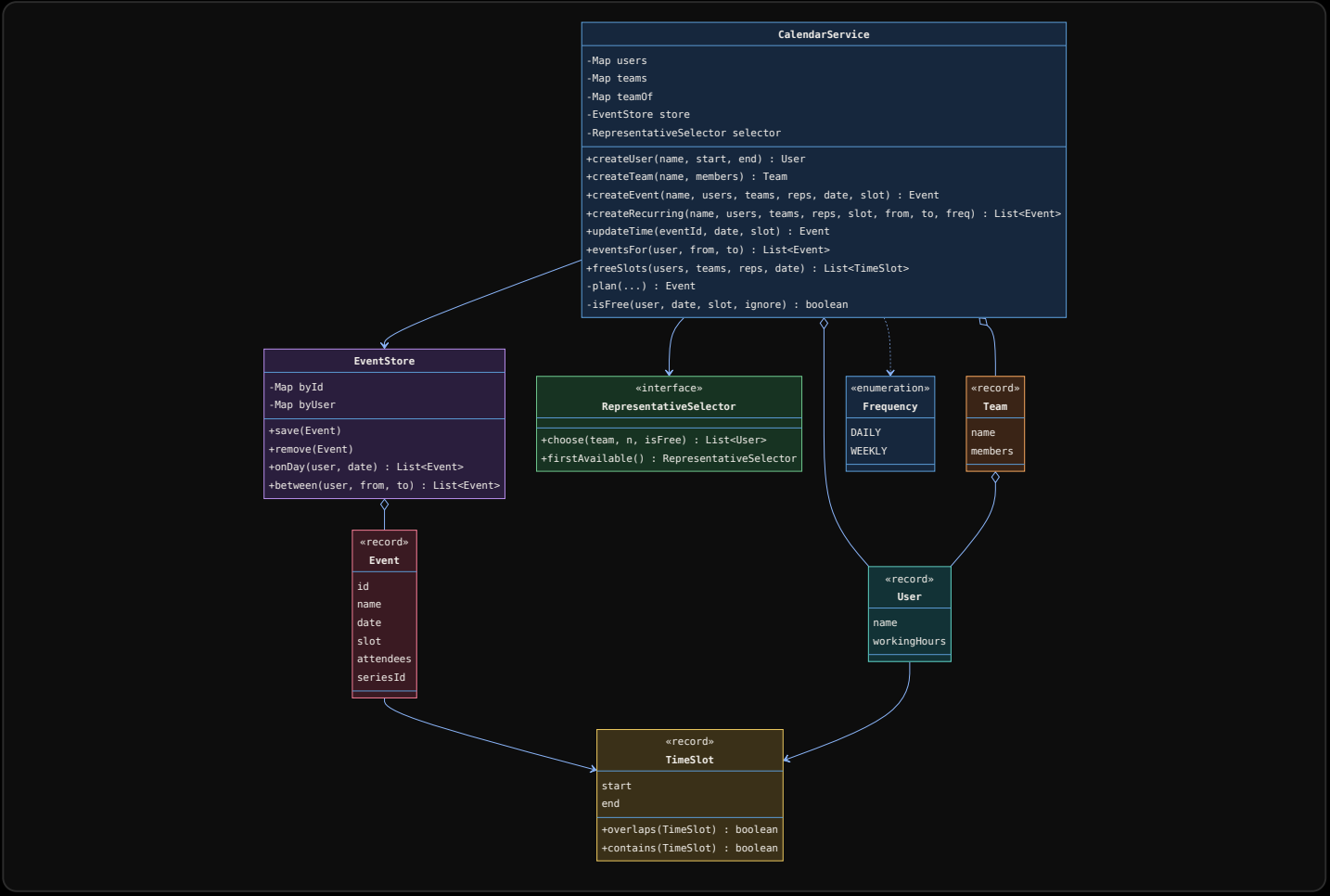
### Use cases

| Actor | Use cases |
|---|---|
| **Admin** | onboard user, create team |
| **Organiser** | create event, create recurring event, move event, find free slots |
| **Attendee** | view own events in a range |

### Core entities

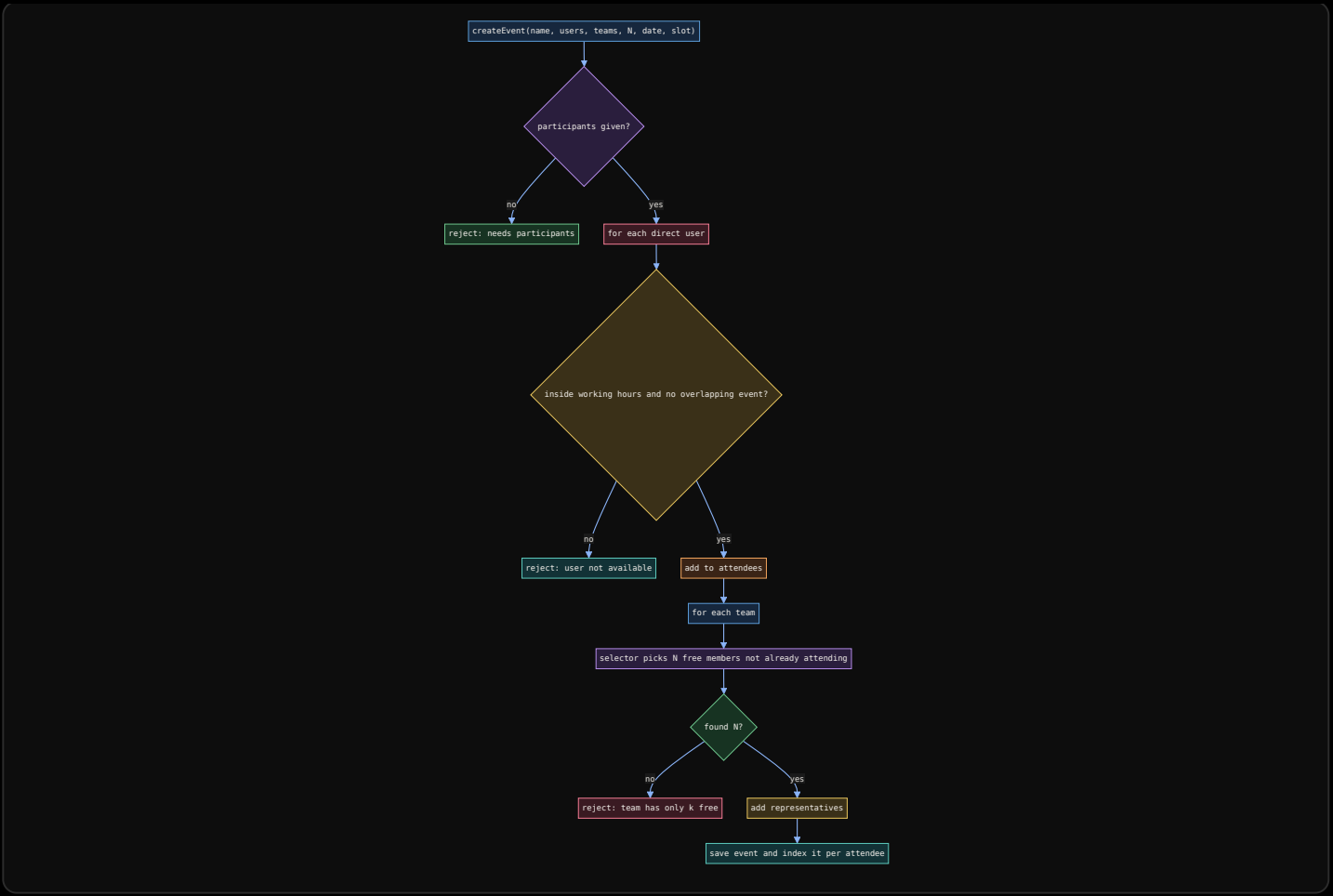
| Class | Kind | Responsibility |
|---|---|---|
| `TimeSlot` | record | Half-open `[start, end)` on one day; `overlaps`, `contains`; validates start < end. |
| `User` | record | Name + working hours. |
| `Team` | record | Name + ordered members. |
| `Event` | record | One concrete occurrence: date, slot, what was requested (users, teams, N) and who is actually blocked ( `attendees` ), optional `seriesId`. |
| `Frequency` | enum | DAILY / WEEKLY — knows the next date. |
| `EventStore` | class | Repository + index `user → date → events` for fast conflict checks and range queries ("everything between X and Y"; `TreeMap.subMap` returns just that slice of a sorted map). |
| `RepresentativeSelector` | interface | Strategy: pick N free members of a team. |
| `CalendarService` | class | All use cases; planning (validation + representative choice) separated from saving. |
| `CalendarException` | exception | Readable reason for every rejection. |

### Class diagram

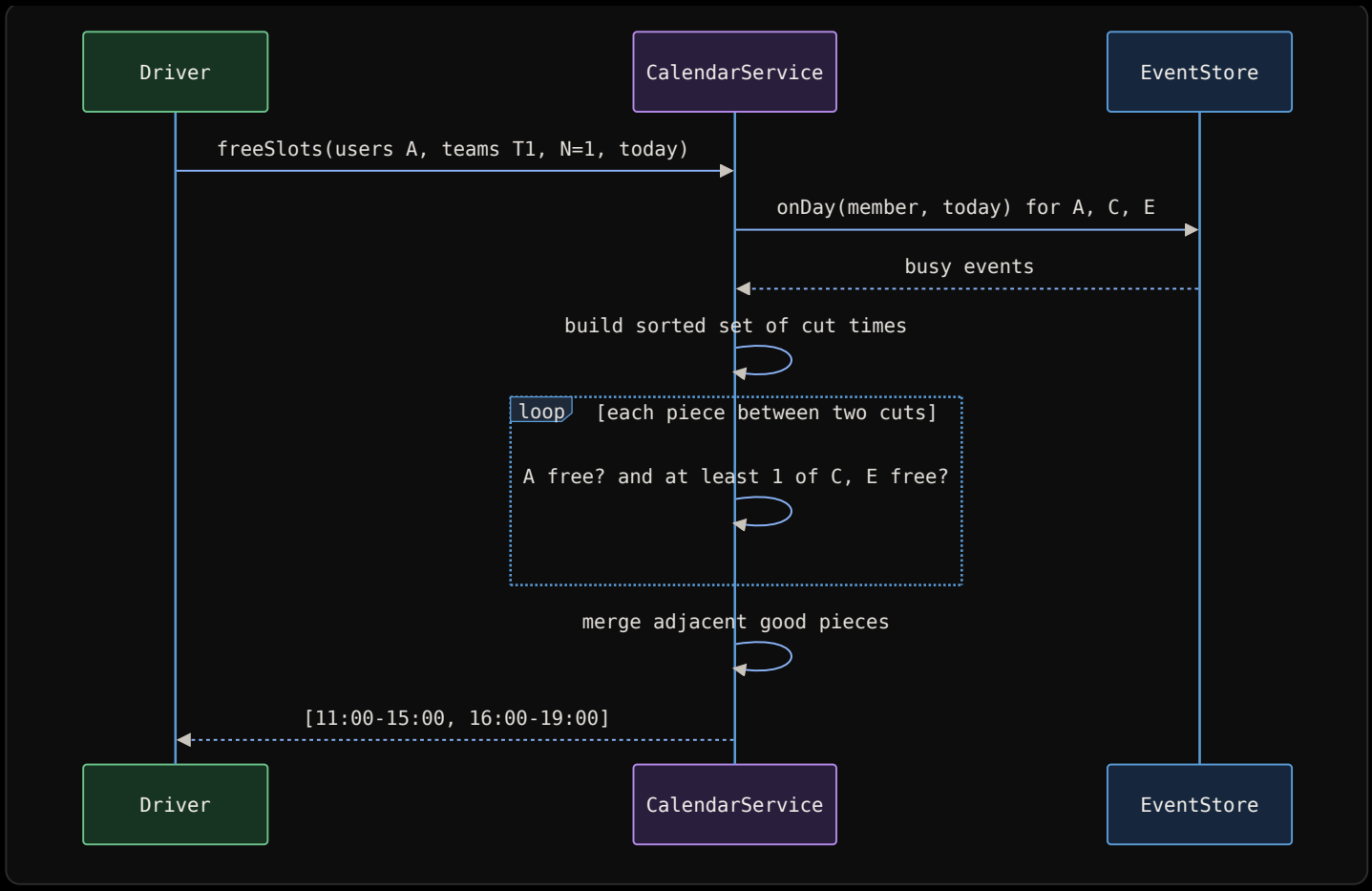



### Key flows

**Create an event (plan, then save):**




**Free-slot sweep.** Collect every "interesting" time on that day: each involved person's working-hour start and end, and the start and end of each of their events. Sort them. Between two consecutive points nothing changes, so test each small piece once: all direct users free, and each team has at least N free members. Merge neighbouring good pieces. For the statement's example (A + T1, N = 1, today) the cut points are 10:00, 11:00, 11:30, 15:00, 16:00, 18:30, 19:00, 19:30, and the good pieces merge into 11:00–15:00 and 16:00–19:00.




### Design decisions & patterns used

| Decision | Why | Alternative rejected |
|---|---|---|
| **Half-open intervals** | Back-to-back meetings do not conflict; `overlaps` is one line: `a.start < b.end && b.start < a.end`. | Closed intervals — 14:00–15:00 would clash with 15:00–16:00. |
| **`plan()` separate from `save()`** | Validation + representative choice produce an `Event` without side effects (no change to any stored state). `createEvent`, `createRecurring` (plan all, then save all) and `updateTime` (plan while ignoring the old event, then swap) all reuse it — all-or-nothing for free. | Saving as you go and rolling back (undoing the saved steps) on failure. |
| **`RepresentativeSelector` strategy** | "Which N members?" is a policy; fairness or least-busy can be plugged in. | Hard-coded loop in the service. |
| **Index `user → TreeMap<date, events>`** | Conflict checks read only one user's one day; range queries use `subMap`. | Scanning all events each time. |
| **Sweep over boundaries for free slots** | Exact, simple, no minute-by-minute loops; works for any N. | Checking every 15 or 30 minutes — misses odd times like 11:30 and is slower. |
| **Service methods `synchronized`** | Check-then-book must be atomic; the store's lists are not thread-safe on their own. | Per-user locks — faster, but an event locks many users, so you need lock ordering; mention it as a scale-up. |

### Java implementation

Full runnable code: `code/lld/ch21/calendar/` (10 files). Below are the files that carry the design; the rest are `User`, `Team`, `Event`, `Frequency`, `EventStore` (index user → date → events), `CalendarException` and `Main`.

```java
// File: lld/ch21/calendar/TimeSlot.java
package lld.ch21.calendar;

import java.time.LocalTime;

/** Half-open interval [start, end) inside one day. 14:00-15:00 and 15:00-16:00 do not overlap. */
public record TimeSlot(LocalTime start, LocalTime end) {
    public TimeSlot {
        if (!start.isBefore(end)) throw new IllegalArgumentException("start must be before end: " + start + "-" +
end);
    }
    public boolean overlaps(TimeSlot o) { return start.isBefore(o.end) && o.start.isBefore(end); }
    public boolean contains(TimeSlot o) { return !o.start.isBefore(start) && !o.end.isAfter(end); }
    @Override public String toString() { return start + "-" + end; }
}
```

```java
// File: lld/ch21/calendar/RepresentativeSelector.java
package lld.ch21.calendar;

import java.util.List;
import java.util.function.Predicate;

/** Strategy: which N free members represent a team. */
public interface RepresentativeSelector {
    List<User> choose(Team team, int n, Predicate<User> isFree);

    /** Default: first N free members in team order (deterministic, easy to explain). */
    static RepresentativeSelector firstAvailable() {
        return (team, n, isFree) -> team.members().stream().filter(isFree).limit(n).toList();
    }
}
```

```java
// File: lld/ch21/calendar/CalendarService.java  (excerpt)
public class CalendarService {
    private final Map<String, User> users = new ConcurrentHashMap<>();
    private final Map<String, Team> teams = new ConcurrentHashMap<>();
    private final Map<String, String> teamOf = new ConcurrentHashMap<>();   // user -> team
    private final EventStore store = new EventStore();
    private final RepresentativeSelector selector;
    private final AtomicInteger seq = new AtomicInteger();

    public CalendarService(RepresentativeSelector selector) { this.selector = selector; }

    // ... createUser, createTeam (a user joins at most one team)

    /** synchronized: the check-then-book sequence must be atomic or two events could grab the same person. */
    public synchronized Event createEvent(String name, Set<String> userNames, Set<String> teamNames,
                                           int reps, LocalDate date, TimeSlot slot) {
        Event e = plan(name, userNames, teamNames, reps, date, slot, null, null);
        store.save(e);
        return e;
    }
    // ... createRecurring (plan every occurrence, save only if all fit), updateTime (re-plan ignoring itself),
eventsFor

    /**
     * Free slots on a day where every listed user is free AND each listed team has at least `reps`
     * free members. Sweep: cut the day at every boundary (working-hour edges, event edges),
     * test each small piece, then merge adjacent good pieces.
     */
    public synchronized List<TimeSlot> freeSlots(Set<String> userNames, Set<String> teamNames, int reps, LocalDate
date) {
        List<User> direct = userNames.stream().map(this::user).toList();
        List<Team> tms = teamNames.stream().map(this::team).toList();
        List<User> everyone = new ArrayList<>(direct);
        tms.forEach(t -> everyone.addAll(t.members()));

        TreeSet<LocalTime> cuts = new TreeSet<>();
        for (User u : everyone) {
            cuts.add(u.workingHours().start());
            cuts.add(u.workingHours().end());
            for (Event e : store.onDay(u.name(), date)) { cuts.add(e.slot().start()); cuts.add(e.slot().end()); }
        }
        List<TimeSlot> result = new ArrayList<>();
        LocalTime prev = null;
        for (LocalTime cut : cuts) {
            if (prev != null) {
                TimeSlot piece = new TimeSlot(prev, cut);
                boolean ok = direct.stream().allMatch(u -> isFree(u, date, piece, null))
                        && tms.stream().allMatch(t -> t.members().stream()
                                .filter(m -> isFree(m, date, piece, null)).count() >= reps);
                if (ok) {
                    int last = result.size() - 1;
                    if (last >= 0 && result.get(last).end().equals(piece.start()))
                        result.set(last, new TimeSlot(result.get(last).start(), piece.end()));
                    else result.add(piece);
                }
            }
            prev = cut;
        }
        return result;
    }

    // ------------------------------------------------------------------ helpers

    private Event plan(String name, Set<String> userNames, Set<String> teamNames, int reps,
                       LocalDate date, TimeSlot slot, String series, Event ignore) {
        if (userNames.isEmpty() && teamNames.isEmpty()) throw new CalendarException("Event needs participants");
        if (!teamNames.isEmpty() && reps < 1) throw new CalendarException("Representatives must be >= 1");
        Set<String> attendees = new LinkedHashSet<>();
        for (String n : new TreeSet<>(userNames)) {          // sorted => deterministic order
            User u = user(n);
            if (!isFree(u, date, slot, ignore))
                throw new CalendarException(name + ": " + n + " is not available " + date + " " + slot);
            attendees.add(n);
```

```
        }
        for (String tn : new TreeSet<>(teamNames)) {
            Team t = team(tn);
            List<User> chosen = selector.choose(t, reps,
                    m -> !attendees.contains(m.name()) && isFree(m, date, slot, ignore));
            if (chosen.size() < reps)
                throw new CalendarException(name + ": team " + tn + " has only " + chosen.size()
                        + " free member(s), needs " + reps);
            chosen.forEach(m -> attendees.add(m.name()));
        }
        String id = ignore != null ? ignore.id() : "E" + seq.incrementAndGet();
        return new Event(id, name, date, slot, Set.copyOf(userNames), Set.copyOf(teamNames), reps,
                List.copyOf(attendees), series);
    }

    private boolean isFree(User u, LocalDate date, TimeSlot slot, Event ignore) {
        if (!u.workingHours().contains(slot)) return false;
        return store.onDay(u.name(), date).stream()
                .filter(e -> ignore == null || !e.id().equals(ignore.id()))
                .noneMatch(e -> e.slot().overlaps(slot));
    }
    // ... user/team lookups, allEvents
}
```

**Output** (of `Main.java`):

```
== 1. onboard users
  6 users created
== 2. teams
  T1=[C,E] T2=[B,D,F]
  FAILED: C already belongs to team T1
== 3. events
  ok: Event1 2026-09-29 14:00-15:00 [A, C, E]
  FAILED: Event2: C is not available 2026-09-29 14:00-15:00
  ok: Event3 2026-09-28 15:00-16:00 [C, E, B, D]
  ok: Event4 2026-09-28 15:00-16:00 [A, F]
  FAILED: Event5: F is not available 2026-09-28 10:00-11:00
== 4. events in a time range (today 10:00 .. tomorrow 17:00)
  A: [Event4, Event1]
  B: [Event3]
  F: [Event4]
== 5. free slots today for A + T1 (1 representative)
  [11:00-15:00, 16:00-19:00]
  free slots today for B + D: [10:00-15:00, 16:00-17:30]
== bonus: update event time
  FAILED: Event1: team T1 has only 1 free member(s), needs 2
  ok: Event1 2026-09-29 16:00-17:00 [A, C, E]
== bonus: recurring daily stand-up (A + D), 3 days
  ok: [Standup@2026-09-28 2026-09-28 10:00-10:15 [A, D], Standup@2026-09-29 2026-09-29 10:00-10:15 [A, D],
Standup@2026-09-30 2026-09-30 10:00-10:15 [A, D]]
  FAILED: Sync@2026-09-28: A is not available 2026-09-28 15:30-16:30
  A today..+2: [Standup@2026-09-28, Event4, Standup@2026-09-29, Event1, Standup@2026-09-30]
```

Checking against the statement: Event2 fails (C is in Event1), Event5 fails (F starts at 11:00), user A sees Event4 and Event1, and user B sees Event3 — because T2's first two free members (B, D) represented it in Event3, which left F as the only free T2 member for Event4. The update to 18:30–19:00 is refused because Event1 needs both T1 members and C stops at 18:30; the old booking stays. The "Sync" series is rejected as a whole because its first day clashes with Event4.

### Concurrency & edge cases

- **Double booking under concurrency:** all mutating and reading methods are `synchronized` on the service, so the "is free?" check and the save are one atomic step.
- **Edge cases handled:** start ≥ end, unknown user/team, user already in a team, empty participant list, N < 1 with teams, direct attendee also in an invited team (not double-counted), event outside working hours, recurring series with any clash (nothing saved), moving an event onto a slot where it would conflict with *itself* (we ignore the event being moved).
- **Time zones:** out of scope; everything is local time on one day. Real calendars store instants (exact moments, in UTC — the world reference time zone) plus the user's zone.

- **Multi-day events:** the statement fixes start and end on the same day; extension would split an event into per-day `TimeSlot`s.

### Extensions & follow-up questions

- **Invites with accept/decline?** Add `InviteStatus {PENDING, ACCEPTED, DECLINED}` per attendee; a declined attendee no longer blocks the slot.
- **Fair representative choice?** A `LeastBusySelector` that sorts free members by number of events this week.
- **Recurring events at scale?** Store one *rule* (like the iCalendar RRULE — the standard text format for "repeats every Tuesday until June") and expand occurrences lazily (only when asked) for the queried range instead of materialising (creating and storing) every occurrence.
- **Rooms?** Treat a room like a user with "working hours" = building hours; the same availability logic applies.
- **Faster free-slot search for big teams?** For each team, a sweep counting free members per piece is O(members × events); with many members, use a count array or segment tree (a tree that answers "how many in this range?" quickly) over minutes.

### Real-world example

Google Calendar's "Find a time" and Outlook's Scheduling Assistant lay attendees' busy blocks side by side and highlight times when everyone is free — the same intersection this sweep computes. Both products store recurring meetings as a rule plus exceptions (following the iCalendar RRULE standard) instead of thousands of copies.

### In the interview

Start by drawing a timeline with two users' working hours and events, and show that free time is "working time minus busy time, intersected". Explain half-open intervals in one sentence. Code `TimeSlot`, `plan()` and `createEvent` first — that satisfies the first three requirements. Then `eventsFor`, then the sweep. Mention the sample inconsistency. Bonus features fall out of `plan()` almost for free, which is the design win to point at.

### Self-check

- **Q:** Why does `plan()` take an `ignore` event? → **A:** When moving an event, its current booking must not count as a conflict with its new time.
- **Q:** Why is the recurring series all-or-nothing easy here? → **A:** We plan every occurrence first (no side effects) and save only if all plans succeed.
- **Q:** What is the overlap test for half-open intervals? → **A:** `a.start < b.end && b.start < a.end`.
- **Q:** Why a `TreeMap` per user? → **A:** Dates are ordered, so a range query is `subMap(from, to)` instead of a full scan.

### TL;DR

Half-open `TimeSlot`s, per-user day index, a side-effect-free `plan()` reused by create/recurring/update, a pluggable representative selector, and a boundary sweep for free slots. Ask about the sample that contradicts the rules.

---

## 21.4 Design a LeetCode-like Coding Platform — "Coding Blox" (Flipkart)

### Problem & clarifying questions

A Flipkart SDE-2 (Software Development Engineer, level 2: a mid-level role) machine-coding statement (kumaransg/LLD, `leetcode-lld-flipkart-coding-blox`). Coding Blox lets users sign up, create contests, and take part in contests hosted by others. Questions and contests each have a level — LOW, MEDIUM or HIGH — and a contest of a level uses **all questions of that level** (the question list is independent of the contest). The platform is driven by commands:

| Command | Meaning |
|---|---|
| `CreateUser <name>` | Register a user with a default score of **1500**. |
| `CreateQuestion <level> <score>` | Add a question; ids auto-increment from 1. |
| `ListQuestion [level]` | All questions, or only that level. |
| `CreateContest <name> <level> <creator>` | Creator automatically attends; contest ids auto-increment from 1. |
| `ListContest [level]` | All contests, or only that level. |
| `AttendContest <contestId> <user>` | Register for a contest. |
| `RunContest <contestId> <creator>` | Only the creator may run it. Simulate which questions each participant solved (random), sum their scores ( `contestPoints` ), then update ratings: LOW → `score + (points − 50)`, MEDIUM → `score + (points − 30)`, HIGH → `score + points`. |
| `LeaderBoard <asc/desc>` | Users with scores, sorted. |
| Bonus `ContestHistory <contestId>` | Per-contest leaderboard: user, points, solved question ids. |
| Bonus `WithdrawContest <contestId> <user>` | Leave before the contest runs; the creator cannot withdraw. |

Expectations in the statement: create the sample data yourself, demo through a driver or tests, modular OO (object-oriented) design with interfaces, extensible, graceful edge-case handling, DRY ("don't repeat yourself": each piece of logic written once).

| Question | Answer used |
|---|---|
| How many questions does a user "solve" in a run? | Random 1..all questions of the contest level. The randomness sits behind a `SolveSimulator` interface and is **seeded** for a reproducible demo. |
| Can a contest run twice? | No — second run is an error, so ratings are never applied twice. |
| Can someone attend after the run? | No. |
| Leaderboard tie-break? | By name. |
| Can a contest be created if no question of that level exists? | No (it would be empty). |

## Requirements

**Functional:** all commands above, including both bonus commands. **Non-functional:** commands from a built-in input list; errors printed, processing continues; scoring rule and solving logic replaceable; rating update safe if two contests finish at once.

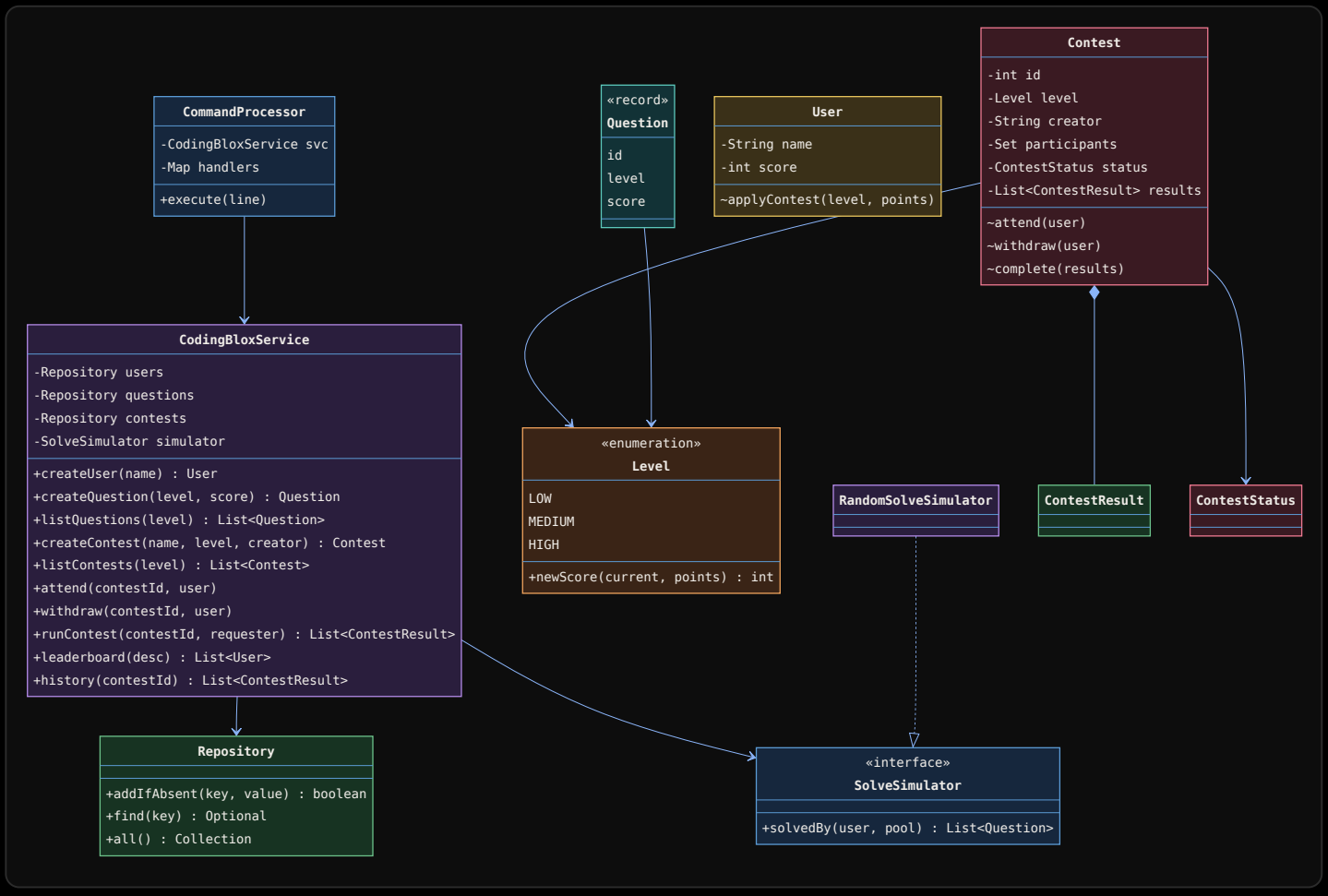
### Use cases

| Actor | Use cases |
|---|---|
| **User** | sign up, attend/withdraw, view leaderboard, view contest history |
| **Contest creator** (a user) | create contest, run contest |
| **Admin / problem setter** | create and list questions |
| **Judge** (system actor, simulated) | decides which questions each participant solved |

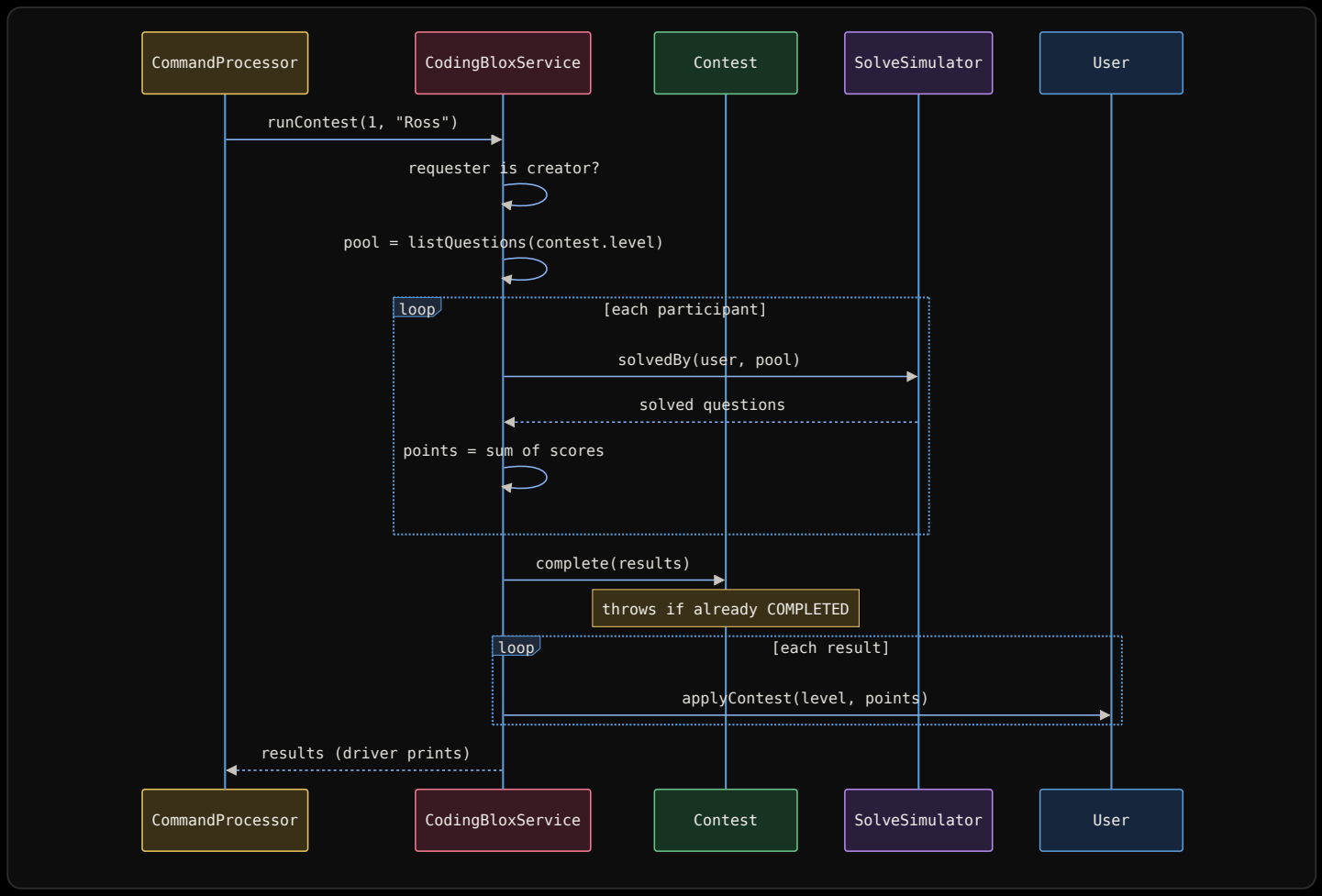
## Core entities

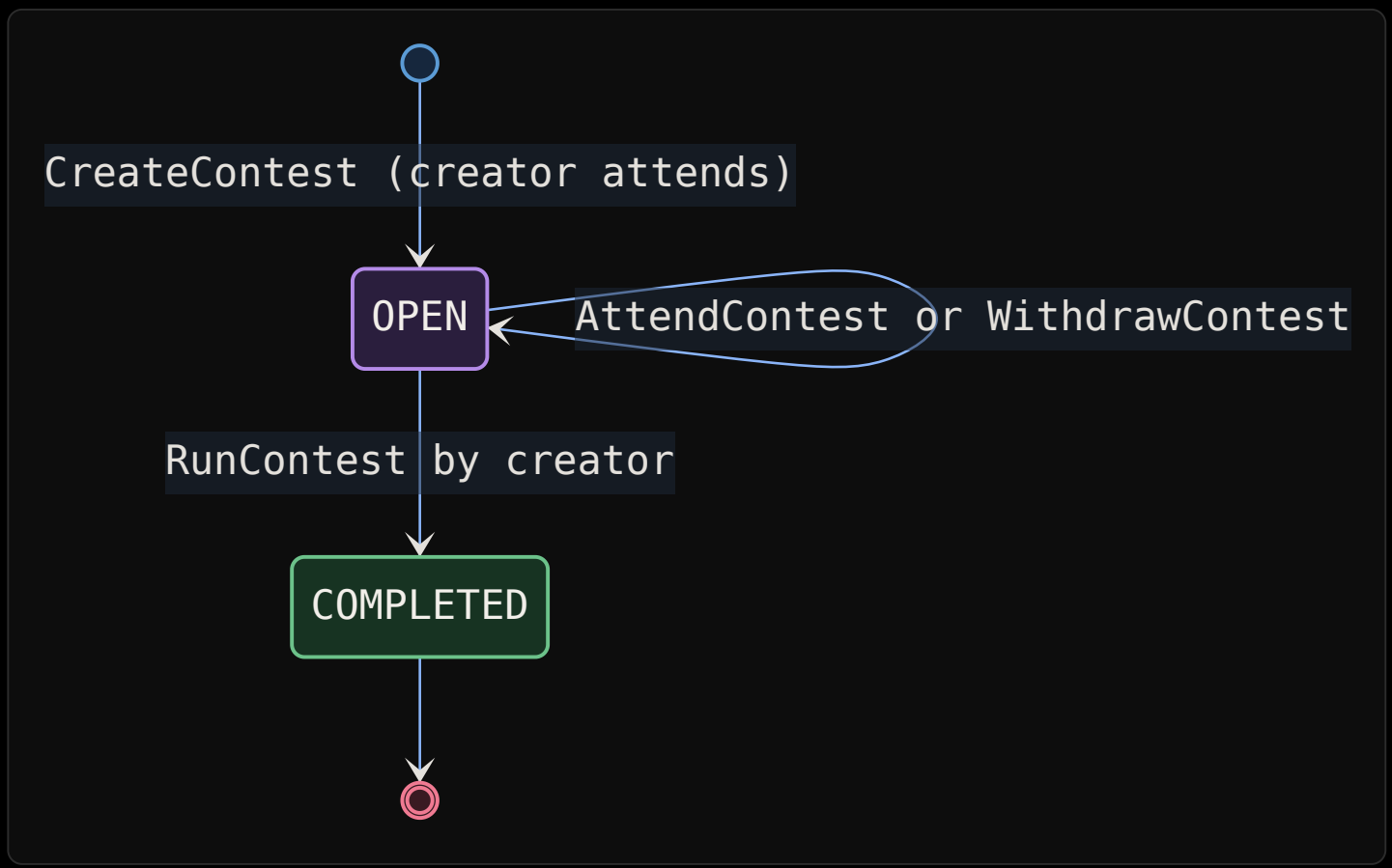
| Class | Kind | Responsibility |
|---|---|---|
| `Level` | enum | LOW/MEDIUM/HIGH, each with its penalty; `newScore(current, points)` implements the rating rule. |
| `User` | class | Name + score; `applyContest` updates the score atomically. |
| `Question` | record | id, level, score. |
| `Contest` | class | id, name, level, creator, participants, status, results; guards "open only" rules. |
| `ContestStatus` | enum | OPEN, COMPLETED. |
| `ContestResult` | record | user, points, solved ids. |
| `SolveSimulator` / `RandomSolveSimulator` | interface / class | Strategy for "who solved what". |
| `Repository<K,V>` | generic class | In-memory storage reused for users, questions and contests. |
| `CodingBloxService` | class | Business rules for every command. |
| `CommandProcessor` | class | Parses (splits and reads) a line, dispatches to (calls) a handler from a map, prints output/errors. |
| `BloxException` | exception | Readable failure. |

## Class diagram



**Key flows**


CommandProcessor
CodingBloxService
Contest
SolveSimulator
User
runContest(1, "Ross")
requester is creator?
pool = listQuestions(contest.level)
loop
[each participant]
solvedBy(user, pool)
solved questions
points = sum of scores
complete(results)
throws if already COMPLETED
loop
[each result]
applyContest(level, points)
results (driver prints)
CommandProcessor
CodingBloxService
Contest
SolveSimulator
User


CreateContest (creator attends)
OPEN
AttendContest or WithdrawContest
RunContest by creator
COMPLETED

### Design decisions & patterns used

| Decision | Pattern | Why | Alternative rejected |
|---|---|---|---|
| Command name → handler map | Command | Adding a command is one `put`; no giant `switch`. Parsing and printing stay out of the service. | Parsing inside the service. |
| Scoring rule inside `Level` enum | Strategy (each enum constant carries its own data) | The rule depends only on the level; a new level brings its own rule. | `if (level == LOW) ... else if ...` in `runContest`. |
| `SolveSimulator` interface | Strategy | Randomness is replaceable: seeded for demo, scripted for tests, a real judge later. | `new Random()` inside the service — untestable. |
| `Contest.complete` checks status | Entity guards invariant | "Run once" enforced where the state lives, so scores are never applied twice. | Checking in the driver. |
| Generic `Repository<K,V>` | Repository | Three stores, one class (DRY — the statement asks for it). | Three copy-pasted maps with lookup code. |
| Results computed, then `complete`, then ratings | Ordering for safety | If `complete` throws (already run), no rating has changed yet. | Updating ratings inside the loop. |

### Java implementation

Full runnable code: `code/lld/ch21/codingblox/` (13 files). Below are the files that carry the design; the rest are `User` (score starts at 1500), `Question`, `Contest` (OPEN → COMPLETED, creator cannot withdraw), `ContestStatus`, `ContestResult`, `RandomSolveSimulator` (seeded), `BloxException` and `Main`.

```java
// File: lld/ch21/codingblox/Level.java
package lld.ch21.codingblox;

/** Difficulty of a question or a contest. Each contest level carries its own scoring rule. */
public enum Level {
    LOW(50), MEDIUM(30), HIGH(0);

    private final int penalty;
    Level(int penalty) { this.penalty = penalty; }

    /** newScore = currentScore + (contestPoints - penalty), exactly as the statement defines. */
    public int newScore(int current, int contestPoints) { return current + contestPoints - penalty; }

    public static Level parse(String s) {
        try { return valueOf(s.toUpperCase()); }
        catch (IllegalArgumentException e) { throw new BloxException("Unknown level: " + s); }
    }
}
```

```java
// File: lld/ch21/codingblox/Repository.java
package lld.ch21.codingblox;

import java.util.Collection;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

/** Tiny generic in-memory repository reused for users, questions and contests. */
public class Repository<K, V> {
    private final Map<K, V> items = new ConcurrentHashMap<>();
    public boolean addIfAbsent(K key, V value) { return items.putIfAbsent(key, value) == null; }
    public Optional<V> find(K key) { return Optional.ofNullable(items.get(key)); }
    public Collection<V> all() { return List.copyOf(items.values()); }
}
```

```java
// File: lld/ch21/codingblox/SolveSimulator.java
package lld.ch21.codingblox;

import java.util.List;

/** Strategy: decides which questions a user solved. Real system = judge results; demo = seeded random. */
public interface SolveSimulator {
    List<Question> solvedBy(String user, List<Question> pool);
}
```

```java
// File: lld/ch21/codingblox/CodingBloxService.java  (excerpt)
public class CodingBloxService {
    private final Repository<String, User> users = new Repository<>();
    private final Repository<Integer, Question> questions = new Repository<>();
    private final Repository<Integer, Contest> contests = new Repository<>();
    private final AtomicInteger questionSeq = new AtomicInteger();
    private final AtomicInteger contestSeq = new AtomicInteger();
    private final SolveSimulator simulator;

    public CodingBloxService(SolveSimulator simulator) { this.simulator = simulator; }

    // ... createUser, createQuestion, listQuestions, createContest, listContests, attend, withdraw

    /** Only the creator may run. Every participant gets random solved questions, then scores update. */
    public List<ContestResult> runContest(int contestId, String requester) {
        Contest c = contest(contestId);
        if (!c.creator().equals(requester)) throw new BloxException(requester + " did not create contest " +
contestId);
        List<Question> pool = listQuestions(c.level());
        List<ContestResult> results = new ArrayList<>();
        for (String name : c.participants()) {
            List<Question> solved = simulator.solvedBy(name, pool);
            int points = solved.stream().mapToInt(Question::score).sum();
            results.add(new ContestResult(name, points, solved.stream().map(Question::id).toList()));
        }
        c.complete(results);                                // throws if already run: scores never double-apply
        for (ContestResult r : results) user(r.user()).applyContest(c.level(), r.points());
        return results;
    }

    // ... leaderboard (sort by score, then name), history, lookups
}
```

```java
// File: lld/ch21/codingblox/CommandProcessor.java
package lld.ch21.codingblox;

import java.util.Arrays;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.function.Consumer;

/**
 * Driver layer. Parses one text line and dispatches to a handler (Command pattern: name -> action).
 * All printing lives here; the service returns data only.
 */
public class CommandProcessor {
    private final CodingBloxService svc;
    private final Map<String, Consumer<String[]>> handlers = new LinkedHashMap<>();

    public CommandProcessor(CodingBloxService svc) {
        this.svc = svc;
        handlers.put("createuser", a -> out("User " + svc.createUser(arg(a, 1)).name() + " created with score 1500"));
        handlers.put("createquestion", a -> out("Question created: " + svc.createQuestion(Level.parse(arg(a, 1)),
Integer.parseInt(arg(a, 2)))));
        handlers.put("listquestion", a -> svc.listQuestions(a.length > 1 ? Level.parse(a[1]) : null).forEach(q ->
out("  " + q)));
        handlers.put("createcontest", a -> out("Contest created: " + svc.createContest(arg(a, 1), Level.parse(arg(a,
2)), arg(a, 3))));
        handlers.put("listcontest", a -> svc.listContests(a.length > 1 ? Level.parse(a[1]) : null).forEach(c -> out("
" + c)));
        handlers.put("attendcontest", a -> { svc.attend(Integer.parseInt(arg(a, 1)), arg(a, 2)); out(a[2] + "
registered for contest " + a[1]); });
        handlers.put("withdrawcontest", a -> { svc.withdraw(Integer.parseInt(arg(a, 1)), arg(a, 2)); out(a[2] + "
withdrew from contest " + a[1]); });
        handlers.put("runcontest", a -> svc.runContest(Integer.parseInt(arg(a, 1)), arg(a, 2))
                .forEach(r -> out("  " + r.user() + " solved " + r.solvedIds() + " -> " + r.points() + " pts")));
        handlers.put("leaderboard", a -> svc.leaderboard(!a[a.length - 1].equalsIgnoreCase("asc"))
                .forEach(u -> out("  " + u.name() + " : " + u.score())));
        handlers.put("contesthistory", a -> svc.history(Integer.parseInt(arg(a, 1)))
                .forEach(r -> out("  " + r.user() + " : " + r.points() + " " + r.solvedIds())));
    }

    public void execute(String line) {
        System.out.println("> " + line);
        String[] a = Arrays.stream(line.trim().split("\\s+")).map(s -> s.replace("\"", "")).toArray(String[]::new);
        Consumer<String[]> h = handlers.get(a[0].toLowerCase());
        try {
            if (h == null) throw new BloxException("Unknown command: " + a[0]);
            h.accept(a);
        } catch (BloxException | NumberFormatException e) {
            out("ERROR: " + e.getMessage());
        }
    }

    private static String arg(String[] a, int i) {
        if (i >= a.length) throw new BloxException("Missing argument #" + i + " for " + a[0]);
        return a[i];
    }

    private static void out(String s) { System.out.println(s); }
}
```

**Output** (of `Main.java`):

```
> CreateUser Ross
User Ross created with score 1500
> CreateUser Monica
User Monica created with score 1500
> CreateUser Joey
User Joey created with score 1500
> CreateUser Chandler
User Chandler created with score 1500
> CreateQuestion "LOW" 10
Question created: Q1(LOW,10)
> CreateQuestion LOW 20
Question created: Q2(LOW,20)
> CreateQuestion LOW 30
Question created: Q3(LOW,30)
> CreateQuestion MEDIUM 40
Question created: Q4(MEDIUM,40)
> CreateQuestion MEDIUM 50
Question created: Q5(MEDIUM,50)
> CreateQuestion HIGH 100
Question created: Q6(HIGH,100)
> ListQuestion LOW
  Q1(LOW,10)
  Q2(LOW,20)
  Q3(LOW,30)
> CreateContest "diwali_contest" LOW "Ross"
Contest created: 1 diwali_contest LOW by Ross [OPEN]
> CreateContest holi_contest MEDIUM Monica
Contest created: 2 holi_contest MEDIUM by Monica [OPEN]
> ListContest
  1 diwali_contest LOW by Ross [OPEN]
  2 holi_contest MEDIUM by Monica [OPEN]
> AttendContest 1 Monica
Monica registered for contest 1
> AttendContest 1 Joey
Joey registered for contest 1
> AttendContest 1 Chandler
Chandler registered for contest 1
> AttendContest 1 Joey
ERROR: Joey already registered for contest 1
> WithdrawContest 1 Chandler
Chandler withdrew from contest 1
> WithdrawContest 1 Ross
ERROR: Creator cannot withdraw from contest 1
> RunContest 1 Monica
ERROR: Monica did not create contest 1
> RunContest 1 Ross
  Ross solved [2] -> 20 pts
  Monica solved [1, 2] -> 30 pts
  Joey solved [1, 2] -> 30 pts
> RunContest 1 Ross
ERROR: Contest 1 has already run
> AttendContest 1 Chandler
ERROR: Contest 1 has already run
> ContestHistory 1
  Joey : 30 [1, 2]
  Monica : 30 [1, 2]
  Ross : 20 [2]
> AttendContest 2 Chandler
Chandler registered for contest 2
> RunContest 2 Monica
  Monica solved [4, 5] -> 90 pts
  Chandler solved [4] -> 40 pts
> LeaderBoard score desc
  Monica : 1540
  Chandler : 1510
  Joey : 1480
  Ross : 1470
> LeaderBoard asc
  Ross : 1470
  Joey : 1480
  Chandler : 1510
  Monica : 1540
> CreateContest x HIGH Rachel
```

```
ERROR: No such user: Rachel
> Dance 1
ERROR: Unknown command: Dance
```

Checking the arithmetic: contest 1 is LOW, so each participant gets `points − 50`: Ross 1500 + (20 − 50) = 1470, Joey and Monica 1500 + (30 − 50) = 1480. Contest 2 is MEDIUM (`points − 30`): Monica 1480 + (90 − 30) = 1540, Chandler 1500 + (40 − 30) = 1510. The leaderboard matches. (Which questions are "solved" comes from the seeded simulator — change the seed and you get different but equally valid results.)

### Concurrency & edge cases

- **Concurrent commands:** repositories are `ConcurrentHashMap`s; `Contest` methods are `synchronized`; `User.applyContest` does its read-modify-write under the user's lock, so two contests finishing together cannot lose a rating update.
- **Double run:** `Contest.complete` throws if already completed; ratings are updated *after* it succeeds.
- **Known gap (say it):** `runContest` copies the participant list, simulates, then calls `complete`. A user who attends in that short window is registered but gets no result. Fix: add a `RUNNING` status and have `Contest` switch to it and hand back the participant list in one `synchronized` step, so no one can attend or withdraw once the run has started.
- **Edge cases handled:** duplicate user, unknown user/contest/level, non-positive question score, contest with no questions of its level, duplicate attend, attend/withdraw after the run, creator withdraw, non-creator run, missing arguments, non-numeric ids, unknown command.

### Extensions & follow-up questions

- **Pick a fixed number of questions per contest?** A `QuestionSelectionStrategy` (random k of level, or by tags) called at contest creation; store the chosen ids in the contest.
- **Real submissions?** Replace `SolveSimulator` with a judge that returns verdicts per submission; add `Submission(user, question, verdict, time)` and score by time/penalty like ICPC (the International Collegiate Programming Contest).
- **Elo-style ratings?** (Elo is the chess rating system: beating a stronger player earns more points.) A `RatingStrategy` interface taking all participants' results (ratings depend on opponents), selected per contest type.
- **Scheduled contests?** Add `startTime/endTime`, a `SCHEDULED → RUNNING → COMPLETED` state machine, and a scheduler that triggers `run`.
- **Leaderboard at scale?** Keep a `TreeSet` (a set that keeps its items sorted) sorted by (score, name) or a Redis sorted set; update on each rating change instead of sorting on every query.

### Real-world example

LeetCode's weekly contests publish a live ranking during the contest, and afterwards each participant's contest rating is recalculated from their rank relative to others — a more elaborate version of the per-level rule in this statement. HackerRank and Codeforces follow the same shape: contests reference a pool of problems, participants register, and ratings change only after the contest is finalised.

### In the interview

Paste the sample commands into the driver list first. Build `CreateUser`, `CreateQuestion`, `CreateContest`, `AttendContest`, `RunContest`, `LeaderBoard` in that order, running after each. Point out the three design moves: command map, scoring rule on the enum, and randomness behind an interface (with a seed). Then bonus commands — both are tiny because `Contest` already stores results and guards its status.

### Self-check

- **Q:** Why must `RunContest` update ratings only after marking the contest complete? → **A:** If the contest had already run, `complete` throws before any rating changes — no double application.
- **Q:** How would you add a `VERY_HIGH` level with bonus +20? → **A:** Add an enum constant with its own rule (e.g. a negative penalty); nothing else changes.
- **Q:** How do you make `RunContest` deterministic in tests? → **A:** Inject a scripted `SolveSimulator` (or a seeded random one).
- **Q:** Where is "creator always attends" enforced? → **A:** In the `Contest` constructor.

### TL;DR

Command map in the driver, rules in the service, invariants in `Contest`, scoring on the `Level` enum, randomness behind a seeded `SolveSimulator`. Complete first, then update ratings.

---

## 21.5 Design a Property Listing Site — "PropertyHunt" (ClearTrip)

### Problem & clarifying questions

A ClearTrip machine-coding statement (kumaransg/LLD, `PropertyHunt`). The folder holds two versions: a text `readme.md` and a longer version as screenshots in `Question Image/`; the summary below follows the longer one, which adds ViewSold, the range rules and the "shortlist *other users'* property" wording. Build a console application (a text-only program run in a terminal) for a property platform where users can: **register**; **list a property** they want to sell or rent; **view their own listings**; **shortlist** other users' properties; **view shortlisted** properties (sold ones still appear, with status SOLD); **search** available properties with any combination of **location** (case-insensitive — capital and small letters treated the same — exact match), **price range**, **listing type** (sell/rent), **size range**, **number of rooms** (1BHK/2BHK/3BHK — bedroom, hall, kitchen: 1BHK is one bedroom), and **sort by** price or size (ascending); **mark sold** (only the owner); and **view sold** properties. Ranges: `X-Y` inclusive; a single `X` means "at least X" for size and "at most X" for price. **Bonus:** sizes in sq ft / sq m / sq yd (buyer and seller may use different units); **nearby locations** on a listing, with nearby matches shown last; prices like `40000`, `40k`, `60L`, `2Cr` (k = thousand; L = lakh = 100,000; Cr = crore = 10,000,000 — the Indian number units); comma-separated values for location and rooms in search. The statement asks for in-memory storage, graceful errors ("Property does not exist"), and a clean, extensible OO design.

| Question | Answer used |
|---|---|
| Input format? | One command per line, fields separated by ` |
| Can I shortlist my own property? | No — the screenshot version says users shortlist *others'* properties. (The text readme's sample has Jack shortlisting his own listing; that is exactly the kind of conflict to ask about.) |
| Can I shortlist a sold property? | No; but a property sold *after* I shortlisted it stays in my list with status SOLD. |
| Units internally? | Price in whole rupees ( `long` ), size in square feet ( `double` ). |
| Sort with location? | Exact location matches first, nearby matches after, then the chosen sort. |
| Mark sold — record buyer and price? | The screenshot sample writes `marksold - 2, xyz, 50000` (property, buyer, price). We store only the status; keeping buyer and sale price is a small extension (a `Sale` record). |

The sample search ( `bellandur, 30000-50000, rent, ...` ) returns a *sarjapur* property. In the screenshot version that is consistent: the sarjapur listing declares `nearby bellandur|kadubeesanahalli|marathahalli`, so it is a nearby match. The text readme's sample drops the `nearby` field, and there the result only makes sense if you know about the bonus. Our demo lists sarjapur with the same three nearby localities, and the search returns it.

### Requirements

**Functional:** RegisterUser, ListProperty, Search, Shortlist, ViewShortlisted, ViewListed, MarkSold, ViewSold — plus all four bonus items. **Non-functional:** search filters combine freely; new filters or units without rewriting search; clear errors; safe under concurrent use.

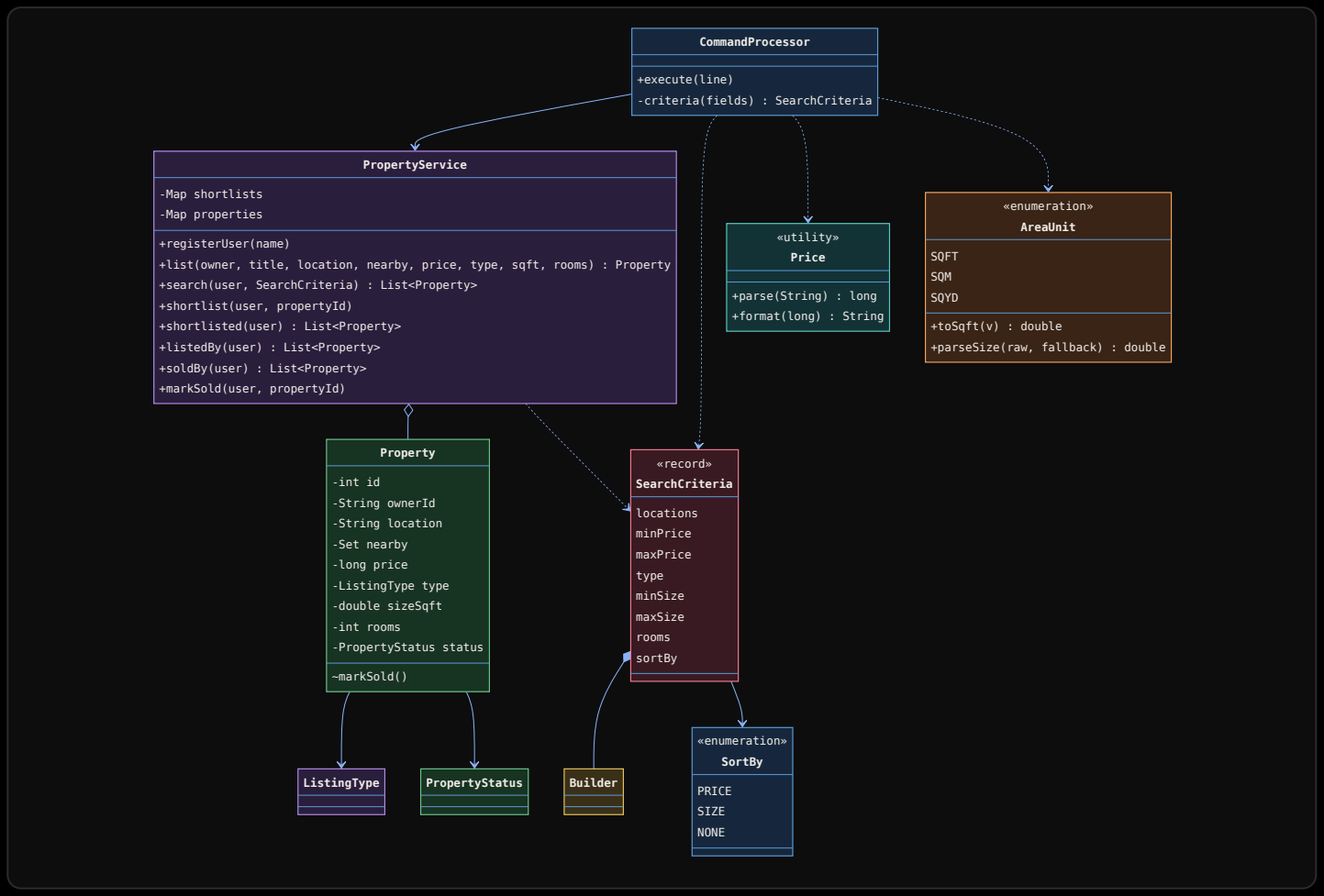
### Use cases

| Actor | Use cases |
|---|---|
| **Seller / landlord** | list property, view listed, mark sold, view sold |
| **Buyer / tenant** | search, shortlist, view shortlisted |
| (Any user can be both — roles are per property, not per account: the owner of a property is its seller.) | |

### Core entities

| Class | Kind | Responsibility |
|---|---|---|
| `Property` | class | Listing data, owner, nearby locations, status; `markSold` guards double sale. |
| `ListingType`, `PropertyStatus`, `SortBy` | enums | SELL/RENT, AVAILABLE/SOLD, sort comparators. |
| `Price` | utility | Parse `40k/60L/2Cr/1.5Cr` → rupees; format back. |
| `AreaUnit` | enum | SQFT/SQM/SQYD with conversion to square feet; parses "1500 sq ft", "70 sqm". |
| `SearchCriteria` (+ `Builder`) | record | All-optional search parameters. |
| `PropertyService` | class | Register, list, search, shortlist, mark sold, views; owner-vs-buyer rules. |
| `CommandProcessor` | class | Parses ` |
| `HuntException` | exception | Readable errors. |

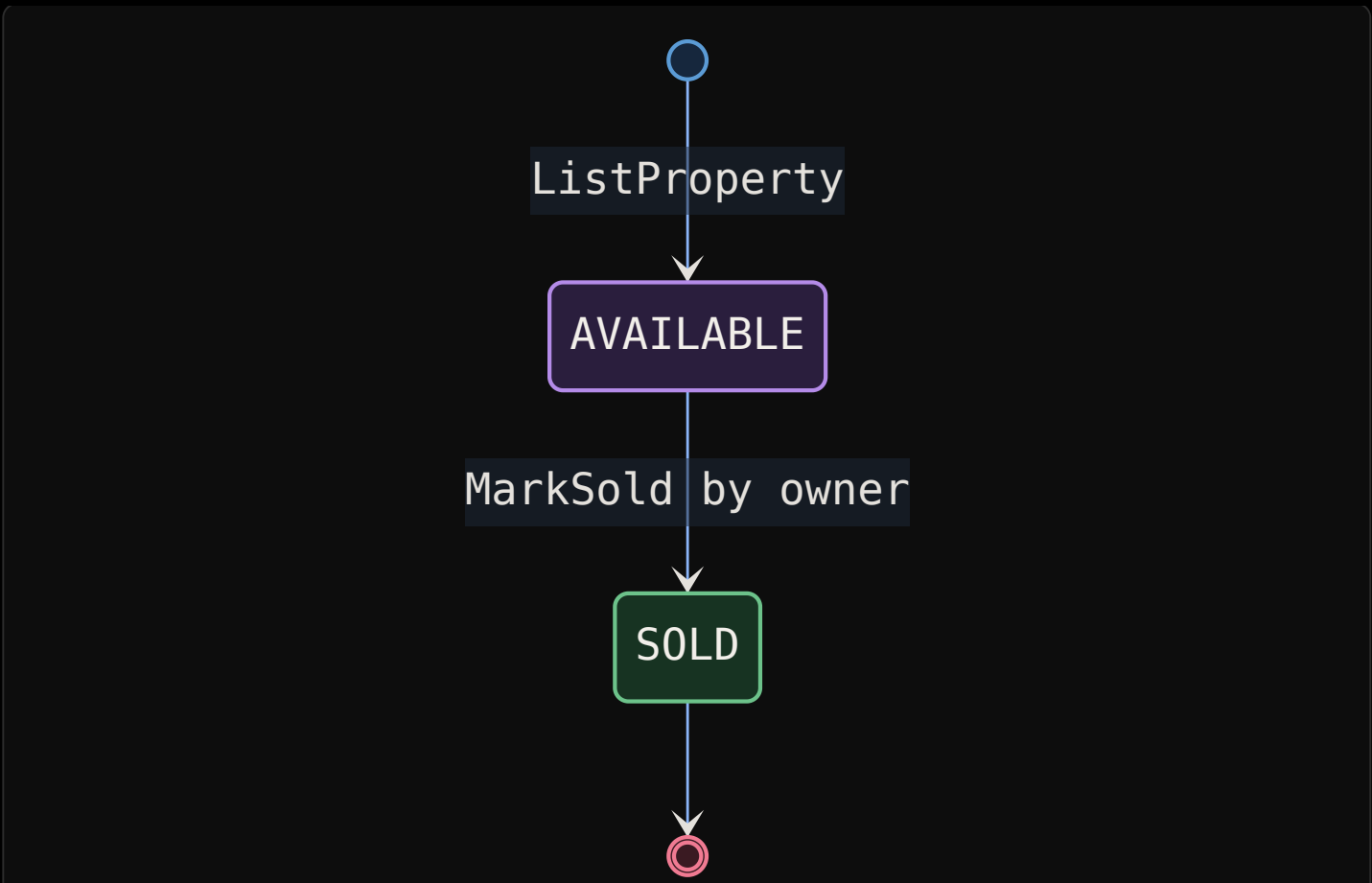
## Class diagram




## Key flows

**Search = build criteria → list of predicates → one combined predicate → sort:**






**Design decisions & patterns used**

| Decision | Pattern | Why | Alternative rejected |
|---|---|---|---|
| All-optional `SearchCriteria` with a Builder | Builder | Eight optional fields; a constructor call with eight nulls is unreadable. | Telescoping constructors (a chain of constructors, each taking one more parameter) / overloads (several methods with the same name but different parameters). |
| Each criterion becomes a `Predicate<Property>`, reduced with `and` | Specification | Any combination works; a new filter is one line; no nested `if` pyramid. | One `if` per combination. |
| `SortBy` enum of comparators | Strategy | Adding "sort by rooms" is one constant. | `switch` inside search. |
| Normalise at the boundary (rupees, sq ft) | — | Business code compares plain numbers; units only matter while parsing. | Storing strings like "90L" and comparing them. |
| Location rank 0/1/2 | — | Exact matches first, nearby second, others filtered out — one small function serves filtering *and* ordering. | Two separate searches merged. |
| Shortlist stored per user as a set of ids | — | View shortlisted shows current status (SOLD appears automatically). | Copying property objects into the shortlist — they would go stale. |

**Java implementation**

Full runnable code: `code/lld/ch21/propertyhunt/` (11 files). Below are the files that carry the design; the rest are `Property`, `ListingType`, `PropertyStatus`, `SortBy`, `HuntException`, the pipe-separated `CommandProcessor` and `Main`.

```java
// File: lld/ch21/propertyhunt/Price.java
package lld.ch21.propertyhunt;

import java.math.BigDecimal;
import java.util.Locale;

/** Parses and prints Indian-style prices: 40000, 40k, 60L, 2Cr, 1.5Cr. Stored as whole rupees. */
public final class Price {
    private Price() { }

    public static long parse(String raw) {
        String s = raw.trim().toLowerCase(Locale.ROOT).replace(",", "").replace(" ", "");
        long mult = 1;
        if (s.endsWith("cr")) { mult = 10_000_000L; s = s.substring(0, s.length() - 2); }
        else if (s.endsWith("l")) { mult = 100_000L; s = s.substring(0, s.length() - 1); }
        else if (s.endsWith("k")) { mult = 1_000L; s = s.substring(0, s.length() - 1); }
        try {
            return new BigDecimal(s).multiply(BigDecimal.valueOf(mult)).longValueExact();
        } catch (ArithmeticException | NumberFormatException e) {
            throw new HuntException("Bad price: " + raw);
        }
    }

    public static String format(long rupees) {
        if (rupees >= 10_000_000L) return shift(rupees, 7) + "Cr";
        if (rupees >= 100_000L) return shift(rupees, 5) + "L";
        if (rupees >= 1_000L) return shift(rupees, 3) + "k";
        return Long.toString(rupees);
    }

    /** Exact decimal shift, e.g. shift(25000000, 7) = "2.5" - no double rounding. */
    private static String shift(long rupees, int places) {
        return BigDecimal.valueOf(rupees).movePointLeft(places).stripTrailingZeros().toPlainString();
    }
}
```

```java
// File: lld/ch21/propertyhunt/AreaUnit.java
package lld.ch21.propertyhunt;

import java.util.Locale;

/** Every size is converted to square feet internally so buyer and seller units can differ. */
public enum AreaUnit {
    SQFT(1.0), SQM(10.7639), SQYD(9.0);

    private final double sqftPerUnit;
    AreaUnit(double f) { this.sqftPerUnit = f; }

    public double toSqft(double v) { return v * sqftPerUnit; }

    /** Accepts "sqft", "sq ft", "sqm", "sq meter", "sqyd", "sq yard" ... (spaces ignored). */
    public static AreaUnit parse(String raw) {
        String s = raw.toLowerCase(Locale.ROOT).replace(" ", "").replace(".", "");
        if (s.isEmpty() || s.startsWith("sqf") || s.equals("ft")) return SQFT;
        if (s.startsWith("sqm")) return SQM;
        if (s.startsWith("sqy")) return SQYD;
        throw new HuntException("Unknown size unit: " + raw);
    }

    /** "1800 sqft" -> 1800.0 ; "200 sqyd" -> 1800.0 ; a bare number uses `fallback`. */
    public static double parseSize(String raw, AreaUnit fallback) {
        String s = raw.trim();
        int i = 0;
        while (i < s.length() && (Character.isDigit(s.charAt(i)) || s.charAt(i) == '.')) i++;
        if (i == 0) throw new HuntException("Bad size: " + raw);
        double value = Double.parseDouble(s.substring(0, i));
        String unit = s.substring(i).trim();
        return (unit.isEmpty() ? fallback : parse(unit)).toSqft(value);
    }
}
```

```java
// File: lld/ch21/propertyhunt/SearchCriteria.java
package lld.ch21.propertyhunt;

import java.util.Set;

/** Every field is optional (null = "don't care"). Built with the nested Builder. */
public record SearchCriteria(Set<String> locations, Long minPrice, Long maxPrice, ListingType type,
                             Double minSize, Double maxSize, Set<Integer> rooms, SortBy sortBy) {

    public static Builder builder() { return new Builder(); }

    public static final class Builder {
        private Set<String> locations; private Long minPrice, maxPrice; private ListingType type;
        private Double minSize, maxSize; private Set<Integer> rooms; private SortBy sortBy = SortBy.NONE;

        public Builder locations(Set<String> l) { locations = l; return this; }
        public Builder price(Long min, Long max) { minPrice = min; maxPrice = max; return this; }
        public Builder type(ListingType t) { type = t; return this; }
        public Builder size(Double min, Double max) { minSize = min; maxSize = max; return this; }
        public Builder rooms(Set<Integer> r) { rooms = r; return this; }
        public Builder sortBy(SortBy s) { sortBy = s; return this; }
        public SearchCriteria build() {
            return new SearchCriteria(locations, minPrice, maxPrice, type, minSize, maxSize, rooms, sortBy);
        }
    }
}
```

```java
// File: lld/ch21/propertyhunt/PropertyService.java  (excerpt)
public class PropertyService {
    private final Map<String, Set<Integer>> shortlists = new ConcurrentHashMap<>();  // user -> ids (the user registry)
    private final Map<Integer, Property> properties = new ConcurrentHashMap<>();
    private final AtomicInteger seq = new AtomicInteger();

    // ... registerUser, list (validates, lower-cases locations)

    /** Only AVAILABLE properties. Exact location matches first, nearby matches last, then the chosen sort. */
    public List<Property> search(String user, SearchCriteria c) {
        requireUser(user);
        Set<String> locs = c.locations() == null ? null : lower(c.locations());
        List<Predicate<Property>> filters = new ArrayList<>();
        filters.add(p -> p.status() == PropertyStatus.AVAILABLE);
        if (locs != null) filters.add(p -> locationRank(p, locs) < 2);
        if (c.minPrice() != null) filters.add(p -> p.price() >= c.minPrice());
        if (c.maxPrice() != null) filters.add(p -> p.price() <= c.maxPrice());
        if (c.type() != null) filters.add(p -> p.type() == c.type());
        if (c.minSize() != null) filters.add(p -> p.sizeSqft() >= c.minSize());
        if (c.maxSize() != null) filters.add(p -> p.sizeSqft() <= c.maxSize());
        if (c.rooms() != null) filters.add(p -> c.rooms().contains(p.rooms()));
        Predicate<Property> all = filters.stream().reduce(x -> true, Predicate::and);

        Comparator<Property> order = locs == null ? c.sortBy().comparator
                : Comparator.<Property>comparingInt(p -> locationRank(p, locs)).thenComparing(c.sortBy().comparator);
        return properties.values().stream().filter(all).sorted(order).toList();
    }
    // ... shortlist, shortlisted, listedBy, soldBy, markSold (owner-only checks)

    /** 0 = exact location match, 1 = nearby match, 2 = no match. */
    private static int locationRank(Property p, Set<String> locs) {
        if (locs.contains(p.location())) return 0;
        return p.nearby().stream().anyMatch(locs::contains) ? 1 : 2;
    }
    // ... lookups
}
```

**Output** (of `Main.java`):

```
-> RegisterUser|Jack
User registered successfully.
-> RegisterUser|Jill
User registered successfully.
-> RegisterUser|Jack
ERROR: User already registered: Jack
-> ListProperty|Jack|A 3BHK for sale|bellandur|90L|sell|1800 sqft|3
Property listed successfully. id=1
-> ListProperty|Jack|A 2BHK for rent|sarjapur|40k|rent|1500 sq ft|2|bellandur,kadubeesanahalli,marathahalli
Property listed successfully. id=2
-> ListProperty|Jill|Cosy 1BHK|HSR|25k|rent|70 sqm|1BHK
Property listed successfully. id=3
-> ListProperty|Jill|Villa|whitefield|2.5Cr|sell|400 sqyd|4
Property listed successfully. id=4
-> Search|Jack|null|null|null|null|null|null
<- Id  Title               Location   Price  Size      Rooms For
<- 1   A 3BHK for sale     bellandur  90L    1800sqft  3BHK  Sale
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent
<- 3   Cosy 1BHK           hsr        25k    753sqft   1BHK  Rent
<- 4   Villa               whitefield 2.5Cr  3600sqft  4BHK  Sale
-> Search|Jack|bellandur|30000-50000|rent|150sqft|2|price
<- Id  Title               Location   Price  Size      Rooms For
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent
-> Search|Jack|bellandur|null|null|null|null|price
<- Id  Title               Location   Price  Size      Rooms For
<- 1   A 3BHK for sale     bellandur  90L    1800sqft  3BHK  Sale
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent
-> Search|Jill|null|1Cr|null|1000-2000 sqft|2,3|size
<- Id  Title               Location   Price  Size      Rooms For
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent
<- 1   A 3BHK for sale     bellandur  90L    1800sqft  3BHK  Sale
-> Search|Jill|hsr,sarjapur|null|rent|null|1BHK,2BHK|price
<- Id  Title               Location   Price  Size      Rooms For
<- 3   Cosy 1BHK           hsr        25k    753sqft   1BHK  Rent
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent
-> Shortlist|Jill|1
Shortlisted property
-> Shortlist|Jill|2
Shortlisted property
-> Shortlist|Jill|9
ERROR: Property does not exist: 9
-> Shortlist|Jack|1
ERROR: You cannot shortlist your own property
-> MarkSold|Jill|1
ERROR: Only the owner can mark property 1 sold
-> MarkSold|Jack|1
Property marked SOLD
-> ViewShortlisted|Jill
<- Id  Title               Location   Price  Size      Rooms For   Status
<- 1   A 3BHK for sale     bellandur  90L    1800sqft  3BHK  Sale  SOLD
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent  AVAILABLE
-> ViewListed|Jack
<- Id  Title               Location   Price  Size      Rooms For   Status
<- 1   A 3BHK for sale     bellandur  90L    1800sqft  3BHK  Sale  SOLD
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent  AVAILABLE
-> ViewSold|Jack
<- Id  Title               Location   Price  Size      Rooms For   Status
<- 1   A 3BHK for sale     bellandur  90L    1800sqft  3BHK  Sale  SOLD
-> Search|Jill|null|null|sell|null|null|price
<- Id  Title               Location   Price  Size      Rooms For
<- 4   Villa               whitefield 2.5Cr  3600sqft  4BHK  Sale
-> ListProperty|Jill|Bad|x|abc|sell|100 sqft|1
ERROR: Bad price: abc
-> Search|Bob|null|null|null|null|null|null
ERROR: User not registered: Bob
```

Things to notice: 70 sq m was stored as 753 sq ft and 400 sq yd as 3600 sq ft; the `bellandur` search returns the bellandur listing first and the sarjapur listing (nearby) last; after `MarkSold`, property 1 disappears from searches but still shows as SOLD in Jill's shortlist; bad input produces an error line and the program continues.

### Concurrency & edge cases

- **Storage:** `ConcurrentHashMap`s; each user's shortlist is a concurrent set; `Property.markSold` is `synchronized` so two "mark sold" calls cannot both succeed.
- **A search racing a sale** may still show a property that is being sold at that instant — acceptable for listings (eventual consistency: different readers may briefly see different values, but they all catch up soon); a purchase flow would re-check status under a lock.
- **Edge cases handled:** duplicate registration, unregistered user, unknown property, own-property shortlist, duplicate shortlist, shortlisting a sold property, non-owner mark sold, double sale, bad price/size/unit/type, non-positive values, missing fields.

### Extensions & follow-up questions

- **Descending sort / multiple sort keys?** `SortBy` holds a comparator; add a direction flag and `.reversed()`, or accept a list of keys and chain `thenComparing`.
- **Fuzzy location ("Bellandur Road")?** Normalise locations to ids through a `LocationResolver` interface (gazetteer / geocoding — a directory of place names / turning an address into map coordinates).
- **"Within 5 km"?** Store latitude/longitude and filter by haversine distance (the standard formula for the distance between two points on a sphere); at scale use a geo-index (a lookup structure for finding things by location: geohash, R-tree) or a search engine with geo queries.
- **Fast search over millions?** Secondary indexes (by location, type, rooms) or Elasticsearch/OpenSearch; price and size as numeric range fields.
- **Notifications on price drop for shortlisted items?** Observer on `Property` price changes → notify users who shortlisted it.

#### Real-world example

Property portals like 99acres, MagicBricks and NoBroker let you filter by locality, budget, BHK count, buy/rent and area, and often suggest nearby localities when results are few; Indian listings commonly quote prices in lakhs and crores — exactly why the statement asks you to parse `60L` and `2Cr`.

#### In the interview

Start with the model and units (show `Price.parse` handling `90L`, and `AreaUnit`). Then implement register → list → search with *one* filter → add filters as predicates → sort. Shortlist, mark sold and the views are small after that. Call out the Builder and predicate list as your extensibility story, and the "normalise at the boundary" principle.

#### Self-check

- **Q:** How does a buyer searching in sq m match a seller who listed in sq yd? → **A:** Both are converted to sq ft at parse time; search compares sq ft.
- **Q:** Why store shortlists as ids rather than objects? → **A:** Status is read live from the property, so SOLD shows without extra updates.
- **Q:** What does a single number mean in the price field? → **A:** "At most X" (for size it means "at least X").
- **Q:** How do you add a "furnished" filter? → **A:** A field on `Property`, an optional field in `SearchCriteria`, and one predicate line in `search`.

#### TL;DR

Normalise price and size at the edges, turn every optional criterion into a predicate and AND them together, sort by (location rank, chosen key, id), and keep shortlists as ids so status stays live.

---

## 21.6 Design a Loan Ledger — "Ledger Co" (Navi)

### Problem & clarifying questions

This problem (kumaransg/LLD, `ledger_company_navi`) is the Geektrust "Ledger Co" backend (server-side logic, no screens) challenge; the repository files it under Navi, which has used it as a hiring exercise. Banks lend money to borrowers; borrowers repay in **EMIs** (Equated Monthly Instalments — the same amount every

month) and may also pay a **lump sum** (a one-off extra payment) at any point. The program reads commands and answers balance questions:

| Command | Meaning |
|---|---|
| `LOAN BANK BORROWER P N R` | Borrower takes principal `P` (the amount borrowed) for `N` years at `R` % simple interest (interest charged only on the original principal, never on interest) per year. |
| `PAYMENT BANK BORROWER LUMP_SUM EMI_NO` | A lump sum paid right after EMI number `EMI_NO`. |
| `BALANCE BANK BORROWER EMI_NO` | Print `BANK BORROWER AMOUNT_PAID EMIS_LEFT` as of (after) EMI number `EMI_NO`. |

Rules from the statement: interest `I = P × N × R / 100` (R is a percentage); total repayable `A = P + I`; monthly EMI = `A / (N × 12)` **rounded up** to a whole number (86.67 → 87, and even 100.10 → 101); amount paid as of EMI *k* = `k × EMI` + every lump sum whose EMI number is *k* or less (a lump sum "after EMI 10" *is* included in `BALANCE ... 10`); EMIs left = remaining amount ÷ EMI, **rounded up** (23.1 → 24); and if the last EMI is more than what is still owed, it shrinks to exactly that amount. A lump sum therefore reduces the *number* of EMIs, never the EMI amount. Only `BALANCE` produces output, in the form `BANK_NAME BORROWER_NAME AMOUNT_PAID NO_OF_EMIS_LEFT`.

| Question | Answer used |
|---|---|
| Can amount paid exceed `A`? | No — capped at `A` (then EMIs left = 0). This is also the "last EMI shrinks" rule: the final payment is only what is left. |
| Is interest a whole number? | The statement defines rounding only for the EMI and the EMI count. We *assume* interest is also rounded up to a whole unit (say so to the interviewer); every official sample has whole-number interest, so both samples come out the same either way. |
| One loan per bank-borrower pair? | Yes — the pair identifies the loan; a second `LOAN` for the same pair is an error. |
| `EMI_NO` beyond the tenure (the loan's total length)? | Error. `EMI_NO = 0` is valid (before any EMI). |

## Requirements

**Functional:** LOAN, PAYMENT, BALANCE exactly as above; reproduce the official sample outputs. **Non-functional:** input from a list/file; one bad line must not stop the rest; interest rule replaceable; exact arithmetic (`BigDecimal`).

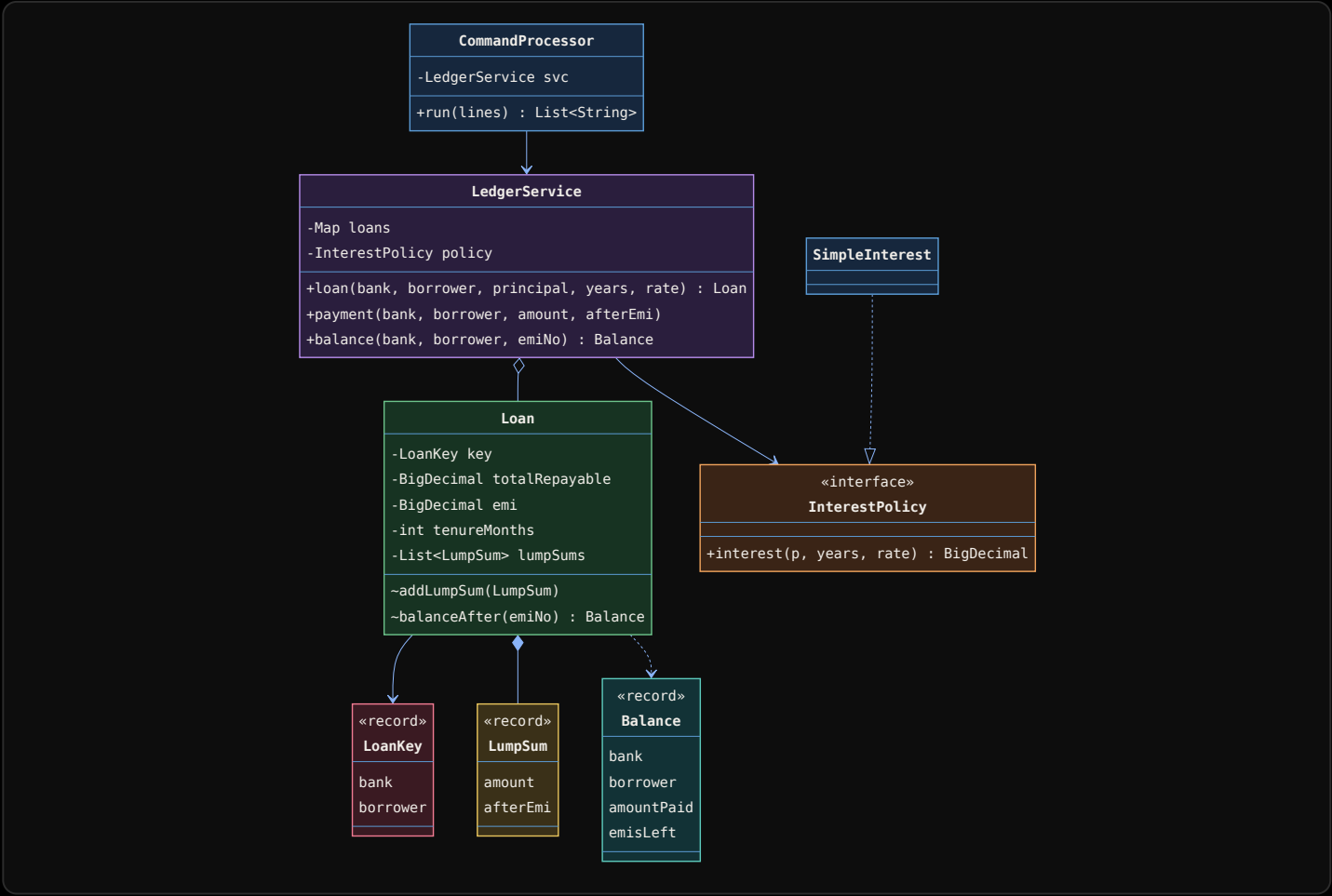
### Use cases

| Actor | Use cases |
|---|---|
| **Bank** | records a loan |
| **Borrower** | pays lump sums |
| **Anyone (support agent, borrower app)** | asks for the balance after EMI *k* |

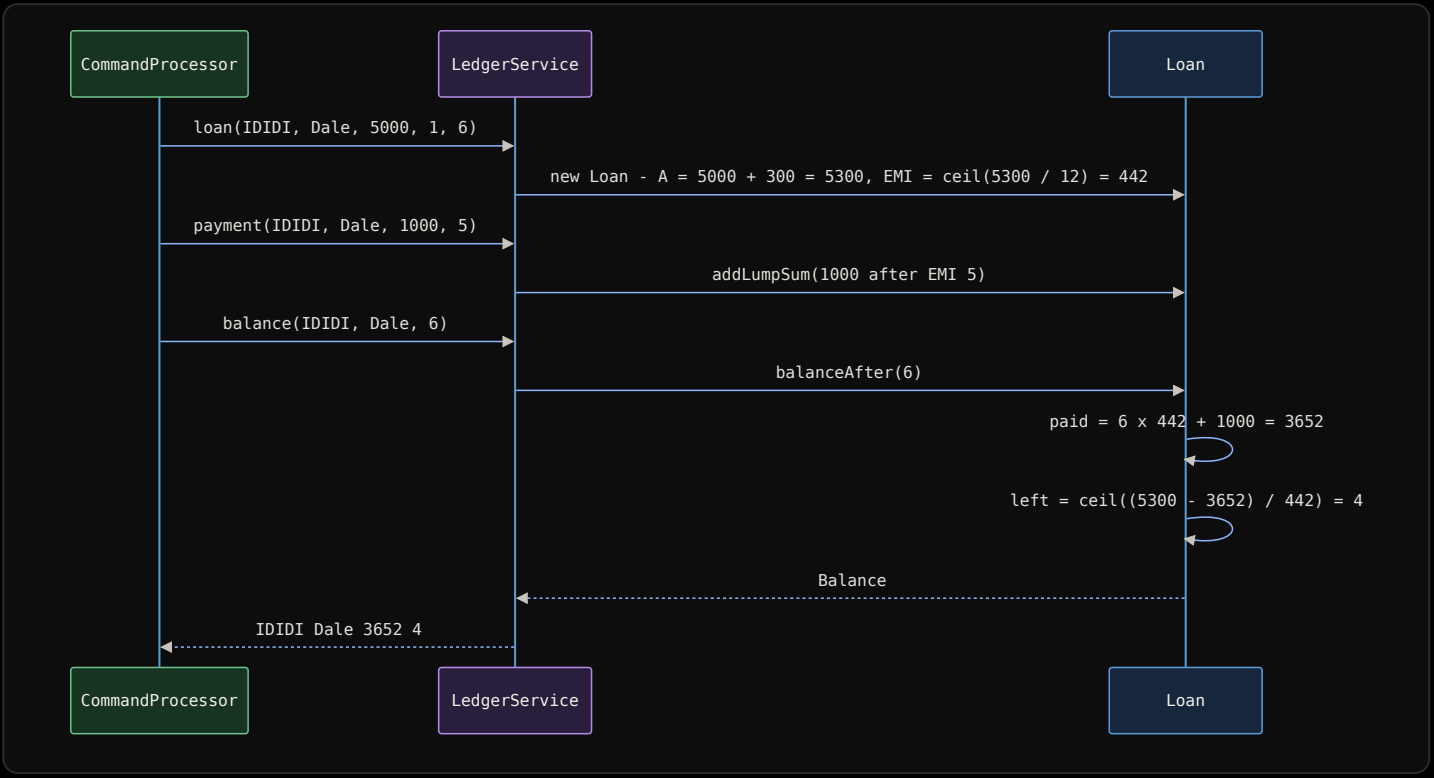
## Core entities

| Class | Kind | Responsibility |
|---|---|---|
| `LoanKey` | record | (bank, borrower) — the loan's identity; a perfect `Map` key because records implement `equals/hashCode` (the two methods a map uses to decide whether two keys are the same). |
| `Loan` | class | Computes `A` and EMI once; stores lump sums; derives a `Balance` for any EMI number. |
| `LumpSum` | record | amount + "after EMI number". |
| `Balance` | record | Output line: bank, borrower, amount paid, EMIs left. |
| `InterestPolicy` / `SimpleInterest` | interface / class | Strategy for total interest. |
| `LedgerService` | class | Store of loans; the three operations. |
| `CommandProcessor` | class | Parses lines, validates argument counts and numbers, collects output. |
| `LedgerException` | exception | Readable failure. |

## Class diagram



### Key flows




**Worked example (Sample 2, Shelly):** P = 15000, N = 2, R = 9 → I = 2700, A = 17700, EMI = ⌈17700 / 24⌉ = ⌈737.5⌉ = 738. After 12 EMIs plus a 7000 lump sum: paid = 12 × 738 + 7000 = 15856; remaining = 1844; EMIs left = ⌈1844 / 738⌉ = ⌈2.499⌉ = 3. Output `UON Shelly 15856 3`.

### Design decisions & patterns used

| Decision | Why | Alternative rejected |
|---|---|---|
| **Derive balance from facts** (A, EMI, lump sums) on every query | Queries can ask about *any* EMI number, even one before a later payment. Storing a running balance would need replays. | Mutable `remaining` field updated per command — breaks when a BALANCE asks about an earlier EMI. |
| `InterestPolicy` Strategy | Simple interest today; reducing-balance or compound tomorrow without touching `Loan`. | Formula hard-coded in `Loan`. |
| `LoanKey` record as map key | Value semantics (two keys with the same fields count as equal) for free (`equals/hashCode`). | Concatenated string `"IDIDI#Dale"` — fragile. |
| `BigDecimal` + explicit `RoundingMode.CEILING` | The statement's rounding is "always up"; ceiling on integers is exact. | `Math.ceil` on doubles — fine for small numbers but floating error (tiny inaccuracies of binary floating-point numbers) can bite (e.g. a value like 4.000000001 ceils to 5). |
| Driver returns `List<String>` | Easy to compare with the expected output in a test. | Printing inside the service. |

### Java implementation

Full runnable code: `code/lld/ch21/ledger/` (10 files). Below are the files that carry the design; the rest are `LoanKey`, `LumpSum`, `Balance`, `LedgerException`, the `CommandProcessor` (LOAN / PAYMENT / BALANCE) and `Main`.

```java
// File: lld/ch21/ledger/InterestPolicy.java
package lld.ch21.ledger;

import java.math.BigDecimal;

/** Strategy for total interest. The problem uses simple interest; a bank could plug in another rule. */
public interface InterestPolicy {
    BigDecimal interest(BigDecimal principal, BigDecimal years, BigDecimal ratePercent);
}
```

```java
// File: lld/ch21/ledger/SimpleInterest.java
package lld.ch21.ledger;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** I = P * N * R / 100, rounded UP to a whole unit (the statement rounds amounts up). */
public class SimpleInterest implements InterestPolicy {
    @Override public BigDecimal interest(BigDecimal p, BigDecimal n, BigDecimal r) {
        return p.multiply(n).multiply(r).divide(BigDecimal.valueOf(100), 0, RoundingMode.CEILING);
    }
}
```

```java
// File: lld/ch21/ledger/Loan.java
package lld.ch21.ledger;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.List;

/**
 * Loan with fixed EMI. Amount repaid A = P + I. EMI = ceil(A / (N*12)).
 * Lump sums shorten the loan (fewer EMIs) but never change the EMI amount.
 */
public class Loan {
    private final LoanKey key;
    private final BigDecimal totalRepayable;
    private final BigDecimal emi;
    private final int tenureMonths;
    private final List<LumpSum> lumpSums = new ArrayList<>();

    Loan(LoanKey key, BigDecimal principal, int years, BigDecimal rate, InterestPolicy policy) {
        if (principal.signum() <= 0 || years <= 0 || rate.signum() < 0)
            throw new LedgerException("Invalid loan terms for " + key);
        this.key = key;
        this.tenureMonths = years * 12;
        this.totalRepayable = principal.add(policy.interest(principal, BigDecimal.valueOf(years), rate));
        this.emi = totalRepayable.divide(BigDecimal.valueOf(tenureMonths), 0, RoundingMode.CEILING);
    }

    public BigDecimal emi() { return emi; }
    public BigDecimal totalRepayable() { return totalRepayable; }

    synchronized void addLumpSum(LumpSum l) {
        if (l.amount().signum() <= 0) throw new LedgerException("Lump sum must be positive");
        if (l.afterEmi() < 0 || l.afterEmi() > tenureMonths) throw new LedgerException("EMI number out of range: " +
l.afterEmi());
        lumpSums.add(l);
    }

    /** Amount paid after `emiNo` EMIs (lump sums made at or before that EMI count), capped at A. */
    synchronized Balance balanceAfter(int emiNo) {
        if (emiNo < 0 || emiNo > tenureMonths) throw new LedgerException("EMI number out of range: " + emiNo);
        BigDecimal lumps = lumpSums.stream().filter(l -> l.afterEmi() <= emiNo)
                .map(LumpSum::amount).reduce(BigDecimal.ZERO, BigDecimal::add);
        BigDecimal paid = emi.multiply(BigDecimal.valueOf(emiNo)).add(lumps).min(totalRepayable);
        BigDecimal remaining = totalRepayable.subtract(paid);
        int left = remaining.divide(emi, 0, RoundingMode.CEILING).intValueExact();
        return new Balance(key.bank(), key.borrower(), paid, left);
    }
}
```

```java
// File: lld/ch21/ledger/LedgerService.java
package lld.ch21.ledger;

import java.math.BigDecimal;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class LedgerService {
    private final Map<LoanKey, Loan> loans = new ConcurrentHashMap<>();
    private final InterestPolicy policy;

    public LedgerService(InterestPolicy policy) { this.policy = policy; }

    public Loan loan(String bank, String borrower, BigDecimal principal, int years, BigDecimal rate) {
        LoanKey key = new LoanKey(bank, borrower);
        Loan loan = new Loan(key, principal, years, rate, policy);
        if (loans.putIfAbsent(key, loan) != null) throw new LedgerException("Loan already exists for " + bank + " " +
borrower);
        return loan;
    }

    public void payment(String bank, String borrower, BigDecimal amount, int afterEmi) {
        find(bank, borrower).addLumpSum(new LumpSum(amount, afterEmi));
    }

    public Balance balance(String bank, String borrower, int emiNo) { return find(bank, borrower).balanceAfter(emiNo);
}

    private Loan find(String bank, String borrower) {
        Loan l = loans.get(new LoanKey(bank, borrower));
        if (l == null) throw new LedgerException("No loan for " + bank + " " + borrower);
        return l;
    }
}
```

**Output** (of `Main.java`):

```
== Sample 1
IDIDI Dale 1000 55
IDIDI Dale 8000 20
MBI Harry 1044 12
MBI Harry 0 24
== Sample 2
IDIDI Dale 1326 9
IDIDI Dale 3652 4
UON Shelly 15856 3
MBI Harry 9044 10
== Edge cases
SBI Asha 1120 0
ERROR Loan already exists for SBI Asha  [LOAN SBI Asha 5 1 1]
ERROR No loan for HDFC Nobody  [BALANCE HDFC Nobody 1]
ERROR EMI number out of range: 13  [BALANCE SBI Asha 13]
ERROR Not a number: ten  [PAYMENT SBI Asha ten 1]
ERROR Unknown command REFUND  [REFUND SBI Asha 1]
```

The first two blocks are exactly the expected outputs of the two official Geektrust samples. In the edge-case block, Asha's 2000 lump sum after EMI 1 overpays a 1120 loan, so the amount paid is capped at 1120 and 0 EMIs remain.

### Concurrency & edge cases

- **Concurrency:** loans live in a `ConcurrentHashMap` (`putIfAbsent` makes "one loan per pair" atomic); `Loan`'s lump-sum list is guarded by `synchronized` methods, so a `PAYMENT` and a `BALANCE` on the same loan do not interfere.
- **Edge cases handled:** duplicate loan, unknown loan, EMI number < 0 or > tenure, non-positive principal/years, negative rate, non-positive lump sum, overpayment (capped), wrong argument count, non-numeric values, unknown command, blank lines.

### Extensions & follow-up questions

- **Reducing-balance (amortised) EMI like real banks?** (Reducing-balance: each month's interest is charged only on what is still owed; amortised: spread into equal payments that cover interest and principal.) New `InterestPolicy`/`EmiCalculator`: `EMI = P·r·(1+r)^n / ((1+r)^n − 1)` with monthly rate `r`; balance after k months uses the amortisation schedule. The `Loan` API stays the same.
- **Lump sum that should reduce the EMI instead of the tenure?** Make that a `PrepaymentPolicy` (REDUCE_TENURE / REDUCE_EMI) chosen per loan.
- **Late payments and penalties?** Record actual payment events with dates instead of assuming EMIs are paid on time; derive "EMIs paid" from them.
- **Read from a file?** `Files.readAllLines(Path.of(args[0]))` fed into the same `run` method — the driver's job, no core change.

### Real-world example

Indian lenders such as Navi, banks and NBFCs (non-banking financial companies: lenders that are not licensed banks) show borrowers an EMI schedule and let them "part-prepay" a loan; most offer a choice between reducing the tenure (fewer EMIs, the Ledger Co rule) or reducing the EMI amount. Real loans use reducing-balance interest rather than the flat simple interest in this puzzle — a good point to raise when asked about extensions.

### In the interview

Show the three formulas first and compute one sample by hand — it proves you understood the rounding. Then code `Loan.balanceAfter` (the heart), the service, and the command loop. Run both samples. Emphasise "state = facts, balances = derived", and the interest Strategy.

### Self-check

- **Q:** Why ceil the EMI? → **A:** A floor would leave a shortfall at the end; rounding up guarantees N × 12 EMIs cover `A` (the final instalment may be smaller).
- **Q:** A `PAYMENT` after EMI 10 — does `BALANCE ... 5` include it? → **A:** No; only lump sums with `afterEmi ≤ 5` count.
- **Q:** Why is the balance computed rather than stored? → **A:** Queries may ask about any EMI number in any order; derivation is always consistent.
- **Q:** Where do you add compound interest (interest that is also charged on earlier interest)? → **A:** A new `InterestPolicy` implementation injected into `LedgerService`.

### TL;DR

EMI = ⌈(P + PNR/100) / 12N⌉. Store loan facts and lump sums; derive paid and EMIs-left on demand with ceiling rounding. Interest rule is a Strategy; the driver is a tiny command loop.

---

## 21.7 Design an Inventory Management System

### Problem & clarifying questions

A very common machine-coding and OOD question (asked in e-commerce — online shopping — and quick-commerce — groceries delivered in minutes — interviews): track **products** (SKUs — stock keeping units, one id per sellable item) stored in several **warehouses**; record every **stock movement** (goods received, shipped, transferred between warehouses, adjusted after a physical count); fulfil orders by choosing warehouses; and raise **reorder alerts** when stock runs low.

| Question | Answer used |
|---|---|
| Is stock tracked per warehouse? | Yes: (SKU, warehouse) → quantity. |
| Can an order be split across warehouses? | Yes; the choice of warehouses is a pluggable strategy (nearest city first, or most stock first). |
| Partial shipments if stock is short? | No — all-or-nothing. |
| When is a reorder alert raised? | When *total* stock of a SKU drops to or below its reorder point. Once per dip (no alert spam); re-armed when stock recovers. |
| How much to reorder? | Up to the product's max level (`max − current`). |
| Negative stock? | Never. |

## Requirements

### Functional

1. Add products (with reorder point and max level) and warehouses.
2. Receive stock (inbound) into a warehouse.
3. Ship an order quantity, allocating across warehouses by a strategy.
4. Transfer stock between warehouses.
5. Adjust stock after a physical count (cycle count).
6. Query stock per warehouse and total; list the movement history of a SKU.
7. Reorder alerts to subscribers.

### Non-functional

- **No overselling** (selling more units than are in stock) under concurrent orders.
- Operations on different products run in parallel.
- Full audit trail (a complete record of every change; here every change is a movement).

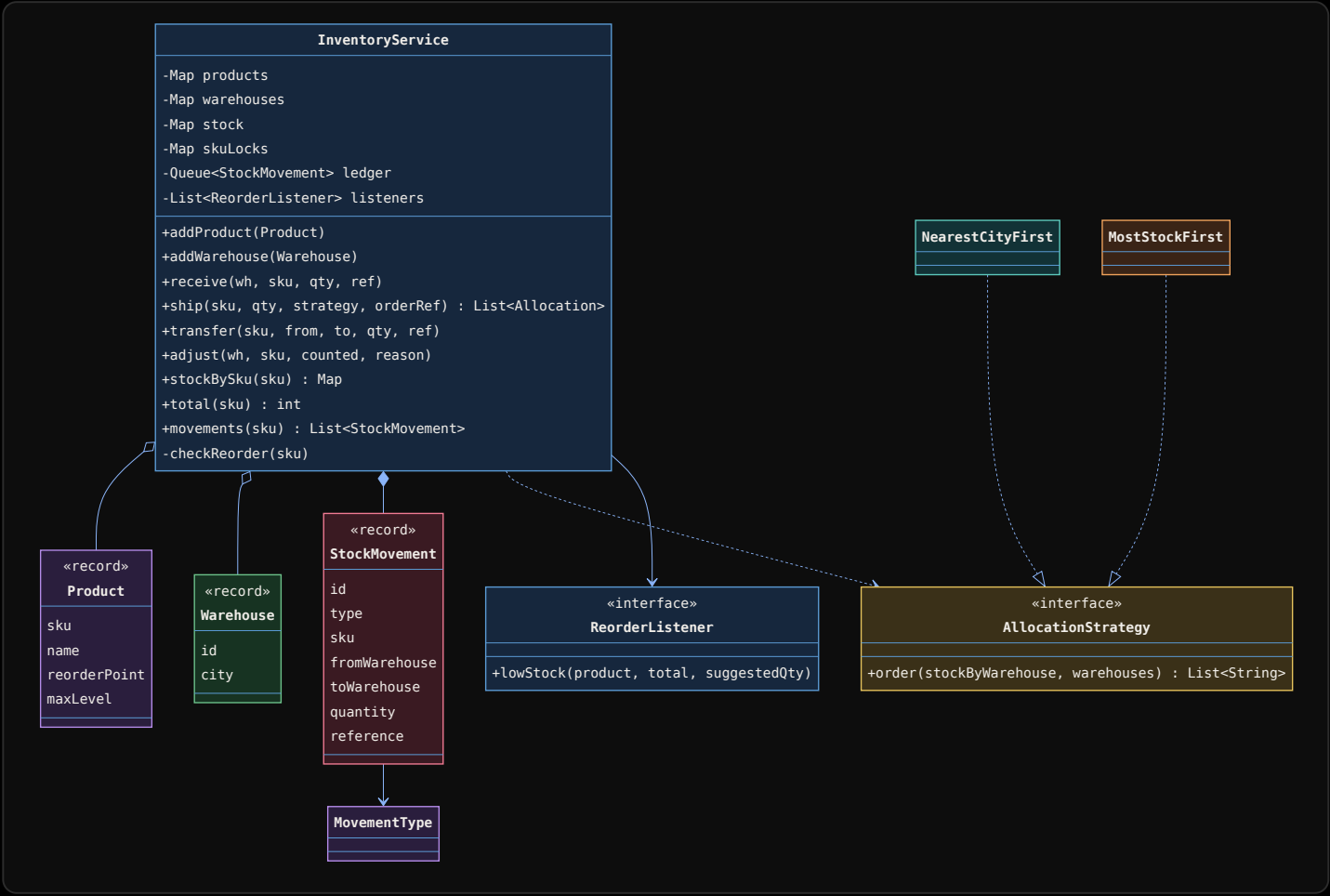
## Use cases

| Actor | Use cases |
|---|---|
| **Warehouse staff** | receive stock, transfer, cycle-count adjust |
| **Order service** (system) | ship / allocate order |
| **Purchasing team** (subscriber) | receives reorder alerts |
| **Manager** | views stock and movement history |

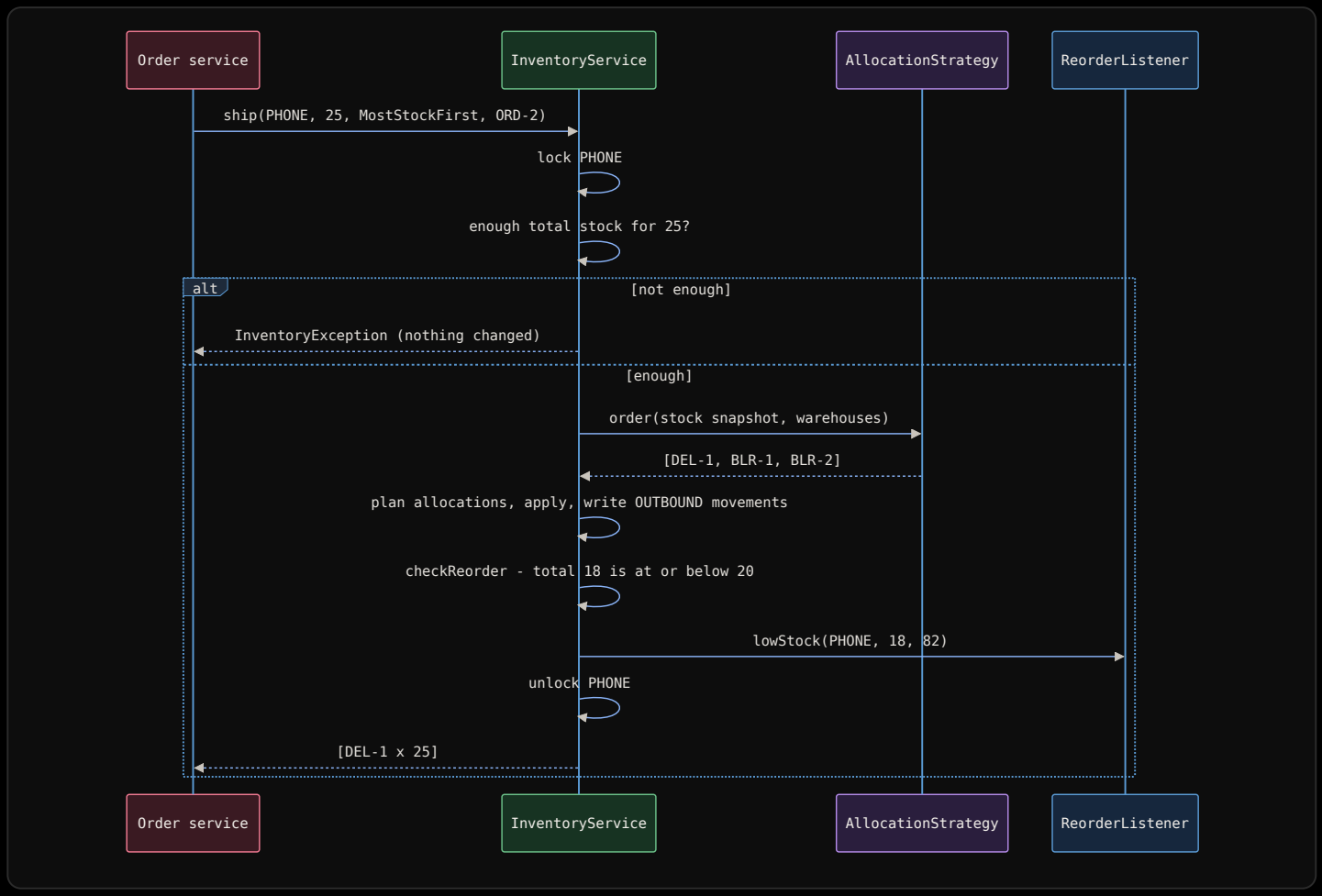
## Core entities

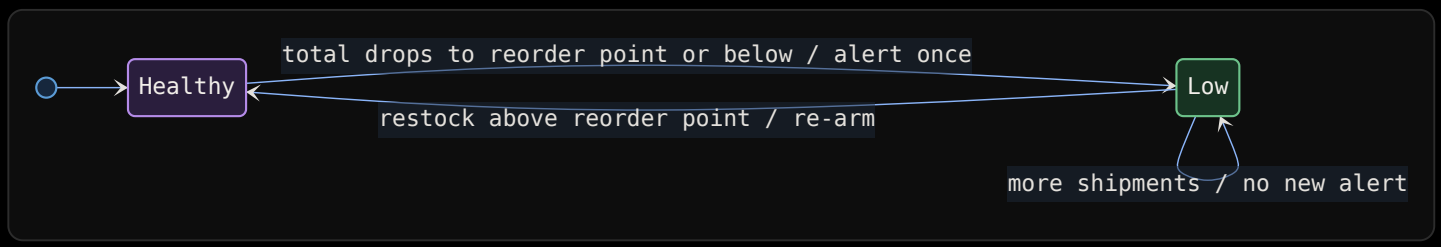
| Class | Kind | Responsibility |
|---|---|---|
| `Product` | record | SKU, name, reorder point, max level (validated). |
| `Warehouse` | record | id, city. |
| `StockMovement` | record | Append-only ledger line (type, sku, from, to, qty, reference). |
| `MovementType` | enum | INBOUND, OUTBOUND, TRANSFER, ADJUSTMENT. |
| `Allocation` | record | "take q from warehouse w" — part of a shipment plan. |
| `AllocationStrategy` + `NearestCityFirst`, `MostStockFirst` | interface + strategies | Order in which warehouses are drained. |
| `ReorderListener` | interface | Observer for low-stock alerts. |
| `InventoryService` | class | Stock levels, per-SKU locking, movements, alerts. |
| `InventoryException` | exception | Readable failure. |

## Class diagram



## Key flows


Order service
InventoryService
AllocationStrategy
ReorderListener
ship(PHONE, 25, MostStockFirst, ORD-2)
lock PHONE
enough total stock for 25?
alt
[not enough]
InventoryException (nothing changed)
[enough]
order(stock snapshot, warehouses)
[DEL-1, BLR-1, BLR-2]
plan allocations, apply, write OUTBOUND movements
checkReorder - total 18 is at or below 20
lowStock(PHONE, 18, 82)
unlock PHONE
[DEL-1 x 25]
Order service
InventoryService
AllocationStrategy
ReorderListener


Healthy
total drops to reorder point or below / alert once
restock above reorder point / re-arm
Low
more shipments / no new alert

### Design decisions & patterns used

| Decision | Pattern | Why | Alternative rejected |
|---|---|---|---|
| **One lock per SKU** | Lock striping by key (a separate lock for each group of keys) | An order for one SKU touches several warehouses; a per-SKU lock covers all of them with no lock-ordering puzzle, while different SKUs never contend. | One lock per (SKU, warehouse) — needs lock ordering for multi-warehouse orders; one global lock — no parallelism. |
| **Plan, then apply** in `ship` | — | Check total first and build the full plan before changing anything → all-or-nothing. | Deduct warehouse by warehouse and roll back on shortage. |
| **`AllocationStrategy`** | Strategy | Business wants different policies (speed vs fewer split parcels); each is a small class. | `if (mode == ...)` in `ship`. |
| **Append-only `StockMovement` ledger** | Event log / audit | Every change is explainable; levels can be rebuilt by replaying; adjustments record the delta (the change, e.g. −3), not just the new number. | Updating quantities with no history. |
| **`ReorderListener` with once-per-dip** | Observer + edge-triggering (fire once when a value crosses a line) | Purchasing is notified once when stock crosses the threshold, not on every sale. | Level-triggered alerts (fire every time while the value is below the line) on every movement — spam. |
| **Strategy gets a snapshot copy** | Defensive copy (hand out a copy so the receiver cannot change your original) | A strategy cannot accidentally modify live stock. | Passing the live map. |

### Java implementation

Full runnable code: `code/lld/ch21/inventory/` (12 files). Below are the files that carry the design; the rest are `Product`, `Warehouse`, `StockMovement`, `MovementType`, `Allocation`, `MostStockFirst`, `ReorderListener`, `InventoryException` and `Main`.

```java
// File: lld/ch21/inventory/AllocationStrategy.java
package lld.ch21.inventory;

import java.util.List;
import java.util.Map;

/** Strategy: in which order should warehouses be drained to fulfil an order? */
public interface AllocationStrategy {
    List<String> order(Map<String, Integer> stockByWarehouse, Map<String, Warehouse> warehouses);
}
```

```java
// File: lld/ch21/inventory/NearestCityFirst.java
package lld.ch21.inventory;

import java.util.Comparator;
import java.util.List;
import java.util.Map;

/** Ship from warehouses in the customer's city first (faster delivery), then the rest by stock. */
public class NearestCityFirst implements AllocationStrategy {
    private final String city;
    public NearestCityFirst(String city) { this.city = city; }

    @Override public List<String> order(Map<String, Integer> stock, Map<String, Warehouse> warehouses) {
        return stock.keySet().stream()
                .sorted(Comparator.comparing((String w) -> !warehouses.get(w).city().equalsIgnoreCase(city))
                        .thenComparing(w -> -stock.get(w)).thenComparing(w -> w))
                .toList();
    }
}
```

```java
// File: lld/ch21/inventory/InventoryService.java  (excerpt)
/**
 * Stock is kept per (sku, warehouse). Locking is per SKU: every operation on one product
 * (even across warehouses) is serialized, while different products proceed in parallel.
 */
public class InventoryService {
    private final Map<String, Product> products = new ConcurrentHashMap<>();
    private final Map<String, Warehouse> warehouses = new ConcurrentHashMap<>();
    private final Map<String, Map<String, Integer>> stock = new ConcurrentHashMap<>();   // sku -> (wh -> qty)
    private final Map<String, ReentrantLock> skuLocks = new ConcurrentHashMap<>();
    private final Set<String> alerted = ConcurrentHashMap.newKeySet();                    // SKUs already flagged low
    private final Queue<StockMovement> ledger = new ConcurrentLinkedQueue<>();              // append-heavy, lock-free
    private final List<ReorderListener> listeners = new CopyOnWriteArrayList<>();
    private final AtomicInteger movementSeq = new AtomicInteger();


    // ... addListener, addProduct (creates the SKU lock), addWarehouse

    public void receive(String warehouseId, String sku, int qty, String ref) {
        positive(qty);
        warehouse(warehouseId);
        withSku(sku, () -> {
            change(sku, warehouseId, qty);
            record(MovementType.INBOUND, sku, null, warehouseId, qty, ref);
        });
    }

    /** All-or-nothing: either the whole quantity is allocated across warehouses, or nothing moves. */
    public List<Allocation> ship(String sku, int qty, AllocationStrategy strategy, String orderRef) {
        positive(qty);
        return withSku(sku, () -> {
            Map<String, Integer> levels = stock.get(sku);
            int total = levels.values().stream().mapToInt(Integer::intValue).sum();
            if (total < qty) throw new InventoryException("Only " + total + " of " + sku + " available, order needs "
+ qty);
            List<Allocation> plan = new ArrayList<>();
            int remaining = qty;
            for (String wh : strategy.order(Map.copyOf(levels), warehouses)) {
                int take = Math.min(remaining, levels.get(wh));
                if (take > 0) { plan.add(new Allocation(wh, take)); remaining -= take; }
                if (remaining == 0) break;
            }
            for (Allocation a : plan) {
                change(sku, a.warehouseId(), -a.quantity());
                record(MovementType.OUTBOUND, sku, a.warehouseId(), null, -a.quantity(), orderRef);
            }
            return plan;
        });
    }
    // ... transfer and adjust follow the same shape: validate, withSku { check, change, record }

    // ------------------------------------------------------------- internals (called with the SKU lock held)

    private void change(String sku, String wh, int delta) {
        stock.get(sku).merge(wh, delta, Integer::sum);
    }

    private void record(MovementType type, String sku, String from, String to, int qty, String ref) {
        ledger.add(new StockMovement(movementSeq.incrementAndGet(), type, sku, from, to, qty, ref));
        checkReorder(sku);
    }

    /** Fires once when stock crosses down to the reorder point; re-arms when stock recovers. */
    private void checkReorder(String sku) {
        Product p = products.get(sku);
        int total = totalUnlocked(sku);
        if (total <= p.reorderPoint()) {
            if (alerted.add(sku)) listeners.forEach(l -> l.lowStock(p, total, p.maxLevel() - total));
        } else {
            alerted.remove(sku);
        }
    }
```

```
    private <T> T withSku(String sku, java.util.function.Supplier<T> action) {
        ReentrantLock lock = skuLocks.get(sku);
        if (lock == null) throw new InventoryException("Unknown SKU " + sku);
        lock.lock();
        try { return action.get(); } finally { lock.unlock(); }
    }
    // ... withSku(Runnable), lookups and validation
}
```

**Output** (of `Main.java`):

```
== receive stock
  PHONE: {BLR-1=30, BLR-2=5, DEL-1=40} total=75
== ship 32 phones to a Bengaluru customer (nearest city first)
  [Allocation[warehouseId=BLR-1, quantity=30], Allocation[warehouseId=BLR-2, quantity=2]]
  PHONE: {BLR-1=0, BLR-2=3, DEL-1=40}
== transfer 15 phones DEL-1 -> BLR-1
  PHONE: {BLR-1=15, BLR-2=3, DEL-1=25}
== ship 25 more (most stock first) -> crosses reorder point
  [ALERT] PHONE low: 18 left, reorder 82 to reach 100
  [Allocation[warehouseId=DEL-1, quantity=25]]
  ship 1 more (no duplicate alert): [Allocation[warehouseId=BLR-1, quantity=1]]
== cycle count finds 2 damaged units in BLR-1
  PHONE: {BLR-1=12, BLR-2=3, DEL-1=0} total=15
== restock re-arms the alert
  [ALERT] PHONE low: 15 left, reorder 85 to reach 100
== edge cases
  error: Only 15 of PHONE available, order needs 999
  error: Source and target warehouse are the same
  error: Unknown warehouse MUM-1
  error: Unknown SKU TABLET
  error: Quantity must be positive: 0
== movement ledger for PHONE
  #1 INBOUND    PHONE qty=+30 -->BLR-1 (PO-1)
  #2 INBOUND    PHONE qty=+40 -->DEL-1 (PO-2)
  #3 INBOUND    PHONE qty=+5 -->BLR-2 (PO-3)
  #4 OUTBOUND   PHONE qty=-30 BLR-1->- (ORD-1)
  #5 OUTBOUND   PHONE qty=-2 BLR-2->- (ORD-1)
  #6 TRANSFER   PHONE qty=+15 DEL-1->BLR-1 (TR-1)
  #7 OUTBOUND   PHONE qty=-25 DEL-1->- (ORD-2)
  #8 OUTBOUND   PHONE qty=-1 BLR-1->- (ORD-3)
  #9 ADJUSTMENT PHONE qty=-2 -->BLR-1 (damaged)
  #10 INBOUND    PHONE qty=+90 -->BLR-1 (PO-4)
  #11 OUTBOUND   PHONE qty=-90 BLR-1->- (ORD-4)
== concurrency: 200 one-unit orders race for 50 cases
  [ALERT] CASE low: 10 left, reorder 190 to reach 200
  shipped=50 rejected=150 left=0
```

Reading the output: the Bengaluru order drained BLR-1 then BLR-2 before touching Delhi; the reorder alert fired once when total PHONE stock fell to 18 (≤ 20) and not again at 17; after a restock pushed the total above 20 the alert re-armed and fired again when stock fell back; the ledger explains every unit; and with 200 concurrent one-unit orders for 50 cases, exactly 50 shipped — no overselling.

### Concurrency & edge cases

- **Overselling:** check-total → plan → apply runs under the SKU's `ReentrantLock`; concurrent orders for the same SKU are serialised.
- **Parallelism:** different SKUs use different locks.
- **Ledger:** a `ConcurrentLinkedQueue` (a thread-safe queue that needs no lock) — lock-free appends from many SKU threads at once, insertion order kept, and iteration never throws `ConcurrentModificationException` (the error Java raises when a collection changes while it is being looped over). (A `CopyOnWriteArrayList` would copy the whole ledger on every movement — right for rarely-changing lists, wrong for an append-heavy log.)
- **Listeners:** `CopyOnWriteArrayList` — read on every alert, changed almost never, which is exactly its use case. Listener calls happen while holding the SKU lock — fine for quick callbacks; for slow ones (email), hand off to an executor (a pool of threads that runs submitted tasks) so the lock is not held during I/O (input/output, such as network or disk calls).

- **Edge cases handled:** unknown SKU/warehouse, non-positive quantities, negative counted quantity, transfer to the same warehouse, transfer or order larger than stock, invalid product thresholds, duplicate SKU/warehouse, "no change" adjustments (not logged).

### Extensions & follow-up questions

- **Reservations (cart holds)?** Track `onHand` and `reserved` per (SKU, warehouse); available = onHand − reserved; reservations expire after a timeout; shipping converts a reservation into an OUTBOUND.
- **Batches / expiry dates (groceries, medicines)?** Stock per batch with expiry; allocate FEFO (first-expiry-first-out) — just another `AllocationStrategy`.
- **Distances instead of cities?** `NearestFirst` sorting by a `DistanceService` result.
- **Automatic purchase orders?** A `ReorderListener` that creates a PO (purchase order: a formal request to a supplier) for `max − current` and marks it pending so a second alert does not create a duplicate.
- **Multiple services / database?** Stock rows with optimistic versioning ( `UPDATE ... SET qty = qty - ? WHERE qty >= ? AND version = ?` ), and the movement ledger as an event stream (e.g. Kafka, a system that stores and delivers an ordered log of events) consumed by analytics.

### Real-world example

Amazon's fulfilment network and Flipkart's warehouses decide which fulfilment centre ships each order based on factors like proximity and stock, and may split an order across centres — the job of `AllocationStrategy`. Shopify lets merchants hold inventory at multiple locations and set a fulfilment priority order among them. Quick-commerce apps (Blinkit, Zepto, Swiggy Instamart) run many small "dark stores" (warehouses closed to the public that only serve online orders), where accurate per-store stock and fast reorder alerts are essential.

### In the interview

Start with the data shape: `stock[sku][warehouse] = qty` plus an append-only movement list. Code receive and ship (with the plan-then-apply and the per-SKU lock) first; they carry most of the marks. Then transfer and adjust (both tiny), then the reorder observer with once-per-dip, then allocation strategies. Run the 200-thread demo to prove "no overselling".

### Self-check

- **Q:** Why does `ship` check the total before building the plan? → **A:** To reject short orders before any warehouse is touched (all-or-nothing).
- **Q:** Why is a per-SKU lock enough for a multi-warehouse order? → **A:** An order involves exactly one SKU, and all of that SKU's warehouses are covered by its one lock.
- **Q:** How do you add FEFO? → **A:** Add batch-level stock and a new `AllocationStrategy`; `ship` does not change.
- **Q:** Why record adjustments as movements? → **A:** So shrinkage/damage is auditable and levels can be rebuilt from the ledger.

### TL;DR

Stock per (SKU, warehouse), every change is a ledger movement, a lock per SKU prevents overselling, allocation is a Strategy, and low-stock alerts are an edge-triggered Observer.

---

## 21.8 Design a Unix File Search ( `find` )

### Problem & clarifying questions

The Unix (the operating-system family that includes Linux and macOS) `find` command walks a directory tree (folders inside folders) and prints every entry that passes a set of tests, for example `find /var/log -type f -size +10M -mtime +7` ("regular files under /var/log bigger than 10 MiB, last changed more than 7 days ago"; 1 MiB = 1,048,576 bytes). The machine-coding version asks you to build a small `find`: a file-system model, a set of **filters**, a way to **combine** them with AND / OR / NOT, and a clean API for the search options. It is a favourite because the obvious solution (one method with ten optional parameters and a pile of `if`s) is exactly what the interviewer wants you to avoid.

Three words used below: a **predicate** (or *test*) is a yes/no question about one entry ("is its name `*.log`?"); a **glob** is a filename pattern with wildcards (`*` = any characters, `?` = one character, `[0-9]` = one character from a set); **depth** is how many levels below the start directory an entry sits (the start itself is depth 0).

| Question | Answer used |
|---|---|
| Real disk or in-memory? | In-memory tree, so runs are identical; a `java.nio` (Java's file and I/O library) adapter (a class that wraps one API to fit another interface) is an extension. |
| Which tests? | Name glob (case-sensitive and insensitive), type (file / directory), size, modification age, owner, empty. |
| How are tests combined? | AND, OR, NOT and parentheses, with `find`'s precedence (which operator is applied first when there are no parentheses): NOT binds tightest, then AND, then OR. Two tests side by side mean AND. |
| Size semantics? | Like GNU `find` (the standard version on Linux, from the GNU project): the size is **rounded up** to the unit before comparing, so `-size -1M` matches only empty files. Size tests apply to files only. |
| Other options? | `mindepth`, `maxdepth`, a result limit, and **pruning** (skip whole directories such as `node_modules`, a JavaScript project's often huge folder of downloaded libraries). |
| API or command line? | Both: a fluent (written as one chain of method calls, `a().b().c()`) **Builder** API for code, and a parser for `find`-style command lines. |
| Output order? | Pre-order depth-first (go as deep as possible down one branch before the next; a directory is listed before its contents), children in name order. |
| Huge trees? | Stream results as they are found; no recursion (a method calling itself; deep trees must not overflow the stack — the limited memory that holds pending method calls); a parallel variant as an extension. |

## Requirements

### Functional

1. Model files and directories (name, owner, modification time, size).
2. Search from a start path with any combination of tests joined by AND / OR / NOT.
3. Options: min/max depth, result limit, pruned directory names.
4. Parse `find` command lines into the same search criteria.
5. Clear errors for bad paths, unknown tests, malformed sizes and unbalanced parentheses.

### Non-functional

- **Open/Closed**: a new test = one new class, no change to the search loop or the parser's structure.
- Memory: stream results; handle very deep trees.
- Thread safety: searching while files are being added must not throw.

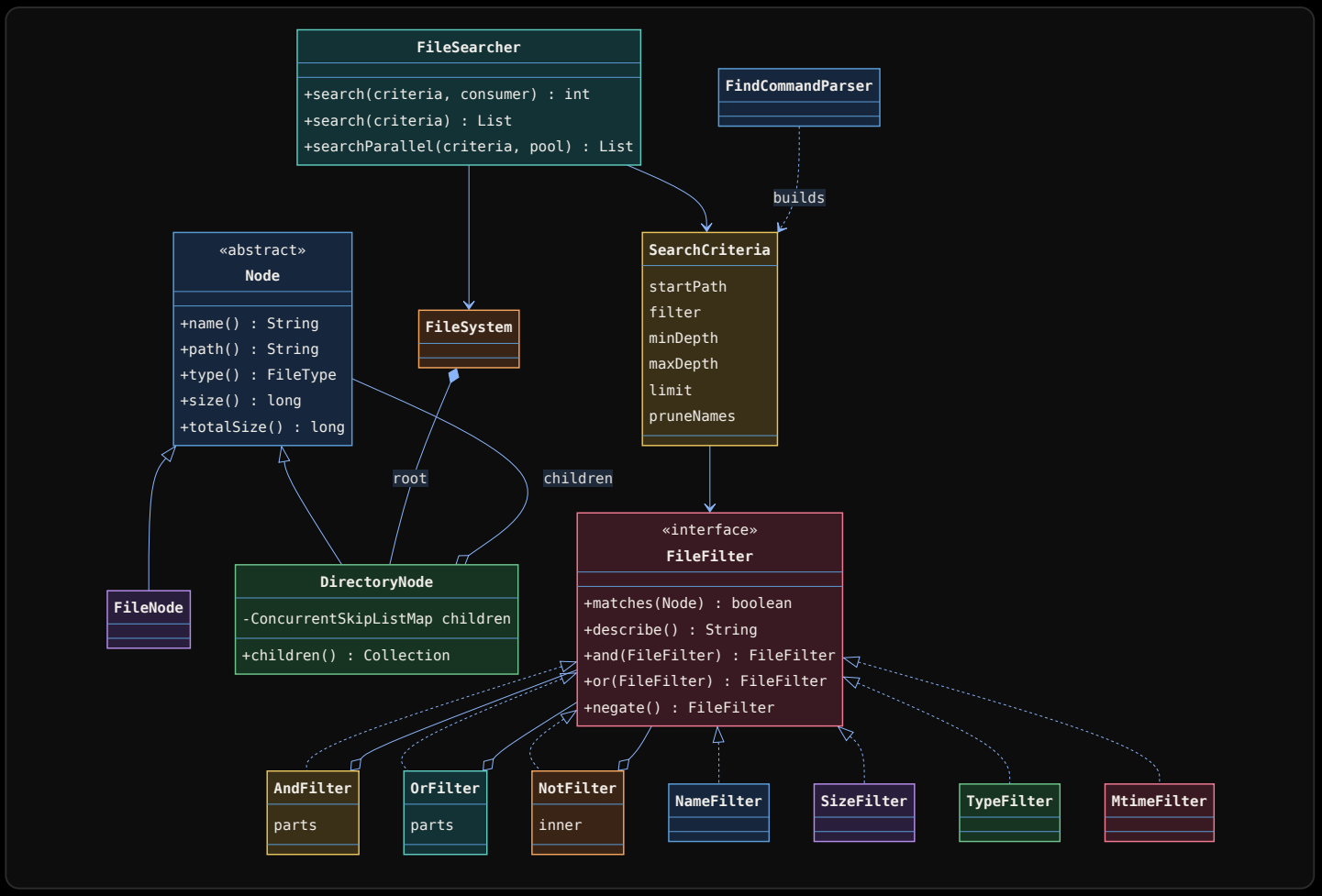
## Use cases

| Actor | Use cases |
|---|---|
| **Developer (API user)** | build criteria with the Builder, run a search, stream results |
| **Shell user** (someone typing commands into a terminal) | type a `find` command line, see matching paths or a clear error |
| **Ops script** (an operations team's automation script) | find big old logs (`-size +10M -mtime +7`) to archive, find empty directories to clean |

### Core entities

| Class | Kind | Responsibility |
|---|---|---|
| `Node` | abstract sealed class (cannot be created directly, and only the listed subclasses may extend it) | **Composite** component: name, parent, owner, modified time, `path()`, `size()`, `totalSize()`. |
| `FileNode` / `DirectoryNode` | classes | Leaf and container; a directory keeps its children sorted in a `ConcurrentSkipListMap` (a sorted map that many threads can use at once). |
| `FileSystem` | class | Root, `mkdirs`, `createFile`, `resolve(path)`. |
| `FileFilter` | interface | **Specification**: `matches(node)`, `describe()`, and default `and` / `or` / `negate`. |
| `AndFilter`, `OrFilter`, `NotFilter` | records | **Composite** filters: a tree of filters that is itself a filter. |
| `Filters` | factory | Leaf tests: name/iname glob, type, size, mtime, owner, empty, match-all. |
| `SearchCriteria` + `Builder` | class | Immutable search description (start, filter, depths, limit, prune set), built fluently and validated once. |
| `FileSearcher` | class | Iterative DFS (depth-first search) with an explicit stack (a last-in, first-out list); streams results to a `Consumer`; optional fork/join version (Java's framework that splits a task into subtasks run in parallel, then joins the results). |
| `FindCommandParser` | class | Recursive-descent parser (one method per grammar rule, the methods calling each other) for `find` expressions → `SearchCriteria`. |
| `SearchException` | exception | Readable failure. |

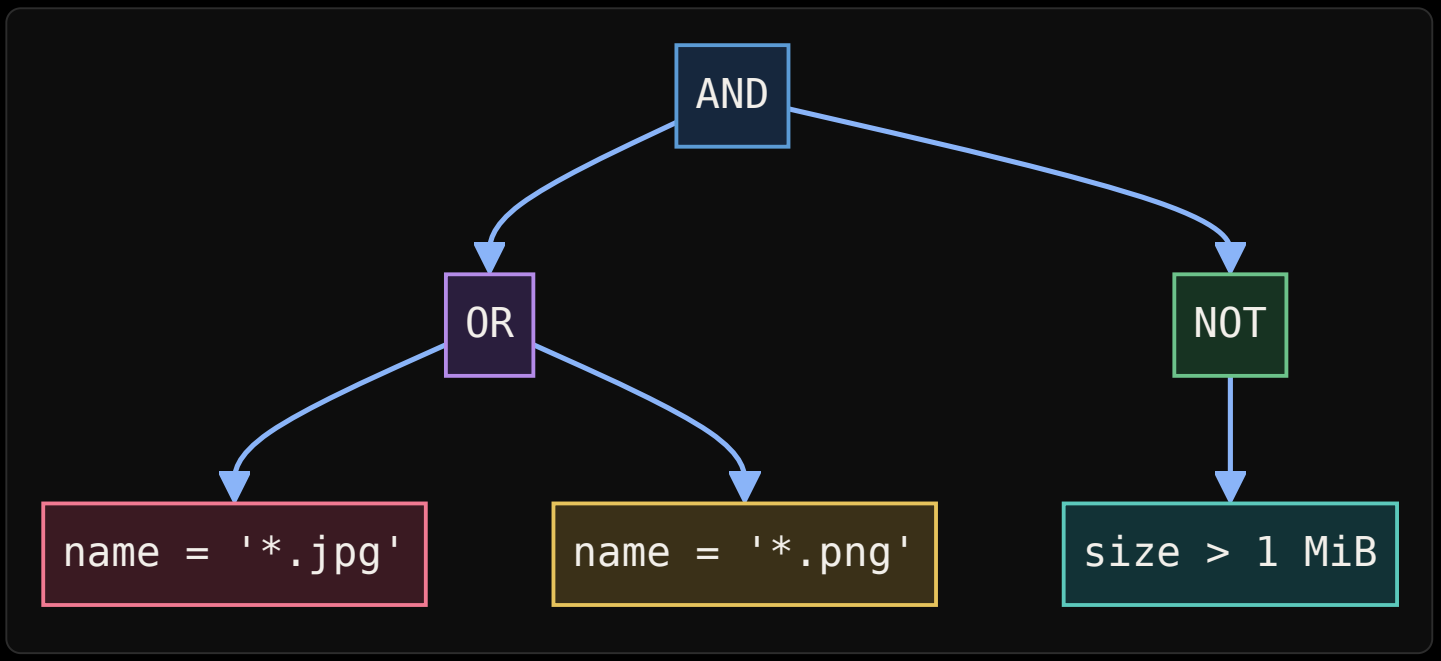
### Class diagram



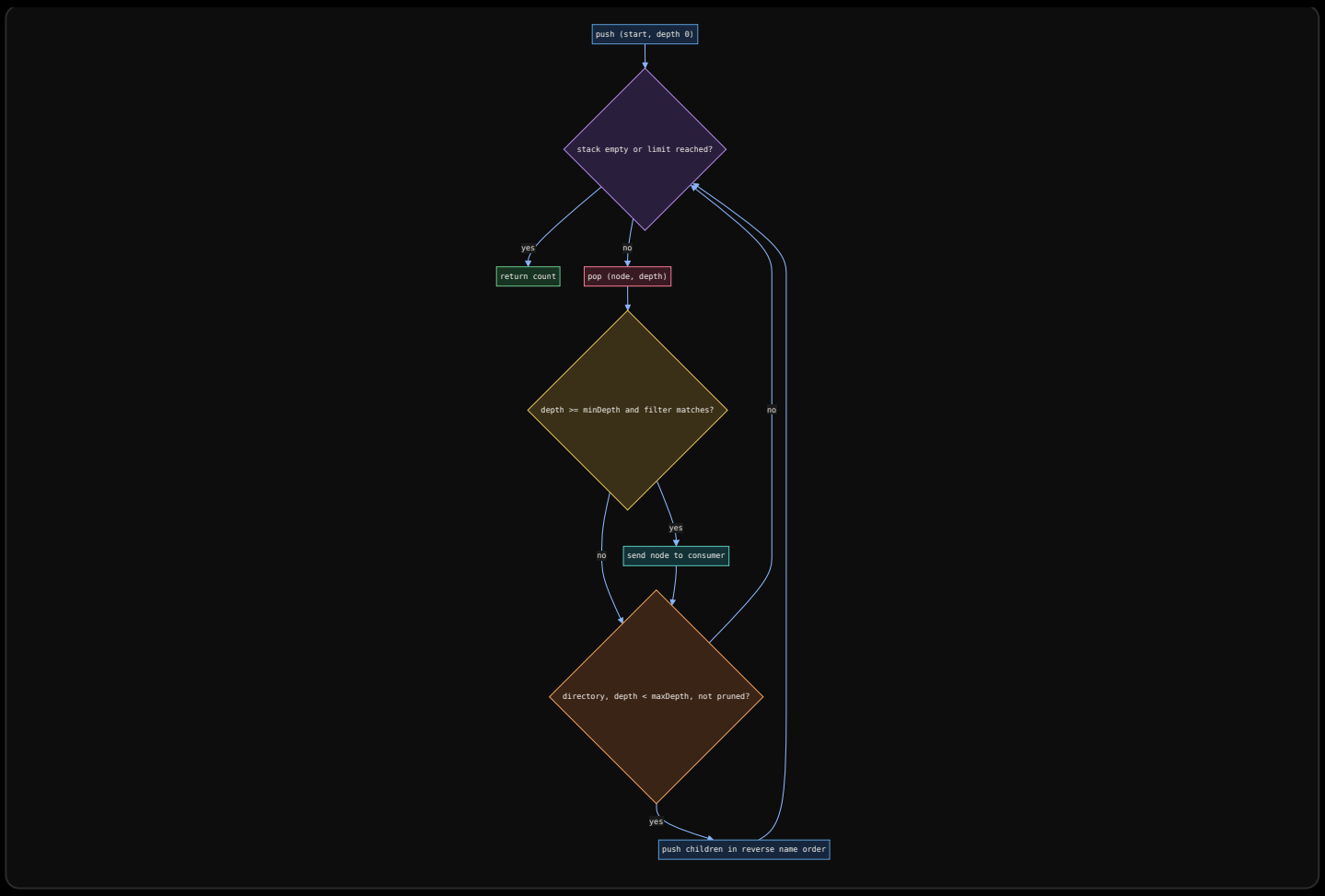

Two Composites sit side by side: the **file tree** (a directory is a node that contains nodes) and the **filter tree** (an AND is a filter that contains filters). The searcher walks the first and asks the second one question per entry.

### Key flows

**Parsing** `find /home ( -name '*.jpg' -o -name '*.png' ) ! -size +1M` produces this filter tree:




**The search loop** (explicit stack, pre-order):




**Operator precedence** (from lowest to highest): `-o` (OR) < `-a` or nothing (AND) < `!` (NOT) < `( … )`. So `-name a -o -name b -type f` means `a OR (b AND file)` — a classic surprise, and the reason `find` users write parentheses.

### Design decisions & patterns used

| Decision | Pattern | Why | Alternative rejected |
|---|---|---|---|
| Each test is a small class with `matches(node)` | **Specification** | A new test is one record; the search loop never changes (Open/Closed) | `search(name, ext, minSize, maxSize, type, …)` with a growing list of `if`s |
| `AndFilter` / `OrFilter` / `NotFilter` hold other filters | **Composite** | Arbitrary nesting (`(a OR b) AND NOT c`) with one interface; the parser's output *is* this tree; `describe()` prints it | Lambdas (short anonymous functions) only (`a.and(b)`) — work, but cannot be printed or inspected (compare §21.1's `TaskFilter`) |
| `Node` / `FileNode` / `DirectoryNode` | **Composite** | The searcher and `totalSize()` treat files and directories uniformly | `instanceof` ("is this object of type X?") checks everywhere, or separate file and directory lists |
| `SearchCriteria.Builder` | **Builder** | Many optional settings; named calls; validation (`mindepth ≤ maxdepth`) once in `build()`; immutable result | Telescoping constructors or a mutable settings bag |
| Recursive-descent `FindCommandParser` | Interpreter-style parser | One method per precedence level makes precedence explicit and testable | Splitting on `-o` with string hacks |
| Iterative DFS with an explicit `ArrayDeque` | — | No `StackOverflowError` (the crash when the call stack runs out) on very deep trees; easy to add a limit | Recursion |
| Results go to a `Consumer<Node>` | Streaming / callback | Prints as it finds, like `find`; memory does not grow with the result count | Always building a full `List` |
| AND / OR short-circuit (stop as soon as the answer is known) | — | Put cheap tests first (`-name` before a size lookup on a real disk) | Evaluating every test |
| `ConcurrentSkipListMap` for children | Concurrent collection | Sorted (deterministic output) and safe to iterate while files are added | `TreeMap` (throws `ConcurrentModificationException`) |

### Java implementation

Full runnable code: `code/lld/ch21/filesearch/` (15 files). Below are the files that carry the design; the rest are the file-system model (`Node`, `FileNode`, `DirectoryNode`, `FileSystem`, `FileType`), `OrFilter`, `NotFilter`, `SearchCriteria` and its Builder, `SearchException` and `Main`.

```java
// File: lld/ch21/filesearch/FileFilter.java
package lld.ch21.filesearch;

/**
 * Specification pattern: one question, "does this entry match?". Leaf filters test one attribute;
 * AndFilter / OrFilter / NotFilter are Composite nodes that combine other filters into a tree.
 */
public interface FileFilter {
    boolean matches(Node n);

    /** Human-readable form of the expression tree, e.g. (name='*.log' AND size>+10M). */
    String describe();

    default FileFilter and(FileFilter other) { return AndFilter.of(this, other); }
    default FileFilter or(FileFilter other) { return OrFilter.of(this, other); }
    default FileFilter negate() { return new NotFilter(this); }
}
```

```java
// File: lld/ch21/filesearch/AndFilter.java
package lld.ch21.filesearch;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

/** Composite: matches when every child matches. Short-circuits like find's -a. */
public record AndFilter(List<FileFilter> parts) implements FileFilter {
    public AndFilter { parts = List.copyOf(parts); }

    /** Flattens nested ANDs so (a AND (b AND c)) is stored as (a AND b AND c). */
    static FileFilter of(FileFilter a, FileFilter b) {
        List<FileFilter> all = new ArrayList<>();
        for (FileFilter f : List.of(a, b)) if (f instanceof AndFilter and) all.addAll(and.parts()); else all.add(f);
        return new AndFilter(all);
    }

    @Override public boolean matches(Node n) {
        for (FileFilter f : parts) if (!f.matches(n)) return false;
        return true;
    }

    @Override public String describe() {
        return parts.stream().map(FileFilter::describe).collect(Collectors.joining(" AND ", "(", ")"));
    }
}
```

```java
// File: lld/ch21/filesearch/Filters.java  (excerpt)
/** Static factories for the leaf filters (one per find test). New test = one new record here; nothing else changes. */
public final class Filters {
    private Filters() { }

    public static FileFilter all() { return new MatchAll(); }
    public static FileFilter name(String glob) { return new NameFilter(glob, false); }
    public static FileFilter iname(String glob) { return new NameFilter(glob, true); }
    public static FileFilter type(FileType t) { return new TypeFilter(t); }
    public static FileFilter owner(String user) { return new OwnerFilter(user); }
    public static FileFilter empty() { return new EmptyFilter(); }
    /** spec like find's -size: "+10M", "-1k", "0c", "20" (no suffix = 512-byte blocks). */
    public static FileFilter size(String spec) { return SizeFilter.parse(spec); }
    /** spec like find's -mtime: "+7" (more than 7 whole days ago), "-1", "0". */
    public static FileFilter mtime(String spec, Instant now) { return MtimeFilter.parse(spec, now); }

    // ... MatchAll, NameFilter (glob -> regex), TypeFilter, OwnerFilter, EmptyFilter, MtimeFilter

    /**
     * GNU find semantics: the file size is rounded UP to whole units, then compared.
     * So "-size -1M" means "fewer than 1 whole MiB after rounding up" = only empty files. A famous gotcha.
     * Size tests apply to files only in this model.
     */
    record SizeFilter(char cmp, long n, char unit, String spec) implements FileFilter {
        static SizeFilter parse(String spec) {
            var m = Pattern.compile("([+-]?)(\\d+)([cbkMG]?)").matcher(spec);
            if (!m.matches()) throw new SearchException("Invalid -size '" + spec + "' (use e.g. +10M, -1k, 100c)");
            char cmp = m.group(1).isEmpty() ? '=' : m.group(1).charAt(0);
            char unit = m.group(3).isEmpty() ? 'b' : m.group(3).charAt(0);
            return new SizeFilter(cmp, Long.parseLong(m.group(2)), unit, spec);
        }
        long unitBytes() {
            return switch (unit) { case 'c' -> 1; case 'b' -> 512; case 'k' -> 1024; case 'M' -> 1024L * 1024; default
-> 1024L * 1024 * 1024; };
        }
        @Override public boolean matches(Node node) {
            if (node.type() != FileType.FILE) return false;
            long units = (node.size() + unitBytes() - 1) / unitBytes();      // round up
            return switch (cmp) { case '+' -> units > n; case '-' -> units < n; default -> units == n; };
        }
        @Override public String describe() { return "size" + (cmp == '=' ? "=" : "") + spec; }
    }
}
```

```java
// File: lld/ch21/filesearch/FileSearcher.java  (excerpt)
/** Walks the tree and reports every entry that passes the filter. */
public final class FileSearcher {
    private final FileSystem fs;

    public FileSearcher(FileSystem fs) { this.fs = fs; }

    private record Frame(Node node, int depth) { }

    /**
     * Pre-order depth-first walk with an explicit stack (no recursion, so a very deep tree cannot
     * overflow the call stack). Results are streamed to the consumer as they are found, like find
     * printing line by line; nothing forces the whole result list into memory.
     */
    public int search(SearchCriteria c, Consumer<Node> out) {
        Deque<Frame> stack = new ArrayDeque<>();
        stack.push(new Frame(fs.resolve(c.startPath()), 0));
        int found = 0;
        while (!stack.isEmpty() && found < c.limit()) {
            Frame f = stack.pop();
            Node n = f.node();
            if (f.depth() >= c.minDepth() && c.filter().matches(n)) { out.accept(n); found++; }
            if (n instanceof DirectoryNode d && f.depth() < c.maxDepth() && !c.pruneNames().contains(d.name())) {
                List<Node> kids = new ArrayList<>(d.children());
                Collections.reverse(kids);                    // push in reverse so we pop in name order
                for (Node k : kids) stack.push(new Frame(k, f.depth() + 1));
            }
        }
        return found;
    }

    // ... search(c) collects into a list; searchParallel forks one RecursiveTask per sub-directory
}
```

```java
// File: lld/ch21/filesearch/FindCommandParser.java  (excerpt)
/**
 * Turns a find-style command line into SearchCriteria, with find's precedence:
 *   expr   := or
 *   or     := and ( ("-o" | "-or") and )*
 *   and    := unary ( ["-a" | "-and"] unary )*      -- two tests side by side mean AND
 *   unary  := ("!" | "-not") unary | "(" expr ")" | test
 *   test   := -name G | -iname G | -type f|d | -size N | -mtime N | -user U | -empty | -true
 * Global options -maxdepth N / -mindepth N may appear anywhere. This is recursive descent:
 * one method per grammar rule, each returning a FileFilter (the Composite tree).
 */
public final class FindCommandParser {
    private final Instant now;
    private List<String> tokens;
    private int pos;
    private SearchCriteria.Builder builder;

    // ... parse(args): read the start path, then builder.where(parseOr()); parse(String) splits on spaces

    private FileFilter parseOr() {
        FileFilter left = parseAnd();
        while (peekIs("-o", "-or")) { pos++; left = left.or(parseAnd()); }
        return left;
    }

    private FileFilter parseAnd() {
        FileFilter left = parseUnary();
        while (true) {
            skipGlobalOptions();
            if (pos >= tokens.size() || peekIs("-o", "-or", ")")) return left;
            if (peekIs("-a", "-and")) pos++;
            left = left.and(parseUnary());
        }
    }

    private FileFilter parseUnary() {
        skipGlobalOptions();
        if (pos >= tokens.size()) throw new SearchException("Expression ends too early");
        String t = tokens.get(pos++);
        switch (t) {
            case "!", "-not" -> { return parseUnary().negate(); }
            case "(" -> {
                FileFilter inner = parseOr();
                if (!peekIs(")")) throw new SearchException("Missing ')'");
                pos++;
                return inner;
            }
            case ")" -> throw new SearchException("Unexpected ')'");
            case "-empty" -> { return Filters.empty(); }
            case "-true" -> { return Filters.all(); }
            case "-name" -> { return Filters.name(arg(t)); }
            case "-iname" -> { return Filters.iname(arg(t)); }
            case "-user" -> { return Filters.owner(arg(t)); }
            case "-size" -> { return Filters.size(arg(t)); }
            case "-mtime" -> { return Filters.mtime(arg(t), now); }
            case "-type" -> {
                String v = arg(t);
                return switch (v) {
                    case "f" -> Filters.type(FileType.FILE);
                    case "d" -> Filters.type(FileType.DIRECTORY);
                    default -> throw new SearchException("-type takes f or d, not '" + v + "'");
                };
            }
            default -> throw new SearchException("Unknown predicate '" + t + "'");
        }
    }
    // ... skipGlobalOptions (-maxdepth/-mindepth anywhere), arg, peekIs
}
```

**How the code works, part by part**

- **Globs.** `NameFilter.toRegex` turns `*` into `.*`, `?` into `.`, keeps `[...]` sets (turning `[!...]` into `[^...]`) and quotes every other character, then compiles the regex once, when the filter is built. Like `find -name`, it matches only the last path component, and `*` also matches names that start with a dot.
- **Size.** `SizeFilter` parses `[+-]N[cbkMG]` (no suffix = 512-byte blocks, as in `find`), rounds the file size **up** to whole units, then compares: `+` means more than N units, `-` fewer, no sign exactly N.
- **Age.** `MtimeFilter` computes the age in whole days (the fraction is dropped) against an injected `now`, so the demo is repeatable.
- **Builder.** Each `where(...)` call is ANDed with earlier ones; `build()` checks depths and the limit and freezes everything into an immutable `SearchCriteria`.
- **Search.** `FileSearcher.search` pushes the start node, then pops, tests, emits and pushes children in reverse name order so they come off the stack in name order. It stops descending at `maxDepth` and at pruned directory names, and stops entirely at the limit.
- **Parser.** `parseOr` → `parseAnd` → `parseUnary`: one method per precedence level. `parseAnd` treats two tests side by side as AND; `-maxdepth`/`-mindepth` are options that can appear anywhere, so every level skips them first.
- **Parallel.** `searchParallel` forks one task per sub-directory (files are tested inline), waits for all, then joins results in name order — the same list, in the same order, as the sequential search.

**Output** (of `Main.java`):

```
== Builder API ==
$ from / where (name='*.log' AND type=f)
   /home/asha/projects/app/build.log
   /var/log/app.log
$ from /home where (iname='*.jpg' AND size+1M)
   /home/asha/photos/a.jpg
   /home/asha/photos/b.JPG
$ from / where (type=d AND empty)
   /home/asha/projects/empty-dir/
   /tmp/
$ from /home where true maxdepth=2
   /home/
   /home/asha/
   /home/asha/notes.txt
   /home/asha/photos/
   /home/asha/projects/
   /home/ravi/
   /home/ravi/report.pdf
   /home/ravi/todo.md
$ from /home/asha/projects where (name='*.java' OR name='*.js') prune=[node_modules]
   /home/asha/projects/app/Main.java
   /home/asha/projects/app/Util.java
$ from / where type=f limit=3
   /home/asha/notes.txt
   /home/asha/photos/a.jpg
   /home/asha/photos/b.JPG

== find command lines ==
$ find /var/log -type f -size +10M -mtime +7
  parsed: from /var/log where (type=f AND size+10M AND mtime+7)
   /var/log/app.log.1
   /var/log/syslog
$ find /home -name '*.java' -o -name '*.md'
  parsed: from /home where (name='*.java' OR name='*.md')
   /home/asha/projects/app/Main.java
   /home/asha/projects/app/Util.java
   /home/ravi/todo.md
$ find /home ( -name '*.jpg' -o -name '*.png' ) ! -size +1M
  parsed: from /home where ((name='*.jpg' OR name='*.png') AND NOT size+1M)
   /home/asha/photos/c.png
   /home/asha/photos/d.jpg
$ find /home -size -1M
  parsed: from /home where size-1M
   /home/ravi/todo.md
$ find / -user ravi -type f
  parsed: from / where (user=ravi AND type=f)
   /home/ravi/report.pdf
   /home/ravi/todo.md
$ find /home/asha -mindepth 2 -maxdepth 2 -type d
  parsed: from /home/asha where type=d mindepth=2 maxdepth=2
   /home/asha/projects/app/
   /home/asha/projects/empty-dir/
$ find /home -iname 'b*'
  parsed: from /home where iname='b*'
   /home/asha/photos/b.JPG
   /home/asha/projects/app/build.log
$ find /var/log -name 'app.log.[0-9]'
  parsed: from /var/log where name='app.log.[0-9]'
   /var/log/app.log.1

== bad input gives a clear message ==
   find / -size 10X                  -> Invalid -size '10X' (use e.g. +10M, -1k, 100c)
   find / ( -name a                  -> Missing ')'
   find /nope -name x                -> No such file or directory: /nope
   find / -frobnicate                -> Unknown predicate '-frobnicate'
   find / -type x                    -> -type takes f or d, not 'x'
   find / -maxdepth 1 -mindepth 3    -> mindepth 3 > maxdepth 1
   find / -name                      -> -name needs an argument

== Composite sizes (du -s) ==
   /home/asha                              13,281,280 bytes
   /home/asha/projects/app/node_modules     1,560,576 bytes
   /var/log                                66,060,288 bytes
```

```
== Parallel search on a big generated tree ==
   from /data where (name='*.csv' AND size+500k AND mtime-10)
   sequential=1600 parallel=1600 same order=true first=/data/d000/sub0/f060.csv last=/data/d199/sub4/f099.csv
```

Reading the output: `-size +1M` kept the two large photos and dropped the 900 KB one; the prune option skipped `node_modules`, so only the two Java files match `*.java OR *.js`; `-size -1M` found only the empty `todo.md` — the rounding gotcha, even though `notes.txt` is only 2 KB; `-mindepth 2 -maxdepth 2 -type d` lists directories exactly two levels below `/home/asha`; the bracket glob `app.log.[0-9]` matches the rotated log but not `app.log`. Every bad input produced a one-line message. The composite `totalSize()` gives `du`-style totals (`du` is the Unix disk-usage command), and on a generated tree of 20,000 files the sequential and parallel searches returned the same 1,600 paths in the same order.

### Concurrency & edge cases

- **Adding files during a search:** children live in a `ConcurrentSkipListMap`, whose iterators are *weakly consistent* — iterators (the objects that step through a collection) never throw, and may or may not show an entry added after the iteration started (like `find` on a live disk).
- **Parallel search:** tasks only read the tree and each builds its own list; there is no shared mutable state, so no locks are needed.
- **Deep trees:** the sequential walk uses a heap-allocated stack (an ordinary object in main memory, not the limited call stack). (The fork/join version recurses one task per directory; fine for normal depths.)
- **Symbolic links / cycles** (real file systems): track visited `(device, inode)` pairs (an inode is the file system's internal id for a file; a symbolic link is a file that points to another path) or do not follow links by default — GNU `find` does not follow symbolic links unless asked (`-L`).
- **Edge cases handled:** start path missing or a file; `mindepth > maxdepth`; unknown predicate; missing argument; bad `-size`/`-mtime`/`-type` values; unbalanced parentheses; an unclosed `[` in a glob is treated as a literal; limit reached mid-walk.

### Extensions & follow-up questions

- **Search a real disk?** An adapter that wraps `java.nio.file.Path` + `BasicFileAttributes` as `Node`s, or use `Files.walkFileTree` with a `FileVisitor` that returns `SKIP_SUBTREE` for pruned directories.
- **Actions (`-delete`, `-exec`)?** A `Consumer<Node>` per action (Command pattern); `-delete` must walk depth-first *post-order*, meaning children are visited before their directory.
- **Content search (`grep -r`, the Unix tool that searches inside files)?** A `ContainsTextFilter` — put it last in the AND so cheap name/size tests reject most files first.
- **Faster repeated searches?** Build an index (name → paths, like `locate`/`updatedb`, Unix tools that pre-build a filename database) and accept that it can be stale.
- **More tests (`-newer file`, `-perm`, `-regex`)?** One record each in `Filters` plus one `case` in the parser.
- **Sort by size or date?** Collect, then sort with a `Comparator` — a Strategy, as in PropertyHunt (§21.5).

### Real-world example

GNU `find` (from GNU findutils) evaluates its expression left to right with exactly this precedence (`!`, then implicit or explicit `-a`, then `-o`) and short-circuits; its manual documents the "`-size -1M` only matches empty files" rounding and the default of not following symbolic links. Java's own `Files.find(start, maxDepth, BiPredicate)` and `Files.walk` take a depth limit and a predicate — the same split between *walking* and *testing* used here. IDE "Find in files" dialogs and desktop search boxes build the same kind of filter tree from checkboxes instead of a command line.

### In the interview

Spend the first 10 minutes on the model (`Node`, `FileNode`, `DirectoryNode`) and the `FileFilter` interface with two leaves (name, size) — that alone lets you demo a search. Then add `AndFilter`/`OrFilter`/`NotFilter`, the Builder with depth options, and the iterative walk. Add the parser last, and only if time allows: interviewers mostly score the filter design and extensibility. Say the Open/Closed point out loud: "a new filter is one class; the searcher never changes".

### Self-check

- **Q:** What does `find -name a -o -name b -type f` mean? → **A:** `a OR (b AND file)`, because AND binds tighter than OR.
- **Q:** Why does `-size -1M` match only empty files? → **A:** Sizes are rounded up to whole MiB, so any non-empty file is at least 1 unit, which is not less than 1.
- **Q:** Why are the AND/OR/NOT filters records rather than lambdas? → **A:** They can be printed (`describe()`), inspected and built by the parser as a visible tree.
- **Q:** How does pruning differ from filtering out a directory? → **A:** Pruning stops the walk from entering it (nothing inside is visited); a filter only hides the directory entry itself.
- **Q:** Why an explicit stack instead of recursion? → **A:** Very deep trees would overflow the call stack; an explicit stack also makes the limit easy.

### TL;DR

Two Composites — the file tree and the filter tree. Filters are Specifications (`matches`) combined with AND / OR / NOT; the Builder describes the search once and validates it; an iterative DFS streams matches; a recursive-descent parser turns `find` command lines into the same criteria, with `!` > AND > OR precedence and GNU's rounded-up `-size`.

---

## Chapter summary

| Problem | Core entities | Key pattern(s) | Concurrency point | Trap to avoid |
|---|---|---|---|---|
| Task Management | Task, TaskStatus, TaskService | Specification filters, enum state machine, Observer | per-task `synchronized` | TODO → DONE jumps; filters as many parameters |
| Digital Wallet | Wallet, Transaction, Offer | Strategy offers, idempotency, funded cashback | lock both wallets in id order | `double` money; check-then-act on keys; cashback created from nothing |
| Event Calendar | TimeSlot, Event, EventStore | Strategy for representatives, plan-then-save | atomic check-and-book | closed intervals; trusting the sample blindly |
| Coding Blox | Contest, Level, User | Command map, Strategy (solver, scoring) | atomic rating update | unseeded randomness; running a contest twice |
| PropertyHunt | Property, SearchCriteria | Builder + predicate list, Strategy sort | synchronized mark-sold | comparing "90L" strings; stale shortlist copies |
| Ledger Co | Loan, LumpSum, Balance | Strategy interest, derived state | synchronized lump sums | storing a running balance; floor instead of ceil |
| Inventory | StockMovement, AllocationStrategy | Strategy allocation, Observer alerts | per-SKU lock | partial shipments; alert spam |
| Unix find | Node (Composite), FileFilter, SearchCriteria | Specification + Composite filters, Builder, recursive-descent parser | weakly consistent children map; read-only parallel walk | parameter-list search methods; `-o` vs AND precedence |

**The pattern behind all eight:** entities guard their own rules, services orchestrate, strategies hold the "X or Y" rules, repositories hide storage, the driver parses and prints, and every error is a clear message — not a crash.

# Chapter 22 — Revision: Cheat Sheets & Glossary

> **What this chapter is.** One-page summaries of the whole handbook — the LLD (low-level design: the classes and code inside one program) answer framework (Ch 1), OOP and UML (Ch 2–3), SOLID, principles and clean code (Ch 4–6), the 24 smells (Ch 7), all 23 GoF patterns plus extras (Ch 8–11), the concurrency toolbox (Ch 12–13), a map of every case study (Ch 14–21), machine-coding and edge-case checklists, and a glossary of about 200 terms.
>
> Nothing here is new: every row summarises a section of Chapters 1–21, and the right-hand column says where to find the full explanation. No code.

**How to read the references.** "§12.17" means section 12.17. Chapters 5 and 6 number their sections from 1 inside the chapter, so "Ch 5 §13" means section 13 of Chapter 5 (Fail Fast). Chapters 16, 17 and 19 have unnumbered case studies, so they are referenced by name, for example "Ch 16 · Car Rental".

**New to a term?** Every technical word in the tables, from `API` and `JVM` to `TTL` and `SKU`, is defined once — in its cheat-sheet row or in the glossary (22.11). A **TL;DR** ("too long; didn't read") heading marks a short summary.

---

## 22.1 How to revise

### In 15 minutes (the morning of the interview)

| Minutes | Read | Why |
|---|---|---|
| 0–3 | 22.2 (the framework) | You will use it in the first minute of the interview |
| 3–6 | 22.4 (SOLID one-pager) and the relationship table in 22.3 | The two things asked about most often by name |
| 6–10 | "Which pattern?" table in 22.7 | Patterns should come from a signal, not from memory |
| 10–13 | Concurrency cheat sheet and deadlock checklist in 22.8 | Every booking/inventory problem has a race |
| 13–15 | The row(s) of 22.9 for the problems you expect | One sentence per problem: entities, pattern, the atomic step |

### The night before (about 2 hours)

1. **Read 22.2–22.8 slowly** (45 min). For each table row, say out loud *one example from the handbook*. If you cannot, open the referenced section.
2. **Pick three case studies** from 22.9 you have not practised recently (45 min). For each: write requirements, list classes, draw the class diagram, and name the one hard method, without looking. Then compare with the chapter's TL;DR.
3. **Scan the glossary** (22.11) and mark any term you cannot explain in one sentence (20 min). Read those sections only.
4. **Stop.** Sleep matters more than one more pattern.

#### Common mistakes when revising

- Re-reading chapters instead of **recalling** first and then checking. Recall is what the interview tests.
- Memorising all 23 patterns equally. Six come up in almost every round (22.7): Strategy, Factory, Observer, State, Singleton, Builder.
- Skipping the "when *not* to use it" column. Interviewers ask "would you always do that?" (§4.13).

---

## 22.2 The LLD interview answer framework (one page)

*Source: Chapter 1 (§1.2, §1.4–§1.10, §1.12, §1.14).*

### TL;DR

**Clarify → requirements → use cases → entities → relationships → class diagram → key flows → patterns/SOLID → code → edge cases/concurrency → extensions.** Say each step out loud as you enter it.

### The eleven steps

| # | Step | Ask yourself | Output |
|---|---|---|---|
| 1 | Clarify | Who uses it? Single site or many? | Shared understanding |
| 2 | Requirements | Functional? Non-functional (thread-safe, extensible, in-memory)? **Out of scope?** | 8–12 numbered lines + out-of-scope list |
| 3 | Use cases | What can each actor do? | Actor → use-case list |
| 4 | Entities | Which nouns hold state or behaviour? Which are just values? | Classes, one sentence each |
| 5 | Relationships | Uses / knows / has / owns / is-a? How many? | Typed lines with multiplicity |
| 6 | Class diagram | Fields, key methods, interfaces, enums | UML class diagram |
| 7 | Key flows | Steps of the 2–3 main use cases; lifecycle of the core object | Sequence / activity / state diagram |
| 8 | Patterns & SOLID | A rule that varies? A lifecycle? Many listeners? Two reasons to change? | 1–2 named patterns + one rejected |
| 9 | Code | Interfaces, enums, models, services, driver | Running code for the main flows |
| 10 | Edge cases & concurrency | Bad input? Full? Two users, one seat? | Validation, exceptions, locks |
| 11 | Extensions | "What if X is added?" | Short answers, maybe one class |

### Time budgets

| **45-minute OOD round** | | **90-minute machine-coding round (write runnable code)** | |
|---|---|---|---|
| Clarify requirements | 5–8 min | Read, requirements, assumptions | 5–10 min |
| Actors and use cases | 3–5 min | Model: entities, enums, service interfaces | 10–15 min |
| Find the classes | 8–10 min | Core flow end to end | 30–40 min |
| Connect the classes | 8–10 min | Remaining features, validation, edge cases | 15–20 min |
| Write the code | 12–15 min | Polish, one extension point, demo | 5–10 min |
| | | Buffer | 5 min (never zero) |

Rule of thumb: roughly **a quarter of the hour goes before you name a class**, and in machine coding you **get something running by the halfway mark** (§1.2).

### Finding classes (§1.6)

- **Nouns → candidate classes; verbs → methods.** A verb with no class to live on means a class is missing.
- Cut candidates with five tests: is it a **value**? the **same thing** under another name? does it have **changing state**? does anything **call** it? does it differ in **behaviour or only data**?
- **Different data → field or enum. Different behaviour → class.**
- One sentence per class, **no "and"**. Put each method where its data is (Information Expert).
- Beginners most often forget the **record-keeping class** (ticket, loan, booking) and the **rule class** (rate, strategy).

### What the interviewer judges (§1.2)

1. Did you **bound** the problem? 2. Does each class have **one job** you can say in a sentence? 3. Are the **relationships** right? 4. Does the **code match the diagram**? 5. Can you explain a **trade-off** (the choice and the rejected option)?

Machine coding adds: does it **run**, is it **modular**, **readable**, does it **handle bad input**, and (senior roles) is shared state **thread-safe**.

### Code habits interviewers notice (§1.10)

| Habit | Instead of |
|---|---|
| Pass time in ( `long nowMinute` , `Instant now` , injected `Clock` ) | Reading the clock inside logic |
| Money as `BigDecimal` from strings, or whole cents in a `long` | `double` (0.1 + 0.2 ≠ 0.3) |
| `IllegalArgumentException` / `IllegalStateException` / a custom exception | Silent failure or error codes |
| `Optional` when "not found" is normal | Returning `null` |
| `private final` fields | Public mutable fields |
| Domain names ( `ParkingTicket` ) | `Manager` , `Helper` , `Data2` |

Skeleton: **Models** → **Repositories** → **Strategies** → **Services** → **Driver** (§1.10). The driver is the `main` class that feeds sample commands to the services and prints the results.

### In the interview

> **30-second answer** (from §1.14): "I clarify and write 8 to 12 requirements with an out-of-scope list, list actors and use cases, pick classes from the nouns and give each one responsibility, connect them and draw a class diagram, sketch the key flow, run a quick SOLID check and pick one or two patterns where the signal is clear, then code the two or three core flows, handle edge cases and concurrency, and finish with extensions and trade-offs."

### Common mistakes (§1.14)

| Mistake | Fix |
|---|---|
| Coding in minute two | 8–13 minutes on requirements and actors first |
| No out-of-scope list | Write it down and agree it |
| One subclass per value ( `CompactSpot` , `LargeSpot` ) | Different data → field/enum |
| God class ( `ParkingLotManager` ) | One sentence per class; split on "and" |
| Inheritance for reuse ("Car extends Engine") | Composition |
| Pattern stuffing | One or two patterns, each with its signal |
| All twelve classes at equal depth | Two or three flows fully |
| Ignoring two users on one resource | Lock or atomic operation on the shared state |
| Non-running code in machine coding | Thin end-to-end path running by halfway |

---

## 22.3 OOP & relationships cheat sheet

*Source: Chapter 2 (OOP) and Chapter 3 (relationships and UML).*

### The four pillars (§2.3–§2.6)

| Pillar | One line | Java tools | Smell when missing |
|---|---|---|---|
| **Encapsulation** (§2.3) | Data is private; only the owning class's methods change it, and they enforce the *invariants* | `private` fields, no setters, validation in constructors | Public fields; setters that break invariants |
| **Abstraction** (§2.4) | Expose only what callers need; hide how | Interfaces, abstract classes, small public API | Callers reach into internals |
| **Inheritance** (§2.5) | Child reuses and specialises parent — **only when "child is a parent" is truly correct** | `extends`, `super`, `@Override` | Subclass refuses or throws on parent methods |
| **Polymorphism** (§2.6) | One call, different behaviour per object; removes type switches | Overriding + dynamic dispatch | `switch` / `instanceof` chains on type |

**Encapsulation vs abstraction:** hiding *data* behind rule-enforcing methods vs hiding *complexity* behind a minimal set of operations (Ch 2 self-check).

### Other OOP facts worth one line each

| Topic | Remember | Where |
|---|---|---|
| Overloading vs overriding | Compile time, by declared parameter types vs run time, by the actual object. Fields and `static` methods are not polymorphic (statics are hidden, not overridden) | §2.7 |
| `this`, `static`, `final` | Current object; one per class; cannot be reassigned/overridden/extended | §2.2 |
| Interfaces | Contracts; can have `default`, `static`, `private` methods; a functional interface has one abstract method (usable with lambdas) | §2.8 |
| Enums | Fixed sets with fields and behaviour ("which spot types fit a truck" lives on `VehicleType`) | §2.10 |
| Composition over inheritance | Avoids fragile base classes; allows run-time swapping | §2.11, Ch 5 §6 |
| `equals` / `hashCode` | Equal objects must have equal hash codes; override both or hash collections break | §2.12 |
| Immutability & records | Final class, private final fields, no setters, defensive copies. Records are only **shallowly** immutable — copy collections in the compact constructor | §2.13 |
| Access modifiers | Who can see a member: `public`, `protected`, package-private (same package only), `private`; default to `private` | §2.14 |
| Generics | Type parameters (`List<T>`) for type-safe reusable infrastructure; **PECS**: producer `extends`, consumer `super` | §2.15 |
| Coupling & cohesion | Low coupling (between classes), high cohesion (inside a class) | §2.16 |
| OOA → OOD → OOP | What the problem is → how, with objects → code. Connections that carry data become objects | §2.17 |

### Abstract class vs interface (§2.9)

| | Abstract class | Interface |
|---|---|---|
| Instantiable? | No | No |
| Instance fields (state) | Yes | No (only `public static final` constants) |
| Constructors | Yes | No |
| Method bodies | Any | `default`, `static`, `private` only |
| How many per class | **One** (`extends`) | **Many** (`implements`) |
| Relationship expressed | "is a" (shared identity and state) | "can do" (capability / role) |
| Typical use | Shared state + partial implementation for a close family (Template Method) | Contracts, roles, strategies, test seams |

**Rule of thumb:** start with an interface; add an abstract class underneath when implementations share real state or a fixed algorithm (JDK: `List` + `AbstractList` + `ArrayList`).

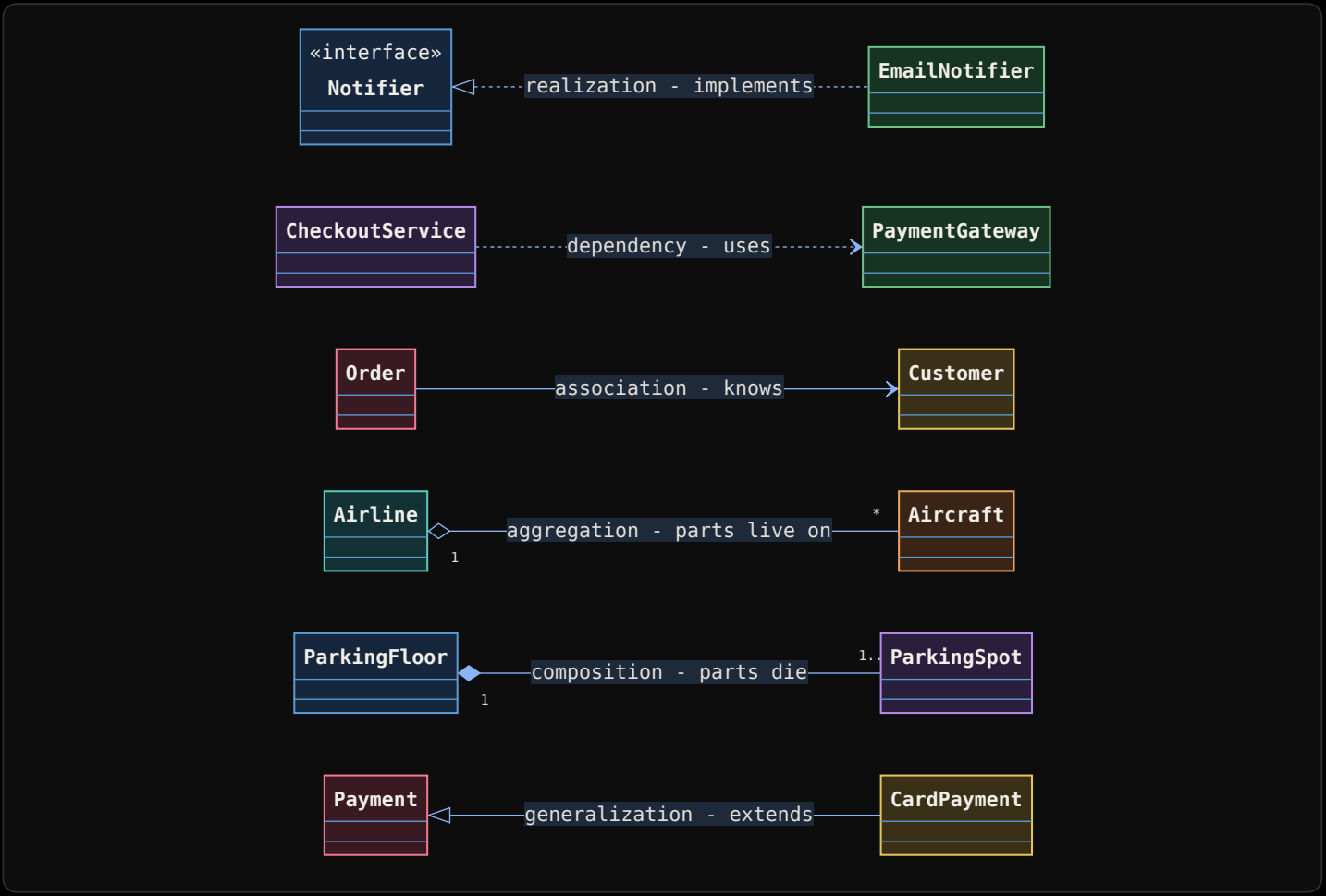
### The six relationships (§3.2–§3.8)

Weakest to strongest coupling: **dependency, association, aggregation, composition, realization, generalization** (§3.16).

| Relationship | Phrase | UML line | Java shape | Lifetime | Example |
|---|---|---|---|---|---|
| Dependency (§3.5) | A *uses* B | dashed, open arrow → B | B as parameter / local / return / static call | none | `CheckoutService` uses `PaymentGateway` |
| Association (§3.2) | A *knows* B | solid line (arrow for one-way) | field of type B, created elsewhere | independent | `Order` → `Customer` |
| Aggregation (§3.3) | A *has* B, parts live on | solid, hollow ◇ at A (the whole) | field/collection, parts passed in | B outlives A | `Airline` ◇— `Aircraft` |
| Composition (§3.4) | A *owns* B | solid, filled ◆ at A (the whole) | A creates B, hides it, B not shared | B dies with A | `ParkingFloor` ◆— `ParkingSpot` |
| Realization (§3.7) | A *implements* B | dashed, hollow △ at B | `implements` | — | `EmailNotifier` ⇢ `Notifier` |
| Generalization (§3.6) | A *is a* B | solid, hollow △ at B | `extends` | — | `CardPayment` → `Payment` |

**Choosing, in order (§3.8):** interface A promises to fulfil → realization. Substitutable with no refused methods → generalization. Not kept in a field → dependency. Part that dies with the whole → composition. Part that outlives it → aggregation. Otherwise → association.

**UML arrow legend (Mermaid)**




| Mermaid | Meaning |
|---|---|
| `A ..> B` | A depends on B |
| `A --> B` | A associated with B, navigable A to B |
| `A -- B` | association (both ways / unspecified) |
| `A o-- B` | aggregation, A is the whole |
| `A *-- B` | composition, A is the whole |
| `B <\|.. A` | A realizes interface B |
| `B <\|-- A` | A extends B |
| `A "1" *-- "1..*" B` | with multiplicities |

**Multiplicity → code (§3.17):** `1` required field; `0..1` `Optional`/nullable; `1..*` non-empty collection checked in the constructor; exact numbers checked in constructors. Visibility: `+` public, `-` private, `#` protected, `~` package.

**UML diagrams in the interview (§3.9–§3.15)**

| Diagram | Use it for | Budget in 45 min |
|---|---|---|
| Use case (§3.10) | Scope: actors + what each does. «include» = always; «extend» = sometimes | Usually spoken, 3–5 min |
| **Class (§3.11)** | **The answer.** 10–15 boxes, every line typed, with multiplicity | Always drawn, 10–15 min |
| Sequence (§3.12) | Call order between objects (3–4 objects, 6–8 messages) | One behaviour diagram only |
| Activity (§3.13) | A flow that branches; fork/join for parallel steps | " |
| State machine (§3.14) | One object's lifecycle; transition label `event [guard] / action` | " |

UML has 14 diagram types: 7 structural, 7 behavioural (§3.9).

**Common mistakes**

- Diamond at the wrong end (it goes at the **whole**); triangle at the wrong end (it goes at the **parent/interface**) (§3.8).
- Lines without type or multiplicity (§3.8).
- Inheritance used to share data; subclasses that only pass a constant to `super` → one class with an enum field (§3.16).

---

## 22.4 SOLID one-pager

*Source: Chapter 4 (§4.2, §4.7–§4.10, §4.12–§4.15), smells from Chapter 7 (§7.4).*

**TL;DR**

SOLID = five names for the ways code becomes painful to change. **Use them to diagnose, not as a checklist**, apply a fix when a real change asks for it, and always say what the fix costs.

| Principle | One-line rule | Smell that signals a violation | Fix | Patterns that help |
|---|---|---|---|---|
| **S** — Single Responsibility (§4.2) | A class should have one reason to change (one actor asks for its changes) | The responsibility sentence needs an "and"; *Divergent Change*, *Large Class* | Split by actor (Extract Class); each new class needs a name that stands alone | Command, Proxy, Decorator (each concern in its own class) |
| **O** — Open/Closed (§4.7) | Add new behaviour by adding code, not by editing working code | A `switch` / `if` chain over a type; *Repeated Switches*, *Shotgun Surgery* | Put the part that *has* varied behind an interface; add variants as classes (Replace Conditional with Polymorphism) | Strategy, Template Method, Decorator, Factory Method, Observer, Command, Visitor |
| **L** — Liskov Substitution (§4.8) | A subtype must be usable anywhere its parent is expected | Subclass throws, does nothing, or refuses a parent method; *Refused Bequest* | Better parent, a narrower interface, or composition (Replace Subclass with Delegate) | Composite, Null Object |
| **I** — Interface Segregation (§4.9) | No client should depend on methods it does not use | Empty method bodies or `UnsupportedOperationException` in implementers; fat interfaces | Split into **role interfaces**, one per kind of caller | Adapter |
| **D** — Dependency Inversion (§4.10) | Depend on abstractions, not concrete details; the policy owns the abstraction | A high-level class does `new` of (or names) a low-level one; cannot test without the real database/provider | Inject an interface owned by the high-level class; `main` wires the details | Strategy, Factory Method / Abstract Factory, Adapter, Observer |

**LSP in detail (§4.8, TL;DR):** a subclass must accept at least as much, promise at least as much, keep invariants and history, and throw nothing new. Square/Rectangle is worse than `Penguin.fly()` throwing because it fails **silently** (wrong area, no error).

### Five-minute pass over your diagram (§4.13, §1.8)

Run S → O → L → I → D over the diagram you just drew. **Two findings is a good pass; five means rework; zero means you did not look.** S and O earn the most credit: O is what interviewers probe with a late requirement ("what if trucks are charged differently?" → "add a class").

### Related ideas (§4.3–§4.6, §4.14)

| Idea | Question it asks |
|---|---|
| SRP | How many actors can ask this class to change? |
| Cohesion | Do the parts of this class belong together? |
| Coupling | How much does A depend on B? |
| Separation of Concerns | Is each area of work in its own module? |
| SRP vs ISP | SRP splits **classes** (guards the worker); ISP splits **interfaces** (guards the caller) |
| DIP vs DI vs IoC | DIP = principle (depend on abstractions); DI = technique (pass dependencies in); IoC = the broader idea that a framework calls your code; IoC containers (e.g. Spring) perform DI |

## Common mistakes: over-engineering (§4.15)

| Warning sign | Principle misapplied | Do instead |
|---|---|---|
| `XHelper`, `XUtils`, `XManager2` after a split | SRP | Split by actor; names must stand alone |
| Six small classes that always change in the same commit | SRP over-split | Merge them back |
| Interface + factory + registry for one implementation | OCP (*Speculative Generality*) | Write the `if`; refactor at the second or third case (rule of three) |
| A subclass that "works" by silently doing nothing | LSP | Better parent or composition |
| One interface per method | ISP over-split | Group by caller into role interfaces |
| An interface in front of stable `String`-like classes | DIP | Invert only volatile dependencies |
| A DI container for a 200-line program | DIP | Wire by hand in `main` |

**YAGNI** is the counterweight: SOLID says *what* a good seam looks like; YAGNI says *when* to cut it.

## In the interview

Say **observation → cost → fix → price of the fix**, and describe before you label: "`Reporting` implements nine methods but calls two — an interface segregation problem" (§4.13).

---

## 22.5 Other design principles and Clean Code rules

### Other principles (Chapter 5)

| Principle | One-line idea | Smell it fixes | Key Java tool | Where |
|---|---|---|---|---|
| DRY | Each piece of knowledge has one authoritative home | Duplicated rules drifting apart | Value objects, constants, methods | Ch 5 §1 |
| KISS | Choose the simplest design that works | Over-engineering | Plain code, `switch` expressions | Ch 5 §2 |
| YAGNI | Don't build features before they are needed | Speculative Generality | Delete unused code | Ch 5 §3 |
| Law of Demeter | Only talk to your immediate friends | Train wrecks `a.getB().getC().do()` | Intent-revealing delegate methods | Ch 5 §4 |
| Tell, Don't Ask | Tell objects what to do; don't pull their data out and decide for them | Anemic model, Feature Envy | Command methods on the owner | Ch 5 §5 |
| Composition over inheritance | Prefer "has-a" to "is-a" for reuse | Fragile base class | Forwarding / decorators | Ch 5 §6 |
| Program to an interface | Depend on supertypes, choose the concrete class once at the edge | Hard-wired concrete dependencies | Interfaces + DI | Ch 5 §7 |
| Encapsulate what varies | Isolate the part that changes behind an interface | Growing type switches | Strategy, lambdas | Ch 5 §8 |
| Loose coupling | Interacting objects know little about each other | God orchestrators | Observer, events, DI | Ch 5 §9 |
| Hollywood Principle / IoC | "Don't call us, we'll call you" — the framework owns the flow | Duplicated skeletons | Template Method, callbacks, DI | Ch 5 §10 |
| Separation of Concerns | Each part handles one concern | Spaghetti code | Layers, MVC | Ch 5 §11, §4.5 |
| Least Astonishment | Do what the name promises and nothing more | Hidden side effects | Honest names, `Optional` | Ch 5 §12 |
| Fail fast | Detect problems early and loudly | Distant `NullPointerException`s | Guard clauses, `requireNonNull` | Ch 5 §13 |
| Immutability | Objects never change after creation | Aliasing, races | Records, `List.copyOf` | Ch 5 §14 |
| Design by Contract | Preconditions (caller), postconditions (method), invariants (always) | Magic error values | Checks at method entry/exit | Ch 5 §15 |
| GRASP | Larman's nine guidelines for "which class should do this?" | God classes | Expert, Creator, Controller… | Ch 5 §16 |
| Domain-Driven Design | Model the business: bounded contexts, entities vs value objects, small aggregates, domain events, repositories per root | Anemic model, big ball of mud, primitive obsession | Records as value objects, aggregate roots, events + outbox | Ch 5 §17 |

**GRASP in one line each (Ch 5 §16.1–§16.9):**

| Guideline | Rule |
|---|---|
| Information Expert | The class with the data does the work |
| Creator | The class that contains, records or has the initial data for A creates A |
| Controller | A thin non-UI object receives each system event and delegates |
| Low Coupling | Choose the design with fewer, more stable dependencies |
| High Cohesion | Each class does one related set of things |
| Polymorphism | Let each type implement the varying behaviour; don't switch on types |
| Pure Fabrication | Invent a helper class (e.g. a repository) when no domain class is a clean home |
| Indirection | Put a middleman between things that shouldn't know each other |
| Protected Variations | Wrap what's unstable behind a stable interface |

**Tensions to mention (Ch 5 summary):** DRY vs loose coupling (sharing code couples its users); YAGNI vs encapsulate-what-varies (abstract only real or clearly predicted variation); Law of Demeter vs cohesion (too many delegates → Middle Man); Information Expert vs SoC (domain objects compute, fabrications persist).

**Clean Code rules checklist (Chapter 6)**

| Area | Rule of thumb | Where |
|---|---|---|
| Names | Reveal intent; pronounceable; searchable; one word per concept; code reads like the requirement | Ch 6 §1 |
| Functions | Small; do one thing; one level of abstraction; top-down (stepdown); ≤ 2–3 arguments; **no flag arguments**; no hidden side effects; commands *or* queries (CQS); exceptions, not error codes | Ch 6 §2 |
| Comments | Say it in code; comment only the *why*, warnings, legal text, public contracts; delete commented-out code | Ch 6 §3 |
| Formatting | Newspaper order (headline first); related things close; one automated team style | Ch 6 §4 |
| Objects vs data structures | Objects hide data and expose behaviour; data structures expose data; no hybrids; no train wrecks | Ch 6 §5 |
| Error handling | Unchecked exceptions with context; wrap vendor exceptions; empty collections / `Optional` / special case instead of `null`; never pass `null`; always clean up | Ch 6 §6 |
| Boundaries | Keep third-party code behind your own interfaces; learn it with tests | Ch 6 §7 |
| Unit tests | Three laws of TDD (Test-Driven Development: write a failing test first); one concept per test; Given–When–Then; **F.I.R.S.T.** (Fast, Independent, Repeatable, Self-validating, Timely) | Ch 6 §8 |
| Classes | Small by responsibility; cohesive; open for extension; depend on abstractions | Ch 6 §9 |
| Systems | Build objects in one place (composition root) and inject them; cross-cutting concerns in proxies/aspects | Ch 6 §10 |
| Simple design | Beck: passes the tests → reveals intention → no duplication → fewest elements (*Clean Code* swaps the middle two) | Ch 6 §11 |
| Concurrency | Separate concurrency code; share as little as possible; use the library; keep locks small; plan shutdown | Ch 6 §12 |
| Smells | The C/E/F/G/J/N/T heuristics catalogue as a review vocabulary | Ch 6 §14 |
| Code review | Correctness → design → readability → tests → security → performance → operability; automate style; small PRs (pull requests: proposed changes sent for review) | Ch 6 §15 |
| Error strategy | Classify, translate at boundaries, retry only transient + idempotent with backoff, one global handler that logs once | Ch 6 §16 |
| Testing strategy | Many unit, some integration, few E2E (end-to-end, whole-system); right test double; verify state over interactions; don't mock what you don't own | Ch 6 §17 |

**Boy Scout Rule:** leave the code a little cleaner than you found it (Ch 6 intro).

---

## 22.6 Code smells → refactorings (all 24)

*Source: Chapter 7 (§7.2 smells, §7.3 catalogue, §7.4 lookup table).* ***Bold*** *= the usual first move.*

| # | Smell | Signal (one line) | Group | Refactorings that cure it |
|---|---|---|---|---|
| 1 | Mysterious Name (§7.2.1) | Name doesn't say what it does/holds ( `data`, `proc()`, `Manager` ) | 2nd-ed extra | **Change Function Declaration**, Rename Variable, Rename Field |
| 2 | Duplicated Code (§7.2.2) | Same structure in more than one place | Dispensables | **Extract Function**, Slide Statements, Pull Up Method |
| 3 | Long Function (§7.2.3) | You want a comment to explain a block | Bloaters | **Extract Function**, Replace Temp with Query, Introduce Parameter Object, Preserve Whole Object, Replace Function with Command, Decompose Conditional, Replace Conditional with Polymorphism, Split Loop |
| 4 | Long Parameter List (§7.2.4) | Many parameters, same types, boolean flags | Bloaters | Replace Parameter with Query, **Preserve Whole Object**, **Introduce Parameter Object**, Remove Flag Argument, Combine Functions into Class |
| 5 | Global Data (§7.2.5) | Readable and writable from anywhere (incl. mutable singletons) | 2nd-ed extra | **Encapsulate Variable** (then shrink its scope) |
| 6 | Mutable Data (§7.2.6) | Shared data changed after creation | 2nd-ed extra | **Encapsulate Variable**, Split Variable, Slide Statements, Extract Function, Separate Query from Modifier, Remove Setting Method, Replace Derived Variable with Query, Combine Functions into Class/Transform, Change Reference to Value |
| 7 | Divergent Change (§7.2.7) | One class changed for many different reasons (SRP) | Change Preventers | **Split Phase**, Move Function, Extract Function, **Extract Class** |
| 8 | Shotgun Surgery (§7.2.8) | One change forces many small edits in many classes | Change Preventers | **Move Function**, **Move Field**, Combine Functions into Class/Transform, Split Phase, Inline Function, Inline Class |
| 9 | Feature Envy (§7.2.9) | Method uses another class's data more than its own | Couplers | **Move Function**, Extract Function |
| 10 | Data Clumps (§7.2.10) | Same group of values always travel together ( `startDate, endDate` ) | Bloaters | **Extract Class**, **Introduce Parameter Object**, Preserve Whole Object |
| 11 | Primitive Obsession (§7.2.11) | Money as `double`, status as `String` | Bloaters | **Replace Primitive with Object**, Replace Type Code with Subclasses, Replace Conditional with Polymorphism, Extract Class, Introduce Parameter Object |
| 12 | Repeated Switches (§7.2.12) | The *same* switch on a type code in several places | OO Abusers | **Replace Conditional with Polymorphism** |
| 13 | Loops (§7.2.13) | Classic loop that filters/maps/aggregates | 2nd-ed extra | **Replace Loop with Pipeline** |
| 14 | Lazy Element (§7.2.14) | A class/function that doesn't pull its weight | Dispensables | **Inline Function**, **Inline Class**, Collapse Hierarchy |

| # | Smell | Signal (one line) | Group | Refactorings that cure it |
|---|---|---|---|---|
| 15 | Speculative Generality (§7.2.15) | Hooks "we might need someday"; abstract class with one subclass | Dispensables | Collapse Hierarchy, Inline Function, Inline Class, Change Function Declaration, Remove Dead Code |
| 16 | Temporary Field (§7.2.16) | Field set only in some situations, `null` otherwise | OO Abusers | **Extract Class**, Move Function, Introduce Special Case |
| 17 | Message Chains (§7.2.17) | `order.getCustomer().getAddress().getCity()` | Couplers | **Hide Delegate**, Extract Function, Move Function |
| 18 | Middle Man (§7.2.18) | Most methods just forward to another class | Couplers | **Remove Middle Man**, Inline Function, Replace Superclass/Subclass with Delegate |
| 19 | Insider Trading (§7.2.19) | Two modules poke at each other's internals | Couplers | **Move Function**, **Move Field**, Hide Delegate, Replace Subclass/Superclass with Delegate |
| 20 | Large Class (§7.2.20) | Too many fields/methods — a God Class | Bloaters | **Extract Class**, Extract Superclass, Replace Type Code with Subclasses |
| 21 | Alternative Classes with Different Interfaces (§7.2.21) | Same job, different method names (`sendMail` vs `dispatch`) | OO Abusers | **Change Function Declaration**, Move Function, Extract Superclass |
| 22 | Data Class (§7.2.22) | Only fields and getters/setters; others manipulate it | Dispensables | **Encapsulate Record**, Remove Setting Method, Move Function, Extract Function, Split Phase |
| 23 | Refused Bequest (§7.2.23) | Subclass doesn't want inherited methods, or throws on them (LSP) | OO Abusers | Push Down Method/Field, **Replace Subclass with Delegate**, **Replace Superclass with Delegate** |
| 24 | Comments (§7.2.24) | Comment used as deodorant for unclear code | Dispensables | **Extract Function**, Change Function Declaration, Introduce Assertion |

### Smell family → principle violated (§7.4)

| Smell family | Principle |
|---|---|
| Divergent Change, Large Class | SRP |
| Repeated Switches, Shotgun Surgery | OCP |
| Refused Bequest | LSP |
| Long Parameter List with flags, fat interfaces | ISP / clean function design |
| Global Data, Mutable Data | Encapsulation |
| Hard-wired `new` of concrete dependencies | DIP |
| Duplicated Code | DRY |
| Speculative Generality | YAGNI |
| Message Chains, Insider Trading | Law of Demeter |
| Feature Envy, Data Class | Tell, Don't Ask |

**Refactoring toward patterns (§7.6)**

| Smell | Refactor toward |
|---|---|
| Switch/if-chain choosing an algorithm | Strategy |
| Methods that check and change a status field | State |
| `if (x == null)` at many call sites | Null Object / Special Case |
| Sibling subclasses with the same skeleton | Template Method |
| Many confusing constructors | Factory Method / creation methods |
| Complex object built by many ordered calls | Builder |
| Undo/queueing needed; huge function with shared locals | Command |
| Logging/caching mixed into a core class | Decorator |
| Objects polling for changes; hard-coded notification | Observer |
| Two classes, same job, different interfaces | Adapter |
| Subclass explosion from feature combinations | Strategy / Decorator / Bridge |
| A pattern that no longer earns its keep | Remove it (Inline Class, Collapse Hierarchy) |

**Refactoring** = changing structure without changing behaviour, in small steps, with tests as the safety net. Wear one of **two hats** at a time: adding function or refactoring (§7.1.1).

---

## 22.7 Design patterns: all 23 GoF + extras

*Source: Chapters 8–11. Catalogue in §8.3; comparisons in §8.15, §9.8, §10.7, §11.6, §11.8.*

**Count:** 5 creational + 7 structural + 11 behavioral = **23**. **Class-scoped** (inheritance) patterns: Factory Method, class Adapter, Template Method, Interpreter; almost everything else is object-scoped (composition) (§8.3).

### Creational (Chapter 8)

| Pattern | Intent (one line) | Use when | Java / JDK example | Commonly confused with | Where |
|---|---|---|---|---|---|
| **Singleton** | Exactly one instance with a global access point | There really must be exactly one (config, pool, registry) | `Runtime.getRuntime()`; Spring singleton-scoped beans | A global variable; a DI singleton scope (preferred for testing) | §8.8 |
| **Factory Method** | Subclasses decide which concrete class to create via an overridable method | Shared procedure, varying product; others will add types | `Collection.iterator()` | Simple Factory (one method with a switch) | §8.10 |
| **Abstract Factory** | Create *families* of related objects without naming concrete classes | Products must match; swap a whole set at once | `DocumentBuilderFactory`; JDBC `Connection` | Factory Method (one product by inheritance vs a family by composition) | §8.11 |
| **Builder** | Build a complex object step by step; one validating `build()` | Many optional fields, cross-field validation, immutable result | `StringBuilder`, `HttpClient.newBuilder()` | A plain constructor (fine for a handful of required fields); telescoping constructors | §8.12 |
| **Prototype** | Create new objects by copying a configured instance | Setup is expensive, or only the interface is known | Copy constructors, `ArrayList.clone()` | Memento (a snapshot to restore, not a copy to use) | §8.13 |

Singleton notes (§8.8, §8.17): eager, holder-class or **`enum`** (most robust: blocks duplicates made through reflection, i.e. code calling private constructors by name, or through serialization, i.e. saving to bytes and reading back); double-checked locking needs a **`volatile`** field. It is global state that hurts testing — prefer injecting one shared instance. Many case studies reject it (Ch 14 self-check).

### Structural (Chapter 9)

| Pattern | Intent (one line) | Use when | Java / JDK example | Commonly confused with | Where |
|---|---|---|---|---|---|
| **Adapter** | Convert one interface into another that clients expect | "The vendor API doesn't match ours" (after the fact) | `Arrays.asList`, `InputStreamReader` | Bridge (same shape, but Bridge is designed up front); Facade | §9.1 |
| **Bridge** | Split abstraction from implementation so both vary independently | Class names with two nouns; two dimensions multiplying classes | JDBC `Connection` + drivers | Adapter; Strategy (same shape) | §9.2 |
| **Composite** | Treat single objects and trees uniformly | Part-whole trees (files, menus, UI) | `java.awt.Container` is a `Component` holding `Component`s | Decorator (also wraps a component, but adds behaviour to one) | §9.3 |
| **Decorator** | Add responsibilities dynamically by wrapping; same interface, stackable | "Sometimes logging, sometimes caching, sometimes both" | `BufferedInputStream`, `Collections.synchronizedList` | Proxy (same shape; Proxy *controls access*); Chain of Responsibility (in CoR someone can stop the flow) | §9.4 |
| **Facade** | One simple entry point to a complex subsystem | Every caller repeats the same multi-step sequence | SLF4J `LoggerFactory`, service layers | Adapter (converts one object's interface); Mediator | §9.5 |
| **Flyweight** | Share intrinsic (immutable) state across huge numbers of objects | Millions of near-identical objects | `Integer.valueOf`, `String.intern()` | Object Pool (reuses objects one borrower at a time; flyweights are shared at once) | §9.6 |
| **Proxy** | A stand-in that controls access (lazy, remote, protective, caching) | It's expensive, remote or restricted | `java.lang.reflect.Proxy`, Spring `@Transactional`, Hibernate lazy load | Decorator | §9.7 |

Memory hooks (§9.8): Adapter *make it fit* · Decorator *add to it* · Proxy *guard it* · Facade *simplify it* · Bridge *split it* · Composite *nest it* · Flyweight *share it*.

## Behavioral (Chapters 10–11)

| Pattern | Intent (one line) | Use when | Java / JDK example | Commonly confused with | Where |
|---|---|---|---|---|---|
| **Chain of Responsibility** | Pass a request along handlers until one handles (or stops) it | Several checks, any may reject; handler not known in advance | Servlet `Filter` chain; `java.util.logging` handlers | Decorator (every layer always runs) | §10.1 |
| **Command** | Turn a request into an object: queue, log, retry, undo | Undo/redo, job queues, scheduling | `Runnable` / `Callable`; Swing `UndoManager` | Strategy (Strategy = *how*; Command = *what*) | §10.2 |
| **Iterator** | Traverse a collection without exposing how it is stored | Walk a collection/tree/paged API | `Iterator`, `Spliterator`, JDBC `ResultSet` | Visitor (what to do at each node) | §10.3 |
| **Mediator** | Centralise communication between peers in one coordinator | Many objects interact in tangled ways; order of reactions matters | Chat servers, UI controllers; `Executor` / `Timer` are a loose fit (§10.4) | Observer (broadcast, no coordination); Facade | §10.4 |
| **Memento** | Capture and restore state without breaking encapsulation | Undo, checkpoints, rollback | A `record` snapshot; DB `ROLLBACK`, Git commits (real-world) | Command undo (store actions, not states); Prototype | §10.5 |
| **Interpreter** | One class per grammar rule; evaluate sentences of a small language | User-written rules/expressions | `java.util.regex.Pattern`, SpEL (Spring's expression language) | Composite (same tree); Visitor (many operations over the AST) | §10.6 |
| **Observer** | Notify many dependents automatically when one object changes | An open-ended set of independent listeners | `PropertyChangeListener`, `java.util.concurrent.Flow`, Spring `@EventListener` | Mediator; pub-sub (Observer through a broker, §20.6) | §11.1 |
| **State** | Behaviour changes with internal state, as if the class changed | Many states/transitions; methods repeat a `switch` on status | TCP connection (GoF's example), order lifecycles. `Thread.State` is only an enum of states, not the pattern | Strategy (same diagram; states know each other and advance themselves) | §11.2 |
| **Strategy** | Make a family of algorithms interchangeable behind one interface | Several ways to do one job, chosen at run time | `Comparator`; `RejectedExecutionHandler` policies | State; Template Method; Decorator (Strategy *replaces* a rule, Decorator *wraps* one) | §11.3 |
| **Template Method** | Fixed algorithm skeleton in a base class; subclasses fill in steps | Long fixed flow, a few varying steps | `AbstractList`, `InputStream.read(byte[],…)`, `HttpServlet.service()` | Strategy (Template = inheritance, one step varies; | §11.4 |

| Pattern | Intent (one line) | Use when | Java / JDK example | Commonly confused with | Where |
|---|---|---|---|---|---|
| | | | | Strategy = composition, whole algorithm varies) | |
| **Visitor** | Add operations to a stable set of classes without changing them (double dispatch) | Many new operations over a fixed type set (ASTs) | `javax.lang.model.element.ElementVisitor`; `FileVisitor` is named after it but is really a tree-walk callback | Iterator; in Java 21, **sealed types + pattern-matching** `switch` usually replace it | §11.5 |

### Beyond GoF: extras you will meet

| Pattern | Intent (one line) | Use when | Java / real example | Commonly confused with | Where |
|---|---|---|---|---|---|
| **Simple Factory** (idiom) | One `create(type)` method maps a type value to a class | Small, stable set of types chosen by config | `Calendar.getInstance()`, `EnumSet.noneOf` | Factory Method | §8.9 |
| **Object Pool** | Borrow and return a bounded set of expensive objects | DB connections, limited resources | HikariCP (a database connection pool), `ThreadPoolExecutor` | Flyweight | §8.14 |
| **Null Object** | A do-nothing implementation instead of `null` | `null` checks everywhere for an optional collaborator | `Collections.emptyList()`, `OutputStream.nullOutputStream()` | `Optional` (for lookups) | §11.7.1 |
| **Dependency Injection** | Receive collaborators (as interfaces) through the constructor | Classes create their own collaborators; hard to test | Spring constructor injection; a hand-wired `main` | Service Locator (anti-pattern) | §11.7.2 |
| **Repository** | Collection-like interface over storage | Business logic mixed with storage code | Spring Data `CrudRepository` | DAO (table-oriented) | §11.7.3 |
| **Specification** | A named, combinable business rule (`isSatisfiedBy`, and/or/not) | Same rule in queries and validation; finder-method explosion | `Predicate`; Spring Data JPA `Specification` | — | §11.7.4 |
| **MVC** | Separate model, view and controller (Observer + Strategy + Composite) | UI, input handling and rules tangled | Spring MVC; Swing `JTable` + `TableModel` | MVP, MVVM (relatives) | §11.7.5 |
| **Event Sourcing** (lite) | Store the events; derive current state by replaying them | Full history, audit, time travel | Append-only logs | Plain audit log | §11.7.6 |
| **Architectural styles** (layered, client-server, modular monolith vs microservices, event-driven, pipe-and-filter, hexagonal/clean) | Fix the shape of the *whole system*: parts, dependency direction, communication | Deciding where classes live and answering "would you split this?" | Spring layered apps; Kafka consumers; Unix pipes; ports and adapters | Design patterns (a few classes inside one part); MVC (only the UI edge) | §11.9 |

### "Which pattern?" by problem symptom

*Combined from §8.5, §8.15, §9.8, §11.8 and §7.6. Look-alikes to rule out are in each pattern's "Commonly confused with" cell above.*

| If you see / need… | Use |
|---|---|
| One operation, several interchangeable rules chosen at run time (pricing, matching, routing) | **Strategy** |
| The class to create depends on a type value; caller shouldn't know the concrete class | **Factory** (Simple Factory / Factory Method) |
| Products must always match as a family | **Abstract Factory** |
| Constructor has ~6+ arguments, many optional | **Builder** |
| Another one of *this*, cheaply or without knowing its class | **Prototype** |
| Exactly one instance must exist | **Singleton** (or DI singleton scope) |
| Expensive objects created and destroyed repeatedly | **Object Pool** |
| Existing class has the wrong interface | **Adapter** |
| Two dimensions multiplying classes (`WindowsButton`, `MacButton`, …) | **Bridge** |
| Tree of part-whole objects treated the same | **Composite** |
| Add behaviour in combinations without editing the class | **Decorator** |
| Every caller repeats the same five steps into a subsystem | **Facade** |
| Millions of near-identical objects | **Flyweight** |
| Lazy load, access control, remote, caching (same interface) | **Proxy** |
| A request must pass several checks; any may reject it | **Chain of Responsibility** |
| Undo/redo, queue, schedule, retry or log *requested actions* | **Command** |
| Evaluate user-written rules in a small language | **Interpreter** (+ Specification) |
| Walk a collection/tree/paged API without exposing structure | **Iterator** |
| Many objects depend on each other and the **order** of reactions matters | **Mediator** |
| Roll back to an earlier state without exposing internals | **Memento** |
| One change must reach an open-ended set of listeners | **Observer** (or a broker) |
| Three or more methods repeat one `switch` over a status; legal actions depend on it | **State** (or enum + guarded method for 3–4 states) |
| Several classes share a long fixed flow, differ in a few steps | **Template Method** |
| Many new operations over a stable set of types | **Visitor**, or sealed types + pattern matching |
| `null` checks everywhere | **Null Object** / `Optional` |
| Classes create their own collaborators | **Dependency Injection** |
| Business logic mixed with storage | **Repository** |
| Same rule duplicated in queries and validation | **Specification** |

**Three questions for the famous look-alikes (§11.8):**

1. Do the alternatives reference each other? Yes → **State**. No → **Strategy** (or Bridge if chosen once at wiring).
2. Can one participant stop the flow? Yes → **Chain of Responsibility**. Everyone always runs → **Decorator**. Everyone is told, nobody stops anyone → **Observer**.
3. Does a central object decide what happens next? Yes → **Mediator**. Each listener decides for itself → **Observer**.

### The six that come up in almost every round (§1.9, §8.5)

| Pattern | Signal | Do not use when |
|---|---|---|
| Strategy | One operation, several swappable rules | Only one rule exists and nothing hints at a second |
| Factory | Creation depends on a type value | Only one concrete class exists |
| Observer | Several listeners the source should not know | A single listener the source already references |
| State | Behaviour depends on status; many transitions | 3–4 states handled by one guarded method |
| Singleton | Genuinely exactly one | You just want a global handle |
| Builder | Many optional fields; long constructor | A handful of required fields |

### Common mistakes (§8.6)

| Anti-pattern | Fix |
|---|---|
| God Object (one class does everything) | Split by responsibility; push behaviour to the data owners |
| Golden Hammer (favourite pattern everywhere) | Start from the problem's signal |
| Pattern overuse ("patternitis") — factories for one class, single-implementation interfaces | YAGNI: add the pattern when the second variant arrives |
| Singletonitis (singletons as globals) | Dependency injection |
| Cargo-cult patterns (structure without intent) | Name the property the pattern gives you, or drop it |

### In the interview

Use **one or two** patterns in a 45-minute design, placed where their signal appears, and **name one you rejected** and why (§8.5, §8.16).

---

## 22.8 Concurrency cheat sheet

*Source: Chapter 12 (concepts) and Chapter 13 (classic problems).*

### TL;DR (§12.28)

Races come from **shared mutable state**. Make the critical section atomic, or remove sharing/mutability. Visibility needs **happens-before** edges. Prevent deadlock with **lock ordering** or **tryLock + back-off**. In LLD: name the contended resource, keep **check + claim atomic**, lock fine-grained and in order, and say what changes with many servers.

**Primitive | use for | gotcha**

| Primitive | Use for | Gotcha | Where |
|---|---|---|---|
| `synchronized` | Making a multi-step update atomic (mutex + visibility) | Don't lock on a changing reference, a boxed value or a string literal; no slow I/O while holding it; readers must use the same lock | §12.7 |
| `ReentrantLock` | Same as `synchronized` plus try/timed/fair locking and `Condition` s | Call `lock()` **just before** `try` , `unlock()` in `finally` ; fairness is noticeably slower | §12.8 |
| `tryLock(timeout)` | Give up if the lock is busy; deadlock prevention with back-off | Never `unlock()` when `tryLock` returned `false` ; don't retry in a tight loop (livelock) | §12.9 |
| `ReentrantReadWriteLock` | Many readers, few writers | Cannot upgrade read → write (self-deadlock); don't modify under the read lock; writer starvation in non-fair mode | §12.10 |
| `StampedLock` | Read-mostly data with optimistic reads | **Not reentrant**; `validate` the stamp before using values read optimistically | §12.10 |
| `volatile` | Visibility of a flag or a published immutable snapshot | Gives visibility, **not** atomicity: `count++` is still a race | §12.6 |
| `AtomicInteger` / `AtomicLong` / CAS | Lock-free counters and single-variable claims | Separate `get()` then `set()` is still a race — use `compareAndSet` / `updateAndGet` ; no side effects in the lambda (may run several times); ABA → `AtomicStampedReference` | §12.15 |
| `LongAdder` | Very hot counters (metrics) | Use instead of `AtomicLong` when contention is high | §12.15 |
| `Semaphore(N)` | Limit concurrent access to N; signalling between threads | Release in `finally` ; don't release when `acquire` never succeeded | §12.12 |
| `wait/notifyAll` , `Condition` | Waiting for a condition on shared state | Always `while` , never `if` (spurious wakeups); hold the monitor; `notify()` can wake the wrong thread; state change without notify = lost wakeup | §12.13 |
| `CountDownLatch` | Wait for N events once | One-shot; `countDown()` in `finally` , `await(timeout)` | §12.14 |
| `CyclicBarrier` / `Phaser` | N threads meet at each phase (reusable) | Fewer threads than parties → everyone waits forever | §12.14 |
| `ConcurrentHashMap` | Shared map | `containsKey` then `put` is a race — use `putIfAbsent` / `computeIfAbsent` / `merge` ; no slow work or recursive updates inside `compute` ; no `null` keys/values | §12.16 |
| `CopyOnWriteArrayList` | Read-mostly lists (listener lists) | Every write copies the array | §12.16 |
| `BlockingQueue` (bounded) | Handing work between threads (producer–consumer) | Unbounded queue → no back-pressure → memory blow-up; one poison pill per consumer | §12.22 |
| `ExecutorService` / `ThreadPoolExecutor` | Running tasks on a bounded pool | Call `shutdown()` ; `newFixedThreadPool` has an **unbounded** queue; exceptions in `submit` ted tasks are lost unless | §12.4, §12.21 |

| Primitive | Use for | Gotcha | Where |
|---|---|---|---|
| | (separates what to run from how) | you call `get()`; tasks waiting on tasks in the same pool → thread-starvation deadlock | |
| Virtual threads | Lightweight JVM-managed threads (Java 21) for many blocking I/O tasks | Pinning (Java 21–23: blocking inside `synchronized`); heavy `ThreadLocal`s waste memory | §12.4, §12.24 |
| `ForkJoinPool` / parallel streams | Splitting CPU work | Use a threshold: tiny tasks cost more to schedule than to run; subtasks must touch disjoint data | §12.26, §13.10 |
| `CompletableFuture` | Combining async results (`thenApply`, `thenCombine`, `allOf`) | Handle errors with `exceptionally` rather than calling `get()` at every step | §12.4 |
| `ThreadLocal` | Per-thread state (thread confinement) | Clear it (`remove()`) in pooled threads, or values leak into the next task | §12.24 |
| Immutability / confinement | Avoiding synchronization entirely | `final List` is not immutable if the list is mutable and exposed | §12.24 |

### Which `java.util.concurrent` class for which job (§12.26)

Read the **Use for** column of the table above as the §12.26 "I need to… → use" lookup (`java.util.concurrent` is Java's standard package of thread-safety tools). Two entries not in that table: to **stop a loop from another thread**, use a `volatile boolean` flag or `interrupt()`; for **many readers, few writers**, an immutable snapshot published through a `volatile` field is often simpler than a read-write lock.

### Deadlock checklist (§12.17–§12.19)

A deadlock needs **all four Coffman conditions**; break any one and it cannot happen.

| # | Condition | How to break it |
|---|---|---|
| 1 | Mutual exclusion | Lock-free/immutable data, shareable resources (read locks) |
| 2 | Hold and wait | Acquire everything at once, or `tryLock` and release everything on failure |
| 3 | No preemption | Timeouts (`tryLock(timeout)`), leases that expire |
| 4 | Circular wait | **Global lock ordering** (always lock the lower id first) |

- [ ] Do I ever hold two locks? If yes, is there **one global order** (account id, seat id), with a tie-breaker for equal keys?
- [ ] Do I call a **listener, callback or another object's synchronized method while holding my lock**? Move it outside the lock.
- [ ] Do tasks in a bounded pool **wait for other tasks in the same pool**? (thread-starvation deadlock)
- [ ] Are retries randomised (**jitter**) so two threads don't back off in lockstep? (livelock, §12.18)
- [ ] Could one thread wait forever while others progress? Consider fairness (§12.19).
- [ ] Detection: a thread dump (`jstack` / `jcmd`) prints "Found one Java-level deadlock"; `ThreadMXBean.findDeadlockedThreads()`.

**Deadlock vs livelock vs starvation:** nobody progresses and all wait / everybody runs but nobody progresses / some progress while one never does (§12.27).

### The LLD concurrency recipe (§12.25)

1. **Name the shared mutable state** (free spots, seat → owner, stock).
2. **State the invariant** ("a seat has at most one booking", "stock ≥ 0").
3. **Find the critical operation** — usually a check-then-act (*find free → assign*).

4. **Pick the smallest tool that makes it atomic:** one item → `queue.poll()`, `putIfAbsent`, `compareAndSet`; several items all-or-nothing → per-item locks **in sorted order**; across servers → DB constraints/transactions, optimistic versioning, or a distributed lock.
5. **Choose granularity:** per floor / show / SKU, not one global lock.
6. **Handle time:** holds with expiry, timeouts, idempotent retries.

### Classic problems (§13.12)

| Problem | Key primitive | Invariant to state |
|---|---|---|
| FooBar (§13.1) | 2 semaphores | Exactly one permit exists across both |
| Zero Even Odd (§13.2) | 3 semaphores | Zero dispatches; one permit circulates |
| FizzBuzz (§13.3) | monitor + `notifyAll` | Predicates partition 1..n |
| H2O (§13.4) | 2 semaphores + barrier | ≤ 2 H and ≤ 1 O per round |
| Dining Philosophers (§13.5) | ordered locks / n−1 semaphore | No circular wait |
| TTL cache (§13.6) | `ConcurrentHashMap` + `compute` + conditional remove | Never return or keep expired data |
| Striped map (§13.7) | lock array | Each bucket belongs to exactly one stripe |
| Blocking queue (§13.8) | lock + 2 conditions | 0 ≤ count ≤ capacity; wait in loops |
| Bloom filter (§13.9) | atomic OR | Bits only go 0 → 1; no false negatives |
| Merge sort (§13.10) | Fork/Join | Subtasks touch disjoint ranges |
| Token-bucket rate limiter (§13.11) | lock or CAS on immutable state | Refill + take is one atomic step |

### Common mistakes (§12.25 and throughout Chapter 12)

- A global `synchronized` on `ParkingLot.park()` — correct, but serialises every gate.
- Checking availability with one call and booking with another (check-then-act).
- Locking seats in the order the user clicked them (deadlock between two multi-seat bookings).
- Forgetting that in-memory locks don't work across multiple app servers.
- Swallowing `InterruptedException` — rethrow, or restore the flag with `Thread.currentThread().interrupt()` (§12.3).
- Calling `run()` instead of `start()` (§12.3).

---

## 22.9 Case-study map

*Source: Chapters 14–21. "Concurrency concern" is the race each chapter guards against, and "atomic step" is how.*

## Everyday objects (Chapter 14)

| Problem | Key entities | Patterns used | Concurrency concern | Where |
|---|---|---|---|---|
| Parking lot | `ParkingLot`, `ParkingFloor`, `ParkingSpot`, `Vehicle`, `Ticket`, `DisplayBoard`, `SpotType` / `VehicleType` enums | Strategy (pricing, spot-finding), Factory (electric spots), Observer (boards); Singleton rejected | Two gates take one spot → atomic `pollFirst` + CAS; per-ticket lock for payment; `putIfAbsent` for plates | §14.1 |
| Vending machine | `VendingMachine`, `Slot`, `Inventory`, `CashBox`, `Denomination`, `VendingState` + 5 states | **State** (the whole design) | `synchronized` machine; change-making by bounded coin-change DP, not greedy | §14.2 |
| Coffee machine | `CoffeeMachine`, `Recipe`, `RecipeBook`, `IngredientStore`, `OrderResult` (sealed) | Observer (low stock), recipes as data | Two orders take the last milk → one `ReentrantLock` around check + deduct; notify outside the lock | §14.3 |
| ATM | `Atm`, `AtmState` + 4 states, `Transaction` (sealed), `NoteHandler`, `CashDispenser`, `BankService`, `Account` | State (session), Command (transactions), Chain of Responsibility (note cassettes) | Overdraw from two ATMs → per-account lock; transfers lock in id order | §14.4 |
| Elevator | `Elevator`, `StopList` (two sorted sets), `Door`, `Dispatcher`, `DispatchStrategy`, `Building` | Strategy (dispatch) + modes; LOOK scheduling | Per-car lock, synchronized dispatcher; tick-driven `step()` | §14.5 |
| Traffic signal | `Intersection`, `TrafficLight`, `Phase`, `SignalState` + 5 states, `TimingPolicy`, `SafetyMonitor` | State (timed), Strategy (timing), Observer (lights) | Synchronized tick; safe with a timer thread plus emergency threads | §14.6 |
| Shipping locker (Amazon Locker) | `LockerStation`, `Compartment`, `Booking`, `CompartmentSelector`, `PickupResult` (sealed), `LockerListener`, `LockerNetwork` | Strategy (door choice), Observer (SMS text messages), enum state machines, sealed results | Two carriers, one door → station lock around choose-and-mark; lazy expiry of holds and pickup windows; hashed single-use pickup and return drop-off codes, keypad lockout | §14.7 |

## Booking & inventory (Chapters 15–16)

| Problem | Key entities | Patterns used | Concurrency concern | Where |
|---|---|---|---|---|
| Library | `Book` (title) vs `BookItem` (copy), `Member`, `BookLending`, `LendingService`, `Catalog` | Strategy (fines), notifier interface; FIFO reservation queue per title | Last copy lent twice → lock member then copy (fixed order) | §15.1 |
| Movie tickets (BookMyShow) | `Show`, `ShowSeat`, `Booking`, `SeatLockProvider`, `BookingService`, `CancellationPolicy` | Strategy (refund rule), gateway behind an interface | Book `ShowSeat`s; all-or-nothing hold with TTL → pay → atomic commit; refund if hold lost | §15.2 |
| Concert tickets | `SeatedInventory`, `GaPool`, `SeatState` (sealed), `Hold`, `VirtualWaitingRoom`, `PurchaseLimiter` | Sealed seat states, waitlist | Extreme contention → CAS per seat, CAS counter for GA, FIFO waiting room, per-user limits, TTL holds with a reaper | §15.3 |
| Hotel | `Hotel`, `Room`, `RoomBooking`, `Invoice`, `RoomCharge`, `Payment` subclasses, `FrontDesk` | Template Method (`RoomCharge`) | Room × night reserved atomically in `synchronized Room.tryReserve` | §15.4 |
| Car rental | `Vehicle`, `DateRange`, `Reservation`, `Bill` / `BillItem`, `ReservationAddOn`, `PricingStrategy` | Template Method (add-ons), Strategy + Decorator (pricing), state machine | Availability is about **dates** → `Vehicle.tryBook(range)` synchronized | Ch 16 · Car Rental |
| Restaurant | `Menu` / `MenuSection` / `MenuItem`, `Table`, `Reservation`, `Order`, `Meal` / `MealItem`, `Kitchen`, `Check` | Observer (order ready), Strategy (chef assignment, payment), state machine | Two receptionists, one table → `Table.tryHold(time)` synchronized; listeners after the lock | Ch 16 · Restaurant |
| Airline | `Flight` vs `FlightInstance`, `Seat` vs `FlightSeat`, `FlightReservation`, `Itinerary`, `FareStrategy` | Strategy + Decorator (fares), Facade | Selling one seat twice → `FlightInstance.holdSeats` all-or-nothing under the instance lock | Ch 16 · Airline |
| Course registration | `Course` vs `Section`, `Student`, `RegistrationRule`s, `Registrar` | Chain of rules / Specification | Overselling a section → section lock inside a student lock (fixed order) | Ch 16 · Course Registration |
| Meeting scheduler | `Interval` (half-open), `Room` (owns a `TreeMap` calendar), `Meeting`, `Invitation`, `RoomSelectionStrategy`, `MeetingObserver`, `Recurrence` | Strategy (room ranking), Observer (invites, RSVPs) | Two organizers, overlapping times → check (`lowerEntry`) and insert under the room lock; rank then try each room; lock order meeting → room | Ch 16 · Meeting Scheduler |

## Games (Chapter 17)

| Problem | Key entities | Patterns used | Concurrency concern | Where |
|---|---|---|---|---|
| Blackjack + deck | `Card`, `Deck<T>`, `Shoe`, `Hand`, `BlackjackScoring`, `PlayerStrategy`, `DealerPolicy` | Strategy (player, dealer); injected randomness | One table = one thread; shared balances need an atomic wallet service | Ch 17 · Blackjack |
| Chess | `Piece` (sealed) + subclasses, `SlidingPiece`, `Board`, `Move`, `Rules`, `Game` | Polymorphic pieces, Factory Method, Command + Memento (undo) | `synchronized` game so a move and a resignation don't interleave | Ch 17 · Chess |
| Tic-tac-toe | `Board` (row/col/diagonal counters), `Game`, `Player`, `MoveStrategy` | Strategy (players) | Many games in a `ConcurrentHashMap`; per-game lock | Ch 17 · Tic-Tac-Toe |
| Snake & ladder | `Board`, `Jump`, `Dice`, `Rules`, `OverflowRule`, `WinStrategy`, `Game` | Strategy (dice, overflow, win) | Each game its own `Random` and its own thread of turns | Ch 17 · Snake and Ladder |
| 2048 | `LineMerger`, `Board`, `TileSpawner`, `Game`, `Direction` | Strategy (spawner); one pure merge function | Single-threaded; `synchronized move` if UI and input threads both call it | Ch 17 · 2048 |
| Jigsaw puzzle | `Side` (FLAT/TAB/BLANK + shape code), `Piece` (immutable), `Board` + `Placement` (piece, rotation), `PuzzleValidator`, `SolverStrategy` | Strategy (brute force vs edge-first with a side index); one placement check for players and solvers | Co-op players → `synchronized` game; solvers carry an attempt budget | Ch 17 · Jigsaw |

### Platforms (Chapters 18–19)

| Problem | Key entities | Patterns used | Concurrency concern | Where |
|---|---|---|---|---|
| Stack Overflow | `Post` (sealed) → `Question` / `Answer` / `Comment`, `Member`, `Tag`, `Bounty`, `QuestionIndex` | Strategy (reputation), Observer (badges), Facade; inverted index | Two votes at once → `synchronized` post; `AtomicInteger` reputation | §18.1 |
| Cricinfo | `Match` → `Innings` → `Over` → `Ball`, `Team`, `Playing11`, `StatQuery` | Template Method (stat queries), Observer (live pages) | Two scorers → `synchronized` match; observers in `CopyOnWriteArrayList` | §18.2 |
| LinkedIn | `Member`, `Profile`, `SocialContent`, `ConnectionInvitation`, `ConnectionService`, `SearchIndex`, `FeedService` | State (invitation), Strategy (feed ranking, notifications), Facade | Symmetric connections → all graph changes synchronized on the service | §18.3 |
| Facebook | `Member`, `Audience` (sealed), `PrivacyList`, `FriendService`, `VisibilityService`, `NewsFeedService` | Strategy (feed ranker), Observer (notifications) | Accept can't leave a one-way friendship → synchronized service, concurrent adjacency sets | §18.4 |
| Amazon | `Product`, `ShoppingCart`, `Order`, `OrderLog`, `Payment`, `Shipment`, `Customer` → `Guest` / `Member` | Strategy (payment), Observer (notifications), order state machine, Facade | Last unit, two buyers → `synchronized Product.reserve`; all-or-nothing multi-line reservation | Ch 19 · Amazon |
| Stock brokerage + exchange | `Order` → Market/Limit/StopLoss/StopLimit, `OrderPart`, `StockLot`, `OrderBook`, `MatchingEngine` | Observer (fills), Template Method (statements), trade-price policy | Over-spending → reserved funds; one lock per symbol; price-then-time matching | Ch 19 · Stock Brokerage |
| Auction | `AuctionListing`, `Bid`, `BidResult`, `AuctionService`, `AuctionListener` | Observer, Specification (search), status state machine | Concurrent bids → all bid rules in one per-listing critical section; notify after unlocking | Ch 19 · Auction |
| Music streaming (Spotify) | `Song` / `Album` / `Artist`, `MusicLibrary`, `Playlist`, `MusicPlayer`, `PlaybackOrder`, `Recommender` | Strategy (play order, recommendations), Observer (plays), plan rules in an enum | Two devices press "next" → `synchronized` player methods | Ch 19 · Music Streaming |
| Food delivery (Swiggy) + FoodKart | `Restaurant`, `Cart`, `Order`, `DeliveryAgent`, `DeliveryAssignmentStrategy`, `RestaurantSelectionStrategy` | Strategy (assignment, selection, payment), Observer, order state machine | Kitchen capacity reserved atomically; riders claimed with CAS | Ch 19 · Food Delivery |
| Grocery store (POS, point of sale) | `Register`, `Sale`, `LineItem`, `PricingEngine`, `ItemPromotion` / `BasketPromotion`, `TaxPolicy`, `Tender` (sealed), `Receipt`, `Inventory` | Strategy (offers, tax, payment gateway), Facade (register), Observer (stock), sale state machine | Registers share stock → atomic `merge`, may go negative, edge-triggered alerts; returns claimed with `compute` | Ch 19 · Grocery Store |

**Pattern-driven components (Chapter 20)**

| Problem | Key entities | Patterns used | Concurrency concern | Where |
|---|---|---|---|---|
| Splitwise | `Expense`, `SplitStrategy` (Equal/Exact/Percent), `Share`, `Ledger`, `DebtSimplifier` | Strategy (splits) | One signed number per pair, one mutator; `BigDecimal`, no lost cents | §20.1 |
| Notification service | `NotificationChannel` (abstract), channels, `SendRule`, `ChannelFactory`, `DeliveryListener` | Template Method (channels), Strategy (send rules), Observer, Simple Factory, Adapter | Rate-limit check-then-act gap; idempotency | §20.2 |
| In-memory cache / LRU | `Cache<K,V>`, `Node`, `DoublyLinkedList`, `EvictionPolicy`, `Cache.Builder`, `ShardedCache` | Strategy (eviction), Observer (removals), Builder | One lock (a `get` is a write in LRU); listeners outside the lock; shard to scale | §20.3 |
| Ride sharing (Uber) | `Rider`, `Driver`, `Trip`, `MatchingStrategy`, `PricingStrategy`, `SurgePricing` | Strategy ×3, Decorator (surge), trip state machine | Double-booking a driver → atomic claim (`AtomicBoolean`) | §20.4 |
| Logging framework | `Logger`, `LogManager`, `LogRecord`, `Appender`, `AsyncAppender`, `Formatter` | Chain of Responsibility (hierarchy), Singleton (`LogManager`), Strategy (formatters), Template Method (appender), Decorator (async) | Thread-safe sinks; never block on a full queue; never crash the app | §20.5 |
| Pub-sub | `Broker`, `Topic`, `Subscription`, `Subscriber`, `Message` | Observer through a broker | One dispatcher per subscription; ack after processing (at-least-once) → idempotent subscribers | §20.6 |
| Rate limiter | `Limiter` (TokenBucket, FixedWindow, SlidingWindowLog/Counter), `CompositeLimiter`, `LimiterFactory` | Strategy, Simple Factory, Composite, Facade | Atomic refill-check-take; one limiter per key via `computeIfAbsent`; Redis + Lua across servers | §20.7 |

**Machine-coding problems (Chapter 21)**

| Problem | Key entities | Patterns used | Concurrency concern | Where |
|---|---|---|---|---|
| Task management | `Task`, `TaskStatus`, `TaskService`, `TaskRepository`, `TaskFilter` | Specification (filters), Strategy (sorts), Observer, enum state machine | Per-task `synchronized` | §21.1 |
| Digital wallet | `Wallet`, `Transaction`, `Money`, `Offer`, `WalletService` | Strategy (offers), idempotency key | Lock both wallets in a global (id) order; debit before credit; total conserved | §21.2 |
| Event calendar (Flipkart) | `TimeSlot` (half-open), `Event`, `EventStore`, `RepresentativeSelector`, `CalendarService` | Strategy (representative), plan-then-save | Atomic check-and-book | §21.3 |
| Coding Blox (Flipkart) | `Contest`, `Level`, `User`, `Question`, `SolveSimulator`, `CommandProcessor` | Command map, Strategy (solver, scoring) | Atomic rating update; don't run a contest twice | §21.4 |
| PropertyHunt (ClearTrip) | `Property`, `SearchCriteria` (+ Builder), `PropertyService` | Builder + predicate list, Strategy (sort) | Synchronized mark-sold | §21.5 |
| Ledger Co (Navi) | `Loan`, `LumpSum`, `Balance`, `InterestPolicy`, `LedgerService` | Strategy (interest), derived state, command loop | Synchronized lump sums | §21.6 |
| Inventory management | `Product`, `Warehouse`, `StockMovement`, `AllocationStrategy`, `ReorderListener` | Strategy (allocation), Observer (edge-triggered alerts) | Overselling → lock per SKU | §21.7 |
| Unix file search (`find`) | `Node` → `FileNode` / `DirectoryNode`, `FileFilter`, `AndFilter` / `OrFilter` / `NotFilter`, `SearchCriteria` (+ Builder), `FileSearcher`, `FindCommandParser` | Specification + Composite (filters and file tree), Builder, recursive-descent parser | Weakly consistent children map; read-only fork/join walk | §21.8 |

**TL;DR — what every case study teaches**

- **Name the real unit of inventory** (copy, show seat, room-night, vehicle-day) (Ch 15 summary).
- **Every "if free then take" is one atomic step** — a monitor, a CAS, or a conditional `UPDATE` (Ch 15 summary).
- **Put rules on the object that owns the data**; the service only orders the steps (Ch 16, Ch 20 summaries).
- **Strategy** for the rule most likely to change; **Observer** for "tell someone when it happens"; **State** (or an enum state machine) for lifecycles.
- **Inject** time and randomness; money in `BigDecimal` or integer cents; call listeners **outside** locks.

---

## 22.10 Machine-coding and edge-case checklists

*Source: §21.0 (and §1.10).*

**Machine-coding checklist**

**Before coding (0–20 min)**

- [ ] Read the statement twice; mark **mandatory vs bonus**; ask clarifying questions.
- [ ] Put the statement's sample input into the driver as a `List<String>` of commands.
- [ ] Ask: "Where does it need to be thread-safe?" (wallets and inventory: always yes).

- [ ] Nouns → entities, verbs → service methods. Pick the 1–2 patterns the statement begs for ("can be sorted by X or Y" → Strategy).
- [ ] Create the six packages: **model, repository, service, strategy, exception, driver**.

**While coding (20–75 min)**

- [ ] Implement mandatory requirements **in the statement's order**; run the driver after each one (at least every 15 minutes).
- [ ] Entities guard their own invariants (e.g. `Task.changeStatus` rejects `TODO → DONE`).
- [ ] Services return data and throw domain exceptions; **no `System.out` in services**.
- [ ] Repository = interface + `ConcurrentHashMap` implementation; IDs from an `AtomicInteger`.
- [ ] Driver = `Map<String, Consumer<String[]>>` command processor; catch domain exceptions per command, **print and continue**.

**Before you stop (75–90 min)**

- [ ] Edge cases and validation done (see below); custom exceptions with readable messages.
- [ ] Thread-safety where money or stock is shared; say where the lock is and why.
- [ ] Bonus only after every mandatory feature runs.
- [ ] Clean names, delete dead code, run everything once more, prepare a 2-minute walkthrough.

**Rubric the interviewer uses (§21.0)**

| # | Criterion | How to score well |
|---|---|---|
| 1 | Working, demo-able code | Thin end-to-end path in the first hour; never finish with code that doesn't compile |
| 2 | Functional completeness | Mandatory first, in order |
| 3 | Clean models | Enums for fixed sets, records for values, classes for things that change |
| 4 | Separation of concerns | Driver → Service → Repository |
| 5 | Extensibility | New rule = new class (Strategy, Specification) |
| 6 | Edge cases & failure handling | Custom exceptions; the driver keeps going |
| 7 | Readability | Names from the statement's nouns and verbs |
| 8 | Concurrency (when asked) | Say where the lock is; demo with a thread pool if time allows |
| 9 | Tests (bonus) | A tiny `assert`-style self-check in `main` is enough |

### Edge-case checklist (§1.10, §21.0, case studies)

| Category | Questions to ask your code |
|---|---|
| Input | `null`, empty, negative, zero, duplicate IDs, unknown IDs, invalid numbers |
| Ownership | Wrong user acting on someone else's object; a player moving out of turn |
| Capacity | Full lot, zero seats left, empty playlist, max loans reached, party larger than any table |
| State | Paying twice, exiting without entering, cancelling a delivered order, moving after game over, running a contest twice |
| Time | Exit before entry, holds/bookings expiring, part hours (round up?), an event exactly at the end instant (half-open intervals), late returns |
| Money | `BigDecimal` scale and rounding; no lost cents in splits; compare with `compareTo`, not `equals` (Ch 4 money note) |
| Concurrency | Two threads booking the same seat/spot/rider; multi-item bookings locking in sorted order; retries that must be idempotent |
| Failure | Payment declined after a hold; hold lost before payment → compensating refund; listener throws |
| Randomness | Seeded `Random` or a scripted strategy so the demo is repeatable |

### Common mistakes (§21.0)

All covered by the checklist above and 22.2: a 20-minute "perfect" class diagram (type enums/records first — the diagram lives in the names), one `Manager` with 15 maps, printing inside services, `double` money, crashing on the first bad command, bonus before mandatory, unseeded `Random`, and `LocalDate.now()` in logic (inject a `java.time.Clock`).

---

## 22.11 Glossary

Alphabetical; "Where" is the section with the full explanation. Besides design vocabulary it covers the everyday Java, tooling and domain words the case studies take for granted (`JVM`, `Redis`, `SKU`, `UPI` …). Terms already defined in a cheat-sheet row above (principles, smells, patterns, concurrency tools) point there instead of being defined twice. System-design terms (latency, throughput, caching, brokers, eventual consistency) go deeper in System Design Fundamentals.

| Term | Plain meaning | Where |
|---|---|---|
| ABA problem | A value changes A → B → A, so a compare-and-swap cannot notice the change; fix with a version stamp (`AtomicStampedReference`) | §12.15 |
| Abstract class | OOP fact — see 22.3 | §2.9 |
| Abstract Factory | Creational pattern — see 22.7 | §8.11 |
| Abstraction | OOP pillar — see 22.3 | §2.4 |
| Access modifiers | OOP fact — see 22.3 | §2.14 |
| Activity diagram | UML flowchart of a process, with decisions, swimlanes and fork/join | §3.13 |
| Actor | Someone or something that uses the system (customer, admin, the system itself) | §1.5 |
| Adapter | Structural pattern — see 22.7 | §9.1 |
| Aggregate (DDD) | A cluster of objects changed as one unit; only its root is reachable from outside, and the root guards the invariants | Ch 5 §17 |
| Aggregation | UML relationship — see 22.3 | §3.3 |
| Anemic domain model | Classes with data but no behaviour; the logic lives in services that pull the data out | Ch 5 §5 |
| Anti-corruption layer | Translation layer at a bounded context's border that keeps a foreign or legacy model out of yours | Ch 5 §17 |
| Anti-pattern | A common "solution" that reliably causes more problems than it solves | §8.6 |
| AOP (aspect-oriented programming) | Adding cross-cutting behaviour (logging, transactions) around many methods without editing them, usually via generated proxies | Ch 5 §11, §9.7 |
| API | Application Programming Interface: the set of public methods (or endpoints) other code is allowed to call | §2.6, §2.14 |
| Appender / formatter | Logging roles: an appender writes a log record somewhere (console, file); a formatter turns it into text | §20.5 |
| Append-only log | Messages are only ever added at the end; readers track their own offset | §20.6 |
| Assertion | A check in code or a test that a condition is true, failing loudly if it is not | Ch 6 §8 |
| Association | UML relationship — see 22.3 | §3.2 |
| Association class | A class that holds data about a link itself (e.g. `Enrollment{grade}`) | §3.11 |
| AST (abstract syntax tree) | The tree a parser builds from an expression or program, one node per piece of grammar | §10.6 |
| Asynchronous vs synchronous | The caller carries on and gets the result later vs the caller waits until the work finishes | §3.12, §20.5 |
| At-least-once delivery | A message may be delivered more than once, never lost; consumers must be idempotent | §20.6 |
| Atomic variable | `AtomicInteger` / `AtomicLong` / `AtomicReference` — concurrency tool, see 22.8 | §12.15 |
| Atomicity / atomic operation | An operation that happens completely or not at all, with no other thread able to see it half-done | §12.5 |
| Audit log | An append-only record of who did what and when, kept for checking later | §10.2, Ch 21 |

| Term | Plain meaning | Where |
|---|---|---|
| Authentication vs authorization | Proving who you are (login, PIN) vs checking what you are allowed to do | Ch 6 §10, §14.4 |
| Back-pressure | Slowing producers when consumers can't keep up, e.g. with a bounded queue | §12.22 |
| BFS / DFS | Breadth-first search (visit neighbours level by level) vs depth-first search (go deep down one path, then back up) | §18.3, §21.8 |
| `BigDecimal` | Exact decimal type used for money; build from strings, compare with `compareTo` | §1.10, Ch 4 (money note) |
| Big-O notation | Shorthand for how running time or memory grows with input size: `O(1)` constant, `O(log n)`, `O(n)` linear | §20.3 |
| Blocking call | A call that makes the thread wait (for I/O, a lock or a queue) before it can continue | §12.4 |
| Bloom filter | Probabilistic set: may say "maybe present" wrongly, never says "absent" wrongly | §13.9 |
| Boilerplate | Repetitive code that must be written but carries no real logic (getters, `equals`, `hashCode`) | §2.13 |
| Bounded context | Boundary inside which one domain model and one vocabulary apply consistently (often one team or service) | Ch 5 §17 |
| Bounded queue | A queue with a fixed capacity; producers wait or are refused when it is full | §12.22 |
| Boy Scout Rule | Clean-code rule — see 22.5 | Ch 6 (intro) |
| Bridge | Structural pattern — see 22.7 | §9.2 |
| Broker | A middleman service that receives messages from publishers and delivers them to subscribers | §20.6 |
| Build tool | A program (Maven, Gradle) that downloads dependencies, compiles, tests and packages the code | §3.5 |
| Builder | Creational pattern — see 22.7 | §8.12 |
| Busy waiting (spinning) | A thread repeatedly checking a condition in a loop instead of sleeping; wastes CPU | §12.6 |
| Bytecode | The compact instruction format `javac` produces and the JVM runs | Ch 12 |
| Callback | A function you hand to other code so it can call you back when something happens | Ch 5 §10 |
| Caretaker / Originator | Memento roles: the originator makes and restores snapshots; the caretaker stores them without looking inside | §10.5 |
| Carrier thread | The real OS thread that a virtual thread runs on while it is not blocked | §12.4 |
| CAS (compare-and-swap) | Atomic "set to new value only if it still equals the expected value" | §12.15 |
| Chain of Responsibility | Behavioral pattern — see 22.7 | §10.1 |
| Characterisation test | A test that records what existing code does now, before you refactor it | §7.8 |
| Checked vs unchecked exception | Checked exceptions must be declared or caught (compiler-enforced); unchecked ones (`RuntimeException` subclasses) need not be | §2.5, Ch 6 §6 |
| Check-then-act | Read a condition, then act on it, as two steps: a race unless made atomic | §12.5, §12.25 |

| Term | Plain meaning | Where |
|---|---|---|
| Circuit breaker | Stops calling a failing dependency for a while after repeated errors, then tries again carefully | Ch 5 §13 |
| Circular wait | T1 waits for T2's lock while T2 waits for T1's; one of the four deadlock conditions | §12.17 |
| Class | A blueprint for objects: fields plus methods | §2.2 |
| Class diagram | UML diagram of classes, fields, methods and relationships — "the answer" in an LLD round | §3.11 |
| Class explosion | Subclasses multiplying across two dimensions ( `WindowsButton` , `MacButton` , …) | §9.2 |
| Classpath | The list of folders and JAR files where the JVM looks for classes | §1.3 |
| Clean code | Code whose intent is obvious to the next reader and which is easy to change safely | Ch 6 (intro) |
| Coarse- vs fine-grained locking | One big lock for everything (simple, slow under load) vs many small locks per item (faster, harder to get right) | §12.11 |
| Code review | A peer reads a change before merge: correctness first, then design, readability, tests, security, performance | Ch 6 §15 |
| Code smell | A surface symptom that usually signals a deeper design problem; not a bug by itself | §7.2 |
| Coffman conditions | Deadlock — see 22.8 | §12.17 |
| Cohesion | How closely the things inside one class belong together; high is good | §2.16, §4.3 |
| Coin-change DP | Finding the fewest coins (or whether change is possible) by building answers for smaller amounts first | §14.2 |
| Collections framework | Java's built-in data structures ( `List` , `Set` , `Map` , `Queue` ) and their implementations | Ch 2 |
| Command | Behavioral pattern — see 22.7 | §10.2 |
| Command–Query Separation (CQS) | A method either changes state or returns a value, not both | Ch 6 §2 |
| Commit / rollback | Make a transaction's changes permanent vs throw all of them away | §10.5, §15.2 |
| `Comparator` / `Comparable` | An object that orders two values from outside vs a class's own natural order ( `compareTo` ) | §1.9, §11.3 |
| Compensation | Undoing a completed step with a new action (e.g. a refund) because money can't be rolled back | §15.2 |
| Compile time vs run time | When `javac` checks and translates the code vs when the program is actually executing | §2.7 |
| `CompletableFuture` | Concurrency tool — see 22.8 | §12.4 |
| Composite | Structural pattern — see 22.7 | §9.3 |
| Composition | UML relationship — see 22.3 | §3.4 |
| Composition over inheritance | Principle — see 22.5 | §2.11, Ch 5 §6 |
| Composition root | The one place (usually `main` ) where objects are created and wired together | Ch 6 §10 |
| Compound interest | Interest charged on the principal plus interest already added | §21.6 |
| Concurrency | Several tasks in progress at once, possibly sharing data | §12.1 |

| Term | Plain meaning | Where |
|---|---|---|
| Concurrency vs parallelism | Many tasks in progress (structure) vs many executing at the same instant (hardware) | §12.1 |
| `ConcurrentHashMap` | Concurrency tool — see 22.8 | §12.16 |
| Condition variable | Concurrency tool — see 22.8 | §12.13 |
| Constructor | Special method that runs when an object is created and sets up its fields | §2.2 |
| Constructor chaining | One constructor calling another with `this(...)` | §2.2 |
| Contention | Many threads competing for the same lock or variable at once, making them wait | §12.7, §12.11 |
| Context switch | The OS pausing one thread and resuming another on a CPU core; costs time | §12.1 |
| Controller (GRASP) | GRASP guideline — see 22.5 | Ch 5 §16.3 |
| `CopyOnWriteArrayList` | Concurrency tool — see 22.8 | §12.16 |
| `CountDownLatch` | Concurrency tool — see 22.8 | §12.14 |
| Coupling | How much one class depends on another's details; low is good | §2.16, §4.4 |
| CPU | The processor; each core runs one thread at a time | §12.1 |
| CQRS | Command Query Responsibility Segregation: separate models for writing data and for reading it | Ch 5 §17 |
| Creator (GRASP) | GRASP guideline — see 22.5 | Ch 5 §16.2 |
| Critical section | Code that touches shared state and must run in one thread at a time | §12.5 |
| Cron | A schedule format (minute, hour, day…) for running a job at fixed times | §10.6 |
| Cross-cutting concern | Behaviour needed in many places (logging, security, transactions) that doesn't belong to one class | §4.5, Ch 6 §10 |
| CRUD | Create, Read, Update, Delete: the four basic storage operations | Ch 5 §17 |
| CSV | Comma-separated values: a plain-text table, one row per line | Ch 5 §3 |
| `CyclicBarrier` | Concurrency tool — see 22.8 | §12.14 |
| Daemon thread | A background thread that does not keep the JVM alive once all normal threads end | §12.4 |
| DAO (Data Access Object) | A class that wraps the SQL for one table; closer to the database than a Repository | §11.7.3 |
| Data Class (smell) | Code smell — see 22.6 | §7.2.22 |
| Data Clumps (smell) | Code smell — see 22.6 | §7.2.10 |
| Database transaction | A group of database changes that succeed or fail together | §12.25, Ch 15 |
| Dead code | Code that is never run or whose result is never used; delete it | Ch 6 §13, Ch 7 |
| Dead-letter queue | Where messages go after retries are exhausted, for later inspection | §20.6 |
| Deadlock | Concurrency hazard — see 22.8 | §12.17 |
| Debit / credit | Taking money out of an account vs putting money in | §14.4, §21.2 |
| Decorator | Structural pattern — see 22.7 | §9.4 |
| Deep vs shallow copy | Copying nested objects too vs sharing them with the original | §8.13 |
| Defensive copy | Copying a mutable object passed in or handed out so outsiders can't change your internals | §2.13 |

| Term | Plain meaning | Where |
|---|---|---|
| Delegation | An object handing a job to another object it holds, instead of doing it itself | §2.11, Ch 5 §6 |
| Dependency | UML relationship — see 22.3 | §3.5 |
| Dependency injection (DI) | Pattern — see 22.7 | §4.10, §11.7.2 |
| Dependency Inversion Principle | SOLID principle — see 22.4 | §4.10 |
| `Deque` / `ArrayDeque` | Double-ended queue: add and remove at both ends; the usual Java stack and queue | §7.7 |
| Derive, don't store | Compute counts and totals from the underlying records so they can't drift | §18.5 |
| Design by Contract | Principle — see 22.5 | Ch 5 §15 |
| Design pattern | A named, reusable solution to a recurring design problem | §8.1 |
| Director | Optional Builder role that runs a fixed sequence of building steps | §8.12 |
| Distributed lock | A lock shared by several servers, kept in an external store (database, Redis) | §12.25 |
| Divergent Change (smell) | Code smell — see 22.6 | §7.2.7 |
| Domain event | Past-tense record of a business fact ( `OrderPlaced` ) that other parts of the system react to | Ch 5 §17 |
| Domain exception | A custom exception named after a business rule ( `InsufficientBalanceException` ) | §21.0 |
| Domain-Driven Design (DDD) | Principle — see 22.5 | Ch 5 §17 |
| Double dispatch | Choosing a method by both the element's and the visitor's type (Visitor's `accept` ) | §11.5 |
| Double-checked locking | Lazy singleton that checks, locks, checks again; the field must be `volatile` | §8.8 |
| Driver (machine coding) | The `main` class that feeds commands to the services and prints results | §1.10, §21.0 |
| DRY | Don't Repeat Yourself — principle, see 22.5 | Ch 5 §1 |
| DTO (Data Transfer Object) | A plain data holder used only to carry values across a boundary (API, layer) | §2.3 |
| Dynamic dispatch | The JVM picks an overridden method from the object's actual class at run time | §2.6–§2.7 |
| Dynamic programming (DP) | Solving a problem by combining saved answers to smaller sub-problems | §14.2 |
| Edge-triggered alert | Fires once when a value crosses a threshold, not on every update while it stays past it | Ch 19 · Grocery Store, §21.7 |
| EMI | Equated monthly instalment: the fixed amount repaid each month on a loan | §21.6 |
| Encapsulate what varies | Principle — see 22.5 | Ch 5 §8 |
| Encapsulation | OOP pillar — see 22.3 | §2.3 |
| Entity (DDD) | Object defined by an identity that survives changes; two entities are equal when their ids are equal | Ch 5 §17 |
| Enum (with behaviour) | A fixed set of named constants that can carry fields and methods | §2.10 |

| Term | Plain meaning | Where |
|---|---|---|
| `EnumMap` / `EnumSet` | Very fast map and set implementations whose keys are the constants of one enum | §2.10 |
| `equals` / `hashCode` contract | Equal objects must have equal hash codes | §2.12 |
| Event Sourcing | Pattern — see 22.7 | §11.7.6 |
| Event-driven | Parts of the system react to published events instead of calling each other directly | Ch 5 §9 |
| Eventual consistency | Copies of data may disagree for a short time but converge once updates stop | Ch 5 §17 |
| Eviction | Removing entries from a cache to make room (LRU, LFU, FIFO) | §20.3 |
| Exception | An object thrown to signal an error, which unwinds the call stack until something catches it | Ch 6 §6 |
| `ExecutorService` | Concurrency tool — see 22.8 | §12.4 |
| Exponential backoff | Waiting longer after each failed retry (1 s, 2 s, 4 s…), usually with jitter | Ch 6 §16 |
| Expression problem | Visitor makes adding operations cheap but adding types expensive (and the reverse for plain polymorphism) | §11.5 |
| Facade | Structural pattern — see 22.7 | §9.5 |
| Factory Method | Creational pattern — see 22.7 | §8.10 |
| Fail fast | Principle — see 22.5 | Ch 5 §13 |
| Fairness | Serving waiting threads roughly in order so none starves; costs throughput | §12.19 |
| Fan-out | One event or post copied to many receivers (followers' feeds, subscribers) | §18.4, §20.6 |
| Feature Envy (smell) | Code smell — see 22.6 | §7.2.9 |
| Field | A variable that belongs to an object (instance field) or to the class (`static` field) | §2.2 |
| FIFO / LIFO | First in, first out (a queue) vs last in, first out (a stack) | §12.8, Ch 15 |
| F.I.R.S.T. | Clean-test rule — see 22.5 | Ch 6 §8 |
| Flag argument | A boolean parameter that makes a function do two things; split it instead | Ch 6 §2 |
| Flyweight | Structural pattern — see 22.7 | §9.6 |
| Fork/Join | Split a task recursively into subtasks run in parallel, then combine results | §12.26, §13.10 |
| Fragile base class | A change in a parent class silently breaks subclasses | Ch 5 §6 |
| Framework | A library that owns the program's flow and calls your code at set points (Spring, JUnit) | Ch 5 §10 |
| Functional interface | An interface with one abstract method, usable with lambdas | §2.8 |
| Functional requirement | A behaviour the system must have ("a customer takes a ticket") | §1.5 |
| GA (general admission) | Unnumbered standing or seating tickets sold as a count, not as named seats | §15.3 |
| Garbage collection (GC) | The JVM automatically freeing memory of objects nothing refers to any more | Ch 12 |
| Generalization | UML relationship — see 22.3 | §3.6 |

| Term | Plain meaning | Where |
|---|---|---|
| Generics | OOP fact — see 22.3 | §2.15 |
| Geohash | Encodes a latitude/longitude as a short string; nearby places share a prefix, so it works as a search key | §20.4 |
| Getter / setter | Methods that read (`getX`) or change (`setX`) a field | §2.3 |
| Git | The version-control tool that records every change to the code as commits | §4.2 |
| Glob | Filename wildcard pattern: `*` any characters, `?` one character, `[0-9]` one character from a set | §21.8 |
| Global Data (smell) | Code smell — see 22.6 | §7.2.5 |
| God class / God Object | Anti-pattern — see 22.7 | §8.6, §7.2.20 |
| GoF | "Gang of Four": the four authors of the 1994 book that catalogued the 23 classic design patterns | §8.2 |
| Golden Hammer | Anti-pattern — see 22.7 | §8.6 |
| Graph | Nodes joined by edges (people and connections, stops and routes) | §18.3 |
| GRASP | Principle — see 22.5 | Ch 5 §16 |
| Greedy algorithm | Always take the best-looking choice right now; fast, but not always correct (e.g. coin change) | §14.2 |
| GST | Goods and Services Tax, India's sales tax | Ch 19 · Grocery Store |
| Guard clause | An early check-and-return/throw at the top of a method instead of nested `if`s | §7.3.5, Ch 5 §13 |
| Half-open interval | A time range `[start, end)` that includes the start but not the end, so back-to-back bookings don't clash | §15.4, §21.3 |
| Happens-before | Java Memory Model rule guaranteeing one action's effects are visible to another | §12.6 |
| Hash function / hash code | Turns a value into a number used to pick a bucket; equal values must give equal hashes | §2.12 |
| Hash table / `HashMap` | Key-value store that finds an entry in about constant time by hashing the key | §2.12 |
| Heap vs stack (memory) | Where objects live (shared by all threads) vs each thread's own area for local variables and calls | §12.2 |
| Heuristic | A rule of thumb that is usually right and cheap to apply, not a guarantee | Ch 6 §14 |
| Hexagonal architecture | Ports and adapters: the domain core defines interfaces; databases, UIs and vendors plug in from outside | §20.4 |
| High Cohesion (GRASP) | GRASP guideline — see 22.5 | Ch 5 §16.5 |
| HLD vs LLD | Many services and machines vs classes inside one application | §1.2 |
| Hold with TTL | A temporary claim (e.g. seats during payment) that expires automatically | §15.2 |
| Holder-class idiom | Lazy singleton created inside a nested static class; the JVM's class loading makes it thread-safe | §8.8 |
| Hollywood Principle | Principle — see 22.5 | Ch 5 §10 |
| HTTP | The request/response protocol browsers and web services use | §3.12 |

| Term | Plain meaning | Where |
|---|---|---|
| IDE | Integrated development environment (IntelliJ, Eclipse): editor plus compiler, debugger and refactoring tools | Ch 7 |
| Idempotency | Running an operation twice has the same effect as once; an idempotency key enforces it | §21.2, §20.6 |
| Identity vs equality | Same object in memory ( `==` ) vs same value ( `equals` ) | §2.12 |
| Immutability | Principle — see 22.5 | §2.13, Ch 5 §14 |
| Indirection (GRASP) | GRASP guideline — see 22.5 | Ch 5 §16.8 |
| Information Expert (GRASP) | GRASP guideline — see 22.5 | Ch 5 §16.1 |
| Inheritance | OOP pillar — see 22.3 | §2.5 |
| In-memory | Data kept only in the program's RAM, lost when it stops; the default in LLD rounds | §1.4 |
| Insider Trading (smell) | Code smell — see 22.6 | §7.2.19 |
| Instance | One object created from a class | §2.2 |
| Integration test | Tests several real parts together (e.g. service plus database), slower than a unit test | Ch 6 §17 |
| Interface | A contract of methods; a class can implement many | §2.8 |
| Interface Segregation Principle | SOLID principle — see 22.4 | §4.9 |
| Interning | Keeping one shared copy of each equal value ( `String.intern()` ) | §9.6 |
| Interpreter | Behavioral pattern — see 22.7 | §10.6 |
| Interrupt | A polite request to a thread to stop what it is doing; blocking calls react by throwing `InterruptedException` | §12.3 |
| Intrinsic vs extrinsic state | Flyweight's shared, immutable part vs the per-use part passed in by the caller | §9.6 |
| Invariant | A fact about an object that must always be true ("balance is never negative") | Ch 5 (key terms), §15 |
| Inversion of Control (IoC) | The framework controls the flow and calls your code; DI is one way to achieve it | Ch 5 §10, §4.14 |
| Inverted index | A map from each word/tag to the list of items containing it, for search | §18.1 |
| Invoker / Receiver | Command roles: the invoker triggers commands; the receiver does the real work | §10.2 |
| I/O | Input/output: reading or writing files, network or database; slow compared with CPU work | §12.4 |
| Iterator | Behavioral pattern — see 22.7 | §10.3 |
| JAR | Java archive: a zip file of compiled classes and resources | §4.7 |
| Java Memory Model (JMM) | Rules for when one thread's writes become visible to another | §12.6 |
| `java.time` | Java's date-time API: `Instant` (a moment), `Duration`, `LocalDate`, and `Clock` (injectable "now") | §1.10 |
| JDBC | Java's standard API for talking to relational databases | §8.11 |
| JDK | Java Development Kit: compiler, tools and the standard library | Ch 2 |
| JIT compiler | Part of the JVM that turns hot bytecode into fast machine code while the program runs | Ch 12 |

| Term | Plain meaning | Where |
|---|---|---|
| Jitter | Random extra delay added to retries so many clients don't retry at the same instant | Ch 6 §16, §12.18 |
| JPA / Hibernate | Java's standard for mapping objects to database tables, and its most popular implementation | §9.7 |
| JSON | A text format for structured data: objects in `{}`, lists in `[]` | §3.2 |
| JUnit | The standard Java unit-testing framework | Ch 6 §8 |
| JVM | Java Virtual Machine: runs compiled Java bytecode and manages memory and threads | Ch 2, Ch 12 |
| Kafka | A distributed append-only log used as a durable message broker | §20.6 |
| KISS | Keep It Simple — principle, see 22.5 | Ch 5 §2 |
| KYC | Know Your Customer: identity checks a financial app must do before allowing higher limits | §21.2 |
| Lambda | A short anonymous function, e.g. `x -> x * 2`, usable wherever a functional interface is expected | §2.8 |
| Large Class (smell) | Code smell — see 22.6 | §7.2.20 |
| Latency | How long one operation takes from start to finish | §1.4 |
| Law of Demeter | Principle — see 22.5 | Ch 5 §4 |
| Lazy Element (smell) | Code smell — see 22.6 | §7.2.14 |
| Lazy expiry | Deleting an expired entry only when someone next touches it, instead of by a timer | §13.6, §14.7 |
| Lazy initialization | Creating an object only when it is first needed | §8.8 |
| Lease | A lock or hold that expires automatically after a set time unless renewed | §12.9 |
| Least Astonishment | Principle — see 22.5 | Ch 5 §12 |
| Ledger | An append-only list of money movements from which balances are derived | §20.1, Ch 21 |
| Legacy code | Existing code, often without tests, that is risky to change | §7.8 |
| LFU / MRU | Cache eviction that removes the least frequently used vs the most recently used entry | §20.3 |
| Linked list | A chain of nodes where each points to the next (doubly linked: also the previous) | §20.3 |
| `LinkedHashMap` | A `HashMap` that also remembers insertion or access order; the shortcut for an LRU cache | §20.3 |
| Linter / static analysis | Tools that read code without running it and flag likely bugs or style issues | Ch 6 §15 |
| Liskov Substitution Principle | SOLID principle — see 22.4 | §4.8 |
| Listener | An object registered to be told when an event happens (the observer in Observer) | §11.1 |
| Livelock | Concurrency hazard — see 22.8 | §12.18 |
| Lock ordering | Always acquire locks in one global order to prevent circular wait | §12.17 |
| Lock striping | Guarding a map with an array of locks, each covering some buckets | §13.7 |
| Lombok | A library that generates boilerplate (getters, builders) from annotations at compile time | §8.12 |

| Term | Plain meaning | Where |
|---|---|---|
| Long Function (smell) | Code smell — see 22.6 | §7.2.3 |
| Long Parameter List (smell) | Code smell — see 22.6 | §7.2.4 |
| `LongAdder` | Concurrency tool — see 22.8 | §12.15 |
| LOOK scheduling | Elevator keeps moving in one direction while stops remain ahead, then reverses | §14.5 |
| Loops (smell) | Code smell — see 22.6 | §7.2.13 |
| Loose coupling | Principle — see 22.5 | Ch 5 §9 |
| Lost wakeup | A signal sent before the waiter started waiting, so the waiter sleeps forever | §12.13 |
| Low Coupling (GRASP) | GRASP guideline — see 22.5 | Ch 5 §16.4 |
| LRU cache | Evicts the least recently used entry; hash map + doubly linked list gives O(1) | §20.3 |
| Lua | A small scripting language; Redis runs a Lua script as one atomic step | §20.7 |
| Machine-coding round | LLD round where working, runnable in-memory code is the answer (90–120 min) | §1.2, §21.0 |
| Market vs limit order | Buy or sell now at the best available price vs only at a given price or better | Ch 19 · Stock Brokerage |
| Matching engine | Per-symbol order book matching buy and sell orders by best price, then oldest first | Ch 19 · Stock Brokerage |
| MDC | Mapped Diagnostic Context: per-thread key-values (request id, user) automatically added to log lines | §20.5 |
| Mediator | Behavioral pattern — see 22.7 | §10.4 |
| Memento | Behavioral pattern — see 22.7 | §10.5 |
| Memory leak | Memory that is never freed because something still holds references to objects no longer needed | §11.1, §12.24 |
| Message Chains (smell) | Code smell — see 22.6 | §7.2.17 |
| Message queue | A buffer service where producers leave messages for consumers to process later | §20.6 |
| Method | A function that belongs to a class | §2.2 |
| Method reference | Shorthand for a lambda that just calls a method, e.g. `String::length` | §2.8 |
| Microservice vs monolith | Many small independently deployed services vs one deployable application | Ch 5 §17 |
| Middle Man (smell) | Code smell — see 22.6 | §7.2.18 |
| Mixin | A small interface with `default` methods that adds a capability to many classes | §4.3 |
| Mock / stub / fake | Test doubles: a mock checks calls were made, a stub returns canned answers, a fake is a simple working version | Ch 6 §17 |
| Mockito | The most common Java library for creating mocks in tests | Ch 6 §8 |
| Module / package | A group of related classes; in Java a package is a folder-level namespace | §2.14 |
| Monitor | The lock built into every Java object, used by `synchronized` and `wait/notify` | §12.7 |

| Term | Plain meaning | Where |
|---|---|---|
| Multiplicity | How many objects take part at each end of a relationship (`1`, `0..1`, `*`, `1..*`) | §3.2 |
| Mutable Data (smell) | Code smell — see 22.6 | §7.2.6 |
| Mutation testing | Deliberately breaking the code in small ways to check the tests notice | Ch 6 §17 |
| Mutex | A lock that lets only one thread into a critical section | §12.7 |
| MVC | Model–View–Controller — pattern, see 22.7 | §11.7.5 |
| Mysterious Name (smell) | Code smell — see 22.6 | §7.2.1 |
| Non-functional requirement | A quality: thread safety, extensibility, in-memory storage, latency | §1.5 |
| Null Object | Pattern — see 22.7 | §11.7.1 |
| `NullPointerException` (NPE) | Thrown when you call a method or read a field through a `null` reference | Ch 5 §13 |
| Object | One instance of a class with real values | §2.2 |
| Object Pool | Creational pattern — see 22.7 | §8.14 |
| Observer | Behavioral pattern — see 22.7 | §11.1 |
| Offset | A position number in a log or list; each reader remembers its own | §20.6 |
| OOA / OOD / OOP | Object-oriented analysis / design / programming — see 22.3 | §2.17 |
| Open/Closed Principle | SOLID principle — see 22.4 | §4.7 |
| Optimistic locking | Read with a version; the write succeeds only if the version is unchanged | §12.10, §12.25 |
| `Optional` | A container that may be empty; return it when "not found" is normal | §1.10 |
| Order book | Per-stock lists of waiting buy and sell orders, sorted by price then time | Ch 19 · Stock Brokerage |
| OTP | One-time password: a short code valid for a single use, usually sent by SMS | Ch 5 §13, §20.2 |
| Outbox (transactional) | Save events in a table in the same transaction as the data, and publish them afterwards, so neither is lost | Ch 5 §17 |
| `OutOfMemoryError` | Thrown when the JVM cannot find memory for a new object | §12.2 |
| Out-of-scope list | Written list of what you will not build; shrinks the problem | §1.5 |
| Overloading | OOP fact — see 22.3 | §2.7 |
| Overriding | OOP fact — see 22.3 | §2.7 |
| Payment gateway | An external service that actually charges cards or UPI; hidden behind your own interface | §15.2, Ch 19 |
| PECS | OOP fact — see 22.3 | §2.15 |
| Pessimistic locking | Lock a row before changing it (e.g. `SELECT … FOR UPDATE`) so nobody else can touch it meanwhile | §12.5, §15.2 |
| Phaser | Concurrency tool — see 22.8 | §12.14, §12.26 |
| PII | Personally identifiable information (names, phone numbers) that must not leak into logs | Ch 6 §15 |
| Pinning | A virtual thread stuck on its carrier thread while it blocks | §12.4 |
| Pipeline | Data flowing through a sequence of steps, each feeding the next (streams, filters, middleware) | §7.2.13 |

| Term | Plain meaning | Where |
|---|---|---|
| PLU code | Four-digit price look-up code for loose produce, keyed at the till together with the scale weight | Ch 19 · Grocery Store |
| Plugin | A piece of code added to a system through a known interface without editing the system | §4.7 |
| PNR | Passenger name record: the booking reference for an airline reservation | Ch 16 · Airline |
| Poison pill | A special message telling a consumer to stop; one per consumer | §12.22 |
| Polymorphism | OOP pillar — see 22.3 | §2.6, Ch 5 §16.6 |
| POS | Point of sale: the till where a sale is rung up and paid | Ch 19 · Grocery Store |
| Precondition / postcondition | What must be true before a call (caller's duty) / what the method promises after it | §4.8, Ch 5 §15 |
| Primitive Obsession (smell) | Code smell — see 22.6 | §7.2.11 |
| `PriorityQueue` | A queue that always hands out the smallest (or highest-priority) element first; a binary heap inside | Ch 16 · Course Registration |
| Process | A running program with its own memory; it contains one or more threads | §12.2 |
| Producer–consumer | Threads hand work through a (bounded) queue | §12.22 |
| Program to an interface | Principle — see 22.5 | Ch 5 §7 |
| Protected Variations (GRASP) | GRASP guideline — see 22.5 | Ch 5 §16.9 |
| Prototype | Creational pattern — see 22.7 | §8.13 |
| Proxy | Structural pattern — see 22.7 | §9.7 |
| Pub-sub | Observer through a broker: publishers and subscribers don't know each other | §20.6 |
| Pure Fabrication (GRASP) | GRASP guideline — see 22.5 | Ch 5 §16.7 |
| Pure function | A function whose result depends only on its inputs and which changes nothing else | §11.7.6, Ch 17 · 2048 |
| Qualified association | An association looked up by a key (`Library [isbn] → Book`, i.e. a `Map`) | §3.11 |
| Race condition | The result depends on the timing of threads touching shared mutable state | §12.5 |
| Rate limiter | Caps how many requests a key may make per time window (token bucket, windows) | §13.11, §20.7 |
| Read-write lock | Many readers at once, or one writer | §12.10 |
| Realization | UML relationship — see 22.3 | §3.7 |
| Record | OOP fact — see 22.3 | §2.13 |
| Recursion | A function that solves a problem by calling itself on a smaller part | §9.3 |
| Recursive-descent parser | A parser with one method per grammar rule, each calling the others | §10.6, §21.8 |
| Redis | A fast shared in-memory key-value store, often used for caches, counters and distributed locks | §20.7 |
| Reentrancy | A thread may re-acquire a lock it already holds | §12.8 |
| `ReentrantLock` | Concurrency tool — see 22.8 | §12.8 |

| Term | Plain meaning | Where |
|---|---|---|
| Refactoring | Refactoring — see 22.6 | §7.1 |
| Reflection | Code inspecting or calling classes, fields and methods at run time by name | §8.8 |
| Refused Bequest (smell) | Code smell — see 22.6 | §7.2.23 |
| Regex | Regular expression: a pattern language for matching text | §10.6 |
| Repeated Switches (smell) | Code smell — see 22.6 | §7.2.12 |
| Repository | Pattern — see 22.7 | §11.7.3 |
| Requirement | One thing the system must do, written so you can check it | §1.5 |
| REST | A style of HTTP API built around resources (URLs) and verbs (GET, POST…) | Ch 5 §1 |
| Retry | Trying a failed operation again; safe only if the operation is idempotent | Ch 6 §16 |
| Role interface | A small interface serving one kind of caller (the ISP fix) | §4.9 |
| Rule of three | Refactor to an abstraction at the second or third case, not the first | §4.15 |
| `RuntimeException` | Base class of unchecked exceptions | §2.5 |
| Saga | A long business process split into local steps, each with a compensating action if a later step fails | §3.12, Ch 15 |
| Scalability | How well a system keeps working as users, data or load grow | §1.14 |
| `ScheduledExecutorService` | Runs tasks after a delay or repeatedly at a fixed rate | §13.6 |
| Schema | The defined structure of stored data (tables, columns, types) | §2.17 |
| SDK | Software development kit: a vendor's ready-made library for calling its service | §9.1 |
| Sealed type | A type whose permitted subtypes are fixed, enabling exhaustive `switch` | §11.5 |
| Seam | A place where you can swap behaviour (e.g. an interface) without editing the code around it; used for testing | §2.9, §7.7 |
| Seed | The starting number for a pseudo-random generator; the same seed gives the same sequence | Ch 17 |
| Semaphore | Concurrency tool — see 22.8 | §12.12 |
| Separation of Concerns | Principle — see 22.5 | §4.5, Ch 5 §11 |
| Sequence diagram | UML diagram of messages between objects over time | §3.12 |
| Serialization | Turning an object into bytes or text (and back) for storing or sending | §8.8 |
| Service Locator | Anti-pattern where a class asks a registry for its dependencies, hiding them | §11.7.2 |
| Sharding | Splitting data across independent parts (each with its own lock) to scale | §20.3 |
| Shotgun Surgery (smell) | Code smell — see 22.6 | §7.2.8 |
| Side effect | Anything a function does besides returning a value (changing a field, writing a file) | Ch 5 §12, Ch 6 §2 |
| Signaling | One thread tells another that something has happened (semaphore, latch) | §12.20 |

| Term | Plain meaning | Where |
|---|---|---|
| Simple Factory | Creational pattern — see 22.7 | §8.9 |
| Single Responsibility Principle | SOLID principle — see 22.4 | §4.2 |
| Singleton | Creational pattern — see 22.7 | §8.8 |
| SKU | Stock-keeping unit: one sellable product variant tracked in inventory | §21.7 |
| SLF4J | A logging facade: code logs through its API and any logging library can sit underneath | §9.5 |
| Sliding window | Rate-limit algorithm counting requests in a moving time window (log or counter) | §20.7 |
| SMS / SMTP | Text messages to phones / the protocol for sending email | §20.2 |
| Snapshot | A copy of state at one moment, used to restore it or to read it safely | §10.5 |
| SOLID | SRP, OCP, LSP, ISP, DIP — see 22.4 | §4.1 |
| Specification | Pattern — see 22.7 | §11.7.4 |
| Speculative Generality (smell) | Code smell — see 22.6 | §7.2.15 |
| Split tender | Paying one bill with several payment methods (e.g. part UPI, part cash) | Ch 19 · Grocery Store |
| Spring | The most widely used Java application framework; creates and wires objects (DI), web, data access | §4.14, §11.7.2 |
| Spurious wakeup | A waiting thread wakes with no signal; why `wait()` sits in a `while` loop | §12.13 |
| SQL | The language for querying and updating relational databases | §11.7.3 |
| Stack trace | The list of method calls active when an exception was thrown; shows where it came from | Ch 6 §6 |
| `StampedLock` | Concurrency tool — see 22.8 | §12.10 |
| Starvation | Concurrency hazard — see 22.8 | §12.19 |
| State (pattern) | Behavioral pattern — see 22.7 | §11.2 |
| State machine diagram | UML diagram of one object's states and transitions ( `event [guard] / action` ) | §3.14 |
| Stateless vs stateful | Keeps no data between calls (safe to share) vs remembers data between calls | §1.6 |
| `static` | Belongs to the class, one per class, not to each object | §2.2 |
| Stop-loss / stop-limit | Orders that wake up when the price hits a trigger, then sell at market / at a limit price | Ch 19 · Stock Brokerage |
| Strategy | Behavioral pattern — see 22.7 | §11.3 |
| Stream (Java) | A pipeline over a collection ( `filter` , `map` , `collect` ) that states what, not how | §2.8, §7.2.13 |
| Subclass / superclass | The child class that `extends` / the parent it extends | §2.5 |
| `synchronized` | Concurrency tool — see 22.8 | §12.7 |
| TCP | The internet protocol that delivers a reliable, ordered byte stream between two machines | §11.2 |

| Term | Plain meaning | Where |
|---|---|---|
| Technical debt | Shortcuts in code that make future changes slower until someone cleans them up | §7.1 |
| Telescoping constructors | A ladder of constructors each adding one more parameter; Builder replaces it | §8.12 |
| Tell, Don't Ask | Principle — see 22.5 | Ch 5 §5 |
| Template Method | Behavioral pattern — see 22.7 | §11.4 |
| Temporary Field (smell) | Code smell — see 22.6 | §7.2.16 |
| Test double | Anything standing in for a real dependency in a test: dummy, stub, spy, mock, fake | Ch 6 §17 |
| Test-Driven Development (TDD) | Failing test → make it pass → refactor, in very short cycles | Ch 6 §8 |
| Thread | An independent path of execution inside a process, sharing its memory | §12.2 |
| Thread confinement | Keeping data inside one thread so it never needs locking | §12.24 |
| Thread dump | A snapshot of every thread's state and call stack (`jstack`, `jcmd`) used to diagnose hangs | §12.3 |
| Thread pool | A fixed set of reusable threads fed from a task queue | §12.21 |
| Thread states | NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED | §12.3 |
| Thread-safe | Gives correct results even when many threads call it at the same time | §12.5 |
| Thread-starvation deadlock | Tasks in a bounded pool wait for tasks queued behind them in the same pool | §12.17, §12.21 |
| Throughput | How much work gets done per unit of time (requests per second) | §12.11 |
| Timeout | A limit on how long to wait before giving up | §12.9 |
| TL;DR | "Too long; didn't read": a short summary at the end of each section or chapter | Every chapter |
| Token bucket | Rate-limit algorithm: tokens refill at a fixed rate; each request takes one; allows bursts | §13.11 |
| `toString` | Method returning a readable text form of an object, used in logs and printing | §2.12 |
| `TreeMap` / `TreeSet` | Sorted map and set (red-black tree inside); support "next higher" and range queries | §14.5 |
| `tryLock` | Concurrency tool — see 22.8 | §12.9 |
| TTL (time to live) | How long an entry or hold stays valid before it expires | §13.6, §15.2 |
| Two hats | Refactoring — see 22.6 | §7.1.1 |
| Type erasure | Java removes generic type arguments after compiling, so `List<String>` and `List<Integer>` are the same class at run time | §2.15 |
| Ubiquitous language | One vocabulary shared by domain experts and the code (`place()`, not `setStatus(2)`) | Ch 5 §17 |
| UI | User interface: the screens or controls a person interacts with | §11.7.5 |
| UML | Unified Modeling Language: 14 diagram types, 7 structural and 7 behavioural | §3.9 |
| Unit test | A fast test of one small piece of code in isolation | Ch 6 §8 |

| Term | Plain meaning | Where |
|---|---|---|
| UPI | Unified Payments Interface: India's instant bank-to-bank mobile payment system | Ch 19, §21.2 |
| Use case | One thing an actor can do with the system | §1.5, §3.10 |
| Use-case diagram | UML diagram of actors, use cases and the system boundary («include», «extend») | §3.10 |
| UTC | Coordinated Universal Time: the time-zone-free reference clock for storing timestamps | Ch 16 · Airline |
| Value object | A small immutable object defined by its values (money, date range), usually a record | §2.13, Ch 5 §1, Ch 5 §17 |
| Virtual threads | Concurrency tool — see 22.8 | §12.4 |
| Virtual waiting room | FIFO queue that admits buyers gradually during a ticket rush | §15.3 |
| Visibility (memory) | Whether a write by one thread can be seen by another; not guaranteed without `volatile`, locks or other happens-before edges | §12.6 |
| Visitor | Behavioral pattern — see 22.7 | §11.5 |
| `volatile` | Concurrency tool — see 22.8 | §12.6 |
| Webhook | A URL a service calls to notify you that something happened (e.g. "payment succeeded") | §15.2 |
| Wildcard (generics) | `?` in a generic type: `? extends T` (read from it), `? super T` (write to it) | §2.15 |
| Work stealing | Idle threads in a fork/join pool take queued tasks from busy threads | §12.26 |
| Wrapper | An object that holds another and adds or changes behaviour around it (decorators, adapters, boxed primitives) | §9.4 |
| Write-through / write-back | Cache writes go to storage immediately vs later in batches | §20.3 |
| YAGNI | You Aren't Gonna Need It — principle, see 22.5 | Ch 5 §3 |

---

## Self-check

- **Q:** Name the eleven framework steps in order. **A:** Clarify, requirements, use cases, entities, relationships, class diagram, key flows, patterns & SOLID, code, edge cases & concurrency, extensions.
- **Q:** Where does the diamond go, and what does filled mean? **A:** At the whole; filled = composition (the part dies with the whole).
- **Q:** A `switch (vehicleType)` computes fees. Which principle, smell and pattern? **A:** OCP; Repeated Switches (if it repeats); Strategy.
- **Q:** Decorator vs Proxy vs Chain of Responsibility? **A:** Decorator adds behaviour and every layer runs; Proxy controls access; in CoR one handler can stop the flow.
- **Q:** State vs Strategy? **A:** States know each other and advance themselves; strategies are chosen from outside.
- **Q:** Four Coffman conditions, and the usual fix? **A:** Mutual exclusion, hold and wait, no preemption, circular wait; global lock ordering.
- **Q:** Why is `volatile` not enough for `count++`? **A:** Visibility, not atomicity; use `AtomicInteger` or a lock.
- **Q:** The six machine-coding packages? **A:** model, repository, service, strategy, exception, driver.
- **Q:** What makes every booking problem safe? **A:** Naming the real unit of inventory and making "if free then take" one atomic step (plus holds that expire).

## TL;DR

- **Framework:** clarify → requirements → use cases → entities → relationships → diagram → flows → patterns/SOLID → code → edge cases/concurrency → extensions.
- **Classes:** different data → field; different behaviour → class; one sentence per class; composition over inheritance.
- **SOLID** diagnoses change pain; always state the cost of the fix, and know when not to apply it.
- **Patterns** come from signals: Strategy (rules vary), Factory (type value → class), Observer (many listeners), State (lifecycle), Singleton (exactly one), Builder (many optional fields). Use one or two, name a rejected one.
- **Concurrency:** find the shared state, state the invariant, make check-then-act atomic with the smallest tool, lock in a fixed order, notify outside locks.
- **Machine coding:** run early and often, mandatory first, services return data, the driver prints and keeps going.

# Chapter 23 — Interview Question Bank

## How to use this bank

This chapter collects the low-level design (LLD) and machine-coding questions that come up again and again in interviews, worded the way they usually appear on LeetCode Discuss interview-experience posts and in the popular public question lists (see the sources at the end). Use it as a checklist so that nothing surprises you.

- ⋆ marks a question that is asked very often. Do those first.
- **Part A** is the "Design X" prompts (parking lot, BookMyShow, Splitwise …). When this book has a full case study, the entry is a short reminder of what to say plus a link to the full design. When it doesn't, the entry is a complete outline you can build from.
- **Part B** is the concept questions (OOP, UML, SOLID, patterns, concurrency, Java specifics). Each has a short answer you can say out loud and a link to the full explanation.
- **Part C** is a minute-by-minute plan for a 60–90 minute machine-coding round.

**How to drill:** read the question, cover the answer, say your answer out loud (or sketch the classes), then compare. Mark the ones you stumbled on and come back to them the next day.

## Part A — "Design X" (LLD and machine coding) questions

These are the "Design X" prompts that show up again and again in object-oriented design (OOD) rounds and in machine-coding rounds. An OOD round is a whiteboard-style talk about classes; a machine-coding round is 90–150 minutes of writing working code. ⋆ marks prompts asked very often, and every machine-coding favourite also gets a ⋆. When this book has a full case study, the entry here is a short reminder of what to say, followed by a link. When the book has no case study, the entry is a complete outline you can build from.

## Everyday objects

**Q1. ⋆ Design a Parking Lot**

*Asked at:* Flipkart (SDE2 LLD round)

- Entities: `ParkingLot` → `Level` → `ParkingSpot` (sized: small, compact, large), `Vehicle`, `Ticket`, `EntryGate` / `ExitGate`, `FeeCalculator`.
- Patterns: Strategy for spot allocation and for pricing; Singleton (or one injected instance) for the lot; Factory for vehicles.
- What interviewers probe: two cars racing for the last free spot (claim the spot atomically), hourly vs flat pricing, and adding EV charging spots without editing old code. → *Full design:* Ch 14 — 14.1 Design a Parking Lot

**Q2. ⋆ Design an Elevator / Lift System**

- Entities: `ElevatorController`, `Elevator` (current floor, direction, door state), `Request` (hall call or car call), `Dispatcher`.
- Patterns: State (moving up, moving down, idle, doors open), Strategy for dispatching (nearest car, SCAN / LOOK).
- What interviewers probe: the scheduling algorithm (why plain first-come-first-served makes cars zig-zag), keeping requests consistent when many buttons are pressed at once, and overload or maintenance modes. → *Full design:* Ch 14 — 14.5 Design an Elevator System

**Q3. ⋆ Design a Vending Machine**

- Entities: `VendingMachine`, `Inventory`, `Product`, `Coin` / `Note`, `ChangeDispenser`.
- Pattern: State (idle, has money, dispensing, sold out) is the whole point of this question.
- What interviewers probe: making change with a limited coin stock, refunding on cancel, and what happens when a product sells out while money is inserted. → *Full design:* Ch 14 — 14.2 Design a Vending Machine

**Q4. Design a Coffee Vending Machine**

- Entities: `CoffeeMachine`, `Recipe` (ingredient → quantity), `IngredientStore`, `Beverage`, add-ons.
- Patterns: Decorator for add-ons (extra shot, milk), Factory for drinks, State for the machine.
- What interviewers probe: checking and deducting every ingredient as one atomic step, and refill alerts (Observer). → *Full design:* Ch 14 — 14.3 Design a Coffee Vending Machine

**Q5. ⋆ Design an ATM**

- Entities: `ATM`, `Card`, `Account`, `BankService`, `CashDispenser`, `Transaction` (withdraw, deposit, balance enquiry).
- Patterns: State (no card, card inserted, PIN verified), Chain of Responsibility for dispensing notes by denomination.
- What interviewers probe: what to do if cash is debited but not dispensed (reversal), PIN retry limits, and two ATMs withdrawing from one account at the same time. → *Full design:* Ch 14 — 14.4 Design an ATM

**Q6. Design a Traffic Signal Control System**

- Entities: `Intersection`, `TrafficLight`, `SignalPhase`, `SignalController`, `Timer`.
- Patterns: State for the light cycle, Strategy for timing plans (fixed, rush hour, sensor-driven), Observer for emergency vehicles.
- What interviewers probe: never letting conflicting directions be green together, and handling emergency override safely. → *Full design:* Ch 14 — 14.6 Design a Traffic Signal Control System

**Q7. Design an Amazon Locker / shipping locker**

- Entities: `LockerLocation`, `Locker` (by size), `Package`, `PickupCode`, `LockerService`.
- Patterns: Strategy for choosing a locker size, State for the locker (free, reserved, occupied).
- What interviewers probe: code expiry and returning the package to the sender, and two deliveries claiming the same locker. → *Full design:* Ch 14 — 14.7 Design a Shipping Locker System

**Q8. Design a toll booth / parking-fee calculator (pricing strategies)**

- This is the pricing part of the parking lot pulled out on its own: a `FeeCalculator` interface with hourly, flat, slab and weekend implementations, chosen by vehicle type and time.
- Patterns: Strategy (one class per pricing rule), optionally Decorator for surcharges and discounts layered on top.
- What interviewers probe: time zones and midnight boundaries, rounding (charge per started hour?), and keeping money as integer paise or `BigDecimal`, never `double`. → *Full design:* Ch 14 — 14.1 Design a Parking Lot (pricing section)

**Q9. Design a home automation system**

*Requirements.* A user owns devices (lights, fans, thermostats, door locks, sensors) made by different vendors. The user can switch a device on or off, set values (brightness, temperature), group actions into a "scene" ("Good night" turns off lights and locks doors), undo the last action, and create rules ("if motion after 23:00, turn on the hall light"). Schedules ("at 07:00") are nice to have.

*Main classes.*

- `Device` (interface): `id()`, `status()`. Subtypes `Light`, `Thermostat`, `Lock`, `MotionSensor`.
- `VendorAdapter`: wraps each vendor's protocol (Zigbee, Wi-Fi API) behind the `Device` interface.
- `DeviceCommand`: one action with `execute()` and `undo()`, for example `SetBrightness(light, 40)` remembers the old brightness.
- `Scene`: a list of commands run as one; it is itself a command.
- `Hub`: registry of devices, runs commands, keeps an undo stack.
- `Rule`: a trigger (sensor event, time) plus a condition plus a command.
- `EventBus`: sensors publish events; rules subscribe.

*Patterns.* Adapter (vendor protocols), Command (actions with undo), Composite (a scene is a command made of commands), Observer (sensor events reach rules), Strategy for the condition check.

*Concurrency and edge cases.* Send commands to one device in order: give each device its own single-threaded executor (or a lock), while different devices run in parallel on virtual threads. A device may be offline: queue the command with a timeout and report failure instead of hanging. Avoid rule loops (rule A turns on a light, which fires rule B, which turns it off) by tracking which rule caused an event and limiting the depth.

*Extension question.* "Support user roles: a guest can unlock the front door only between 10:00 and 18:00." Add an `AccessPolicy` checked by the `Hub` before running any command (a Proxy around devices works well).

**Q10. Design a Jackpot / slot machine**

*Requirements.* A player inserts credit, places a bet, and pulls the lever. Three (or five) reels spin and stop on symbols. A pay table decides the win. The machine tracks credit, pays out, and refuses to play when it is out of service or cannot cover the payout. Optional: a progressive jackpot shared by many machines.

*Main classes.*

- `SlotMachine`: holds the credit balance and the current state, and exposes `insert`, `bet`, `spin` and `cashOut`.
- `MachineState` (sealed interface): `Idle`, `HasCredit`, `Spinning`, `OutOfService`. Each state allows only its own actions.
- `Reel`: a list of symbols with weights, so a cherry can be more common than a seven.
- `RandomSource`: an interface wrapping `SecureRandom`. Tests swap in a fixed sequence, so results are repeatable.
- `PayTable`: maps a symbol combination to a multiplier. A Strategy, so different machines can pay differently.
- `JackpotPool`: one shared progressive pool. Each bet adds a small cut.

*Patterns.* State (the machine lifecycle), Strategy (pay rules, random source), Singleton-like shared `JackpotPool` (inject one instance rather than a static).

*Concurrency and edge cases.* Two machines can hit the jackpot at the same moment. Take the pool with one atomic step, `AtomicLong.getAndSet(seed)`, so only one machine wins the full amount. Use whole numbers for money. Reject a bet bigger than the credit. Go out of service if the machine's cash cannot cover the largest possible win. Log every spin (seed, result) for audits.

*Extension question.* "Add a bonus round after three scatter symbols." Add a `BonusRound` state that the machine moves into after the normal payout. No existing state class needs to change.

**Q11. Design a music playlist**

- Entities: `Song`, `Playlist` (ordered list), `Player` (current position, play mode), `User`.
- Patterns: Iterator (sequential, shuffle, repeat as different iterators) or Strategy for play order.
- What interviewers probe: shuffle without repeats until all songs played, and editing the playlist while it is playing. → *Full design:* Ch 1 — 1.3 A complete answer in five minutes

## Booking and reservations

**Q12. ⋆ Design BookMyShow / a movie ticket booking system**

*Asked at:* Flipkart

- Entities: `Cinema` → `Screen` → `Seat`, `Movie`, `Show`, `ShowSeat` (seat state per show), `Booking`, `Payment`.
- Patterns: State for a seat (available, held, booked), Strategy for pricing, Observer for confirmations.
- What interviewers probe: two users picking the same seat (hold with expiry, then confirm), releasing holds when payment fails or times out. → *Full design:* Ch 15 — 15.2 Design a Movie Ticket Booking System

**Q13. ⋆ Design a Library Management System**

- Entities: `Book` (title info) vs `BookItem` (physical copy), `Member`, `Loan`, `Reservation`, `Fine`, `Catalog` (search).
- Patterns: Strategy for fine rules, Observer for "your reserved book is back".
- What interviewers probe: the book vs copy split, borrowing limits, and the reservation queue. → *Full design:* Ch 15 — 15.1 Design a Library Management System

**Q14. ⋆ Design a Hotel Management System**

- Entities: `Hotel`, `Room` (type, rate), `Guest`, `Reservation` (date range), `Invoice`, `HousekeepingTask`.
- Patterns: State for reservation and room status, Strategy for room pricing.
- What interviewers probe: date-range overlap checks, overbooking rules, and late check-out charges. → *Full design:* Ch 15 — 15.4 Design a Hotel Management System

**Q15. Design a concert ticket booking system**

- Like BookMyShow, but one big event with a sudden rush: seat sections, general admission, and a waiting room.
- Patterns: State for seats, plus a queue in front of booking.
- What interviewers probe: thousands of buyers at the same second, hold timeouts, and per-user ticket limits. → *Full design:* Ch 15 — 15.3 Design a Concert Ticket Booking System

**Q16. ⋆ Design a car / vehicle rental system (Zoomcar)**

*Asked at:* Flipkart (SDE2, 2024)

- Entities: `Branch`, `Vehicle`, `Reservation` (pickup and drop time and branch), `Customer`, `PricingPolicy`, `Invoice`.
- Patterns: Strategy for pricing and for vehicle selection, State for reservation lifecycle.
- What interviewers probe: availability across overlapping time windows, one-way rentals, and damage or late-return charges. → *Full design:* Ch 16 — Design a Car Rental System

**Q17. Design a restaurant management system (table booking / waiting queue)**

- Entities: `Table`, `Reservation`, `WaitlistEntry`, `Order`, `MenuItem`, `Bill`, `Kitchen`.
- Patterns: Command for orders sent to the kitchen, Observer for "your table is ready", State for orders.
- What interviewers probe: fitting party sizes to tables, walk-ins vs reservations, and splitting the bill. → *Full design:* Ch 16 — Design a Restaurant Management System

**Q18. Design an airline management / flight booking system**

- Entities: `Flight` (route) vs `FlightInstance` (a dated departure), `Aircraft`, `Seat`, `Itinerary`, `Passenger`, `Booking`.
- Patterns: State for booking, Strategy for fare rules.
- What interviewers probe: the route vs instance split, multi-leg trips booked all-or-nothing, and seat holds. → *Full design:* Ch 16 — Design an Airline Management System

**Q19. Design a course registration system**

- Entities: `Course`, `Section` (time slot, capacity), `Student`, `Enrollment`, `Prerequisite`, `Waitlist`.
- Patterns: Specification or Chain of Responsibility for eligibility checks (prerequisites, clashes, credit limit).
- What interviewers probe: the last seat taken by two students at once, and automatic waitlist promotion. → *Full design:* Ch 16 — Design a Course Registration System

**Q20. Design a learning management system / online exam portal**

- Start from course registration and add `Exam`, `Question` (single choice, multiple choice, text), `Attempt`, `Grader`.
- Patterns: Strategy for grading each question type, State for an attempt (started, submitted, graded), and a timer that auto-submits.
- What interviewers probe: saving answers often so a dropped connection loses nothing, and randomising question order per student. → *Related design:* Ch 16 — Design a Course Registration System

**Q21. ⋆ Design a Meeting Scheduler / meeting room booking**

- Entities: `Interval` (half-open), `Room` (owns a sorted calendar), `Meeting`, `Invitation` (RSVP), `MeetingScheduler`, `RoomSelectionStrategy`, `MeetingObserver`.
- Patterns: Strategy for room ranking, Observer for invitations and RSVPs.
- What interviewers probe: overlapping bookings of one room (check and insert under the room lock), room clash blocks while a busy person only warns, reschedule without losing the slot, and "first time everyone is free" (merge busy intervals, scan the gaps). → *Full design:* Ch 16 — Design a Meeting Scheduler

**Q22. ⋆ Design a doctor appointment booking system**

*Asked at:* Flipkart (SDE2, 2021)

*Requirements.* Doctors register with a speciality (cardiologist, dermatologist). Each doctor declares available one-hour slots for the day. Patients search by speciality and see free slots, sorted by start time by default. A patient books, cancels, and views their bookings. A patient may not hold two appointments at the same time. When a slot is full, the patient can join its waitlist, and a cancellation promotes the first person waiting.

*Main classes.*

- `Doctor` (id, name, speciality, slots) and `Patient` (id, name, appointments).
- `Slot`: doctor, start time, status (`FREE`, `BOOKED`), booked patient, and a waitlist (a `Deque<Patient>`).
- `Appointment`: a record (id, patient, slot).
- `BookingService`: `book`, `cancel` and `joinWaitlist`. It enforces "no double booking for the patient".
- `SearchService` with a `SlotRankingStrategy`. Order by start time by default, or by doctor rating later.
- Repositories: in-memory maps, as the machine-coding round expects.

*Patterns.* Strategy (ranking), Observer (tell a waitlisted patient they were promoted), a simple State on `Slot`.

*Concurrency and edge cases.* Two patients may book the last slot together. Guard the slot status with a per-slot lock, or a compare-and-set on an `AtomicReference<SlotStatus>`. Check the patient's own clash under the patient's lock. Lock patient then slot, always in that order, so no deadlock can form. Validate that slots are whole hours inside working hours. Cancelling an already-cancelled booking is a no-op.

*Extension question.* "Show the trending doctor: the one with the most bookings today." Keep a counter per doctor that `book` updates, and read it through a max-heap or a sorted view.

**Q192. Meeting scheduler: why can't a concurrent map keyed by start time stop double-booking, and how do you find a clash fast?** Overlapping meetings usually start at different times (10:00–11:00 and 10:30–11:15), so their keys differ and `putIfAbsent` lets both in. Keep each room's bookings in a `TreeMap` keyed by start; because stored bookings never overlap, only the one returned by `lowerEntry(newEnd)` can clash, an O(log n) check. Do that check and the insert under the room's lock. → *More:* Ch 16 — Design a Meeting Scheduler

## Games

**Q23. ⋆ Design Snake and Ladder**

- Entities: `Board` (size, snakes and ladders as jumps), `Player`, `Dice`, `Game` (turn order queue).
- Patterns: Strategy for dice (one die, two dice, a loaded die in tests), a simple game loop.
- What interviewers probe: input-driven setup in machine coding, exact-roll-to-win rules, and chained jumps. → *Full design:* Ch 17 — Design Snake and Ladder

**Q24. ⋆ Design Chess**

- Entities: `Board`, `Square`, `Piece` subclasses, `Move`, `Player`, `Game` (status, history).
- Patterns: Strategy (or polymorphism) for each piece's moves, Command for moves with undo, State for game status.
- What interviewers probe: check / checkmate detection (simulate the move, then test the king), castling and en passant, and why moves should be objects. → *Full design:* Ch 17 — Design Chess

**Q25. ⋆ Design a 2048 game**

*Asked at:* Flipkart (2021)

- Entities: `Board` (grid), `Tile`, `Game` (score, state), `MoveDirection`, `TileSpawner`.
- Pattern: write one "slide and merge a row left" routine and rotate the grid for the other three directions.
- What interviewers probe: each tile merging only once per move, spawning only after a real change, and detecting game over. → *Full design:* Ch 17 — Design 2048

**Q26. ⋆ Design Tic-Tac-Toe**

- Entities: `Board`, `Player`, `Game`, `WinChecker`.
- Patterns: Strategy for players (human, AI), and per-row / column / diagonal counters so a win check is O(1).
- What interviewers probe: N×N boards, the O(1) win check, and keeping input handling out of the game rules. → *Full design:* Ch 17 — Design Tic-Tac-Toe

**Q27. Design a deck of cards / Blackjack**

- Entities: `Card` (rank, suit enums), `Deck`, `Shoe`, `Hand`, `Player`, `Dealer`, `Game`.
- Patterns: Template Method or Strategy for the dealer's rule, and a generic deck reused by other card games.

- What interviewers probe: ace counted as 1 or 11, and making a deck that works for any card game. → *Full design:* Ch 17 — Design Blackjack and a Deck of Cards

**Q28. Design Minesweeper**

*Requirements.* A rows × cols grid hides a number of mines. The player reveals a cell or toggles a flag. Revealing a mine loses the game. Revealing a number shows how many of the eight neighbours are mines. Revealing a zero opens all connected empty cells and their numbered borders. The player wins when every non-mine cell is revealed. The first click must never be a mine.

*Main classes.*

- `Cell`: `mine` (boolean), `adjacentMines`, and `state` (`HIDDEN`, `REVEALED`, `FLAGGED`).
- `Board`: the grid, `neighbours(r, c)`, `revealedCount`, `mineCount`.
- `MinePlacer` (interface): random placement with a seed. Tests use a fixed layout. Mines are placed *after* the first click, avoiding that cell (and its neighbours, if you like).
- `Game`: `reveal(r, c)`, `toggleFlag(r, c)`, and `status` (`PLAYING`, `WON`, `LOST`).
- `Renderer`: prints the board. Keep it separate from the rules.

*Patterns.* Strategy (mine placement), State for game status (no moves after a win or loss), and Observer if a UI must redraw.

*Algorithm.* For reveal, use a breadth-first search with a queue, not recursion. A recursive flood fill on a 1000×1000 empty board can overflow the stack. Mark each cell revealed when you add it to the queue, so no cell is processed twice. Flagged cells are not opened by the flood. Win check: `revealedCount == rows * cols - mineCount`, an O(1) test.

*Edge cases.* Ignore clicks outside the board, on revealed cells, or after the game ended. Reject more mines than cells minus one.

*Extension question.* "Add chording: clicking a number whose flags are all placed opens its other neighbours." Write it as one more method that reuses `reveal`, so no existing rule changes.

**Q29. Design a Bowling Alley scoring system**

- Entities: `Game`, `Frame` (rolls), `TenthFrame` (up to three rolls), `Player`, `Lane`.
- Key logic: a strike adds the next two rolls, a spare adds the next one. Score from the stored list of rolls instead of keeping running totals, so bonuses resolve naturally.
- What interviewers probe: the tenth frame, partial scores while bonuses are pending, and validating pin counts per frame.
- The scoring core for a finished game (validate rolls first: 0–10 each, at most 10 per frame outside the tenth):

```
static int score(List<Integer> rolls) {
    int total = 0, i = 0;
    for (int frame = 1; frame <= 10; frame++) {
        if (rolls.get(i) == 10) {                                  // strike
            total += 10 + rolls.get(i + 1) + rolls.get(i + 2); i += 1;
        } else if (rolls.get(i) + rolls.get(i + 1) == 10) {        // spare
            total += 10 + rolls.get(i + 2); i += 2;
        } else { total += rolls.get(i) + rolls.get(i + 1); i += 2; }
    }
    return total;   // 12 strikes -> 300; 21 rolls of 5 -> 150
}
```

This book has no Bowling Alley case study, so this outline is the whole answer here.

**Q30. Design an online game engine**

*Requirements.* Provide the core that any 2D game can build on: a loop that updates and draws the world at a steady rate, game objects with position and behaviour, keyboard and mouse input, collision detection, events between objects, loading images and sounds once, and screens (menu, playing, paused).

*Main classes.*

- `GameLoop`: runs a fixed-timestep update (say 60 times a second) and renders as often as possible. It catches up with several updates if a frame was slow.
- `Entity`: an id that holds `Component`s (`Position`, `Velocity`, `Sprite`, `Collider`). This is the Entity-Component-System (ECS) idea: behaviour comes from parts, not from a deep inheritance tree.

- `System`: `MovementSystem`, `CollisionSystem` and `RenderSystem`. Each one walks the entities that have the components it needs.
- `InputHandler`: maps keys to `Command` objects (`Jump`, `MoveLeft`), so key bindings can change.
- `EventBus`: lets objects publish "player died" or "coin collected" without knowing each other.
- `AssetManager`: loads each texture once and shares it.
- `SceneManager`: holds a stack of `Scene`s. Each scene is a state, such as `Menu`, `Level` or `Pause`.

*Patterns.* Component (composition over inheritance), Command (input), Observer (events), Flyweight (shared assets), State (scenes), Game Loop.

*Concurrency and edge cases.* Keep the world single-threaded. Input arrives from other threads into a `ConcurrentLinkedQueue`, and the loop drains it at the start of each tick. Asset loading can run on virtual threads, publishing "loaded" events. Use a fixed time step, so physics gives the same result on fast and slow machines.

*Extension question.* "Make it multiplayer." The server owns the true state and runs ticks. Clients send commands, predict movement locally, and correct themselves when the server state arrives.

**Q193. Design a Jigsaw Puzzle**

- Entities: `Side` (FLAT, TAB or BLANK plus a shape code), immutable `Piece` (four sides clockwise), `Board` with `Placement(piece, rotation)`, `PuzzleValidator`, `SolverStrategy`.
- Patterns: Strategy for solvers (brute-force backtracking vs edge-first with a `Side → pieces` index); all placement rules in one `Board.check` used by players and solvers.
- What interviewers probe: where rotation lives (in the placement, so pieces stay immutable), the facing side after `r` turns (`sides[(d − r) mod 4]`), why corners need no special rule, and the cost of solving (about n attempts with unique shapes, exponential with look-alikes, hence an attempt budget). → *Full design:* Ch 17 — Design a Jigsaw Puzzle

## Platforms and marketplaces

**Q31. ⋆ Design a cab booking / ride-sharing service (Uber; Flipkart "offer and select a ride")**

*Asked at:* Flipkart

- Entities: `Rider`, `Driver`, `Vehicle`, `Location`, `RideRequest`, `Trip`, `FareCalculator`, `MatchingService`.
- Patterns: Strategy for matching (nearest, best rated) and for fares (surge), State for the trip, Observer for live updates.
- What interviewers probe: two riders matched to one driver, cancellation rules, and the carpool variant (offer a ride with seats, select the best ride). → *Full design:* Ch 20 — 20.4 Design a Ride Sharing Service (Uber)

**Q32. ⋆ Design a food delivery system (Swiggy / Zomato; Flipkart "FoodKart")**

*Asked at:* Flipkart

- Entities: `Restaurant`, `Menu`, `Item`, `Cart`, `Order`, `DeliveryAgent`, `RestaurantSelectionStrategy`.
- Patterns: State for the order, Strategy for choosing the restaurant (lowest cost, highest rating) and assigning agents, Observer for tracking.
- What interviewers probe: restaurant capacity (max orders at once) freed when an order completes, and one-restaurant-per-cart rules. → *Full design:* Ch 19 — Design an Online Food Delivery Service (Swiggy) + FoodKart

**Q33. ⋆ Design CricInfo / a cricket scoreboard (Udaan "Cricket Match Dashboard")**

*Asked at:* Udaan

- Entities: `Match`, `Team`, `Player`, `Innings`, `Over`, `Ball` (the event that everything derives from), `Scorecard`.
- Patterns: Template Method for match formats, Observer for live updates, "derive stats from balls, don't store totals".
- What interviewers probe: extras (wide, no-ball), strike rotation at the end of an over, and the machine-coding version that prints the scorecard after every over. → *Full design:* Ch 18 — 18.2 Design Cricinfo

**Q34. ⋆ Design a stock exchange / order matching engine (LLD)**

- Entities: `Order` (buy or sell, limit or market, quantity, price, time), `OrderBook` per symbol, `Trade`, `MatchingEngine`.
- Data structure: for each side, a `TreeMap<price, Deque<Order>>`. The best bid is the highest key and the best ask is the lowest. Match by price first, then by time.
- What interviewers probe: partial fills, cancelling an order sitting in the book, and a single writer thread per symbol so no locks are needed. → *Related design:* Ch 19 — Design an Online Stock Brokerage System

**Q35. Design an online stock brokerage system**

- Entities: `Account`, `Portfolio`, `Position`, `Order` (market, limit, stop), `Exchange` adapter, `Watchlist`.
- Patterns: Strategy/Factory for order types, State for order lifecycle, Observer for price alerts, Adapter for the exchange.
- What interviewers probe: blocking funds when an order is placed, partial fills, and order status kept in step with the exchange. → *Full design:* Ch 19 — Design an Online Stock Brokerage System

**Q36. Design a cryptocurrency exchange**

- Same core as Q34 (order book and matching) plus wallets per currency, deposits that wait for blockchain confirmations, and withdrawals that need approval.
- Patterns: State for deposits and withdrawals, Strategy for fees.
- What interviewers probe: exact decimal amounts (`BigDecimal` with each coin's scale), and never letting a balance go negative under concurrent trades. → *Related design:* Ch 19 — Design an Online Stock Brokerage System

**Q37. Design Stack Overflow**

- Entities: `User` (reputation, badges), `Question`, `Answer`, `Comment`, `Vote`, `Tag`, `Bounty`.
- Patterns: Observer for reputation changes and notifications, Strategy for search and ranking.
- What interviewers probe: one vote per user per post, accepting an answer, and permissions that depend on reputation. → *Full design:* Ch 18 — 18.1 Design Stack Overflow

**Q38. Design LinkedIn**

- Entities: `Member`, `Profile`, `Connection`, `Invitation`, `Post`, `Job`, `Message`.
- Patterns: State for invitations, BFS for 1st/2nd/3rd degree connections, Strategy for feed ranking.
- What interviewers probe: two-sided operations (accepting adds both sides), and degree computation limits. → *Full design:* Ch 18 — 18.3 Design LinkedIn

**Q39. Design Facebook / a social network**

- Entities: `User`, `Friendship`, `Follow`, `Post`, `Audience` (privacy), `Page`, `Group`, `NewsFeed`.
- Patterns: one visibility policy used by feed and search, Strategy for feed ranking, pull-model feed.
- What interviewers probe: friends vs followers, privacy checks on every read, and friend suggestions. → *Full design:* Ch 18 — 18.4 Design Facebook

**Q40. ⋆ Design Twitter (post / follow / feed; LeetCode 355 "Design Twitter")**

- The LeetCode version: `postTweet`, `getNewsFeed` (the 10 newest tweets from self and followees), `follow`, and `unfollow`.
- Data: a map from user to followees, and per user a list of tweets with a global increasing timestamp. Build the feed by merging the users' newest-first lists with a max-heap, k-way merge style, and stop after 10.
- What interviewers probe: why you pull (merge on read) rather than push here, self-follow, and unfollowing yourself. → *Related design:* Ch 18 — 18.4 Design Facebook (feed and follow)

**Q41. ⋆ Design Amazon (online shopping system / cart / checkout)**

- Entities: `Product`, `Catalog`, `Cart`, `Order`, `OrderItem`, `Payment`, `Shipment`, `Inventory`.
- Patterns: State for orders, Strategy for payment and shipping, Observer for status emails.
- What interviewers probe: reserving stock at checkout vs at payment, price changing while the item sits in the cart, and partial cancellations. → *Full design:* Ch 19 — Design Amazon — Online Shopping System

**Q42. Design an online auction system (eBay; live IPL auction)**

- Entities: `Auction` (item, reserve price, end time), `Bid`, `Bidder`, `Seller`, `AuctionHouse`.

- Patterns: State (scheduled, live, closed), Observer for outbid alerts, Strategy for auction type (English, sealed bid).
- What interviewers probe: two bids arriving at the same moment (serialise per auction), last-second sniping and extending the end time. → *Full design:* Ch 19 — Design an Online Auction System

**Q43. Design Spotify / a music streaming service (LLD)**

- Entities: `Track`, `Album`, `Artist`, `Playlist`, `User`, `Subscription`, `Player`, `PlayQueue`.
- Patterns: Strategy for recommendations and shuffle, State for the player, Iterator for queues.
- What interviewers probe: free vs premium limits, collaborative playlists, and the play-next queue vs the playlist order. → *Full design:* Ch 19 — Design a Music Streaming Service (Spotify)

**Q44. Design a grocery store / POS / checkout (BigBasket)**

- Entities: `Product` (priced by unit or by weight), `Inventory`, `Basket`, `Receipt`, `Discount`, `Payment`.
- Patterns: Strategy and Composite for discounts (buy-2-get-1, percent off a category), Decorator for tax.
- What interviewers probe: the order in which discounts apply, and returns. → *Full design:* Ch 19 — Design a Grocery Store (POS & Checkout)

**Q45. Design a subscription-based sports score website**

- Reuse the Cricinfo model for matches and add `Subscription` (plan, expiry), `Entitlement` checks and live score pushes.
- Patterns: Observer (subscribers to a match), Proxy for checking entitlements before content is served.
- What interviewers probe: free vs paid content per request, and fan-out of score updates to many subscribers. → *Related design:* Ch 18 — 18.2 Design Cricinfo

**Q46. Design a logistics / shipment tracking system**

- Entities: `Shipment`, `Parcel`, `Hub`, `Route`, `TrackingEvent`, `Courier`.
- Patterns: State for shipment status, Observer for customer updates, Strategy for route choice.
- What interviewers probe: events that arrive out of order (sort by event time, not arrival), and returns. → *Related design:* Ch 14 — 14.7 Design a Shipping Locker System

**Q47. Design a newsletter service**

- Entities: `Publication`, `Issue`, `Subscriber`, `Subscription` (topics, frequency), `Delivery`.
- Patterns: Observer / Pub-Sub (subscribers per topic), Strategy for channel (email, in-app), Template Method for rendering.
- What interviewers probe: unsubscribe taking effect at once, retries without double delivery, and daily digests. → *Related design:* Ch 20 — 20.6 Design a Pub-Sub System

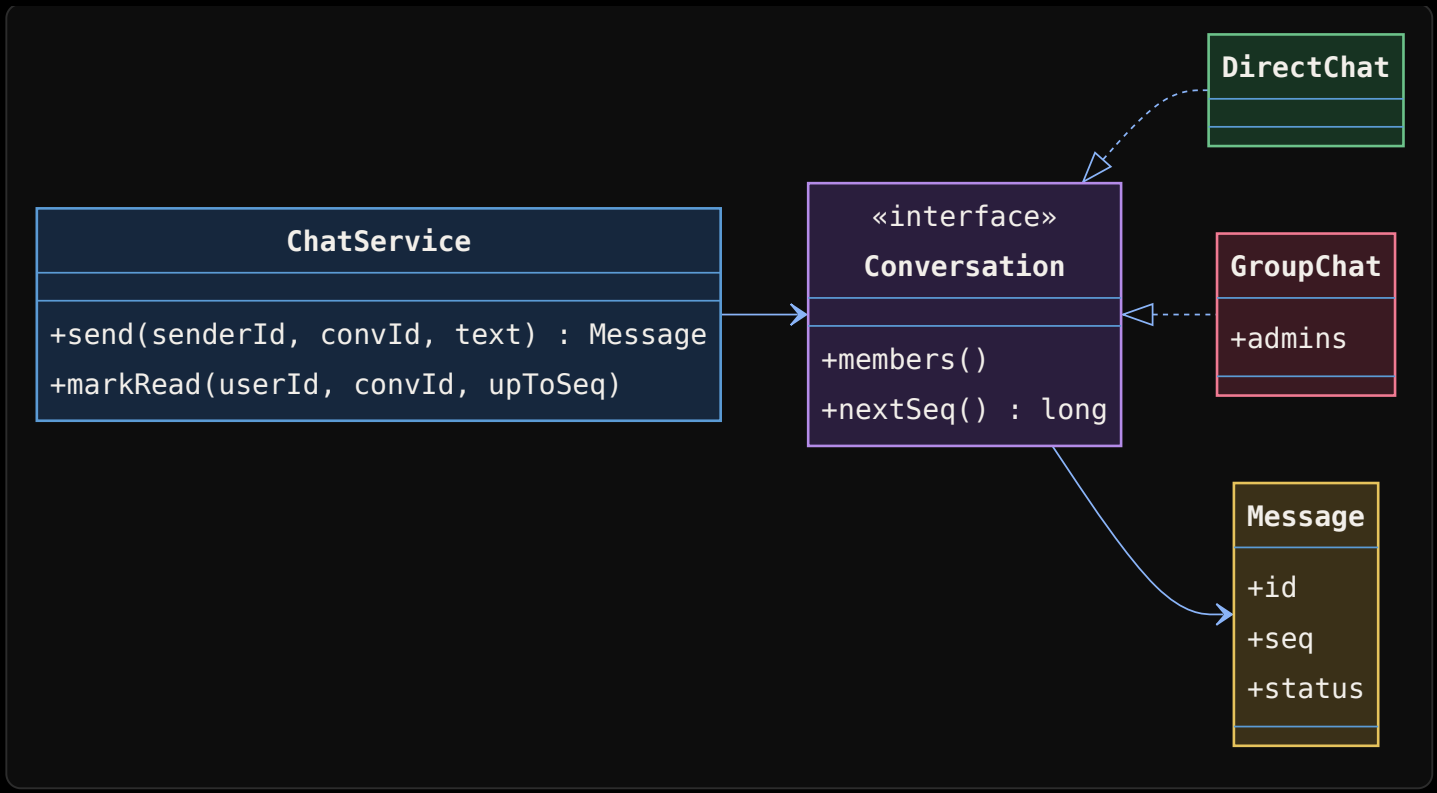
**Q48. Design a maps navigator client (walk / car / bus / bike)**

- Entities: `Navigator`, `Route`, `RouteStrategy` (one per travel mode), `Location`.
- Pattern: Strategy is the whole question. The navigator swaps strategies at runtime without `if` chains.
- What interviewers probe: adding a new travel mode without touching old code, and combining modes (walk + bus). → *Related design:* Ch 11 — 11.3 Strategy

**Q49. Design a chat server / WhatsApp (LLD)**

*Requirements.* Users send text messages one-to-one and in groups. Messages show sent, delivered and read ticks. Offline users receive their messages when they come back. Each conversation keeps its order. Groups have admins who add and remove members. Online and last-seen status is a nice extra.

*Main classes.*




- `Conversation` (sealed interface): `DirectChat` (two members) and `GroupChat` (members, admins).
- `Message`: a record (id, conversationId, senderId, seq, text, sentAt). Its delivery status is kept per recipient, because in a group each person reads at a different time.
- `Inbox`: per user, a queue of messages not yet delivered.
- `PresenceService`: online or offline, and last seen.
- `ConnectionRegistry`: the user's live sessions. Delivery listeners hang off it (Observer).

*Patterns.* Observer (deliver to connected sessions), Strategy (delivery channel: socket or push notification), and Mediator (`ChatService` coordinates instead of users talking to each other).

*Concurrency and edge cases.* Give each conversation an `AtomicLong` sequence. Sort messages by sequence, not by clock time, so senders on two phones cannot reorder history. Make delivery idempotent: a client acknowledges the highest sequence it has seen, and re-sent messages are ignored. A removed group member must not receive later messages, so check membership when the message is sent.

*Extension question.* "Add message editing and deletion." Store edits as new events that point at the original id, and let clients apply them, so the order stays clean.

**Q50. Design Truecaller**

*Requirements.* Users sign up with a phone number and may upload their contacts. Anyone can look up a number to see the most likely name and a spam score. Users report numbers as spam and block numbers. Users can search by name. A user can mark their own number as private.

*Main classes.*

- `PhoneNumber`: a value object. Its constructor normalises the input to E.164 format (for example `+919876543210`), so the same number can never be stored two ways.
- `User`: an id, an own number, contacts and a block list.
- `Contact`: a record (number, savedName).
- `GlobalDirectory`: maps each number to a name-count table (name → how many users saved it under that name). A lookup returns the most frequent name, and a registered user's own name always wins.
- `SpamService`: counts reports per number and gives a score. A number is "likely spam" past a threshold, weighted by how trustworthy each reporter is.
- `CallerIdService`: answers `lookup(number)` with `CallerInfo(name, spamScore, blocked)`.
- `NameSearchIndex`: a trie (prefix tree) from name prefixes to numbers.

*Patterns.* Value object, Strategy (spam scoring, name ranking), Observer (a new contact upload updates the directory and the index), Facade (`CallerIdService`).

*Concurrency and edge cases.* Uploads and reports arrive at the same time from many users. Use `ConcurrentHashMap<PhoneNumber, ConcurrentHashMap<String, LongAdder>>` for name counts and `LongAdder` for spam counts, so updates never block reads. Count one report per reporter per number, so repeat reports don't inflate the score. A private user's name should not be served from the crowd data.

*Extension question.* "Mark verified business numbers." Add a `VerifiedBusiness` record checked before the crowd data, so a verified name always beats crowd guesses.

**Q51. Design a mentorship / mock-interview platform (Pramp)**

*Requirements.* Users sign up with the topics they practise (data structures, system design) and their available time slots. The platform pairs two users who share a topic and a free slot. Each person interviews the other for half the session. Afterwards both leave feedback. Users can cancel, and they are reminded before a session.

*Main classes.*

- `User`: id, topics, a calendar of `AvailabilitySlot`s, and a rating.
- `AvailabilitySlot`: a record (start, end).
- `MatchRequest`: user, topic, preferred slots.
- `MatchingService`: keeps the open requests per topic and slot, and pairs requests through a `MatchingStrategy` (first come first served, similar rating).
- `Session`: two participants, topic, slot, question set, and a state (`SCHEDULED`, `IN_PROGRESS`, `COMPLETED`, `CANCELLED`, `NO_SHOW`).
- `Feedback`: a record (from, to, scores, notes).
- `ReminderScheduler`: uses a `ScheduledExecutorService` to fire reminders.

*Patterns.* Strategy (matching), State (session), Observer (send notifications on state change), Builder for sessions.

*Concurrency and edge cases.* One user must never be matched twice for the same slot. Run the matching in one step: lock both users' calendars in a fixed order (lowest id first) to avoid deadlock, check both are still free, then book both. A cancellation within an hour counts against the user's reliability, and the partner goes back into the pool. A no-show is detected when a session never moves to `IN_PROGRESS`.

*Extension question.* "Add paid sessions with expert mentors." Add a `Mentor` role with a price. Payment must succeed before the session moves to `SCHEDULED`. The rest of the flow stays the same.

**Q52. Design an Application Tracking System (Greenhouse)**

*Requirements.* A company posts jobs. Each job has a pipeline of stages (Applied → Phone screen → Onsite → Offer → Hired). Candidates apply, and recruiters move applications forward or reject them. Interviewers submit scorecards. Hiring managers see a dashboard of each job's pipeline. Every action is audited.

*Main classes.*

- `Job`: id, title, department, status (`OPEN`, `ON_HOLD`, `CLOSED`), and its own `Pipeline` (an ordered list of `Stage`s).
- `Candidate`: a person, who may have several `Application`s.
- `Application`: candidate + job, the current stage, a status (`ACTIVE`, `REJECTED`, `WITHDRAWN`, `HIRED`), and a history of stage changes.
- `Interview`: application, interviewers and time. `Scorecard`: interviewer, ratings and a recommendation.
- `ApplicationService`: `advance`, `reject` and `hire`. It checks the rules (no skipping to offer without scorecards, unless overridden).
- `PermissionPolicy`: decides what a recruiter, an interviewer or a hiring manager may see or do.
- `AuditLog` and `NotificationService`.

*Patterns.* State (application status), Template Method or configurable stage lists (each job has its own pipeline), Chain of Responsibility (the checks before advancing), Observer (emails and audit entries on each change).

*Concurrency and edge cases.* Two recruiters may move the same application at the same time. Use optimistic locking: a version number on `Application`, where an update fails if the version changed. Duplicate applications (same candidate, same job) are merged or rejected. Closing a job should not delete applications, only stop new ones.

*Extension question.* "Auto-reject candidates who fail a knock-out question." Add a rule to the advance chain that runs on `Applied`.

### Q53. Design a call center

*Requirements.* The center has three levels of employee: respondents, managers and directors. An incoming call goes to a free respondent. If the respondent can't handle it, or none is free, it escalates to a manager, then to a director. If nobody is free, the call waits in a queue. When an employee finishes a call, they take the next waiting call at their level.

*Main classes.*

- `Employee` (abstract or sealed): id, `Rank`, and a current call. Subtypes `Respondent`, `Manager`, `Director`. Each can `canHandle(call)`.
- `Call`: caller, required minimum rank (it rises when escalated), state (`WAITING`, `IN_PROGRESS`, `ESCALATED`, `ENDED`).
- `CallDispatcher`: holds a queue of free employees per rank (`BlockingQueue<Employee>`) and a queue of waiting calls per rank. `dispatch(call)` tries the call's rank and above.
- `EscalationHandler`: a chain from respondent to manager to director.

*Patterns.* Chain of Responsibility (escalation), Strategy (routing: least busy, skills-based), State (call lifecycle).

*Concurrency and edge cases.* Calls arrive and end on many threads. Use `poll()` on the free-employee queues, so taking an employee is atomic. When an employee becomes free, first check the waiting-call queue for their rank (and lower ranks), and only then return to the free pool. Do these two steps under one lock, or a call could be missed. If a call is dropped mid-escalation, end it cleanly and free the employee. A call that waits too long can be escalated or offered a callback.

*Extension question.* "Add priority customers." Change the waiting queue to a `PriorityBlockingQueue` ordered by priority, then by arrival time.

### Q54. Design a Survey / Google Forms system

*Requirements.* An owner builds a form with a title and questions of several types: short text, paragraph, single choice, multiple choice, rating, and date. Questions can be required and can have validation (email format, number range). The form can show or skip a section based on an earlier answer. Respondents submit responses. The owner sees a summary per question and can close the form.

*Main classes.*

- `Form`: id, owner, settings (one response per user, deadline), an ordered list of `Section`s, a version, and a status.
- `Question` (sealed interface): `TextQuestion`, `ChoiceQuestion` (options, multiSelect), `RatingQuestion` (scale), `DateQuestion`. Each has `validate(Answer)`.
- `ValidationRule`: composable rules (required, regex, range).
- `BranchRule`: "if the answer to question 3 is 'No', go to section 5".
- `Response`: form id, form version, respondent, and `Map<questionId, Answer>`.
- `FormBuilder`: builds a form step by step and checks it is valid.
- `Summary`: per question type, a different aggregation (counts, average, word list).

*Patterns.* Builder (forms), Composite (form → sections → questions), Strategy/Specification (validation), Visitor or a pattern-matching `switch` over the sealed `Question` types for summaries.

*Edge cases.* Editing a form after responses exist: create a new version, so old responses still point at the questions they answered. Enforce one response per user with a set of respondent ids, updated atomically with `ConcurrentHashMap.newKeySet().add(...)`. Validate every answer on the server, never trusting the client.

*Extension question.* "Add a quiz mode with scores." Add a correct answer and points to each question, plus a `Grader` that walks the response. Forms that are not quizzes are unaffected.

### Q55. Design an employee management / payroll system

*Requirements.* Store employees with department, role and manager. Show the org chart and everyone who reports (directly or indirectly) to a manager. Track leave balances and requests. Run monthly payroll for salaried, hourly and commission employees, with deductions (tax, provident fund) and payslips.

*Main classes.*

- `Employee`: id, name, department, role, managerId, join date, and a `PayPolicy`.
- `OrgChart`: manager id → list of direct reports. `allReports(id)` walks the tree breadth-first.

- `PayPolicy` (interface): `SalariedPolicy` (monthly amount, prorated for part months), `HourlyPolicy` (rate × timesheet hours, overtime), `CommissionPolicy` (base + percent of sales).
- `Deduction` (interface): `IncomeTax`, `ProvidentFund`, `LeaveWithoutPay`, applied in order.
- `PayrollRun`: a month and the list of `Payslip`s it produced. Its status is `DRAFT`, `APPROVED` or `PAID`.
- `LeaveService`: balances per leave type, and requests that a manager approves.

*Patterns.* Strategy (pay policies), Chain of Responsibility or a list of deductions, Composite (org tree), State (payroll run and leave request).

*Edge cases.* Hold money as `long` paise or `BigDecimal`, and round only once, at the end. Mid-month joiners and leavers get prorated pay. A manager cycle (A reports to B, B to A) must be rejected when a manager is assigned. Once a payroll run is `PAID`, it is never edited. Corrections go into next month as adjustments.

*Extension question.* "Change an employee's salary from the middle of the month." Store pay policies with an effective date, and prorate between the two policies.

## Components and infrastructure

**Q56. ⋆ Design a Rate Limiter (LLD)**

*Asked at:* Atlassian (P4 LLD round)

- Entities: `RateLimiter` interface, `TokenBucket`, `SlidingWindowLog`, `FixedWindowCounter`, a `RateLimiterRegistry` (one limiter per client key), and a config.
- Patterns: Strategy for the algorithm, Factory to build from config.
- What interviewers probe: thread safety (per-key lock or CAS), lazy refill based on elapsed time instead of a timer thread, and the burst behaviour of each algorithm. → *Full design:* Ch 20 — 20.7 Design a Rate Limiter; also Ch 13 — 13.11 Rate limiter — token bucket

**Q57. ⋆ Design an LRU Cache**

- Data structure: a `HashMap<key, Node>` plus a doubly linked list. Every `get` or `put` moves the node to the front, and eviction removes the tail. Both operations are O(1).
- In Java you can also show `LinkedHashMap` with `accessOrder = true` and an overridden `removeEldestEntry`.
- What interviewers probe: writing the linked list by hand with dummy head and tail nodes, and making it thread-safe. → *Full design:* Ch 20 — 20.3 Design an In-Memory Cache (and an LRU Cache from scratch)

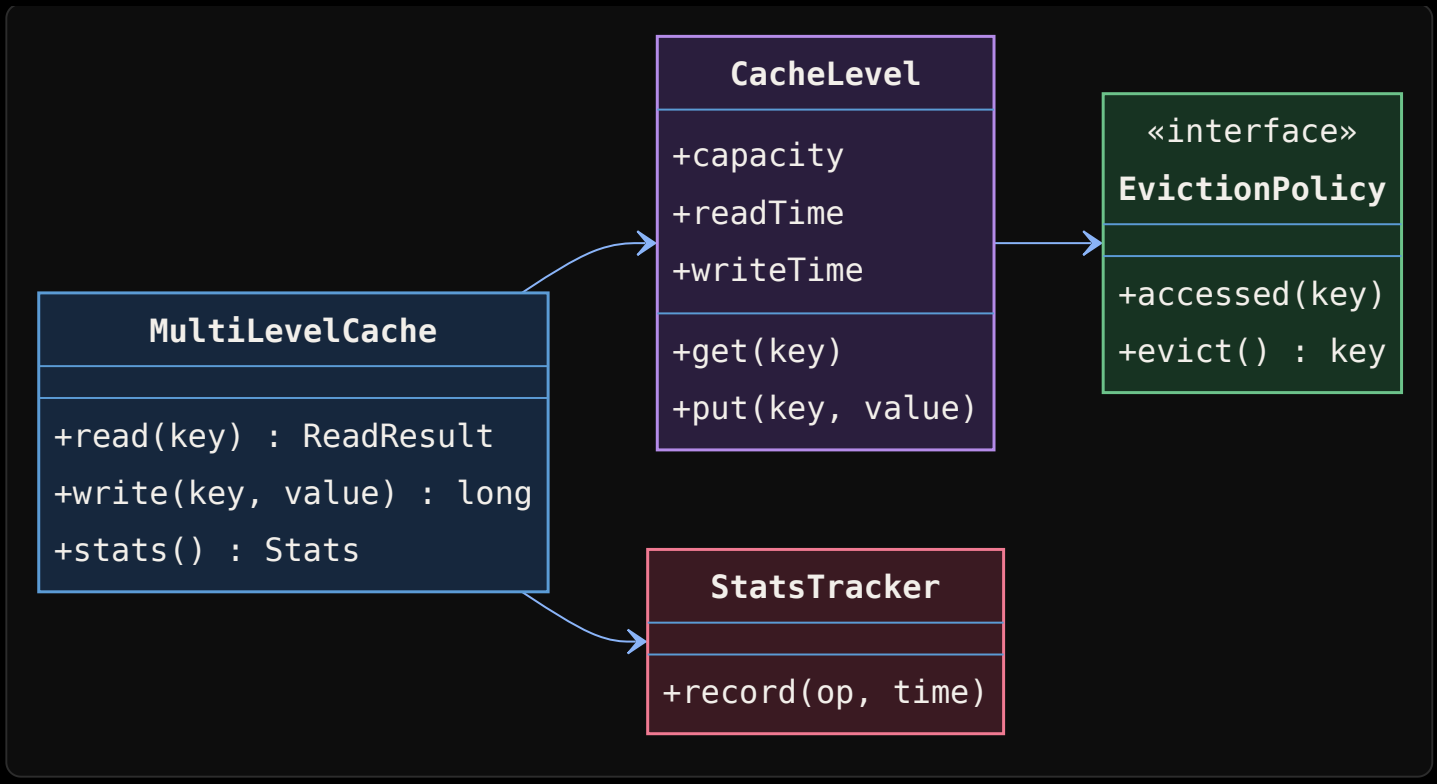
**Q58. ⋆ Design a cache with pluggable eviction policies (LRU / LFU)**

- Entities: `Cache<K, V>`, `Storage`, `EvictionPolicy` (`keyAccessed`, `evictKey`) with LRU, LFU and FIFO implementations.
- Pattern: Strategy for eviction. LFU in O(1) uses a frequency → linked set map plus a `minFreq` pointer (LeetCode 460).
- What interviewers probe: tie-breaking in LFU, and keeping the policy and the storage in step. → *Full design:* Ch 20 — 20.3 Design an In-Memory Cache

**Q59. ⋆ Design a multi-level cache**

*Requirements.* A cache has N levels (L1, L2, … Ln). Each level has a capacity, a read time and a write time. Reading a key checks L1 first, then each lower level, until the key is found. When found at level i, write the value into every level above i. Writing a key writes it to every level, skipping a level that already holds the same value. Report the time taken by every read and write, plus the average read and write time over the last N operations. Show usage per level.

*Main classes.*




- `CacheLevel`: one level, with its own storage and `EvictionPolicy` (LRU by default).
- `MultiLevelCache`: holds the ordered list of levels and runs the read and write paths. Time is the sum of each level's read time visited, plus the write times added.
- `StatsTracker`: a ring buffer of the last N timings for reads and for writes, with running sums so averages are O(1).

*Patterns.* Chain of Responsibility (a level passes a miss to the next one), Strategy (eviction), Builder (configure levels).

*Concurrency and edge cases.* In a machine-coding round, guard the whole cache with one lock (`synchronized` or a `ReentrantLock`). A read-write lock does not help here, because a read is not read-only: it updates each level's LRU order and copies the value into the levels above, so every read would need the write lock anyway. For more throughput, lock levels one at a time, always top-down, so no deadlock can form. An eviction at L1 does not remove the key from L2. Reject capacity 0 or an empty list of levels.

*Extension question.* "Make it write-back instead of write-through." Writes go to L1 only and are marked dirty. They reach lower levels when evicted, and a flush is forced on shutdown.

**Q60. ⋆ Design an in-memory key-value store (with transactions / TTL / versioning)**

- Base: `ConcurrentHashMap<String, Entry>` where `Entry` holds value and expiry; expired entries are removed lazily on read plus by a periodic sweeper.
- Transactions (`begin`, `set`, `get`, `commit`, `rollback`, nestable): keep a stack of change maps. Reads check the top map, then the maps below it, then the base store. `commit` merges the top map down, and `rollback` pops it.
- Versioning: store a list of `(version, value)` per key and answer `get(key, version)` by binary search.
- What interviewers probe: isolation between clients (one transaction stack per session), and deletes inside a transaction (store a tombstone marker). → *Related design:* Ch 20 — 20.3 Design an In-Memory Cache; Ch 13 — 13.6 Thread-safe cache with TTL

**Q61. ⋆ Design a Logging Framework / Logger library**

- Entities: `Logger`, `LogLevel`, `LogMessage`, `Appender` (console, file, database), `Formatter`, `LoggerConfig`.
- Patterns: Singleton or factory for loggers, Strategy for appenders and formatters, Chain of Responsibility for level filtering.
- What interviewers probe: async logging through a queue without losing messages on shutdown, and per-package log levels. → *Full design:* Ch 20 — 20.5 Design a Logging Framework

**Q62. ⋆ Design a Pub-Sub system / message queue (LLD)**

- Entities: `Broker`, `Topic`, `Message`, `Publisher`, `Subscriber`, `Subscription` (with offset).

- Patterns: Observer, and a per-subscriber offset so slow consumers don't block fast ones.
- What interviewers probe: at-least-once delivery with acks and retries, ordering within a topic, and replay from an offset. → *Full design:* Ch 20 — 20.6 Design a Pub-Sub System

**Q63. ⋆ Design a Notification System (LLD)**

*Asked at:* Flipkart

- Entities: `Notification`, `Channel` (email, SMS, push), `Template`, `UserPreference`, `NotificationService`, `RetryPolicy`.
- Patterns: Strategy per channel, Template Method or Builder for messages, Observer for events, Decorator for retries.
- What interviewers probe: user opt-outs, retries with backoff, falling back to another channel, and not sending duplicates. → *Full design:* Ch 20 — 20.2 Design a Notification Service

**Q64. Design a HashMap**

- Array of buckets; each bucket is a linked list (Java turns long chains into balanced trees); index = `hash(key) & (capacity - 1)`; resize and rehash when size exceeds capacity × load factor (0.75).
- For the concurrent version: lock striping (a lock per group of buckets) or CAS on empty buckets.
- What interviewers probe: `equals` / `hashCode` contract, why capacity is a power of two, and resizing under concurrency. → *Full design:* Ch 13 — 13.7 Design a (simplified) Concurrent HashMap

**Q65. ⋆ Design a thread-safe blocking queue / producer-consumer**

- A bounded buffer: a lock with two conditions, `notFull` and `notEmpty`. `put` waits while the queue is full, and `take` waits while it is empty. Always wait inside a `while` loop, never an `if`.
- Alternative: two `Semaphore`s (free slots, filled slots) plus a mutex.
- What interviewers probe: spurious wakeups, `signal` vs `signalAll`, and shutdown with a poison pill. → *Full design:* Ch 13 — 13.8 Thread-safe bounded Blocking Queue; Ch 12 — 12.22 Producer-consumer pattern

**Q66. Design a thread-safe cache with TTL**

- `ConcurrentHashMap` of entries with expiry time; lazy expiry on `get`; a `ScheduledExecutorService` or a `DelayQueue` for active cleanup.
- What interviewers probe: get-or-compute without two threads loading the same key (`computeIfAbsent`), and clock choice (`System.nanoTime` for durations). → *Full design:* Ch 13 — 13.6 Thread-safe cache with TTL

**Q67. Design a concurrent Bloom filter**

- A bit array with k hash functions. "Might contain" can give false positives but never false negatives.
- Concurrent version: an `AtomicLongArray` with CAS on each word, so adds are lock-free.
- What interviewers probe: choosing the size and k for a target false-positive rate, and why you cannot delete (use a counting Bloom filter for that). → *Full design:* Ch 13 — 13.9 Concurrent Bloom Filter

**Q68. Design a thread pool / executor**

- Entities: a work queue (`BlockingQueue<Runnable>`), worker threads looping on `take()`, and `submit` returning a `Future`. Shutdown has a graceful mode and an immediate mode.
- What interviewers probe: what happens when the queue is full (rejection policy), worker death on exceptions, and when virtual threads make a pool unnecessary. → *Full design:* Ch 12 — 12.21 Thread pool pattern

**Q69. Dining philosophers / print FooBar alternately / building H2O (LeetCode concurrency)**

- Short puzzles about making threads take turns or meet at a point. The tools are semaphores, `CyclicBarrier`, lock ordering, and conditions.
- Dining philosophers: break the deadlock by always taking the lower-numbered fork first, or by letting at most four philosophers sit at once.
- What interviewers probe: proving there is no deadlock and no starvation. → *Full design:* Ch 13 — 13.1–13.5

**Q70. Design a connection pool / object pool**

- Entities: `Pool` with idle and in-use sets, `acquire(timeout)` and `release`, max size, validation on borrow, and idle eviction.
- Pattern: Object Pool, with a `Semaphore` capping how many objects can be borrowed at once.

- What interviewers probe: leaked connections (borrow timeouts, try-with-resources wrapper), and broken connections returned to the pool. → *Full design:* Ch 8 — 8.14 Bonus — Object Pool

**Q71. Design a payment gateway (Razorpay) — LLD**

- Entities: `PaymentRequest`, `Payment` (state: created, authorised, captured, failed, refunded), `PaymentMethod` (card, UPI, net banking), `ProcessorAdapter` per bank, `Refund`, `WebhookNotifier`.
- Patterns: State (payment lifecycle), Strategy/Adapter (methods and processors), Observer (webhooks to merchants).
- What interviewers probe: an idempotency key so retries never double-charge, timeouts with an unknown result (check status before retrying), and refunds that are partial. → *Related design:* Ch 21 — 21.2 Design a Digital Wallet Service

**Q72. Design a text editor with undo / redo (Notion / Google Docs LLD)**

*Asked at:* Notion

- Entities: `Document` (a buffer, such as a list of lines or a gap buffer), `Cursor`, `EditCommand` (insert, delete), and `History` (an undo stack and a redo stack).
- Patterns: Command (each edit knows how to undo itself) or Memento (save snapshots). Any new edit clears the redo stack.
- What interviewers probe: Command vs Memento memory cost, grouping keystrokes into one undo step, and collaborative editing (operational transforms) as a stretch. → *Full design:* Ch 10 — 10.2 Command and 10.5 Memento

**Q73. Design a business rule engine**

- Entities: `Rule` (condition + action, priority), `Condition` (composable with and / or / not), `Fact` (input data), `RuleEngine` (evaluate in priority order, first-match or all-match).
- Patterns: Specification (composable conditions), Chain of Responsibility (rule order), Interpreter if rules come from a text language.
- What interviewers probe: loading rules from config without redeploying, and conflicts when two rules match. → *Related design:* Ch 11 — 11.7 Beyond GoF (Specification); Ch 10 — 10.1 Chain of Responsibility

**Q74. Design a circuit breaker (LLD)**

- States: `Closed` (count failures), `Open` (fail fast until a cool-down ends), `HalfOpen` (let a few trial calls through; success closes, failure re-opens).
- Patterns: State, plus Proxy/Decorator wrapping the protected call.
- What interviewers probe: a sliding window of failure rate vs a plain count, and thread-safe state changes (`AtomicReference` with CAS). → *Related design:* Ch 20 — 20.2 Design a Notification Service (provider circuit breaker)

**Q75. Design a Hit Counter (LeetCode 362)**

*Requirements.* `hit(timestamp)` records a hit at a time given in seconds. `getHits(timestamp)` returns how many hits happened in the past 300 seconds, including the current second. Timestamps arrive in non-decreasing order. Follow-ups: what if there are millions of hits per second, and what if it must be thread-safe?

*Main classes.*

- `HitCounter` (interface) with `hit` and `getHits`.
- `QueueHitCounter`: a deque of timestamps. `getHits` drops entries older than `t - 299` and returns the size. It is simple, but memory grows with the number of hits.
- `BucketHitCounter`: two arrays of size 300, `times[]` and `counts[]`. The index is `t % 300`. If `times[i] != t`, the bucket is stale, so reset it to `(t, 1)`; otherwise increment. `getHits` sums the buckets whose `times[i] > t - 300`. Memory is fixed, and `getHits` costs O(300).
- `HitCounterRegistry`: one counter per key (per URL or per user), in a `ConcurrentHashMap`.

*Patterns.* Strategy (swap implementations), with the sliding window as the core idea. This is the same window as a rate limiter, and interviewers often link the two.

*Concurrency and edge cases.* For thread safety, guard each bucket's reset-or-increment as one step. Lock per bucket (stripes), or store `(time, count)` packed into one `AtomicLong` updated with CAS. For very high hit rates, use a `LongAdder` per bucket. Hits from the future should not happen, but clamp them anyway. If time jumps forward by more than 300 seconds, the stale check already clears old buckets.

*Extension question.* "Make the window configurable and also report per minute and per hour." Keep several bucket arrays at different resolutions (seconds, minutes), like a time-series database's roll-ups.

**Q76. Design a URL shortener (LLD / class design)**

*Requirements.* `shorten(longUrl)` returns a short code such as `aZ3k9Q`. `resolve(code)` returns the long URL. Users may request a custom alias. Links can expire. The service counts clicks. The same long URL may get the same code or a new one; ask which.

*Main classes.*

- `ShortUrl`: a record (code, longUrl, ownerId, createdAt, expiresAt).
- `CodeGenerator` (interface): `CounterBase62Generator` (an `AtomicLong` counter encoded in base 62, short with no collisions, but guessable), `HashGenerator` (the first 7 characters of a hash of the URL, retrying with a salt on collision), and `RandomGenerator`.
- `UrlRepository`: `saveIfAbsent(ShortUrl)`, `find(code)`. The in-memory version is a `ConcurrentHashMap`.
- `UrlShortenerService`: validates the URL, applies the alias or generates a code, and handles expiry.
- `ClickTracker`: a `LongAdder` per code, with events sent on asynchronously for analytics.

*Patterns.* Strategy (code generation), Repository, Facade (the service), Observer (click events).

*Concurrency and edge cases.* Two users may ask for the same custom alias. `putIfAbsent` decides the winner atomically, and the loser gets "alias taken". The counter generator never collides, while hash and random generators must retry. Reject non-HTTP(S) URLs and links to the shortener itself (loops). Expired codes return "not found". Remove them lazily, plus a sweeper. Keep codes case-sensitive and avoid look-alike characters if humans type them.

*Extension question.* "Hide the order of codes, so people cannot enumerate links." Scramble the counter with a reversible mixing function before encoding. It stays collision-free but no longer looks sequential.

**Q77. Design a JSON / CSV parser**

*Requirements.* Parse JSON text into an in-memory tree: objects, arrays, strings (with escapes and `\uXXXX`), numbers (sign, fraction, exponent), `true`, `false` and `null`. Give clear errors with line and column. Serialise the tree back to text. For CSV: split rows into fields, where quoted fields may contain commas, newlines and doubled quotes (`""`).

*Main classes.*

- `JsonValue` (sealed interface) with records `JsonObject(Map)`, `JsonArray(List)`, `JsonString`, `JsonNumber(BigDecimal)`, `JsonBool`, `JsonNull`.
- `Lexer`: turns characters into `Token`s (`LBRACE`, `STRING`, `NUMBER`, …) and tracks position.
- `Parser`: recursive descent. `parseValue` looks at the next token and calls `parseObject`, `parseArray` and so on. One method per grammar rule.
- `JsonWriter`: walks the tree with a pattern-matching `switch` over the sealed types. The compiler (Java 21 and later) checks that every case is covered.
- `ParseException`: message, line, column.
- `CsvReader`: a small state machine with the states `FIELD_START`, `IN_UNQUOTED`, `IN_QUOTED`, `QUOTE_IN_QUOTED`.

*Patterns.* Interpreter/Composite (the value tree), Visitor-style traversal (here a sealed `switch`), State (the CSV reader), Builder for writing.

*Edge cases.* Duplicate keys (last wins, or an error). Very deep nesting, so add a depth limit against stack overflow. Leading zeros (`01` is invalid in JSON). Trailing commas (invalid). Big numbers (use `BigDecimal`). Text left over after the root value (an error). For huge inputs, offer a streaming, event-based API (`startObject`, `key`, `value`) instead of building the whole tree.

*Extension question.* "Map JSON onto Java records." Use reflection on record components (`getRecordComponents()`) to match keys to constructor parameters.

**Q78. Design a configuration management system (LLD)**

*Requirements.* Services read typed config values (`int`, `boolean`, `duration`) by key. Values can be set at several layers: defaults, then environment (dev, staging, prod), then service, then instance. The most specific layer wins. Every change creates a new version and records who changed what. Users can roll back to an earlier version. Running services are told about changes without restarting.

*Main classes.*

- `ConfigKey`: a record (namespace, name), plus a `ConfigSchema` that holds the type, default, and validation per key.
- `ConfigSnapshot`: an immutable map for one version: version number, author, time, values.
- `ConfigStore`: keeps the list of snapshots per scope, and an `AtomicReference<ConfigSnapshot>` for the current one. `set` builds a new snapshot (copy-on-write) and swaps it in with CAS.
- `ConfigResolver`: looks a key up through the layers in order (instance → service → environment → default).
- `ConfigWatcher` (Observer): a client registers a callback on keys or prefixes.
- `AuditLog`: who, what, when, old value and new value.

*Patterns.* Chain of Responsibility (layered lookup), Observer (change listeners), Memento/immutable snapshots (versions and rollback), Strategy for value parsers.

*Concurrency and edge cases.* Readers never lock: they read the current snapshot reference. Writers race through CAS and retry on conflict, or use compare-by-version so a stale edit is rejected. Validate against the schema before publishing, so a bad value never goes live. A listener that throws must not stop the other listeners.

*Extension question.* "Add feature flags rolled out to 10% of users." Add a `FlagRule` with a percentage. Hash the user id into 0–99 and compare it, so each user gets a stable result.

**Q79. Design AWS Lambda (serverless) — LLD**

*Requirements.* Users register a function: name, runtime (Java, Python), memory, timeout, code location. `invoke(name, payload)` runs it synchronously and returns the result, or runs it asynchronously with retries. Warm environments are reused, and new ones are started (cold start) when none is free. A per-function limit caps how many run at once. Idle environments shut down after a while.

*Main classes.*

- `FunctionConfig`: a record (name, runtime, memoryMb, timeout, maxConcurrency, codeRef).
- `FunctionRegistry`: create, update and delete functions, with versions.
- `ExecutionEnvironment`: one sandbox. Its state goes `INITIALISING` → `READY` → `BUSY` → `READY` …, and finally `TERMINATED`.
- `RuntimeFactory`: builds an environment for each runtime type.
- `EnvironmentPool`: per function, a deque of warm environments (newest first, so old ones age out).
- `Invoker`: takes a concurrency permit, borrows or creates an environment, runs with a timeout, and returns the environment to the pool.
- `AsyncQueue` + `RetryPolicy` + `DeadLetterQueue` for asynchronous calls.
- `IdleReaper`: a scheduled task that terminates environments idle longer than a limit.

*Patterns.* Object Pool (warm environments), Factory (runtimes), State (environment lifecycle), Proxy (the `Invoker` hides pooling and limits), Strategy (retry backoff).

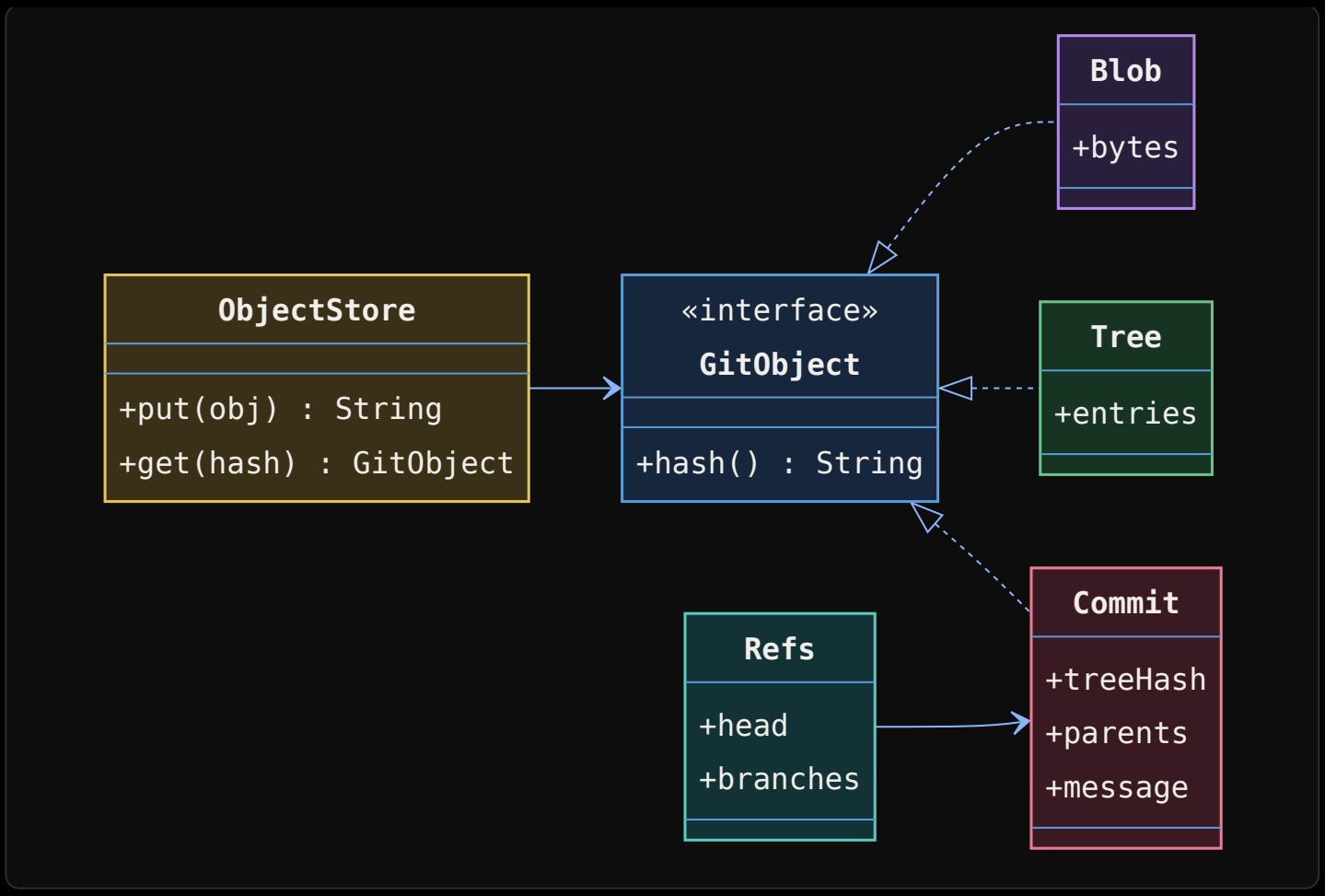
*Concurrency and edge cases.* A per-function `Semaphore` enforces `maxConcurrency`. Throttle by failing fast, or queue async calls. Run the handler on a virtual thread and wait with `Future.get(timeout)`. On timeout, kill and discard that environment, never reuse it. A crashed environment is removed, not returned. An update to the code marks old warm environments as stale.

*Extension question.* "Add provisioned concurrency." Pre-create N environments at deploy time, and keep the reaper from removing them.

**Q80. Design a version-control / Git-like system**

*Requirements.* Support `init`, `add` (stage files), `commit`, `log`, `branch`, `checkout`, `diff`, and `merge` for a single local repository. Identical content is stored once. History is never changed by normal commands.

*Main classes.*




- `Blob`: file content. `Tree`: a directory with name → (blob or tree) hash entries. `Commit`: tree hash, parent hashes, author, time, message. All of them are immutable records.
- `ObjectStore`: content-addressed storage, a map from hash (SHA-256 of the content) to object. The same content always gets the same hash, so duplicates cost nothing.
- `Index` (staging area): path → blob hash for the next commit.
- `Refs`: branch name → commit hash, and `HEAD` → a branch name (or a commit, the "detached" state).
- `Repository`: a facade exposing the commands, with each command as a `Command` object for the CLI.

*Patterns.* Composite (trees of trees), Command (CLI operations), immutable value objects, Facade.

*Algorithms and edge cases.* `log` follows parent links. `merge` first finds the merge base, the lowest common ancestor of the two commits (BFS from both sides). If one side is the base, just move the branch pointer (a fast-forward). Otherwise do a three-way merge per file and mark conflicts. `checkout` with uncommitted changes must refuse, or warn. Moving a branch is one atomic reference update.

*Extension question.* "Add `revert`." Create a new commit that undoes the changes of an earlier one. Never rewrite history.

## Machine-coding round favourites

**Q81. ⋆ Design Splitwise (expense sharing)**

*Asked at:* Swiggy (SDE1; SDE2, twice)

- Entities: `User`, `Group`, `Expense` with a `Split` type (equal, exact, percent), `BalanceSheet` (who owes whom), `SettlementService`.
- Patterns: Strategy/Factory for split types, plus validation that the splits add up to the total.
- What interviewers probe: rounding (give the leftover paise to one person), simplifying debts (greedy matching of the biggest creditor with the biggest debtor), and the command-line input format. → *Full design:* Ch 20 — 20.1 Design Splitwise

**Q82. ⋆ Design a task scheduler / distributed task scheduler (LLD)**

*Asked at:* Flipkart (SDE2, Jan 2026)

- Entities: `Task` (id, payload, run-at time, recurrence, retries), `Scheduler` (`schedule`, `cancel`), a `DelayQueue` or `PriorityQueue` ordered by run time, a `Worker` pool, and an `ExecutionRecord`.
- Design: one dispatcher thread waits for the next due task (a `Condition` with a timed wait, woken early when a sooner task arrives) and hands it to a worker pool. A recurring task schedules its next run after finishing.
- What interviewers probe: cancelling a queued task, retries with backoff, and the distributed version (lease a task in storage with a timeout, so a crashed worker's task is picked up again). → *Related design:* Ch 12 — 12.21 Thread pool pattern

**Q83. ⋆ Design an in-memory file system (mkdir / ls / find)**

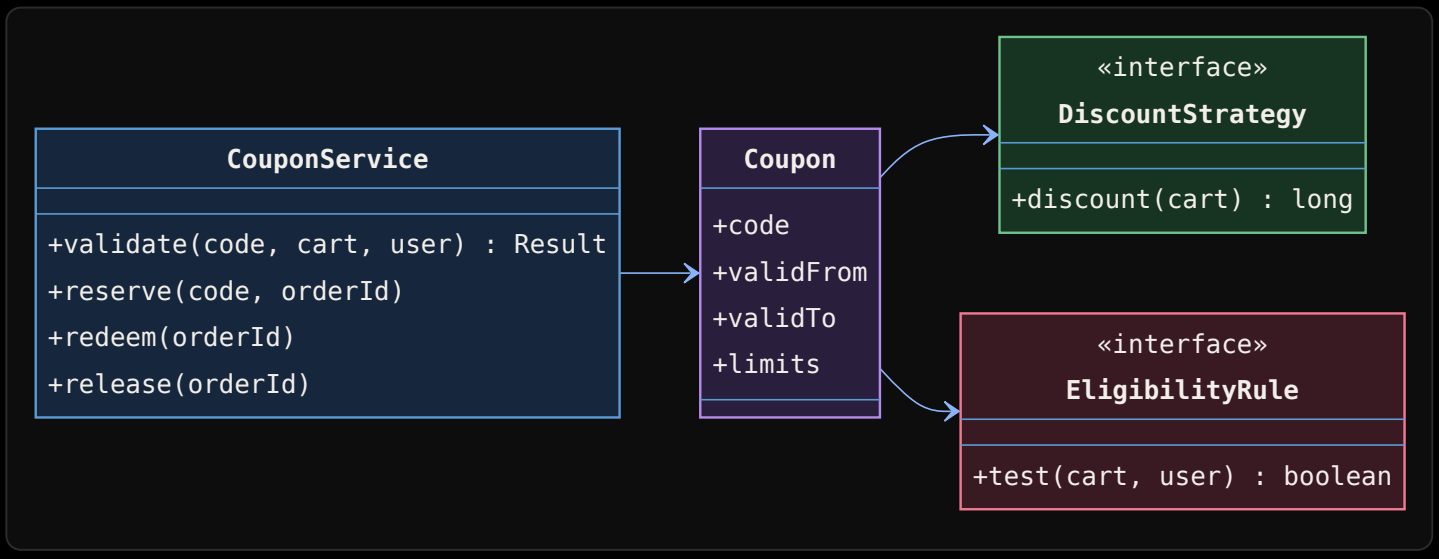
*Asked at:* ClearTrip and others (Unix `find`)

- Entities: `FileSystemNode` (sealed: `File`, `Directory`), `Path` parsing, and `FileSystem` (`mkdir -p`, `ls`, `addContent`, `readContent`).
- Patterns: Composite for the tree, Specification/Strategy for `find` filters (name, extension, size) combined with and / or.
- What interviewers probe: adding a new filter without changing existing ones, and walking a deep tree without recursion limits. → *Full design:* Ch 21 — 21.8 Design a Unix File Search (`find`); Ch 9 — 9.3 Composite Pattern

**Q84. ⋆ Design a coupon / promo-code system**

*Requirements.* Admins create coupons: a code, a discount type (flat ₹100 off, 20% off up to ₹150, buy-one-get-one), a validity window, a total usage limit, and a per-user limit. Eligibility rules include a minimum cart value, certain categories, first order only, and certain user segments. At checkout the user applies a code. The system validates it, computes the discount, and redeems it when payment succeeds. Coupons can or cannot be combined (stacking).

*Main classes.*




- `DiscountStrategy`: `FlatDiscount`, `PercentDiscount(cap)`, `BuyXGetY`. The result never exceeds the eligible amount.
- `EligibilityRule`: small rules joined with `and` / `or` (Specification). Each failure gives a reason the user can read.
- `Redemption`: a record (code, userId, orderId, status: `RESERVED`, `REDEEMED`, `RELEASED`).
- `UsageCounter`: total and per-user counts.

*Patterns.* Strategy (discounts), Specification / Chain of Responsibility (eligibility), Factory (build a coupon from an admin request).

*Concurrency and edge cases.* A "first 1,000 users" coupon must not be used 1,001 times. At checkout, *reserve* a use: increment the counter atomically only while it is below the limit (a CAS loop on an `AtomicInteger`). Move it to `REDEEMED` when payment succeeds, and *release* it on failure or timeout. Make redeeming idempotent per order id. Codes are case-insensitive. Check validity windows with a time zone in mind.

*Extension question.* "Auto-apply the best coupon." Evaluate every eligible coupon's discount against the cart and pick the biggest, and allow stacking only within the stacking rules.

**Q85. ⋆ Design an Event Calendar (Flipkart; Google Calendar)**

*Asked at:* Flipkart

- Entities: `User`, `Event` (time range, organiser, participants), `Team`, `Invite` (accept / reject), `Calendar` view.
- Patterns: Observer for invites, and an interval-overlap check for conflicts.
- What interviewers probe: finding a common free slot for a team, recurring events, and the per-user agenda for a day. → *Full design:* Ch 21 — 21.3 Design an Event Calendar (Flipkart)

**Q86. ⋆ Design a LeetCode-like coding platform ("Coding Blox", Flipkart)**

*Asked at:* Flipkart

- Entities: `User` (score), `Contest`, `Question` (difficulty level, score), `Submission`, `Leaderboard`.
- Patterns: Strategy for scoring and question selection, Observer for leaderboard updates.
- What interviewers probe: contest states (created, running, ended), only the creator can start a contest, and ordering the leaderboard. → *Full design:* Ch 21 — 21.4 Design a LeetCode-like Coding Platform

**Q87. ⋆ Design a property listing site ("PropertyHunt", ClearTrip)**

*Asked at:* ClearTrip

- Entities: `User`, `Property` (location, price, size, type, status), `Shortlist`, `SearchQuery`.
- Patterns: Specification/Builder for search filters, Strategy for sorting.
- What interviewers probe: combining many optional filters cleanly, and marking a property sold so it leaves search results. → *Full design:* Ch 21 — 21.5 Design a Property Listing Site

**Q88. ⋆ Design a loan ledger ("Ledger Co", Navi)**

*Asked at:* Navi

- Entities: `Loan` (principal, rate, years, EMI), `Payment` (lump sum after EMI n), `Bank`, `LedgerService`, and commands `LOAN`, `PAYMENT` and `BALANCE`.
- Patterns: Command for each input line, and pure functions for the EMI and balance maths.
- What interviewers probe: rounding EMIs up, how a lump sum shortens the remaining EMIs, and parsing the input file robustly. → *Full design:* Ch 21 — 21.6 Design a Loan Ledger

**Q89. ⋆ Design a Task Management System / Jira / task planner**

- Entities: `Task` (title, assignee, status, due date, priority), `Story`, `Bug`, `Sprint`, `User`, `TaskService`.
- Patterns: State for status transitions, Factory for task types, Observer for assignment notifications.
- What interviewers probe: valid transitions per task type, subtasks, and filtering or sorting by many fields. → *Full design:* Ch 21 — 21.1 Design a Task Management System

**Q90. ⋆ Design a To-Do list app**

*Asked at:* PhonePe

- A smaller task manager: `TodoItem` (text, done, due date, tags), `TodoList`, add / edit / complete / delete, filters, and an activity log.
- Patterns: Command for actions (gives undo and the activity log for free), Strategy for sorting.
- What interviewers probe: soft delete, undo, and recurring tasks. → *Related design:* Ch 21 — 21.1 Design a Task Management System

**Q91. ⋆ Design a Digital Wallet (Paytm / PhonePe)**

- Entities: `User`, `Wallet` (balance), `Transaction` (credit, debit, transfer), `Offer`, `Statement`.
- Patterns: Strategy for offers and cashback, Command or a ledger of immutable transactions.
- What interviewers probe: transfers between two wallets without deadlock (lock in id order), money as whole paise, and idempotent transfers. → *Full design:* Ch 21 — 21.2 Design a Digital Wallet Service

**Q92. ⋆ Design an Inventory / Warehouse Management System**

- Entities: `Product` / `SKU`, `Warehouse`, `StockLevel`, `StockMovement` (in, out, transfer), `Reservation`, `ReorderRule`.

- Patterns: Observer for low-stock alerts, Strategy for picking a warehouse, and reserve-then-commit for orders.
- What interviewers probe: preventing overselling under concurrent orders, and keeping a movement log so stock can be audited. → *Full design:* Ch 21 — 21.7 Design an Inventory Management System

## Part B — Concept questions

These are the "explain X" and "X vs Y" questions that come up in a low-level design (LLD) round, usually in the first 10 minutes or while you are coding. Answer in two to four sentences, then give a tiny example from your design.

## OOP basics

**Q93. ⋆ What are the four pillars of OOP? Explain each with an example.** **Encapsulation** keeps an object's data private and changes it only through methods that check the rules (a `BankAccount` whose `withdraw` refuses overdrafts). **Abstraction** shows *what* something does and hides *how* (a `PaymentGateway` interface with `pay()`). **Inheritance** lets a subclass reuse and specialise a parent (`SavingsAccount extends Account`). **Polymorphism** means one call runs different code depending on the real object type (`account.monthlyFee()`). → *More:* Ch 02 — 2.3–2.6 The four pillars

**Q94. ⋆ Encapsulation vs abstraction: what is the difference?** Encapsulation is about *protecting state*: fields are private and every change goes through a method that keeps the object valid. Abstraction is about *hiding complexity*: callers depend on a simple contract (an interface or a small public API) and never see the details. Encapsulation is done inside one class, while abstraction is a design decision about what to expose at all. → *More:* Ch 02 — 2.3 Encapsulation, 2.4 Abstraction

**Q95. ⋆ Compile-time vs runtime polymorphism; overloading vs overriding.** **Overloading** means several methods share one name but have different parameter lists. The compiler picks one from the declared argument types, so this is compile-time (static) polymorphism. **Overriding** means a subclass redefines an inherited instance method with the same signature. The JVM picks the version from the object's actual class while the program runs (dynamic dispatch), so this is runtime polymorphism. Return types may be covariant (a subtype) when overriding, and the override cannot reduce visibility or throw broader checked exceptions. → *More:* Ch 02 — 2.7 Overloading vs overriding

**Q96. Can you override a static or a private method?** No. A `static` method with the same signature in a subclass *hides* the parent's method, and the call is chosen by the declared type, not the object. A `private` method is invisible to subclasses, so a method with the same name there is a brand-new method. `final` methods cannot be overridden either. → *More:* Ch 02 — 2.7 Overloading vs overriding

**Q97. ⋆ Interface vs abstract class: when do you use which?** Use an **interface** for a capability that unrelated classes can share (`Comparable`, `PaymentMethod`). A class can implement many interfaces, and an interface has no instance state. Use an **abstract class** when closely related classes share state or a fixed algorithm skeleton (constructor, protected fields, Template Method). A class can extend only one class. In LLD rounds, default to interfaces and add an abstract base class only when real shared code appears. → *More:* Ch 02 — 2.9 Abstract classes vs interfaces

**Q98. What are default, static and private methods in interfaces?** A **default** method (Java 8) has a body that implementing classes inherit, so you can add methods to an interface without breaking existing implementers. A **static** interface method is a helper called as `Shape.unit()` and is not inherited. **Private** interface methods (Java 9) share code between default methods. Interfaces still cannot hold instance fields, only constants. → *More:* Ch 02 — 2.8 Interfaces

**Q99. Can an abstract class have a constructor? Can it be instantiated?** It can have constructors, and they run when a subclass is created (via `super(...)`) to initialise the shared fields. You cannot write `new` on it directly, although an anonymous subclass works. Since Java 25 (flexible constructor bodies, JEP 513), a constructor may run statements *before* calling `super(...)`: it can validate or prepare arguments and assign its own fields, but it cannot read fields, call instance methods or otherwise use `this`. → *More:* Ch 02 — 2.2 Classes, objects, constructors

**Q100. ⋆ Composition vs inheritance: why "favor composition"?** Inheritance ties a subclass to its parent's internals at compile time, so a parent change can break every child (the "fragile base class"), and Java allows only one parent. Composition means an object *holds* collaborators behind interfaces and delegates to them. You can swap them at runtime and combine behaviours without an explosion of subclasses. Use

inheritance only for a true, stable IS-A relationship where the subclass honours the parent's contract. → *More:* Ch 02 — 2.11 Composition over inheritance

**Q101. IS-A vs HAS-A.** IS-A is inheritance or interface implementation: a `Car` *is a* `Vehicle`, so it can be used wherever a `Vehicle` is expected. HAS-A is a field reference (association, aggregation or composition): a `Car` *has an* `Engine`. If you can't say "every X is a Y and can stand in for it" truthfully, use HAS-A. → *More:* Ch 03 — 3.8 Decision guide

**Q102. ★ What is the diamond problem? How does Java handle multiple inheritance?** The diamond problem appears when class D inherits the same method from two parents B and C that both override A's version, so it is unclear which one D gets. Java avoids it for state by allowing only one superclass. With interfaces, if two default methods clash, the class *must* override the method and may pick one with `B.super.method()`. A class method always beats an interface default. → *More:* Ch 02 — 2.8 Interfaces

**Q103. ★ Explain the `equals()` and `hashCode()` contract.** Two objects that are `equals` must return the same `hashCode`. The reverse is not required, because collisions are allowed. Hash collections find the bucket with `hashCode` and then compare with `equals`. If you override only `equals`, a `HashSet` will hold "duplicates" and `map.get` will miss keys. Build both from the same fields, never from mutable fields of a key while it sits in a map, and use records for value objects. → *More:* Ch 02 — 2.12 equals/hashCode contract

**Q104. ★ How do you make a class immutable? What do records give you?** Make the class `final`, all fields `private final`, set them once in the constructor, provide no setters, and make defensive copies of mutable inputs and outputs (`List.copyOf`). A **record** gives final fields, a canonical constructor, accessors, `equals`, `hashCode` and `toString` in one line. You still copy mutable components in a compact constructor.

```
record Order(String id, List<String> items) {
    Order { items = List.copyOf(items); }   // defensive copy, also rejects nulls
}
```

→ *More:* Ch 02 — 2.13 Immutability and records

**Q105. Explain the access modifiers, and `static` vs `final`.** `private` means this class only. *No modifier* (package-private) means the same package. `protected` means the package plus subclasses. `public` means everyone. `static` means "belongs to the class, not to each object" (one shared counter, a factory method). `final` means "cannot change": a final variable is assigned once, a final method cannot be overridden, a final class cannot be extended. They are independent, so `static final` is a class-level constant. → *More:* Ch 02 — 2.2 and 2.14

**Q106. ★ Coupling vs cohesion. Cohesion** measures how closely the parts of one class belong together. Aim high: one clear job. **Coupling** measures how much one class depends on others' details. Aim low: depend on small interfaces. High cohesion plus low coupling means a change stays in one place and is easy to test. These two yardsticks sit behind almost every principle and pattern. → *More:* Ch 02 — 2.16 Coupling and cohesion

## Relationships and UML

**Q107. ★ Association vs aggregation vs composition. Association** means two classes know about each other (`Doctor` – `Patient`). **Aggregation** is a weak whole–part link where the part can live on its own and be shared (`Team` has `Player`s). **Composition** is a strong whole–part link where the part is created and destroyed with the whole and belongs to only one owner (`Order` owns its `OrderLine`s). In UML, aggregation is a hollow diamond and composition is a filled diamond on the whole's side. → *More:* Ch 03 — 3.2–3.4

**Q108. Dependency vs association.** A **dependency** is a short-lived use: the class receives another as a method parameter, local variable or return value (a dashed arrow in UML). An **association** is a lasting structural link stored in a field (a solid line). If deleting the field removes the link, it was an association. → *More:* Ch 03 — 3.5 Dependency

**Q109. What is UML? Class vs sequence vs activity vs state diagrams.** UML (Unified Modeling Language) is a standard notation for drawing software designs. **Class diagrams** show static structure: classes, fields, methods, relationships. **Sequence diagrams** show which objects call which, in time order, for one scenario. **Activity diagrams** are flowcharts of a process. **State machine diagrams** show an object's states and the events that move it between them, as for a vending machine or an order. → *More:* Ch 03 — 3.9–3.14

**Q110. Draw a class diagram (or use-case diagram) for X.** Start with a **use-case diagram** only if actors matter: actors as stick figures, use cases as ovals ("Park vehicle", "Pay fee"). For the class diagram, list the nouns from the requirements as classes, put two or three key fields and methods in each, and draw only the relationships that carry meaning, with multiplicities (`1`, `0..*`). Mark interfaces and enums. Keep it to what you will actually code. → *More:* Ch 03 — 3.10 Use-case, 3.11 Class diagrams

**Q111. ⋆ How do you approach an LLD interview, and how much UML should you draw?** First clarify the requirements and agree on the scope (core use cases, what is out of scope). Then find the entities, then draw a lean class diagram with the key interfaces. Walk through one main flow, and only then write code with the patterns that fit. Add concurrency and extensibility last. Draw just enough UML to agree on the shape, usually one class diagram and maybe one sequence or state sketch. Interviewers judge the reasoning and the code, not notation perfection. → *More:* Ch 01 — 1.4 The step-by-step answer framework

**Q112. ⋆ LLD vs HLD: what is the difference?** HLD (high-level design) chooses the services, databases, caches, queues and how they scale across machines, with boxes and arrows. LLD (low-level design) works inside one service: classes, interfaces, relationships, patterns, method signatures and thread safety, often ending in working code. HLD asks "which components and how do they scale?", LLD asks "how is this component built cleanly?" → *More:* Ch 01 — 1.1 What this round is

**Q113. ⋆ Machine-coding round: how do you structure code in 90 minutes, and what do evaluators look for?** They want working, runnable code that shows the core flows end to end with a demo `main`. They also look for clean separation into models, services and strategies, sensible names, input validation with clear errors, and an easy path to the extension they will ask for. Keep the code in memory (maps for storage), get a vertical slice working early, then add features one at a time. Part C below gives a minute-by-minute plan. → *More:* Ch 21 — 21.0 How the machine-coding round works

## SOLID

**Q114. ⋆ Explain SOLID with examples.** **S**ingle Responsibility: one reason to change. **O**pen/Closed: add behaviour with new classes, not by editing tested ones. **L**iskov Substitution: subtypes must work wherever the parent is expected. **I**nterface Segregation: small role-based interfaces. **D**ependency Inversion: high-level code depends on abstractions, not on concrete details. Together they keep changes local and code testable. Apply them where change is likely, not everywhere. → *More:* Ch 04 — SOLID principles

**Q115. ⋆ Single Responsibility Principle with an example.** A class should have one reason to change, meaning one actor or concern drives its edits. An `Invoice` that calculates totals, prints PDFs and saves to the database changes for finance, design and DBA reasons. Split it into `Invoice` (rules), `InvoicePrinter` and `InvoiceRepository`. The test: describe the class without the word "and". → *More:* Ch 04 — 4.2 SRP

**Q116. ⋆ Open/Closed Principle with an example.** Code should be open for extension but closed for modification. A `FareCalculator` with a growing `switch (vehicleType)` must be edited for every new type. Instead, define a `PricingStrategy` interface and add `EvPricing` as a new class while the calculator stays untouched. Strategy, Decorator and Factory registries are the usual tools. → *More:* Ch 04 — 4.7 OCP

**Q117. ⋆ Liskov Substitution: explain the rectangle/square problem.** LSP says a subtype must keep the parent's promises: no stronger preconditions, no weaker postconditions, no surprise exceptions. If `Square extends Rectangle`, then `setWidth(5)` silently changes the height too, so code that expects area = width × height breaks. The fix is not to make Square a subtype of a mutable Rectangle. Use a common `Shape` interface or immutable shapes instead. → *More:* Ch 04 — 4.8 LSP

**Q118. Interface Segregation with an example.** Clients should not depend on methods they don't use. A fat `Machine` interface with `print`, `scan` and `fax` forces a basic printer to throw `UnsupportedOperationException`. Split it into `Printer`, `Scanner` and `Fax`, and let a multifunction device implement all three. → *More:* Ch 04 — 4.9 ISP

**Q119. ⋆ Dependency Inversion vs Dependency Injection vs Inversion of Control.** **DIP** is a principle: high-level policy and low-level details both depend on an abstraction that the high-level side owns (`OrderService` → `PaymentGateway` interface ← `StripeGateway`). **Dependency Injection** is a technique: dependencies are handed in from outside, usually through the constructor, instead of being created with `new`. **IoC** is the broader idea that a framework or container calls your code and wires objects ("don't call us, we'll call you"). DI is one way to do IoC, and both help you apply DIP. → *More:* Ch 04 — 4.10 DIP

**Q120. Can SOLID be overdone? How do you use it in an interview?** Yes. Interfaces with a single implementation that will never vary, factories for trivial objects and ten-class hierarchies for a two-case `if` are over-engineering. In an interview, apply SOLID at the points the interviewer is likely to

extend (pricing, notification channels, strategies) and say why. Leave stable parts simple. → *More:* Ch 04 — 4.15 Over-engineering

## Design principles and clean code

**Q121. ⋆ What are DRY, KISS and YAGNI? DRY** (Don't Repeat Yourself): each piece of knowledge, such as a rule or formula, lives in one place. **KISS** (Keep It Simple): choose the simplest design that meets the need. **YAGNI** (You Aren't Gonna Need It): don't build features or hooks for imagined future needs. They balance each other, because too much DRY or flexibility hurts simplicity. → *More:* Ch 05 — §1–3

**Q122. Law of Demeter and Tell, Don't Ask.** The **Law of Demeter** says a method should talk only to its direct friends (its fields, its parameters, objects it creates), not to strangers through chains like `order.getCustomer().getWallet().debit(x)`. **Tell, Don't Ask** says tell an object to do the work (`order.pay(x)`) instead of pulling its data out and deciding for it. Both keep logic next to its data and cut coupling. → *More:* Ch 05 — §4–5

**Q123. What does "program to an interface, not an implementation" mean?** Declare fields, parameters and return types with the abstraction (`List`, `NotificationSender`), and create the concrete class in one place (a factory or the composition root). Callers then work with any implementation, including test fakes, without changes. → *More:* Ch 05 — §7

**Q124. What does "encapsulate what varies" mean?** Find the parts of the design that are likely to change, such as pricing rules, payment providers or eviction policies. Hide each one behind an interface so that the stable code around it doesn't change when they do. Most behavioural patterns (Strategy, State, Observer) are this idea applied to one kind of change. → *More:* Ch 05 — §8

**Q125. What are the GRASP principles?** GRASP (General Responsibility Assignment Software Patterns) are nine guidelines for deciding which class gets which job. **Information Expert**: give the job to the class that has the data. **Creator**: the class that holds or aggregates B creates B. **Controller**: one object receives system events. **Low Coupling** and **High Cohesion**. **Polymorphism**. **Pure Fabrication**: invent a service class when no domain class fits. **Indirection**. **Protected Variations**. → *More:* Ch 05 — §16 GRASP

**Q126. DDD basics: entity vs value object, and what is an aggregate?** An **entity** has an identity that lasts while its attributes change (a `Customer` with an ID). A **value object** is defined only by its values, is immutable and is compared by value (`Money`, `Address`). An **aggregate** is a cluster of objects changed as one unit through a single **root** (`Order` with its lines), which protects the cluster's rules. Outsiders hold references only to the root. → *More:* Ch 05 — §17 DDD essentials

**Q127. What does "fail fast" mean?** Detect invalid input or state as early as possible and stop with a clear exception, instead of carrying bad data forward until it fails somewhere confusing. Examples are `Objects.requireNonNull` in constructors, validating in a record's compact constructor and rejecting an invalid state transition right away. → *More:* Ch 05 — §13 Fail Fast

**Q128. ⋆ What are anti-patterns? What is a God class? What is over-engineering?** An **anti-pattern** is a common "solution" that causes more problems than it solves. A **God class** knows and does too much (a `ParkingLotManager` with 40 methods), so every change touches it and it can't be tested in isolation. **Over-engineering** adds layers, patterns and configurability nobody asked for. Other anti-patterns are Singleton abuse, anemic models, magic numbers and copy-paste code. → *More:* Ch 08 — 8.6 Anti-patterns

**Q129. ⋆ What are code smells? Name a few and their refactorings.** A **code smell** is a surface sign of a deeper design problem. **Long Method** → Extract Method. **Large Class** → Extract Class. **Long Parameter List** → Introduce Parameter Object. **Switch on type** → Replace Conditional with Polymorphism. **Feature Envy** (a method that uses another class's data more than its own) → Move Method. **Duplicate Code** → Extract and reuse. **Refactoring** means changing structure in small, behaviour-preserving steps, with tests as the safety net. → *More:* Ch 07 — 7.2 Code smells, 7.4 Lookup table

**Q130. What makes code "clean" (naming, functions, error handling)?** Names reveal intent (`activeBookings`, not `list2`). Functions are small, do one thing at one level of abstraction and take few arguments. Errors are reported with exceptions that carry context, not with error codes or `null`. Comments explain *why*, not *what*. Clean code reads like the problem description. → *More:* Ch 06 — Clean code

**Q131. Checked vs unchecked exceptions: which do you use? Checked** exceptions extend `Exception` (but not `RuntimeException`). The compiler forces callers to catch or declare them, which suits recoverable conditions from outside the program, such as `IOException`. **Unchecked** exceptions extend `RuntimeException` and usually signal programming errors or broken rules (`IllegalArgumentException`, a custom `SeatAlreadyBookedException`).

Modern Java design prefers unchecked domain exceptions with clear names, plus checked ones only where the caller can really recover. → *More:* Ch 06 — §6 Error Handling

**Q132. Unit testing: mocks vs stubs, and how do you make a design testable?** A **stub** returns canned answers so the code under test can run. A **mock** also *verifies* interactions (for example, "`send` was called once"). A **fake** is a working lightweight version, such as an in-memory repository. Designs are testable when dependencies are injected through interfaces, time and randomness are injected (`Clock`), and logic sits apart from I/O. → *More:* Ch 06 — §17 Testing strategies

**Q194. ⋆ Layered vs hexagonal architecture: what is the difference?** Both separate concerns. In **layered** architecture (controller → service → repository) the domain depends *down* on persistence, so storage tends to shape the model. In **hexagonal** (ports and adapters) the core owns interfaces such as `OrderRepository`, and adapters (JDBC, in-memory, REST) depend *in* on them, so the core imports no framework and tests run with in-memory adapters. Layered suits CRUD; hexagonal pays off when the rules are valuable. → *More:* Ch 11 — 11.9 Architectural patterns

**Q195. Event-driven architecture: broker vs mediator topology? Broker** (choreography): no coordinator; each consumer reacts to events and may publish new ones. Very decoupled, but no single place shows the whole flow. **Mediator** (orchestration): a central coordinator sends ordered commands and handles failures and compensation. Use a broker for independent reactions (email, analytics) and a mediator for an ordered process with undo (order fulfilment). Either way, delivery is usually at least once, so consumers must be idempotent. → *More:* Ch 11 — 11.9.5 Event-driven architecture

**Q196. Pipe-and-filter vs Chain of Responsibility?** In a **pipeline** every filter runs and *transforms* the data, possibly into a new type (`String → fields → command`); a rejection is just one possible output. In a **chain of responsibility** each handler may handle or *stop* the request, and usually one handler deals with it. Decorator differs from both: it wraps one object behind the *same* interface. → *More:* Ch 11 — 11.9.6 Pipe-and-filter

**Q197. ⋆ Modular monolith or microservices for a new system?** Start with a **modular monolith**: one deployable split into business modules with small public interfaces, which gives clear boundaries without network failures. Extract a module into a **microservice** only for a concrete reason (independent team releases, very different scaling, fault isolation), and accept the costs: timeouts and retries, idempotency, sagas instead of cross-service transactions, eventual consistency. Services that must be deployed together are a "distributed monolith", the worst of both. → *More:* Ch 11 — 11.9.4 Monolith, modular monolith and microservices

## Creational patterns

**Q133. ⋆ What is a design pattern? What are the three GoF categories?** A design pattern is a named, reusable solution to a recurring design problem. It is a template you adapt, not a library you import. The "Gang of Four" (GoF) book lists 23 patterns in three groups. **Creational** patterns deal with how objects are made (Singleton, Factory Method, Abstract Factory, Builder, Prototype). **Structural** patterns deal with how objects are composed (Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy). **Behavioral** patterns deal with how objects communicate and share work (Strategy, Observer, State, Command and others). → *More:* Ch 08 — 8.1–8.3

**Q134. ⋆ Singleton: how do you make it thread-safe?** Use an **enum** (simplest, and safe against reflection and serialization), or the **holder idiom**: a static nested class that the JVM loads, thread-safely, on first use. **Double-checked locking** also works but needs a `volatile` field. Without `volatile`, another thread can see a half-constructed object.

```
final class Config {
    private Config() {}
    private static class Holder { static final Config INSTANCE = new Config(); }
    static Config get() { return Holder.INSTANCE; }
}
```

→ *More:* Ch 08 — 8.8 Singleton

**Q135. How can a Singleton be broken?** Reflection can call the private constructor. Deserialization creates a new instance unless you add `readResolve`. Cloning breaks it if the class is `Cloneable`. Each class loader can also hold its own copy. An enum singleton blocks the first three. → *More:* Ch 08 — 8.8 Singleton

**Q136. ⋆ Why is Singleton often called an anti-pattern?** It is global mutable state in disguise. It hides dependencies (callers reach out with `getInstance()` instead of receiving them), it makes unit tests hard because you can't swap or reset it, and it creates concurrency hot spots. Prefer creating one instance at the composition root (or a DI container's singleton scope) and injecting it. → *More:* Ch 08 — 8.8 Singleton

**Q137. ⋆ Simple Factory vs Factory Method vs Abstract Factory.** A **Simple Factory** is one method with a `switch` that returns the right subtype (`VehicleFactory.create(type)`). It is an idiom, not a GoF pattern. **Factory Method** lets subclasses decide which product to create by overriding a creation method. **Abstract Factory** creates *families* of related products that must match (a `WindowsUiFactory` makes `WinButton` + `WinCheckbox`). In interviews, a simple factory or a `Map<Type, Supplier<T>>` registry is usually enough. → *More:* Ch 08 — 8.9–8.11, 8.15

**Q138. ⋆ When would you use Builder?** Use it when an object has many parameters, several of them optional (the "telescoping constructor" problem), or when you want to validate everything before building an immutable object. The builder collects values with readable chained calls and `build()` checks them once. Records with a few fields don't need one.

```
var pizza = Pizza.builder().size(LARGE).cheese(true).topping("olive").build();
```

→ *More:* Ch 08 — 8.12 Builder

**Q139. Prototype pattern; shallow vs deep copy. Prototype** creates new objects by copying a configured existing one, which helps when construction is expensive or the configuration is complex. A **shallow copy** copies fields as they are, so the copy shares nested mutable objects with the original. A **deep copy** also copies those nested objects, so changes don't leak between them. In Java, prefer copy constructors or factory methods over `clone()`. → *More:* Ch 08 — 8.13 Prototype

**Q140. What is an object pool, and when is it worth it?** A pool keeps a fixed set of expensive, reusable objects (database connections, threads). It lends one out and takes it back, so you avoid repeated creation and cap the total. It needs a thread-safe borrow/return (often a `BlockingQueue` or a `Semaphore`) and must reset each object's state on return. For cheap objects it only adds bugs. → *More:* Ch 08 — 8.14 Object Pool

## Structural patterns

**Q141. ⋆ Adapter: when would you use it? Adapter vs Facade vs Bridge.** An **Adapter** converts one interface into another that the client expects. A typical use is wrapping a third-party `LegacyPaymentApi` so it implements your `PaymentGateway`. A **Facade** gives a *simpler* front to a whole subsystem (`HomeTheater.watchMovie()`) without changing the interfaces. A **Bridge** splits an abstraction from its implementation up front, so the two can vary independently (`Shape` × `Renderer`). Adapter fixes a mismatch after the fact, Facade simplifies, and Bridge is planned in from the start. → *More:* Ch 09 — 9.1, 9.2, 9.5, 9.8

**Q142. ⋆ Decorator vs Proxy (and vs inheritance).** Both wrap an object behind the same interface. A **Decorator** *adds* behaviour, and decorators can be stacked at runtime (`new Logging(new Compressing(stream))`), which avoids a subclass for every combination. A **Proxy** *controls access* to the same behaviour: lazy loading, caching, security checks, remote calls. Usually the proxy, not the client, decides whether to create the real object. → *More:* Ch 09 — 9.4 Decorator, 9.7 Proxy

**Q143. ⋆ Composite pattern (file system / org chart).** Composite treats single objects and groups of objects through one interface, so the client doesn't care which it has. A `File` and a `Directory` both implement `FileSystemNode.size()`, and the directory sums its children recursively. Use it for any tree: menus, org charts, expression trees. → *More:* Ch 09 — 9.3 Composite

**Q144. What is the Flyweight pattern?** Flyweight saves memory when there are huge numbers of similar objects. It shares the *intrinsic* (unchanging, shareable) state and passes the *extrinsic* (per-use) state in from outside. Examples: one shared `Glyph` per character in a text editor, with the position stored separately, or tree types shared across a million trees in a game. `Integer.valueOf` caching small values is a JDK example. → *More:* Ch 09 — 9.6 Flyweight

**Q145. How does Java support Proxy at runtime (dynamic proxies, Spring AOP)?** `java.lang.reflect.Proxy` creates, at runtime, a class that implements given interfaces and sends every call to an `InvocationHandler`. That is how logging, transactions and retries get added around calls. Spring AOP builds either a JDK proxy (interfaces only) or a generated subclass (CGLIB). Plain Spring uses a JDK proxy when the bean has an

interface, while Spring Boot defaults to CGLIB subclasses for everything. Either way the advice lives in the proxy, so a `@Transactional` method called from inside the same class (through `this`) bypasses it. → *More:* Ch 09 — 9.7 Proxy

## Behavioral patterns

**Q146. ⋆ Strategy pattern: where did you use it?** Strategy puts each interchangeable algorithm behind an interface and lets the context pick one at runtime. Examples are a parking fee per vehicle type, Splitwise's equal/exact/percent splits, cache eviction (LRU/LFU) and payment methods. It removes `switch` statements and follows Open/Closed. In modern Java a strategy is often just a lambda.

```
interface SplitStrategy { Map<User, Money> split(Money total, List<User> users); }
```

→ *More:* Ch 11 — 11.3 Strategy

**Q147. ⋆ Strategy vs State.** They have the same class shape but a different intent. With **Strategy**, the *client* picks the algorithm, and strategies don't know about each other. With **State**, the *object itself* changes behaviour as its internal state changes, and the states usually trigger the transitions (`IdleState` → `HasCoinState`). Ask "who chooses?": the caller means Strategy, the object's own lifecycle means State. → *More:* Ch 11 — 11.8 Choose the right behavioral pattern

**Q148. Strategy vs Template Method. Template Method** fixes the steps of an algorithm in a base class and lets subclasses fill in some of them (inheritance, chosen at compile time). **Strategy** swaps the whole algorithm through composition, at runtime. Prefer Strategy when variants change independently or need to be combined. Template Method suits a stable skeleton with small hooks. → *More:* Ch 11 — 11.4 Template Method

**Q149. ⋆ Observer pattern; Observer vs Pub-Sub.** In **Observer**, a subject keeps a list of observers and calls them directly when its state changes (a stock price notifying displays). The subject knows its observers' interface, and delivery is usually synchronous and in-process. **Pub-Sub** puts a broker or event bus between them: publishers send to a topic without knowing the subscribers, and delivery can be asynchronous and cross-process. Pub-Sub decouples more, at the cost of extra infrastructure. → *More:* Ch 11 — 11.1 Observer

**Q150. ⋆ Command pattern: how does it give you undo/redo?** Command turns a request into an object with `execute()` and `undo()`, so it can be queued, logged, retried or reversed. For undo/redo, push each executed command on an undo stack. Undo pops it, calls `undo()` and pushes it on the redo stack. Any new command clears the redo stack. Text editors, remote controls and job queues use it. → *More:* Ch 10 — 10.2 Command

**Q151. Chain of Responsibility (middleware, approval flows).** A request passes along a chain of handlers. Each one either handles it or passes it to the next, so the sender doesn't know who will deal with it. Examples are expense approval (manager → director → CFO by amount), ATM cash dispensing by note value, and servlet filters or HTTP middleware (auth → rate limit → logging). Handlers can be added or reordered without changing the client. → *More:* Ch 10 — 10.1 Chain of Responsibility

**Q152. Iterator, Mediator, Memento, Interpreter, Visitor: when would you use each? Iterator**: walk a collection without exposing how it is stored. **Mediator**: objects talk through a hub instead of to each other (an air-traffic control tower, a chat room). **Memento**: save and restore an object's state without breaking encapsulation (snapshots for undo). **Interpreter**: evaluate sentences of a small grammar (rule expressions). **Visitor**: add new operations to a stable class hierarchy without editing it (export or tax calculation over node types). In modern Java, sealed types with pattern-matching `switch` often replace Visitor. → *More:* Ch 10 — 10.3–10.6, Ch 11 — 11.5 Visitor

**Q153. ⋆ State pattern (vending machine / ATM / elevator).** Each state becomes a class that implements the same interface (`insertCoin`, `selectItem`, `dispense`). The machine delegates to its current state object, and states switch the machine to the next state. This replaces large `if (state == ...)` blocks, makes illegal actions explicit per state, and lets you add a state without touching the others. A small enum-based state machine is fine when the states have little behaviour. → *More:* Ch 11 — 11.2 State

**Q154. What are Null Object, Repository, Specification and MVC? Null Object**: a do-nothing implementation (`NoDiscount`) used instead of `null`, which removes null checks. **Repository**: a collection-like interface for storing and finding aggregates (`BookingRepository.findById`) that hides the storage. **Specification**: a business rule as a combinable object (`isAvailable.and(inCity)`). **MVC** (Model–View–Controller): the model holds data and rules, the view displays it, and the controller turns input into model updates. → *More:* Ch 11 — 11.7 Beyond GoF

**Q155. ⋆ Which patterns would you use for a parking lot, an elevator or a vending machine? Parking lot**: Strategy for pricing and spot allocation, Factory for vehicles or spots, Singleton or injected for the lot, Observer for display boards. **Elevator**: State for each car (moving/idle/doors open), Strategy for the dispatch algorithm, Command for requests. **Vending machine**: State for idle/has-money/dispensing, and Chain of Responsibility for giving change. Name a pattern only when you can say what change it absorbs. → *More:* Ch 22 — 22.9 Case-study map

**Q156. How have lambdas and modern Java changed classic patterns?** Patterns with a single method, such as Strategy, Command, Observer callbacks and Template Method hooks, are often just functional interfaces with lambdas (`Comparator`, `Runnable`, `Function`). Records cover simple value objects, and sealed interfaces with pattern-matching `switch` give exhaustive case handling in place of many Visitor uses. The *ideas* stay the same, with less boilerplate. → *More:* Ch 11 — 11.6 Quick-reference summary

## Concurrency

**Q157. ⋆ Process vs thread.** A **process** is a running program with its own memory space. Processes are isolated and costly to create, and they communicate through IPC (inter-process communication: pipes, sockets). A **thread** is a path of execution *inside* a process. Threads share the heap with their siblings but each has its own stack, so they are cheap to switch and communicate through shared memory, which is why they need synchronisation. → *More:* Ch 12 — 12.2

**Q158. ⋆ Concurrency vs parallelism. Concurrency** means dealing with many tasks over the same period by interleaving them. It is about structure, and it works on one core. **Parallelism** means literally running tasks at the same instant on several cores. It is about speed. A web server handling 10,000 waiting connections is concurrent, while a multi-core merge sort is parallel. → *More:* Ch 12 — 12.1

**Q159. What are the states in a thread's lifecycle?** `Thread.State` has six values. **NEW** means created but not started. **RUNNABLE** means running or ready to run. **BLOCKED** means waiting to enter a `synchronized` monitor. **WAITING** means waiting without a time limit (`wait()`, `join()`, `LockSupport.park`). **TIMED_WAITING** means waiting with a timeout (`sleep(ms)`, `wait(ms)`). **TERMINATED** means finished. → *More:* Ch 12 — 12.3

**Q160. Why call `start()` and not `run()`?** `start()` asks the JVM to create a new thread, which then calls `run()`. Calling `run()` directly just executes the method on the *current* thread, so nothing runs concurrently. Calling `start()` twice on the same thread throws `IllegalThreadStateException`. → *More:* Ch 12 — 12.4

**Q161. ⋆ Runnable vs Callable; ExecutorService; CompletableFuture.** A `Runnable` returns nothing and can't throw checked exceptions. A `Callable<V>` returns a value and may throw. An `ExecutorService` runs tasks on a managed pool and hands back a `Future` for the result. `CompletableFuture` adds non-blocking chaining and combining (`thenApply`, `thenCombine`, `allOf`) and error handling. Always shut executors down. Since Java 19, `ExecutorService` is `AutoCloseable`, so try-with-resources works: `close()` shuts the executor down and waits for submitted tasks to finish. → *More:* Ch 12 — 12.4

**Q162. ⋆ What are virtual threads, and when should you not use them?** Virtual threads (final in Java 21) are cheap JVM-managed threads. When one blocks on I/O it unmounts from its OS carrier thread, so you can run one per request or task, even millions. They help blocking, I/O-heavy work, not CPU-bound work. Don't pool them, and limit access to scarce resources with a `Semaphore`. Since JDK 24 (JEP 491), blocking inside `synchronized` or in `Object.wait()` no longer pins the carrier. Pinning remains in rare cases: blocking while a native frame is on the stack, and blocking in class loading or a class initializer. → *More:* Ch 12 — 12.4

**Q163. ⋆ What is a race condition? What is a critical section?** A **race condition** happens when the result depends on the timing of threads touching shared mutable data. A classic case is `count++`, which is read–modify–write: two threads read 5 and both write 6. Another is "check-then-act", where two users both see seat A1 free. A **critical section** is the code that touches the shared state and must run by one thread at a time (or be made atomic). → *More:* Ch 12 — 12.5

**Q164. ⋆ What does `volatile` do? What is happens-before (the Java Memory Model)?** The Java Memory Model (JMM) defines when one thread's writes become visible to another. **Happens-before** is its guarantee: if A happens-before B, then B sees A's effects. Releasing a lock happens-before the next acquire of it, `Thread.start` and `join` create the edge too, and so do volatile writes and reads. `volatile` gives visibility and ordering for one variable but **not** atomicity, so `volatile int count; count++` is still a race. It is good for flags like `running = false`. → *More:* Ch 12 — 12.6

**Q165. ⋆ `synchronized` vs `ReentrantLock`.** Both give mutual exclusion and are **reentrant**, meaning the thread holding the lock can enter it again. `synchronized` is simpler and releases automatically. `ReentrantLock` adds

`tryLock` with a timeout, interruptible locking, optional fairness and several `Condition`s, but you must `unlock()` in `finally`. Start with `synchronized` and switch when you need those features.

```
lock.lock();
try { /* critical section */ } finally { lock.unlock(); }
```

→ *More:* Ch 12 — 12.7, 12.8

**Q166. ReadWriteLock and StampedLock: when do you use them?** A `ReentrantReadWriteLock` lets many readers in at once but a writer gets exclusive access, which helps when reads far outnumber writes (a config cache). `StampedLock` adds **optimistic reads**: read without locking, then `validate(stamp)`, and retry under a real read lock if a write happened. It is faster for read-heavy data but not reentrant and harder to use correctly. → *More:* Ch 12 — 12.10

**Q167. ★ Semaphore vs mutex.** A **mutex** (lock) lets exactly one thread in and has an owner: only the locker should unlock. A **semaphore** holds N permits, so up to N threads proceed at once (limiting a pool to 10 connections). It has no owner, so one thread can release a permit another acquired, which is useful for signalling. A binary semaphore looks like a mutex but lacks ownership and reentrancy. → *More:* Ch 12 — 12.12 Semaphores

**Q168. `wait()` / `notify()` vs `Condition`; `notify()` vs `notifyAll()`.** `wait()` releases the object's monitor and sleeps until notified. It must be called while holding the monitor (`synchronized`), and always inside a `while` loop that re-checks the condition, because of spurious wakeups. `notify()` wakes one arbitrary waiter, `notifyAll()` wakes all. `notifyAll` is safer when waiters wait for different conditions. A `Condition` from a `Lock` does the same job, but one lock can have several conditions (`notFull`, `notEmpty`), so you wake only the right group. → *More:* Ch 12 — 12.13 Condition variables

**Q169. ★ `wait()` vs `sleep()`.** `Object.wait()` **releases** the monitor, must be called inside `synchronized` (otherwise it throws `IllegalMonitorStateException`), and returns on `notify` / `notifyAll`, a timeout, an interrupt or a spurious wakeup. Use it to wait for a condition. `Thread.sleep()` is a static method that pauses the current thread for a time and **keeps** any locks it holds. Use it for delays, not coordination. Both throw `InterruptedException`. → *More:* Ch 12 — 12.3, 12.13

**Q170. ★ `CountDownLatch` vs `CyclicBarrier`.** A `CountDownLatch` lets threads wait until a count reaches zero. It is one-shot, and the threads counting down are usually different from the waiters ("start when all 5 services are ready"). A `CyclicBarrier` makes a fixed group of threads wait for *each other* at a point and can be reused round after round, with an optional barrier action ("each simulation step"). `Phaser` is the flexible, dynamic version. → *More:* Ch 12 — 12.14

**Q171. ★ What is CAS? How do atomics like `AtomicInteger` work?** CAS (compare-and-swap) is a CPU instruction: "set the value to new **only if** it still equals expected", which is atomic and lock-free. Atomic updates are a CAS loop: read, compute, CAS, and retry if another thread changed the value (`updateAndGet` works exactly like this, and on x86 the JIT turns `incrementAndGet` into a single atomic add instruction). That beats locks under low contention, and `LongAdder` spreads hot counters across cells under high contention. Watch for the **ABA problem** (a value changes A→B→A unnoticed). `AtomicStampedReference` solves it with a version stamp. → *More:* Ch 12 — 12.15

**Q172. ★ How does `ConcurrentHashMap` work? How is it different from `Hashtable` and `synchronizedMap`?** `Hashtable` and `Collections.synchronizedMap` lock the whole map for every call. Since Java 8, `ConcurrentHashMap` uses CAS to place the first node in an empty bucket and otherwise locks only that bucket's head node. Reads take no lock at all, and writes to different buckets run in parallel. It forbids `null` keys and values. Its iterators are weakly consistent and never throw `ConcurrentModificationException`. Use `compute` / `merge` / `putIfAbsent` for atomic read-modify-write. → *More:* Ch 12 — 12.16, Ch 13 — 13.7

**Q173. ★ What is a deadlock? What are its conditions, and how do you prevent and detect it?** A deadlock happens when threads wait on each other forever, for example T1 holds lock A and wants B while T2 holds B and wants A. The four Coffman conditions must all hold: mutual exclusion, hold-and-wait, no preemption and circular wait. To prevent it, break one condition. The most common way is to always acquire locks in a global order (for example by account ID). Others are `tryLock` with a timeout and backoff, or holding fewer locks. To detect it, take a thread dump (`jstack`), use `ThreadMXBean.findDeadlockedThreads()`, or use a wait-for graph. → *More:* Ch 12 — 12.17 Deadlock

**Q174. Livelock vs starvation.** In a **livelock**, threads are active but make no progress because they keep reacting to each other, like two people stepping aside the same way in a corridor. The fix is random backoff. In **starvation**, a thread never gets the CPU or the lock because others keep winning. The fix is

fair locks (`new ReentrantLock(true)`), bounded queues and avoiding priority tricks. → *More:* Ch 12 — 12.18, 12.19

**Q175. ⋆ Explain the producer-consumer problem and how to solve it.** Producers put items into a bounded buffer and consumers take them out. Producers must wait when the buffer is full, and consumers must wait when it is empty, without busy-spinning or racing. The easy solution is a `BlockingQueue` (`ArrayBlockingQueue`), whose `put` and `take` block. Hand-rolled versions use a lock with `notFull`/`notEmpty` conditions, or `wait`/`notifyAll` in `while` loops. → *More:* Ch 12 — 12.22, Ch 13 — 13.8

**Q176. The reader-writer problem.** Many readers may read shared data at once, but a writer needs exclusive access. A naive solution can starve writers while readers keep arriving. In Java, use `ReentrantReadWriteLock` (with the fair option if writer starvation matters), `StampedLock`, or copy-on-write structures when writes are rare. → *More:* Ch 12 — 12.23

**Q177. ⋆ How do you prevent double booking of a seat or parking spot?** The danger is check-then-act: two requests both see the seat free and both book it. In memory, make the check and the claim one atomic step, for example `seatHolds.putIfAbsent(seatId, userId)` on a `ConcurrentHashMap`, or lock per show or seat (fine-grained), acquiring several seats in sorted order. With a database, use **optimistic locking** (a version column, and the update fails if the version changed, so you retry or report "taken"), **pessimistic locking** (`SELECT ... FOR UPDATE`) or a unique constraint. Add temporary holds with a timeout during payment. → *More:* Ch 12 — 12.25

**Q178. Optimistic vs pessimistic locking. Pessimistic** locking takes the lock *before* working, assuming conflicts are likely. It is safe but blocks others. **Optimistic** locking works without a lock and checks at commit that nobody changed the data (a version number or CAS), retrying if they did. It is fast when conflicts are rare. Use pessimistic locking for hot, contended rows (the last seats of a flash sale) and optimistic locking for most other updates. → *More:* Ch 12 — 12.25

**Q179. Coarse-grained vs fine-grained locking. Coarse-grained** means one lock guards a lot (the whole parking lot). It is simple and deadlock-free but serialises everything. **Fine-grained** means many locks guard small parts (per floor, per spot, per bucket). Throughput goes up, but so do complexity and deadlock risk. Start coarse, measure, and split only the hot path. Lock striping is the middle ground. → *More:* Ch 12 — 12.11

**Q180. Name the ways to make a class thread-safe.** In rough order of preference: make it **immutable** (records, final fields). **Confine** the state to one thread (local variables, `ThreadLocal`, or a single-threaded owner reached via a queue). Use **thread-safe building blocks** (`ConcurrentHashMap`, atomics, `BlockingQueue`). Last, **lock** carefully around every invariant that spans several fields. Document which one you chose. → *More:* Ch 12 — 12.24 Immutability & thread confinement

## Java specifics for LLD rounds

**Q181. ⋆ Why is `String` immutable in Java?** Strings are shared everywhere, so immutability makes that safe. Literals live in the **string pool** and are reused, which would be dangerous if one holder could change them. Immutable strings are automatically **thread-safe**. Their `hashCode` can be **cached**, which makes them fast, stable `HashMap` keys. They are also **secure** for values like file paths, URLs and class names, which cannot be changed after they are checked. "Modifying" methods such as `concat` and `replace` return new strings. → *More:* Ch 02 — 2.13 Immutability

**Q182. ⋆ Is Java pass-by-value or pass-by-reference?** Always **pass-by-value**. For objects, the value copied is the *reference*. So a method can change the object's fields through its copy (`list.add(x)` is visible to the caller), but reassigning the parameter (`list = new ArrayList<>()`) does not affect the caller's variable. That is why a `swap(a, b)` method can't swap the caller's objects.

```
void reset(StringBuilder sb) { sb.setLength(0); sb = new StringBuilder("x"); }
// the caller's sb is now empty, not "x"
```

→ *More:* Ch 02 — 2.2 Classes and objects

**Q183. ⋆ `==` vs `equals()` for strings (and the string pool).** `==` compares references (is this the same object?), while `equals` compares contents. `"ab" == "ab"` is true because string literals (and compile-time constants such as `"a" + "b"`) are interned into the pool. `new String("ab") == "ab"` is false, and so is a string built at runtime unless you call `intern()`. Always compare strings with `equals` (or `Objects.equals` for null-safety). → *More:* Ch 02 — 2.12 Identity vs equality

**Q184.** **`String` vs `StringBuilder` vs `StringBuffer`.** `String` is immutable, so concatenating in a loop creates many temporary objects. `StringBuilder` is mutable and fast but not thread-safe, which makes it the default for building text. `StringBuffer` is the old synchronised version and is rarely needed, since builders are almost always used inside one thread. → *More:* Ch 02 — 2.13 Immutability

**Q185.** **`final` vs `finally` vs `finalize`.** `final` is a modifier for a constant variable, a non-overridable method or a non-extendable class. `finally` is the block after `try` that always runs, used for cleanup, though try-with-resources is better. `finalize()` was a GC cleanup hook. It has been deprecated for removal since JDK 18 (JEP 421) because it is unpredictable and slow. Use `AutoCloseable` or `java.lang.ref.Cleaner` instead. → *More:* Ch 02 — 2.2 static and final

**Q186.** ⋆ **How does `HashMap` work internally?** `HashMap` keeps an array of buckets. It spreads the key's `hashCode` bits and masks the result to a bucket index, then compares with `equals` inside the bucket. Collisions form a linked list. When a bucket already holding 8 entries (`TREEIFY_THRESHOLD`) gets another, the list becomes a red-black tree, but only if the table has at least 64 slots (`MIN_TREEIFY_CAPACITY`); a smaller table is resized instead. A tree shrinks back to a list at 6 entries or fewer. When the size exceeds capacity × load factor (0.75 by default), the table doubles and entries are redistributed. It allows one `null` key and is not thread-safe. → *More:* Ch 13 — 13.7 Concurrent HashMap

**Q187.** **`Comparable` vs `Comparator`.** `Comparable` gives a class its single natural order through `compareTo`, for example `Money` by amount. A `Comparator` is an external, swappable ordering, which makes it a Strategy (`Comparator.comparing(Ride::fare).thenComparing(Ride::eta)`). Keep `compareTo` consistent with `equals`, or sorted sets and maps will behave oddly. → *More:* Ch 11 — 11.3 Strategy

**Q188.** **Fail-fast vs fail-safe iterators.** **Fail-fast** iterators (`ArrayList`, `HashMap`) throw `ConcurrentModificationException` if the collection is structurally changed during iteration, other than through the iterator's own `remove`. **Fail-safe** (weakly consistent or snapshot) iterators (`ConcurrentHashMap`, `CopyOnWriteArrayList`) never throw, but may not show the latest changes. To remove while looping, use `iterator.remove()` or `removeIf`. → *More:* Ch 12 — 12.16 Concurrent collections

**Q189.** ⋆ **Sealed classes, records and pattern matching: how do they help in LLD?** A `sealed` interface lists its only allowed implementations (Java 17). Combined with records and pattern-matching `switch` (Java 21), the compiler checks that every case is handled, with no `default` needed. This is a clean way to model closed sets like payment results or vehicle types.

```
sealed interface Payment permits Card, Upi {}
record Card(String last4) implements Payment {}
record Upi(String vpa) implements Payment {}
String describe(Payment p) { return switch (p) {
    case Card c -> "Card " + c.last4();
    case Upi u  -> "UPI " + u.vpa(); }; }
```

→ *More:* Ch 02 — 2.13 Immutability and records

**Q190.** **Generics: what are wildcards and PECS?** `List<? extends Shape>` accepts lists of any Shape subtype and lets you *read* Shapes, but you can't add to it. `List<? super Circle>` lets you *add* Circles. **PECS** ("Producer Extends, Consumer Super") is the rule: use `extends` for sources you read from and `super` for sinks you write to, as in `Collections.copy(List<? super T> dst, List<? extends T> src)`. Generics are erased at runtime, so `new T()` is impossible. → *More:* Ch 02 — 2.15 Generics basics

**Q191.** **When should you use `Optional`, and how do you avoid `null` in a design?** Return `Optional<T>` from lookups that may find nothing (`findSpot()`), so callers must handle absence. Don't use it for fields, parameters or collections; return an empty collection instead. Elsewhere, avoid `null` with validated constructors, the Null Object pattern and `Objects.requireNonNull`. → *More:* Ch 11 — 11.7.1 Null Object

## Part C — How to run a 60–90 minute machine-coding round

A machine-coding round asks you to build a small working system, such as Splitwise, a parking lot or a ride-sharing service, in one sitting, usually in an IDE. It is judged on whether the code runs, how it is structured, and how easily it absorbs a change. The plan below assumes 90 minutes. For a 60-minute round, roughly halve each step.

**Minutes 0–10: clarify and cut scope.** Read the problem twice. Write down the must-have features (usually 3–5 commands), the input/output format, and what you will leave out (persistence, auth, UI). Ask about the

tricky rules, such as rounding, ties, invalid input and concurrent users. Confirm that in-memory storage and a console demo are fine.

**Minutes 10–20: model.** List the entities (nouns) and the services (verbs). Decide the few interfaces where change is likely, such as a pricing, split or allocation strategy. Sketch the packages: `model`, `service`, `strategy`, `repository` (in-memory maps), `exception`, and `Main` for the demo. Don't draw full UML. A short list of classes is enough.

**Minutes 20–60: build a vertical slice first.** Get the main flow working end to end (create → act → show) before adding anything else. Use records and enums for the models, constructor injection for the services, and clear custom exceptions for invalid operations. Commit or save each working state. Write code that compiles at every step, and don't leave a half-built feature that breaks the demo.

**Minutes 60–75: complete the features and handle edge cases.** Add the remaining commands one by one. Validate input, handle "not found" and duplicates, and make any shared state thread-safe if concurrency was asked (`ConcurrentHashMap`, per-entity locks).

**Minutes 75–85: demo and test.** Run `Main` on the sample input plus two or three edge cases. If time allows, add a few unit tests for the core rule, such as the split math or the fare calculation.

**Minutes 85–90: walk through and extend.** Explain the structure in one minute, then show how the likely extension fits: "a new split type is one new class". Be honest about anything you skipped.

Common ways to lose marks: one giant class, no working demo, over-engineering before the basics run, ignoring input validation, and staying silent. Talk while you code.

→ *More:* Ch 01 — The LLD interview approach, Ch 21 — 21.0 How the machine-coding round works

## Sources of the question lists

Question wording and frequency only; every answer above is written for this book.

- LeetCode Discuss interview-experience posts (Flipkart SDE2, Swiggy SDE1/SDE2, Atlassian P4, PhonePe, Meta E5) and the "How do you prepare for machine coding rounds" thread
- prasadgujar/low-level-design-primer — `questions.md` (LeetCode-Discuss-sourced list)
- ashishps1/awesome-low-level-design — "LLD Interview Problems" and concurrency problems
- kumaransg/LLD — machine-coding questions tagged Flipkart / Udaan / Navi / ClearTrip
- donnemartin/system-design-primer — "Object-oriented design interview questions"
- workat.tech machine-coding practice problems
- Course tables of contents (titles only): Grokking the Object-Oriented Design Interview

# Chapter 24 — Mock Interview Drills

> **Goal.** Turn each of the 23 classic object-oriented design problems into a timed, spoken rehearsal. A drill is the interview script only: the prompt, the questions to ask back, a plan, a checklist, the usual follow-ups and a link. The designs themselves are taught in the case-study chapters; this chapter trains you to present them.

An **object-oriented design (OOD) round**, also called a low-level design (LLD) round, is a 45–60 minute interview in which you turn a one-line product idea into classes, relationships and a little code while explaining your choices. It grades the **model** and the **explanation**. These drills train the explanation.

## 24.0 How to use these drills

### Running a drill

1. Read only the prompt and the clarifying questions. Cover the rest.
2. Start a 45-minute timer and talk out loud, sketching on paper and typing the core code.
3. When the timer stops, score yourself with the rubric below **before** looking at anything.
4. Uncover the checklist and follow-ups, then open the linked case study. Re-score, and write down the one dimension you will fix next time.
5. Repeat a drill the next day if you scored under 24.

### The timebox

Every drill's "Minute-by-minute plan" uses these phases. Only the problem-specific content is listed in each drill.

| 45-min round | 60-min round | Phase | What exists at the end |
|---|---|---|---|
| 0–5 | 0–6 | **Clarify** | 6–10 numbered requirements, an out-of-scope line, 2–4 actors (people or systems that use the design) |
| 5–8 | 6–10 | **Use cases** | 4–8 actions, one line each |
| 8–18 | 10–22 | **Entities and relationships** | Classes with one sentence of responsibility each; a class diagram with kinds and multiplicities (how many on each side) |
| 18–23 | 22–28 | **Walk one flow** | The main use case traced through the classes |
| 23–35 | 28–45 | **Code** | The 2–3 classes that carry the main flow, written properly; the rest only named |
| 35–45 | 45–60 | **Follow-ups** | Concurrency, one extension, trade-offs |

Two rules for every problem. **Say the plan in the first minute**, so the interviewer can steer early. **Have a class diagram by minute 18**; if you don't, cut scope out loud ("refunds later") and move on. Extra time in a 60-minute round goes to code and follow-ups, never to longer clarifying.

### How to explain clearly

Narrate **decisions, not actions**. "I'm drawing Ticket" is an action. "Ticket is its own class because it outlives the parking session and payment needs something to point at" is a decision, and decisions are what get graded.

- **Each class:** name, one sentence of responsibility, why it is separate.
- **Each relationship:** kind (composition, aggregation, association; §3.2–3.8) and multiplicity.
- **Each design pattern:** name the **change it absorbs**. "Strategy for pricing, because pricing changes most often and a new rule should be a new class" scores; "I'll use Strategy" alone does not.
- **At each phase boundary**, check in: "That's my class list. Shall I walk the booking flow, or would you rather I start on concurrency?"

### Terms the drills use

Pattern and concurrency terms (Strategy, Command, compare-and-set, idempotent, half-open interval …) are defined in the glossary, §22.11. In the plans, "`A` owns many `B`" means composition, one to many, and "walk" means trace one use case through the classes.

### Scoring rubric

Score each dimension 1–4 (total out of 28).

| Dimension | 1 — Weak | 2 — Partial | 3 — Solid | 4 — Strong |
|---|---|---|---|---|
| **Requirements** | Drew classes at once | Asked questions, wrote nothing | Numbered functional list plus out of scope | Also non-functional (concurrency, failure), confirmed with the interviewer |
| **Entities** | One "Manager" does everything | Right nouns, blurred jobs | One-sentence responsibility per class | Also the non-obvious split (type vs instance, book vs copy) |
| **Relationships** | Unlabelled lines | Arrows without kind or multiplicity | Correct kinds and multiplicities | Also inheritance only where substitution holds, with reasons |
| **Patterns** | None, or named for show | Named, change not stated | One or two, each tied to the change it absorbs | Also one considered and rejected, with the reason |
| **Concurrency** | Not mentioned | "Add synchronized" with no target | Names the shared resource; locks at the right grain | Also the failure mode: hold expiry, lock order, idempotent retry |
| **Extensibility** | A new type edits many classes | Claims extensibility without showing it | Shows one extension as "one new class" | Also says what would *not* extend cheaply and why that is acceptable |
| **Communication** | Silent or rambling | Narrates actions | Narrates decisions, checks in | Also keeps to time and invites steering |

**Reading the total:** treat 24–28 as interview-ready. 18–23 is borderline, usually dragged down by Requirements or Communication. Below 18, study the case and repeat the drill tomorrow.

### Suggested order

Small state machines first, platforms last: Parking Lot, Vending Machine, ATM, Elevator, Amazon Locker → Library, Movie Tickets, Hotel, Car Rental, Airline, Restaurant, Meeting Scheduler → Blackjack, Chess, Jigsaw → Amazon Shopping, Stack Overflow, Facebook, LinkedIn, ESPNcricinfo, Stock Brokerage → Ride Sharing and Food Delivery with no notes at all. The sections below follow the original course order so you can find a problem quickly.

## 24.1 Drill — Parking Lot

### The prompt as the interviewer says it

> "Design a parking lot. Cars come in, get a ticket, park, and pay when they leave. Take it wherever you like."

The open ending tests whether you bound the problem.

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Levels? | Several, each with a fixed set of spots |
| Vehicle and spot types? | Motorbike/car/truck; spots small/medium/large; a smaller vehicle may take a bigger spot |
| Gates? | Several entries and exits running at once |
| Fee? | Hourly, rate per vehicle type, easy to change |
| Display? | Free spots per level and size |
| Out of scope? | Advance reservations, valet, plate cameras |

### Minute-by-minute plan

- **0–8:** Actors: driver, attendant, admin. Use cases: take ticket, park, pay, exit, view free spots.
- **8–18:** `ParkingLot` owns many `Level` owns many `ParkingSpot`; `Vehicle`, `Ticket`, `EntryGate`, `ExitGate`, `SpotAllocationStrategy`, `FeeStrategy`, `Payment`.
- **18–23:** Walk "car enters": gate → lot → strategy picks spot → spot claimed → ticket issued.
- **23–35:** Code `ParkingSpot.tryOccupy`, one allocation strategy, `FeeStrategy.fee(ticket, exitTime)`.

### What a strong answer must include

- [ ] Size enums for vehicle and spot, with one fit rule.
- [ ] `Ticket` as its own class (spot, vehicle, entry time).
- [ ] Allocation and fee each behind an interface; fee separate from payment method.
- [ ] Atomic spot claim across gates.
- [ ] Free counts maintained in one place, not by scanning.

### Likely follow-ups and how to answer

- **Two gates, last spot:** compare-and-set on the spot (or a per-size free queue); the loser retries, then shows "full". A lot-wide lock is correct but serialises every gate.
- **EV charging spots:** a new spot kind plus a charging fee class; no gate code changes.
- **Weekend rates / first hour free:** a new `FeeStrategy`, or a decorator around an existing one.
- **Lost ticket:** flat daily maximum; look up by plate if stored.

### Where to study it

Ch 14 — 14.1 Design a Parking Lot

## 24.2 Drill — Elevator System

### The prompt as the interviewer says it

> "Design the elevators for a building. People press buttons on floors and inside cars. How do you decide which elevator goes where?"

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Size? | 3–4 cars, ~20 floors |
| Buttons? | Up/down per floor (hall calls), floor panel per car (car calls), door and emergency buttons |
| Capacity? | Weight limit; an overloaded car holds its doors open |
| Dispatch goal? | Short waits, few direction changes, swappable |
| Special modes? | Maintenance, fire mode (all cars to ground) |
| Out of scope? | Motor physics, access cards |

### Minute-by-minute plan

- **0–8:** Actors: passenger, operator. Use cases: call car, choose floor, open/close door, maintenance, fire mode.
- **8–18:** `ElevatorSystem` → many `ElevatorCar` (state, current floor, up-stops and down-stops as sorted sets), `Request` (floor, direction, kind), `Dispatcher` interface, `Door`.
- **18–23:** Walk "floor 7 presses up": dispatcher scores cars, adds a stop to one.
- **23–35:** Code the dispatcher's cost function and `ElevatorCar.step()` using the LOOK rule (keep going while stops lie ahead, then reverse).

### What a strong answer must include

- [ ] Hall calls assigned to one car; car calls belong to their car.
- [ ] Sorted stop sets per direction, not a FIFO queue; you can say why FIFO zig-zags.
- [ ] Dispatch as Strategy (nearest, least busy, zoned).
- [ ] Car states with legal transitions; doors never open while moving.
- [ ] Button presses from many threads handled safely.

### Likely follow-ups and how to answer

- **Why not nearest idle car?** A car already heading towards the caller in the same direction is often better. Cost = distance + direction-change penalty + load penalty.
- **Many presses at once:** one controller thread drains a request queue, so dispatch logic stays single-threaded.
- **Fire mode:** a system-wide mode that clears stops and sends every car down; cars consult it each step.
- **A car breaks mid-trip:** maintenance state, re-dispatch its hall calls to other cars.

### Where to study it

Ch 14 — 14.5 Design an Elevator System

## 24.3 Drill — Library Management System

### The prompt as the interviewer says it

> "Design a library system. Members borrow and return books, and late returns are fined."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Multiple copies of a title? | Yes; each copy has a barcode and rack |
| Loan rules? | 14 days, max 5 books, renew once |
| Reservations? | Per-title queue; next in line is notified on return |
| Fines? | Per day late; may differ by membership type |
| Search? | Title, author, subject, date |
| Out of scope? | E-books, inter-library loans, payment processing |

### Minute-by-minute plan

- **0–8:** Actors: member, librarian, system. Use cases: search, borrow, return, renew, reserve, pay fine.
- **8–18:** `Book` (title data) owns many `BookItem` (physical copy), `Member`, `Loan`, `Reservation`, `Fine`, `Catalog`, `FinePolicy`, `Notifier`.
- **18–23:** Walk "return a late book": close loan → compute fine → check reservation queue → notify.
- **23–35:** Code `borrow` (limit, availability, reservation honoured) and `returnItem`.

### What a strong answer must include

- [ ] The **book vs copy** split; this is what the interviewer looks for first.
- [ ] `Loan` as a class with its own dates, not fields on `Member`.
- [ ] Borrow limit enforced in one place, per membership type.
- [ ] Fine rule behind an interface; notifications decoupled.
- [ ] Search separated from lending.

### Likely follow-ups and how to answer

- **Two members, last copy:** the copy's available → loaned change is atomic; the loser is offered a reservation.
- **Reserved copy never collected:** hold N days, then a scheduled sweep passes it on.
- **Add DVDs and magazines:** generalise to a catalog item only where lending rules match; a non-lendable magazine must not inherit `borrow`.

### Where to study it

Ch 15 — 15.1 Design a Library Management System

## 24.4 Drill — Amazon Locker Service

### The prompt as the interviewer says it

> "Customers can have parcels delivered to a self-service locker. Design delivery, pickup, and what happens when nobody collects."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Sites and sizes? | Many sites; lockers small/medium/large |
| Opening a locker? | One-time code sent to the customer |
| Pickup window? | 3 days, then return to sender |
| Returns? | Customer drops an item with a code; courier collects |
| Site full? | Courier redirected to the nearest site with space |
| Out of scope? | Ordering, payments, hardware drivers |

### Minute-by-minute plan

- **0–8:** Actors: customer, delivery agent, ops admin. Use cases: assign locker, drop parcel, pick up, return item, expire.
- **8–18:** `LockerSite` owns many `Locker` (size, state), `Parcel`, `LockerAssignment` (parcel, locker, code, deadline), `LockerSelectionStrategy`, `CodeGenerator`, `Notifier`, `ExpiryJob`.
- **18–23:** Walk "pick up with code": find assignment by code → check deadline → open → free locker.
- **23–35:** Code `LockerSite.assign(parcel)` and `pickUp(code)`.

### What a strong answer must include

- [ ] Ordered size enums; "smallest that fits" behind an interface.
- [ ] Locker states: free → reserved → occupied → free, plus out of service.
- [ ] Code on the assignment, single-use, with a deadline.
- [ ] Scheduled expiry that frees lockers and starts return-to-sender.
- [ ] Customer returns reuse the same assignment mechanism.

### Likely follow-ups and how to answer

- **Two couriers, last medium locker:** reserve under a per-site lock or poll a per-size free queue; fall back to a larger size, then another site.
- **Repeated wrong codes:** count failures per keypad (a wrong code matches no assignment, so a per-assignment count cannot work) and lock the keypad for a few minutes after five in a row.
- **Jammed locker on arrival:** mark out of service and reassign; the code belongs to the assignment, so it survives.

### Where to study it

Ch 14 — 14.7 Design a Shipping Locker System (Amazon Locker)

## 24.5 Drill — Vending Machine

### The prompt as the interviewer says it

> "Design a vending machine. A user inserts money, picks a product, and gets the product and change."

This is a State-pattern question; reach the states quickly.

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Payment? | Coins and notes; cards later |
| Order? | Pay first, then pick |
| Cancel? | Full refund of what was inserted |
| No change available? | Refuse and refund before dispensing |
| Products? | Slots (A1, B3) with product and count |
| Out of scope? | Telemetry, cooling |

### Minute-by-minute plan

- **0–8:** Actors: customer, operator. Use cases: insert, select, cancel, dispense, restock.
- **8–18:** `VendingMachine` (context) → `VendingState` with `Idle`, `HasMoney`, `Dispensing`, `SoldOut`, `Service`; `Inventory`, `Slot`, `Coin`, `ChangeMaker`.
- **18–23:** Walk the state diagram, including cancel and sold-out.
- **23–35:** Code the state interface, two states, and `ChangeMaker.makeChange`.

### What a strong answer must include

- [ ] Each state lists the events it accepts; illegal events fail clearly.
- [ ] State pattern justified: each mode's rules in one class, no growing `switch`.
- [ ] Money as integer minor units, never `double`.
- [ ] Change checked against real coin stock **before** dispensing.
- [ ] Restock as a state, so sales stop during it.

### Likely follow-ups and how to answer

- **Greedy change?** Works for normal coin sets, fails for odd ones or depleted stock; a small search is the fallback behind the same interface.
- **Card payment:** a payment-method abstraction; card authorises the exact price, so no change.
- **Product jams:** back to `HasMoney`, offer refund or another pick.
- **Enum vs State classes:** an enum with a switch is fine for four stateless modes; classes pay off once states carry data or multiply.

### Where to study it

Ch 14 — 14.2 Design a Vending Machine

## 24.6 Drill — Online Blackjack Game

### The prompt as the interviewer says it

> "Design an online Blackjack game: several players at a table against a dealer, with betting."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Players per table? | 1–5 plus the dealer |
| Decks? | Six-deck shoe, reshuffled at a cut card |
| Actions? | Hit, stand, double; split later |
| Dealer rule? | Hits below 17, stands on all 17s |
| Payouts? | 1:1 win, 3:2 blackjack, push returns bet |
| Out of scope? | Side bets, insurance, networking details |

### Minute-by-minute plan

- **0–8:** Actors: player, dealer (system). Use cases: join, bet, deal, hit, stand, double, settle.
- **8–18:** `Card` (`Rank`, `Suit`), `Deck`, `Shoe`, `Hand`, `Player`, `Dealer`, `Table`, `Round`, `Bet`, `DealerPolicy`.
- **18–23:** Walk one round: bets → deal → players act in turn → dealer plays → settle.
- **23–35:** Code `Hand.bestValue()` and `Round.settle()`.

### What a strong answer must include

- [ ] A game-agnostic `Card`/`Deck`, reusable for other card games.
- [ ] The ace rule inside `Hand` only.
- [ ] `Round` with phases, separate from the long-lived `Table`.
- [ ] Dealer rule as a policy, so "hit soft 17" is a new class.
- [ ] Bets debited once at start, settled once; an injectable seeded shuffle for tests.

### Likely follow-ups and how to answer

- **Ace value:** count aces as 1; if total + 10 ≤ 21, add 10 once. That also yields the "soft" flag.
- **Splitting:** a player holds a list of `Hand`s, each with its own `Bet`; design for the list from the start.
- **Online multiplayer:** each table processes player commands from a queue on one thread, so game state needs no locks.
- **Reuse for Poker:** deck, card and shoe carry over; hand evaluation and round phases do not.

### Where to study it

Ch 17 — Design Blackjack and a Deck of Cards

## 24.7 Drill — Ride Sharing (practice session)

### The prompt as the interviewer says it

> "Design a ride-sharing service like Uber. Riders request rides, drivers accept them, and the rider pays at the end."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Matching rule? | Nearest available driver within a radius; swappable |
| Ride types? | Standard and premium; carpool later |
| Fare? | Base + distance + time, with surge multiplier |
| Cancellation? | Either side may cancel; a fee after driver assignment |
| Ratings? | Both sides rate after the trip |
| Out of scope? | Maps/routing, real payment gateway |

### Minute-by-minute plan

- **0–8:** Actors: rider, driver, payment system. Use cases: request, match, accept, start, complete, pay, rate, cancel.
- **8–18:** `Rider`, `Driver` (status, `Vehicle`, `Location`), `RideRequest`, `Trip` (state), `MatchingStrategy`, `FareStrategy`, `DriverLocator`, `TripService`, `Payment`.
- **18–23:** Walk "request to completion": request → match → offer → accept → start → complete → fare → pay.
- **23–35:** Code `TripService.request` with matching and atomic driver claim, plus `Trip` state transitions.

### What a strong answer must include

- ☐ `Trip` state machine: requested → driver assigned → in progress → completed, with cancelled from the early states.
- ☐ Matching and fare each as Strategy (nearest/best-rated; normal/surge).
- ☐ Driver availability claimed atomically.
- ☐ Location lookup behind its own class (a grid or geohash index), not a scan.
- ☐ Observer for live status updates to the rider.

### Likely follow-ups and how to answer

- **Two requests match one driver:** compare-and-set the driver's status from available to assigned; the loser is re-matched to the next candidate.
- **Driver declines or times out:** offer to the next candidate; cap attempts, then tell the rider.
- **Carpool:** a trip holds several riders and seats; matching checks detour cost. A new strategy plus a seats field, not a new service.
- **Surge:** a fare decorator using a demand/supply ratio per area.

### Where to study it

Ch 20 — 20.4 Design a Ride Sharing Service (Uber)

## 24.8 Drill — Meeting Scheduler

### The prompt as the interviewer says it

> "Design a meeting scheduler for an office. People book rooms for meetings and invite colleagues."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Room choice? | System picks a free room with enough capacity; user may name one |
| Time model? | Start and end on one day; half-open intervals, so 10–11 and 11–12 do not clash |
| Invitations? | Invitees accept or decline; organiser sees replies |
| Notifications? | On invite, change and cancel |
| Recurring meetings? | Later extension |
| Out of scope? | External calendars, time zones across offices |

### Minute-by-minute plan

- **0–8:** Actors: organiser, invitee, admin. Use cases: book, invite, respond, cancel, view room or person calendar.
- **8–18:** `Scheduler`, `Room` (capacity, own `Calendar` of bookings), `Meeting` (interval, organiser, room), `Invitation` (status), `Interval`, `RoomSelectionStrategy`, `Notifier`.
- **18–23:** Walk "book": filter rooms by capacity → strategy orders them → first room with no overlap is booked → invitations sent.
- **23–35:** Code `Interval.overlaps`, `Room.tryReserve` (sorted bookings, one `lowerEntry(end)` check) and `Scheduler.book`.

### What a strong answer must include

- [ ] An `Interval` value type with a correct half-open overlap test.
- [ ] Bookings per room in a sorted map; clash check in O(log n).
- [ ] Check and insert under one per-room lock.
- [ ] Room selection as Strategy (smallest fitting, nearest).
- [ ] Invitations with RSVP status; notifications through Observer.

### Likely follow-ups and how to answer

- **Two users, same room, overlapping times:** lock per room; when trying rooms in turn, release one lock before taking the next, so no deadlock.
- **First slot when all attendees are free:** gather busy intervals, sort, merge overlaps, scan the gaps for the required length.
- **Recurring meetings:** expand into instances; fail the whole booking on any clash, or offer to skip clashing dates.
- **Room goes out of service:** cancel or move its future meetings and notify organisers.

### Where to study it

Ch 16 — Design a Meeting Scheduler

## 24.9 Drill — Movie Ticket Booking System

### The prompt as the interviewer says it

> "Design BookMyShow: users find a movie, pick a show and seats, and pay."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Scope? | Many cities, cinemas, screens and shows |
| Seat selection? | User picks exact seats on a seat map |
| Seat types and pricing? | Regular/premium; price per show and seat type |
| Hold time? | Seats held 10 minutes while the user pays |
| Cancellation? | Allowed up to a cutoff, with refund rules |
| Out of scope? | Reviews, food ordering, real payment gateway |

### Minute-by-minute plan

- **0–8:** Actors: customer, cinema admin, payment system. Use cases: search, view seats, hold, pay, cancel, add show.
- **8–18:** `Cinema` owns `Screen` owns `Seat`; `Movie`; `Show` (screen, movie, time); `ShowSeat` (seat state per show); `Booking`; `Payment`; `PricingStrategy`.
- **18–23:** Walk "book two seats": hold seats → create pending booking → pay → confirm, or release on timeout.
- **23–35:** Code `ShowSeat` hold/confirm/release and `BookingService.hold(show, seats, user)` as all-or-nothing.

### What a strong answer must include

- [ ] `Seat` (physical) vs `ShowSeat` (state for one show); this is the key split.
- [ ] Seat states: available → held → booked, with hold expiry back to available.
- [ ] Holding several seats is all-or-nothing.
- [ ] Pricing as Strategy; booking confirmation via Observer.
- [ ] Payment failure releases the hold.

### Likely follow-ups and how to answer

- **Two users click the same seat:** lock the show's seats in a fixed order (by seat id) and check-and-hold together; the second user sees "taken". Fixed order prevents deadlock between overlapping selections.
- **Payment succeeds after the hold expired:** check hold ownership at confirm; if lost, refund automatically.
- **Show cancelled by cinema:** cancel all bookings for the show and refund, notifying each customer.
- **Many servers:** the hold must live in shared storage with a conditional update, not in one JVM's memory.

### Where to study it

Ch 15 — 15.2 Design a Movie Ticket Booking System

## 24.10 Drill — Car Rental System

### The prompt as the interviewer says it

> "Design a car rental system. Customers reserve a car at a branch for some dates and return it later."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Reserve a model or a specific car? | A vehicle type; the car is assigned at pickup |
| Branches? | Several; one-way rentals allowed |
| Pricing? | Daily rate per type, plus add-ons (insurance, GPS) |
| Late return / damage? | Extra charges at return |
| Cancellation? | Free until 24 hours before pickup |
| Out of scope? | Fleet maintenance scheduling, loyalty |

### Minute-by-minute plan

- **0–8:** Actors: customer, branch agent, admin. Use cases: search, reserve, pick up, return, cancel, pay.
- **8–18:** `Branch` holds many `Vehicle` (type, status); `Customer`; `Reservation` (type, dates, pickup and drop branch, state); `RentalAgreement`; `PricingPolicy`; `Invoice`; `AddOn`.
- **18–23:** Walk "return late": close rental → price base + add-ons + late fee → invoice → vehicle available at drop branch.
- **23–35:** Code availability for a date range and `Reservation` state transitions.

### What a strong answer must include

- [ ] Availability as interval overlap, not a single "is free" flag.
- [ ] Reservation lifecycle: reserved → picked up → returned, plus cancelled and no-show.
- [ ] Pricing as Strategy; add-ons composable (Decorator or a list).
- [ ] One-way rental moves the vehicle to the drop branch.
- [ ] Atomic reserve against overlapping requests for the last car of a type.

### Likely follow-ups and how to answer

- **Last SUV, two customers:** per-branch, per-type lock (or a count with compare-and-set) around check and reserve.
- **Overbooking:** allow a small configured margin per type; upgrade when short.
- **Dynamic pricing:** a new pricing class reading demand; nothing else changes.
- **Car breaks during rental:** swap vehicle on the same agreement; the agreement, not the car, carries the money.

### Where to study it

Ch 16 — Design a Car Rental System

## 24.11 Drill — ATM

### The prompt as the interviewer says it

> "Design an ATM. A customer inserts a card, enters a PIN, and withdraws cash or checks the balance."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Transactions? | Withdraw, deposit, balance, transfer between own accounts |
| Who holds balances? | The bank, via a remote service |
| PIN rules? | Three wrong tries keep the card |
| Cash? | Cassettes per denomination; limited stock |
| Limits? | Daily withdrawal limit per card |
| Out of scope? | Bank's internal ledger, hardware drivers |

### Minute-by-minute plan

- **0–8:** Actors: customer, bank system, technician. Use cases: authenticate, withdraw, deposit, balance, refill cash.
- **8–18:** `ATM` (context) with states `Idle`, `CardInserted`, `Authenticated`, `Dispensing`; `Card`; `BankService` interface; `CashDispenser` with a chain of denomination handlers; `Transaction` subclasses.
- **18–23:** Walk "withdraw": check cash → debit via bank → dispense → print receipt; reverse on dispense failure.
- **23–35:** Code the state transitions and the dispenser chain.

### What a strong answer must include

- [ ] ATM states with events allowed in each.
- [ ] Bank behind an interface; ATM never holds balances.
- [ ] Note dispensing as Chain of Responsibility (largest denomination first).
- [ ] Order: check cash → debit → dispense → on failure, reverse the debit.
- [ ] PIN retry limit and card retention.

### Likely follow-ups and how to answer

- **Debited but no cash:** send a reversal with the same transaction id; the bank treats it idempotently.
- **Two ATMs, one account:** the bank serialises debits per account; the ATM just handles a "declined" reply.
- **Network drop mid-transaction:** retry with the same id; idempotency stops a double debit.
- **Add cardless (QR) withdrawal:** a new authentication state; the rest of the flow is unchanged.

### Where to study it

Ch 14 — 14.4 Design an ATM

## 24.12 Drill — Chess Game

### The prompt as the interviewer says it

> "Design a two-player chess game."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Players? | Two humans; computer player later |
| Full rules? | All moves, including castling, en passant, promotion |
| End conditions? | Checkmate, stalemate, resignation, draw by agreement |
| Clocks? | Optional time control |
| Undo? | Yes, for practice games |
| Out of scope? | Ratings, online matchmaking |

### Minute-by-minute plan

- **0–8:** Actors: two players. Use cases: start, move, undo, resign, offer draw.
- **8–18:** `Game` (status, turn, history), `Board` owns 64 `Square`s, abstract `Piece` with subclasses, `Move` (from, to, captured, special kind), `Player`, `MoveValidator`.
- **18–23:** Walk "make a move": piece's pseudo-legal moves → apply to a copy → reject if own king in check → commit → update status.
- **23–35:** Code `Piece.candidateMoves` for two pieces and `Game.makeMove` with the check test.

### What a strong answer must include

- [ ] Piece movement through polymorphism, one subclass each; sliding pieces share a helper.
- [ ] `Move` as an object (Command) storing enough to undo.
- [ ] Legality = pseudo-legal + "own king not in check after the move".
- [ ] Game status computed after every move (check, mate, stalemate).
- [ ] Turn enforcement in `Game`, not in pieces.

### Likely follow-ups and how to answer

- **Detect checkmate:** in check and no legal move exists; generate all moves and test each.
- **Castling rules:** needs "has moved" flags, empty path, and no attacked squares on the king's path.
- **Add an AI player:** a `Player` strategy that picks a move; the game loop does not change.
- **Why not a giant switch on piece type?** Adding a fairy piece would edit it; a subclass is closed for modification.

### Where to study it

Ch 17 — Design Chess

## 24.13 Drill — Hotel Management System

### The prompt as the interviewer says it

> "Design a hotel management system: booking rooms, check-in, check-out, and billing."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Book a room type or a room? | A type; the room is assigned at check-in |
| Date model? | Nights as half-open date ranges |
| Extra services? | Room service, laundry, added to the bill |
| Cancellation and no-show? | Free until a cutoff; one night charged for no-show |
| Housekeeping? | Room must be cleaned before the next guest |
| Out of scope? | Channel managers, loyalty |

### Minute-by-minute plan

- **0–8:** Actors: guest, receptionist, housekeeper. Use cases: search, reserve, check in, add charge, check out, cancel.
- **8–18:** `Hotel` owns `Room` (type, status); `Guest`; `Reservation` (type, date range, state); `Stay`; `Invoice` with `Charge` lines; `RoomPricing`; `HousekeepingTask`.
- **18–23:** Walk "check out": total charges → pay → room marked dirty → housekeeping task created.
- **23–35:** Code availability per room type over a date range and the reservation state machine.

### What a strong answer must include

- [ ] Availability = count of rooms of a type minus overlapping reservations, per night.
- [ ] Reservation states: booked → checked in → checked out, plus cancelled and no-show.
- [ ] Room status separate from reservation status (occupied, dirty, clean, out of order).
- [ ] Pricing as Strategy (season, weekend).
- [ ] Charges as line items, so new services need no invoice change.

### Likely follow-ups and how to answer

- **Last room, two bookings:** lock per room type and date range check-and-insert; or a per-night counter decremented atomically.
- **Early check-in / late check-out:** a charge line plus a room-status check.
- **Overbooking:** allow a configured margin and a "walk" procedure to a partner hotel.
- **Group bookings:** a parent reservation holding several room reservations, booked all-or-nothing.

### Where to study it

Ch 15 — 15.4 Design a Hotel Management System

## 24.14 Drill — Food Delivery (practice session)

### The prompt as the interviewer says it

> "Design a food delivery app like Swiggy: customers order from restaurants, and delivery agents bring the food."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| One restaurant per order? | Yes; the cart resets if the user switches restaurant |
| Restaurant capacity? | A kitchen cooks at most N items at once |
| Agent assignment? | Nearest free agent when the food is ready; swappable |
| Tracking? | Customer sees order status changes |
| Cancellation? | Until cooking starts; prepaid orders are refunded |
| Out of scope? | Routing maps, payment gateway internals |

### Minute-by-minute plan

- **0–8:** Actors: customer, restaurant, delivery agent, admin. Use cases: browse, add to cart, place order, accept, prepare, assign agent, deliver, cancel.
- **8–18:** `Restaurant` (menu, capacity, rating), `MenuItem`, `Cart`, `Order` (state), `DeliveryAgent` (status, location), `RestaurantSelectionStrategy`, `AgentAssignmentStrategy`, `OrderService`, `Notifier`.
- **18–23:** Walk "place order to delivered": cart → order placed → restaurant accepts (capacity taken) → ready → agent assigned → picked up → delivered (capacity freed).
- **23–35:** Code `Order` state transitions and capacity handling in `OrderService`.

### What a strong answer must include

- [ ] Order state machine with who may trigger each transition.
- [ ] Kitchen capacity taken on accept, released when the food is ready or the order is cancelled; a refusal for a full kitchen refunds a prepaid order.
- [ ] Restaurant selection and agent assignment as separate Strategies.
- [ ] Atomic claim of a free agent.
- [ ] Status updates through Observer.

### Likely follow-ups and how to answer

- **Two orders, one agent:** compare-and-set the agent's status; the loser goes to the next candidate.
- **Restaurant never accepts:** a timeout auto-cancels and refunds.
- **Batch two orders to one agent:** assignment strategy that allows a second pickup within a detour limit.
- **Pick the cheapest restaurant for a whole cart (FoodKart):** a selection strategy across restaurants serving every item.

### Where to study it

Ch 19 — Design an Online Food Delivery Service (Swiggy) + FoodKart

## 24.15 Drill — Amazon Online Shopping System

### The prompt as the interviewer says it

> "Design the core of Amazon: browse products, add to cart, check out, and track the order."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Sellers? | Many; a product may be sold by several |
| Stock? | Per product per warehouse |
| Reserve stock when? | At checkout, released if payment fails |
| Payments? | Card, wallet, cash on delivery |
| Order changes? | Cancel before shipping; return after delivery |
| Out of scope? | Recommendations, search ranking |

### Minute-by-minute plan

- **0–8:** Actors: customer, seller, admin, payment system. Use cases: search, add to cart, checkout, pay, track, cancel, return, review.
- **8–18:** `Product`, `Catalog`, `Cart` with `CartItem`, `Order` with `OrderItem` (price copied), `Inventory`, `Payment` (Strategy), `Shipment`, `Customer`, `Notifier`.
- **18–23:** Walk "checkout": validate cart → reserve stock → create order → pay → confirm, or release on failure.
- **23–35:** Code `CheckoutService.checkout` and `Order` state transitions.

### What a strong answer must include

- [ ] `OrderItem` copies price at purchase time; the cart reads live prices.
- [ ] Stock reserved atomically for all lines, released on failure.
- [ ] Order states: placed → paid → shipped → delivered, plus cancelled and returned.
- [ ] Payment and shipping methods as Strategy.
- [ ] Notifications via Observer on state change.

### Likely follow-ups and how to answer

- **Price changes while in the cart:** recheck at checkout and show the new price before paying.
- **Last unit, two buyers:** conditional decrement on stock; the loser gets "out of stock" at checkout.
- **Partial cancellation:** cancel per `OrderItem`, refund that line, release its stock.
- **Payment timeout:** reservation has an expiry; a sweeper releases stock.

### Where to study it

Ch 19 — Design Amazon — Online Shopping System

## 24.16 Drill — Stack Overflow

### The prompt as the interviewer says it

> "Design Stack Overflow: users ask questions, answer them, vote, and earn reputation."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Content types? | Question, answer, comment; tags on questions |
| Voting? | Up/down on questions and answers, one vote per user per post |
| Reputation? | Changes on votes and accepted answers; unlocks privileges |
| Bounties and badges? | Yes, as extensions |
| Moderation? | Close, delete, flag |
| Out of scope? | Full-text search engine internals, chat |

### Minute-by-minute plan

- **0–8:** Actors: guest, member, moderator, system. Use cases: ask, answer, comment, vote, accept, search, close.
- **8–18:** `User` (reputation), abstract `Post` → `Question`, `Answer`; `Comment`; `Vote`; `Tag`; `Badge`; `Bounty`; `ReputationService`; `SearchService`.
- **18–23:** Walk "upvote an answer": check privilege → record vote (unique per user and post) → update score → reputation event to the author.
- **23–35:** Code `VoteService.vote` and the reputation listener.

### What a strong answer must include

- [ ] Shared `Post` base for voting and commenting; `Question` adds title, tags and accepted answer.
- [ ] `Vote` keyed by (user, post) so a second vote replaces, never adds.
- [ ] Reputation changed by events (Observer), not inline in the vote code.
- [ ] Privileges as reputation thresholds in one place.
- [ ] Search separate from posting.

### Likely follow-ups and how to answer

- **Concurrent votes on a hot post:** atomic score counter; the unique (user, post) key stops double voting.
- **Changing a vote:** reverse the old reputation delta, then apply the new one.
- **Bounties:** reputation is escrowed when offered and paid on award or expiry.
- **Self-voting / vote rings:** a rule in the vote service; detection can run later as a job.

### Where to study it

Ch 18 — 18.1 Design Stack Overflow

## 24.17 Drill — Restaurant Management System

### The prompt as the interviewer says it

> "Design a restaurant system: table reservations, taking orders, the kitchen, and the bill."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Reservations and walk-ins? | Both; walk-ins join a waitlist |
| Tables? | Sizes 2/4/6; tables may be joined |
| Ordering? | Waiter enters orders per table; items go to kitchen stations |
| Bill? | Per table, with tax and tip; may be split |
| Menu changes? | Items can be marked unavailable |
| Out of scope? | Inventory purchasing, payroll |

### Minute-by-minute plan

- **0–8:** Actors: guest, host, waiter, chef, cashier. Use cases: reserve, seat, order, prepare, serve, bill, pay.
- **8–18:** `Table` (size, status), `Reservation`, `WaitlistEntry`, `Menu` with `MenuItem`, `Order` with `OrderItem` (state), `Kitchen` with stations, `Bill`, `TableAllocator`.
- **18–23:** Walk "order to bill": waiter places order → items sent to stations → ready → served → bill.
- **23–35:** Code `TableAllocator` (smallest table that fits) and `OrderItem` state transitions.

### What a strong answer must include

- [ ] Table allocation as Strategy, with waitlist fallback.
- [ ] Orders sent to the kitchen as Command objects (queued, can be cancelled).
- [ ] `OrderItem` state: ordered → cooking → ready → served.
- [ ] "Table ready" and "dish ready" via Observer.
- [ ] Bill built from order items plus tax/tip policy; split as a separate step.

### Likely follow-ups and how to answer

- **Reservation vs walk-in for the same table:** hold reserved tables from a time before the slot; walk-ins only get tables free for a full sitting.
- **Split the bill:** by items or evenly; a split strategy producing several bills.
- **Item runs out mid-service:** mark unavailable; pending orders with it are flagged to the waiter.
- **Several waiters ordering for one table:** lock the order per table, or append-only items.

### Where to study it

Ch 16 — Design a Restaurant Management System

## 24.18 Drill — Facebook

### The prompt as the interviewer says it

> "Design the core of Facebook: profiles, friends, posts, a news feed and privacy."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Friends or followers? | Both: mutual friendship and one-way follow |
| Post visibility? | Public, friends, only me, custom list |
| Feed? | Recent posts from friends, followed pages and groups, ranked |
| Interactions? | Like, comment, share |
| Groups and pages? | Yes, simple versions |
| Out of scope? | Ads, messaging, media storage |

### Minute-by-minute plan

- **0–8:** Actors: user, page admin, system. Use cases: send/accept friend request, post, comment, like, view feed, search.
- **8–18:** `User`, `Profile`, `FriendRequest` (state), `Friendship`, `Follow`, `Post` (audience), `Comment`, `Reaction`, `Group`, `Page`, `VisibilityPolicy`, `FeedService` with `FeedRanker`.
- **18–23:** Walk "load feed": collect candidate posts → filter by visibility → rank → page.
- **23–35:** Code `VisibilityPolicy.canSee(viewer, post)` and the feed assembly.

### What a strong answer must include

- [ ] Friendship (two-sided, via request) distinct from follow (one-sided).
- [ ] One visibility policy reused by feed, profile and search.
- [ ] Feed ranking as Strategy.
- [ ] A stated choice of pull (merge at read) vs push (fan-out at write), with the trade-off.
- [ ] Friend request state: pending → accepted / rejected.

### Likely follow-ups and how to answer

- **Push or pull?** Pull is simpler and right for an in-memory design; push makes reads fast but costs writes for users with many friends. Hybrid: push for normal users, pull for celebrities.
- **Privacy changed after posting:** check visibility at read time, never cache it into the feed.
- **Friend suggestions:** friends-of-friends counted by mutual friends (two-hop search).
- **Unfriend:** remove both sides of the friendship atomically.

### Where to study it

Ch 18 — 18.4 Design Facebook

## 24.19 Drill — Online Stock Brokerage System

### The prompt as the interviewer says it

> "Design an online stock brokerage: users fund an account, place buy and sell orders, and see their portfolio."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Order types? | Market and limit; stop-loss later |
| Who matches orders? | An external exchange; we route and track |
| Partial fills? | Yes |
| Funds? | Blocked when a buy is placed, released on cancel |
| Alerts? | Price alerts and watchlists |
| Out of scope? | Margin trading, tax reports |

### Minute-by-minute plan

- **0–8:** Actors: investor, exchange, admin. Use cases: deposit, place order, cancel, receive fill, view portfolio, set alert.
- **8–18:** `Account` (cash, blocked cash), `Portfolio` with `Position`s, abstract `Order` → `MarketOrder`, `LimitOrder` (state), `Fill`, `ExchangeGateway` (Adapter), `OrderService`, `Watchlist`, `PriceAlert`.
- **18–23:** Walk "buy with partial fills": validate → block funds → send → fills update position and cash → filled or cancelled remainder releases funds.
- **23–35:** Code `OrderService.place` and `onFill`.

### What a strong answer must include

- [ ] Funds or shares blocked at placement, not at fill.
- [ ] Order states: new → open → partially filled → filled / cancelled / rejected.
- [ ] Exchange behind an Adapter interface.
- [ ] Money as `BigDecimal` or integer units.
- [ ] Price alerts through Observer.

### Likely follow-ups and how to answer

- **Two orders spend the same cash:** lock per account around check-and-block.
- **Fill arrives after cancel:** the exchange is the truth; apply the fill, cancel only the remainder.
- **Duplicate fill messages:** keep processed fill ids; ignore repeats (idempotency).
- **How would the exchange match?** Per symbol, a price-sorted book per side, matched by price then time.

### Where to study it

Ch 19 — Design an Online Stock Brokerage System

## 24.20 Drill — Jigsaw Puzzle

### The prompt as the interviewer says it

> "Design a jigsaw puzzle: pieces, the board, how to tell whether two pieces fit, and an algorithm to solve it."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Piece shape? | Rectangular grid; each piece has four sides |
| Side kinds? | Flat (border), tab (sticks out), blank (goes in) |
| How do two sides fit? | A tab fits a blank with the same shape id |
| Rotation? | Pieces may be rotated |
| Solve or just play? | Both: a user places pieces; a solver can finish |
| Out of scope? | Image processing, 3-D puzzles |

### Minute-by-minute plan

- **0–8:** Actor: player (the solver is part of the system). Use cases: start (includes validating the box), place piece (includes choosing a rotation), remove, auto-solve.
- **8–18:** `Puzzle`, `Board` (grid of `Placement(piece, rotation)`), immutable `Piece` (four `Side`s), `Side` (kind, shape id), `Direction` enum (TOP, RIGHT, BOTTOM, LEFT), `SolverStrategy`.
- **18–23:** Walk the solver: corners (two flat sides) → borders (one flat side) → inside, each by matching neighbours.
- **23–35:** Code `Side.fits(other)`, `Piece.side(direction, rotation)`, `Board.check` and the solver loop with a side index.

### What a strong answer must include

- [ ] Sides as their own type with a fit rule, not booleans on the piece.
- [ ] Rotation stored in the placement, not the piece; the side facing `d` after `r` turns is `sides[(d − r) mod 4]`.
- [ ] Piece category from flat-side count: corner, border, interior.
- [ ] Solver that fills cells in order and looks up candidates by shape id (hash map), not by scanning all pieces.
- [ ] Board validates a placement against already-placed neighbours.

### Likely follow-ups and how to answer

- **Complexity:** with unique shapes and an index from side to pieces, each cell is O(1) average, so O(n) total; scanning every piece is $O(n^2)$. Look-alike pieces force backtracking, so budget the search.
- **Two pieces fit the same spot:** add a picture-similarity score as a tie-breaker, or backtrack.
- **Undo a placement:** placements as Command objects on a stack.
- **Non-rectangular puzzles:** sides become a list and `Edge` a direction count; most code is unchanged.

### Where to study it

Ch 17 — Design a Jigsaw Puzzle

## 24.21 Drill — Airline Management System

### The prompt as the interviewer says it

> "Design an airline reservation system: search flights, book seats, check in."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Route vs dated departure? | A flight number recurs; each date is its own instance |
| Multi-leg trips? | Yes, booked all-or-nothing |
| Seat classes? | Economy, business; seat selection optional |
| Fares? | Per class, varies with time to departure |
| Changes? | Cancel with refund rules; crew assignment for admin |
| Out of scope? | Loyalty, baggage tracking |

### Minute-by-minute plan

- **0–8:** Actors: customer, agent, admin, crew. Use cases: search, book, pay, select seat, check in, cancel, assign crew.
- **8–18:** `Airport`, `Flight` (route, number), `FlightInstance` (date, aircraft, status), `Aircraft` with `Seat`s, `FlightSeat` (per instance), `Itinerary` with `Reservation`s, `Passenger`, `FarePolicy`, `Payment`.
- **18–23:** Walk "book a two-leg itinerary": hold seats on both instances → pay → confirm both, or release both.
- **23–35:** Code the itinerary hold as all-or-nothing and `FlightSeat` states.

### What a strong answer must include

- [ ] `Flight` vs `FlightInstance` split, and seat state per instance.
- [ ] All-or-nothing booking across legs with release on failure.
- [ ] Fare rules as Strategy.
- [ ] Booking states: held → confirmed → checked in / cancelled.
- [ ] Flight status changes notify passengers (Observer).

### Likely follow-ups and how to answer

- **Two people, same seat:** lock seats in a fixed order across instances to avoid deadlock; check-and-hold together.
- **Flight cancelled:** rebook passengers on the next instance or refund.
- **Overbooking:** allow a margin per class; deny boarding by a priority rule at check-in.
- **Multi-airline itineraries:** an adapter per partner airline behind one booking interface.

### Where to study it

Ch 16 — Design an Airline Management System

## 24.22 Drill — ESPNcricinfo

### The prompt as the interviewer says it

> "Design a cricket site like ESPNcricinfo: live scores, scorecards, commentary and player stats."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Formats? | Test, ODI, T20 |
| Data entry? | An admin enters each ball |
| Live view? | Scorecard and commentary update per ball |
| Stats? | Career stats per player, per format |
| Tournaments? | Series and tournaments with points tables |
| Out of scope? | Video, fantasy games |

### Minute-by-minute plan

- **0–8:** Actors: viewer, scorer/admin, system. Use cases: record ball, view scorecard, follow match, view stats, manage tournament.
- **8–18:** `Tournament`, `Match` (format), `Team`, `Player`, `Innings`, `Over`, `Ball` (runs, extras, wicket), `Wicket`, `Commentary`, `Scorecard`, `MatchFormatRules`.
- **18–23:** Walk "record a ball": validate → append ball → update innings totals and strike → notify subscribers.
- **23–35:** Code `Innings.record(ball)` including extras and strike rotation.

### What a strong answer must include

- [ ] `Ball` as the source event; totals and stats derived from balls.
- [ ] Extras handled correctly: wide and no-ball do not count as legal deliveries.
- [ ] Strike rotation on odd runs and at the end of an over.
- [ ] Format rules (overs, powerplays) behind an interface or Template Method.
- [ ] Live updates via Observer.

### Likely follow-ups and how to answer

- **Scorer corrects a ball:** replace that event and recompute derived totals.
- **Career stats fast to read:** maintain aggregates updated from ball events, rebuildable from the log.
- **Many viewers:** one writer per match; readers get immutable snapshots.
- **Rain-reduced match:** a format rule computes the revised target; the ball model is unchanged.

### Where to study it

Ch 18 — 18.2 Design Cricinfo

## 24.23 Drill — LinkedIn

### The prompt as the interviewer says it

> "Design LinkedIn: profiles, connections, a feed, jobs and messages."

### Clarifying questions to ask

| Ask | Assume |
|---|---|
| Connections? | Two-sided, by invitation; show 1st/2nd/3rd degree |
| Profile? | Experience, education, skills, endorsements |
| Jobs? | Companies post; members apply |
| Feed? | Posts from connections and followed companies |
| Messaging? | One-to-one, between connections |
| Out of scope? | Ads, premium analytics |

### Minute-by-minute plan

- **0–8:** Actors: member, recruiter, company admin. Use cases: edit profile, connect, accept, post, view feed, post job, apply, message.
- **8–18:** `Member`, `Profile` (`Experience`, `Education`, `Skill`), `Invitation` (state), `Connection`, `Company`, `Job`, `Application` (state), `Post`, `Message`, `FeedService`, `ConnectionGraph`.
- **18–23:** Walk "accept invitation": invitation → accepted → connection stored for both sides → feed and suggestions update.
- **23–35:** Code `ConnectionGraph.degree(a, b)` with a bounded breadth-first search, and `Invitation` transitions.

### What a strong answer must include

- [ ] Invitation states: pending → accepted / ignored / withdrawn.
- [ ] Connection symmetric and created for both members in one step.
- [ ] Degree by BFS limited to depth 3.
- [ ] Job applications with their own state (applied → reviewed → rejected / offered).
- [ ] Feed ranking as Strategy; notifications via Observer.

### Likely follow-ups and how to answer

- **Degree on a huge graph:** bidirectional BFS from both ends, stop at depth 3.
- **"People you may know":** second-degree members ranked by mutual connections.
- **Messages to non-connections:** a permission check (premium or recruiter) before sending.
- **Endorsements:** a (skill, endorser) pair stored once per endorser.

### Where to study it

Ch 18 — 18.3 Design LinkedIn

## Chapter summary

- The patterns repeat across problems: an atomic claim of a scarce resource (spot, seat, room, driver, locker), a state machine for the main entity, a Strategy for the rule most likely to change, and Observer for notifications. Recognise which one the interviewer is probing.